Nostr Book of NIPs

compiled by Adam Shannon

Abstract

The Nostr Book of NIPs is a readable compilation of Nostr Implementation Possibilities, the specifications that describe how Nostr clients and relays talk to each other. Instead of the original numerical order, related NIPs are grouped into chapters on communication, social features, groups, relays, payments, and more.

The NIP text is unchanged from the community-maintained nostr-protocol/nips repository. Compiled by Adam Shannon.

Introduction

Welcome to the Nostr Book of NIPs, a readable compilation of the [Nostr Implementation Possibilities](). NIPs are the community's notes on how clients and relays can interoperate. They are possibilities, not a checklist. Nothing forces any app to implement every NIP, and you should not implement one just because it exists.

I did not write the NIPs. Credit belongs to the original authors and to everyone who keeps nostr-protocol/nips up to date. What I have done is take those documents out of numerical order and group them by theme, so you can read "how messaging works" or "how relays advertise themselves" as a chapter instead of hopping between NIP numbers.

Each chapter starts with a short introduction in my own words. The NIP text that follows is unchanged. Where a NIP is marked unrecommended upstream, I leave it in place and say so in the chapter intro, because you will still run into it in old clients and old events.

This snapshot tracks a specific commit of the NIPs repository, recorded on the next page. The protocol moves. If something here disagrees with upstream, upstream wins.

Thanks for picking this up. I hope the grouping makes the protocol easier to learn, and that it helps more people build on Nostr.

How this book is organized

The NIPs in this book are the same documents you would find in the nostr-protocol/nips repository. I have not rewritten them. I have grouped them by what they help you do, and written a short introduction to each group.

Start with the overview if you are new. NIP-01 is the protocol. Everything else is optional. Skip around once you know the event shape.

Unrecommended NIPs are still here. You will meet them in old clients, old events, and old blog posts. The chapter intro will say so, briefly, and point at whatever replaced them.

Chapters

- **Overview** — What Nostr is, and NIP-01, the basic protocol.
- **Communication** — Notes, articles, comments, media, drafts, and messages.
- **Social** — Follows, identity, reactions, reposts, badges, polls, and bookmarks.
- **Groups** — Public chat, relay-based groups, chats, and forum threads.
- **Moderation** — Labels, lists, reports, sensitive content, and proof of work.
- **Relays** — How relays describe themselves, authenticate clients, sync, and get found.
- **Clients** — URIs, bech32 encodings, timestamps, and a BLE experiment.
- **Payments** — Lightning zaps, wallet connect, Cashu, and peer-to-peer orders.
- **Third Parties** — Gift wrap, remote signers, and other services that act on your behalf.
- **Application Features** — Calendars, live streams, wikis, files, git, and other app kinds.
- **Security** — Key handling, payload encryption, HTTP auth, and requests to vanish.
- **Developers** — Browser and Android signers, unknown events, and app handlers.

Source snapshot

This book was built from the following commit of [nostr-protocol/nips](). If something here disagrees with upstream, upstream wins.

```
commit 488b787848fcf1c6c3498c253264b8121b1a9692
Author: hodlbod <jstaab@protonmail.com>
Date:   Tue Sep 1 10:12:31 2026 -0700

    Add i tags to highlights, clarify that r is whatever (#2454)
```

Nostr Overview

Nostr is a simple protocol for publishing notes and other stuff over the internet. The name stands for "Notes and Other Stuff Transmitted by Relays." There is no company in the middle, no single server that can shut you down, and no username you have to ask permission for. You make a keypair, you write events, and you send those events to relays — ordinary servers that store them and hand them to whoever asks.

Everything in Nostr is an event. An event is a small JSON object: a public key (who), a timestamp (when), a kind number (what), some content, optional tags, and a signature that proves you wrote it. Kind 0 is your profile. Kind 1 is a short text note. Later chapters add kinds for messages, zaps, groups, videos, and a long tail of application-specific things. The shape of the event does not change. New features are almost always a new kind, or a new convention for tags.

Your identity is that keypair. The public key is who you are; the private key is how you sign. Anyone can check a signature. Only you can create one. Relays are intentionally dumb. They accept events, store them, and return them to clients. Clients decide which relays to talk to, which events to show, and how to present them. That split — smart clients, simple relays — is the whole design. If a relay misbehaves, you leave it. If a client is bad, you switch clients. Your keys, and the events you signed, go with you.

The next page is NIP-01, the basic protocol. If you only read one NIP, read that one. It defines events, signatures, the WebSocket messages between clients and relays, and the `REQ` filters that make the rest of this book possible. Everything after it is an optional extension of the same idea.

NIP-01

Basic protocol flow description

`draft` `mandatory` `relay`

This NIP defines the basic protocol that should be implemented by everybody. New NIPs may add new optional (or mandatory) fields and messages and features to the structures and flows described here.

Events and signatures

Each user has a keypair. Signatures, public key, and encodings are done according to the [Schnorr signatures standard for the curve `secp256k1`].

The only object type that exists is the `event`, which has the following format on the wire:

```
{
  "id": <32-bytes lowercase hex-encoded sha256 of the serialized event data>,
  "pubkey": <32-bytes lowercase hex-encoded public key of the event creator>,
  "created_at": <unix timestamp in seconds>,
  "kind": <integer between 0 and 65535>,
  "tags": [
    [<arbitrary string>...],
    // ...
  ],
  "content": <arbitrary string>,
  "sig": <64-bytes lowercase hex of the signature of the sha256 hash of the serialized event
        data, which is the same as the "id" field>
}
```

To obtain the `event.id`, we `sha256` the serialized event. The serialization is done over the UTF-8 JSON-serialized string (which is described below) of the following structure:

```
[
  0,
  <pubkey, as a lowercase hex string>,
  <created_at, as a number>,
  <kind, as a number>,
  <tags, as an array of arrays of non-null strings>,
  <content, as a string>
]
```

To prevent implementation differences from creating a different event ID for the same event, the following rules MUST be followed while serializing: - UTF-8 should be used for encoding. - Whitespace, line breaks or other unnecessary formatting should not be included in the output JSON. - The following characters in the content field must be escaped as shown, and all other characters must be included verbatim: - A line break (`0x0A`), use `\n` - A double quote (`0x22`), use `\"` - A backslash (`0x5C`), use `\\` - A carriage return (`0x0D`), use `\r` - A tab character (`0x09`), use `\t` - A backspace, (`0x08`), use `\b` - A form feed, (`0x0C`), use `\f`

Tags

Each tag is an array of one or more strings, with some conventions around them. Take a look at the example below:

```json
{
  "tags": [
    ["e", "5c83da77af1dec6d7289834998ad7aafbd9e2191396d75ec3cc27f5a77226f36", "wss://
        nostr.example.com"],
    ["p", "f7234bd4c1394dda46d09f35bd384dd30cc552ad5541990f98844fb06676e9ca"],
    ["a", "30023:f7234bd4c1394dda46d09f35bd384dd30cc552ad5541990f98844fb06676e9ca:abcd", "wss://
        nostr.example.com"],
    ["alt", "reply"],
    // ...
  ],
  // ...
}
```

The first element of the tag array is referred to as the tag *name* or *key* and the second as the tag *value*. So we can safely say that the event above has an `e` tag set to `"5c83da77af1dec6d7289834998ad7aafbd9e2191396d75ec3cc27f5a77226f36"`, an `alt` tag set to `"reply"` and so on. All elements after the second do not have a conventional name.

This NIP defines 3 standard tags that can be used across all event kinds with the same meaning. They are as follows:

- The `e` tag, used to refer to an event: `["e", <32-bytes lowercase hex of the id of another event>, <recommended relay URL, optional>, <32-bytes lowercase hex of the author's pubkey, optional>]`
- The `p` tag, used to refer to another user: `["p", <32-bytes lowercase hex of a pubkey>, <recommended relay URL, optional>]`
- The `a` tag, used to refer to an addressable or replaceable event
  - for an addressable event: `["a", "<kind integer>:<32-bytes lowercase hex of a pubkey>:<d tag value>", <recommended relay URL, optional>]`
  - for a normal replaceable event: `["a", "<kind integer>:<32-bytes lowercase hex of a pubkey>:", <recommended relay URL, optional>]` (note: include the trailing colon)

As a convention, all single-letter (only english alphabet letters: a-z, A-Z) key tags are expected to be indexed by relays, such that it is possible, for example, to query or subscribe to events that reference the event `"5c83da77af1dec6d7289834998ad7aafbd9e2191396d75ec3cc27f5a77226f36"` by using the `{"#e": ["5c83da77af1dec6d7289834998ad7aafbd9e2191396d75ec3cc27f5a77226f36"]}` filter. Only the first value in any given tag is indexed.

Kinds

Kinds specify how clients should interpret the meaning of each event and the other fields of each event (e.g. an `"r"` tag may have a meaning in an event of kind 1 and an entirely different meaning in an event of kind 10002). Each NIP may define the meaning of a set of kinds that weren't defined elsewhere. NIP-10, for instance, specifies the `kind:1` text note for social media applications.

This NIP defines one basic kind:

- 0: **user metadata**: the content is set to a stringified JSON object {name: <nickname or full name>, about: <short bio>, picture: <url of the image>} describing the user who created the event. Extra metadata fields may be set. A relay may delete older events once it gets a new one for the same pubkey.

And also a convention for kind ranges that allow for easier experimentation and flexibility of relay implementation:

- for kind n such that 1000 <= n < 10000 || 4 <= n < 45 || n == 1 || n == 2, events are **regular**, which means they're all expected to be stored by relays.
- for kind n such that 10000 <= n < 20000 || n == 0 || n == 3, events are **replaceable**, which means that, for each combination of pubkey and kind, only the latest event MUST be stored by relays, older versions MAY be discarded.
- for kind n such that 20000 <= n < 30000, events are **ephemeral**, which means they are not expected to be stored by relays.
- for kind n such that 30000 <= n < 40000, events are **addressable** by their kind, pubkey and d tag value – which means that, for each combination of kind, pubkey and the d tag value, only the latest event MUST be stored by relays, older versions MAY be discarded.

In case of replaceable events with the same timestamp, the event with the lowest id (first in lexical order) should be retained, and the other discarded.

When answering to REQ messages for replaceable events such as {"kinds":[0],"authors":[<hex-key>]}, even if the relay has more than one version stored, it SHOULD return just the latest one.

These are just conventions and relay implementations may differ.

Communication between clients and relays

Relays expose a websocket endpoint to which clients can connect. Clients SHOULD open a single websocket connection to each relay and use it for all their subscriptions. Relays MAY limit number of connections from specific IP/client/etc.

Relays MUST only accept connections to a single endpoint when additional path segments do not influence its behavior.

From client to relay: sending events and creating subscriptions

Clients can send 3 types of messages, which must be JSON arrays, according to the following patterns:

- ["EVENT", <event JSON as defined above>], used to publish events.
- ["REQ", <subscription_id>, <filters1>, <filters2>, ...], used to request events and subscribe to new updates.
- ["CLOSE", <subscription_id>], used to stop previous subscriptions.

<subscription_id> is an arbitrary, non-empty string of max length 64 chars. It represents a subscription per connection. Relays MUST manage <subscription_id>s independently for each WebSocket connection. <subscription_id>s are not guaranteed to be globally unique.

`<filtersX>` is a JSON object that determines what events will be sent in that subscription, it can have the following attributes:

```json
{
  "ids": <a list of event ids>,
  "authors": <a list of lowercase pubkeys, the pubkey of an event must be one of these>,
  "kinds": <a list of a kind numbers>,
  "#<single-letter (a-zA-Z)>": <a list of tag values, for #e — a list of event ids, for #p — a
        list of pubkeys, etc.>,
  "since": <an integer unix timestamp in seconds. Events must have a created_at >= to this to
        pass>,
  "until": <an integer unix timestamp in seconds. Events must have a created_at <= to this to
        pass>,
  "limit": <maximum number of events relays SHOULD return in the initial query>
}
```

Upon receiving a `REQ` message, the relay SHOULD return events that match the filter. Any new events it receives SHOULD be sent to that same websocket until the connection is closed, a `CLOSE` event is received with the same `<subscription_id>`, or a new `REQ` is sent using the same `<subscription_id>` (in which case a new subscription is created, replacing the old one).

Filter attributes containing lists (`ids`, `authors`, `kinds` and tag filters like `#e`) are JSON arrays with one or more values. At least one of the arrays' values must match the relevant field in an event for the condition to be considered a match. For scalar event attributes such as `authors` and `kind`, the attribute from the event must be contained in the filter list. In the case of tag attributes such as `#e`, for which an event may have multiple values, the event and filter condition values must have at least one item in common.

The `ids`, `authors`, `#e` and `#p` filter lists MUST contain exact 64-character lowercase hex values.

The `since` and `until` properties can be used to specify the time range of events returned in the subscription. If a filter includes the `since` property, events with `created_at` greater than or equal to `since` are considered to match the filter. The `until` property is similar except that `created_at` must be less than or equal to `until`. In short, an event matches a filter if `since <= created_at <= until` holds.

All conditions of a filter that are specified must match for an event for it to pass the filter, i.e., multiple conditions are interpreted as `&&` conditions.

A `REQ` message may contain multiple filters. In this case, events that match any of the filters are to be returned, i.e., multiple filters are to be interpreted as `||` conditions.

The `limit` property of a filter is only valid for the initial query and MUST be ignored afterwards. When `limit: n` is present it is assumed that the events returned in the initial query will be the last `n` events ordered by the `created_at`. Newer events should appear first, and in the case of ties the event with the lowest id (first in lexical order) should be first. Relays SHOULD use the `limit` value to guide how many events are returned in the initial response. Returning fewer events is acceptable, but returning (much) more should be avoided to prevent overwhelming clients.

From relay to client: sending events and notices

Relays can send 5 types of messages, which must also be JSON arrays, according to the following patterns:

- ["EVENT", <subscription_id>, <event JSON as defined above>], used to send events requested by clients.
- ["OK", <event_id>, <true|false>, <message>], used to indicate acceptance or denial of an EVENT message.
- ["EOSE", <subscription_id>], used to indicate the *end of stored events* and the beginning of events newly received in real-time.
- ["CLOSED", <subscription_id>, <message>], used to indicate that a subscription was ended on the server side.
- ["NOTICE", <message>], used to send human-readable error messages or other things to clients.

This NIP defines no rules for how NOTICE messages should be sent or treated.

- EVENT messages MUST be sent only with a subscription ID related to a subscription previously initiated by the client (using the REQ message above).
- OK messages MUST be sent in response to EVENT messages received from clients, they must have the 3rd parameter set to true when an event has been accepted by the relay, false otherwise. The 4th parameter MUST always be present, but MAY be an empty string when the 3rd is true, otherwise it MUST be a string formed by a machine-readable single-word prefix followed by a : and then a human-readable message. Some examples:
  - ["OK", "b1a649ebe8...", true, ""]
  - ["OK", "b1a649ebe8...", true, "pow: difficulty 25>=24"]
  - ["OK", "b1a649ebe8...", true, "duplicate: already have this event"]
  - ["OK", "b1a649ebe8...", false, "blocked: you are banned from posting here"]
  - ["OK", "b1a649ebe8...", false, "blocked: please register your pubkey at https://my-expensive-relay.example.com"]
  - ["OK", "b1a649ebe8...", false, "rate-limited: slow down there chief"]
  - ["OK", "b1a649ebe8...", false, "invalid: event creation date is too far off from the current time"]
  - ["OK", "b1a649ebe8...", false, "pow: difficulty 26 is less than 30"]
  - ["OK", "b1a649ebe8...", false, "restricted: not allowed to write."]
  - ["OK", "b1a649ebe8...", false, "error: could not connect to the database"]
  - ["OK", "b1a649ebe8...", false, "mute: no one was listening to your ephemeral event and it wasn't handled in any way, it was ignored"]
- CLOSED messages MUST be sent in response to a REQ when the relay refuses to fulfill it. It can also be sent when a relay decides to kill a subscription on its side before a client has disconnected or sent a CLOSE. This message uses the same pattern of OK messages with the machine-readable prefix and human-readable message. Some examples:
  - ["CLOSED", "sub1", "unsupported: filter contains unknown elements"]
  - ["CLOSED", "sub1", "error: could not connect to the database"]
  - ["CLOSED", "sub1", "error: shutting down idle subscription"]
- The standardized machine-readable prefixes for OK and CLOSED are: duplicate, pow, blocked, rate-limited, invalid, restricted, mute and error for when none of that fits.

Communication

Most people meet Nostr as a feed of short notes. NIP-10 is that feed: kind 1 text notes, plus the `e` and `p` tags that turn a pile of notes into threads. A subject line (NIP-14) is optional. When a note is too small, NIP-23 defines long-form articles, and NIP-24 collects extra profile and note metadata that clients have grown used to.

Not every reply belongs in a kind 1 thread. NIP-22 comments can hang off almost any event — an article, a video, a product listing — without taking over the original conversation. Picture-first feeds (NIP-68), video events (NIP-71), and voice messages (NIP-A0) give those conversations other shapes. NIP-92 describes how to attach media so clients can display it well. Drafts (NIP-37) let you keep unfinished work encrypted until you are ready to publish.

Messaging has a few flavors. Public messages (NIP-A4) are plaintext notes aimed at someone, meant for notification screens rather than a chat history. Private direct messages (NIP-17) are the current recommendation for DMs; they sit on top of the encryption in NIP-44, which appears later in the security chapter. The older encrypted DM scheme (NIP-04) is unrecommended — the encryption is weak by modern standards. NIP-EE's MLS-based messaging is unrecommended too, superseded by the Marmot protocol outside this repository.

Two housekeeping NIPs round out the chapter. NIP-40 lets an event expire so relays can drop it. NIP-09 is how you ask relays to delete something you published. Neither is a guarantee — relays can ignore you — but they are the protocol's polite way of saying "I'm done with this."

In this chapter

- NIP-10 — Text notes and threads
- NIP-14 — Subject tag in text events
- NIP-23 — Long-form content
- NIP-24 — Extra metadata fields and tags
- NIP-22 — Comments
- NIP-68 — Picture-first feeds
- NIP-71 — Video events
- NIP-A0 — Voice messages
- NIP-92 — Media attachments
- NIP-37 — Draft events
- NIP-A4 — Public messages
- NIP-17 — Private direct messages
- NIP-04 — Encrypted direct messages (unrecommended; use NIP-17)
- NIP-EE — MLS messaging (unrecommended; superseded by Marmot)
- NIP-40 — Expiration timestamp
- NIP-09 — Event deletion request

NIP-10

Text Notes and Threads

`draft` `optional`

This NIP defines `kind:1` as a simple plaintext note.

Abstract

The `.content` property contains some human-readable text.

`e` tags can be used to define note thread roots and replies. They SHOULD be sorted by the reply stack from root to the direct parent.

`q` tags MAY be used when citing events in the `.content` with [NIP-21](21.md).

```
["q", "<event-id> or <event-address>", "<relay-url>", "<pubkey-if-a-regular-event>"]
```

Authors of the `e` and `q` tags SHOULD be added as `p` tags to notify of a new reply or quote.

Markup languages such as markdown and HTML SHOULD NOT be used.

Marked "e" tags (PREFERRED)

Kind 1 events with `e` tags are replies to other kind 1 events. Kind 1 replies MUST NOT be used to reply to other kinds, use [NIP-22](22.md) instead.

```
["e", <event-id>, <relay-url>, <marker>, <pubkey>]
```

Where:

- `<event-id>` is the id of the event being referenced.
- `<relay-url>` is the URL of a recommended relay associated with the reference. Clients SHOULD add a valid `<relay-url>` field, but may instead leave it as `""`.
- `<marker>` is optional and if present is one of `"reply"`, `"root"`.
- `<pubkey>` is optional, SHOULD be the pubkey of the author of the referenced event

Those marked with `"reply"` denote the id of the reply event being responded to. Those marked with `"root"` denote the root id of the reply thread being responded to. For top level replies (those replying directly to the root event), only the `"root"` marker should be used.

A direct reply to the root of a thread should have a single marked "e" tag of type "root".

> This scheme is preferred because it allows events to mention others without confusing them with `<reply-id>` or `<root-id>`.

`<pubkey>` SHOULD be the pubkey of the author of the `e` tagged event, this is used in the outbox model to search for that event from the authors write relays where relay hints did not resolve the event.

The "p" tag

Used in a text event contains a list of pubkeys used to record who is involved in a reply thread.

When replying to a text event E the reply event's "p" tags should contain all of E's "p" tags as well as the `"pubkey"` of the event being replied to.

Example: Given a text event authored by `a1` with "p" tags [`p1`, `p2`, `p3`] then the "p" tags of the reply should be [`a1`, `p1`, `p2`, `p3`] in no particular order.

Deprecated Positional "e" tags

This scheme is not in common use anymore and is here just to keep backward compatibility with older events on the network.

Positional `e` tags are deprecated because they create ambiguities that are difficult, or impossible to resolve when an event references another but is not a reply.

They use simple `e` tags without any marker.

`["e", <event-id>, <relay-url>]` as per NIP-01.

Where:

- `<event-id>` is the id of the event being referenced.
- `<relay-url>` is the URL of a recommended relay associated with the reference. Many clients treat this field as optional.

The positions of the "e" tags within the event denote specific meanings as follows:

- No "e" tag:
  This event is not a reply to, nor does it refer to, any other event.

- One "e" tag:
  `["e", <id>]`: The id of the event to which this event is a reply.

- Two "e" tags: `["e", <root-id>]`, `["e", <reply-id>]`
  `<root-id>` is the id of the event at the root of the reply chain. `<reply-id>` is the id of the article to which this event is a reply.

- Many "e" tags: `["e", <root-id>]` `["e", <mention-id>]`, …, `["e", <reply-id>]`
  There may be any number of `<mention-ids>`. These are the ids of events which may, or may not be in the reply chain. They are citing from this event. `root-id` and `reply-id` are as above.

NIP-14

Subject tag in Text events

`draft` `optional`

This NIP defines the use of the "subject" tag in text (kind: 1) events. (implemented in more-speech)

```
["subject": <string>]
```

Browsers often display threaded lists of messages. The contents of the subject tag can be used in such lists, instead of the more ad hoc approach of using the first few words of the message. This is very similar to the way email browsers display lists of incoming emails by subject rather than by contents.

When replying to a message with a subject, clients SHOULD replicate the subject tag. Clients MAY adorn the subject to denote that it is a reply. e.g. by prepending "Re:".

Subjects should generally be shorter than 80 chars. Long subjects will likely be trimmed by clients.

NIP-23

Long-form Content

`draft` `optional`

This NIP defines `kind:30023` (an *addressable event*) for long-form text content, generally referred to as "articles" or "blog posts".

Deprecated: `kind:30024` was used for long-form drafts (self-encrypted nip04, same format as `kind:30023`). The preferred way of doing long-form drafts is to use [NIP-37](NIP-37) instead.

"Social" clients that deal primarily with `kind:1` notes should not be expected to implement this NIP.

Format

The `.content` of these events should be a string text in Markdown syntax. To maximize compatibility and readability between different clients and devices, any client that is creating long form notes:

- MUST NOT hard line-break paragraphs of text, such as arbitrary line breaks at 80 column boundaries.

- MUST NOT support adding HTML to Markdown.

Metadata

For the date of the last update the `.created_at` field should be used, for "tags"/"hashtags" (i.e. topics about which the event might be of relevance) the `t` tag should be used.

Other metadata fields can be added as tags to the event as necessary. Here we standardize 4 that may be useful, although they remain strictly optional:

- `"title"`, for the article title
- `"image"`, for a URL pointing to an image to be shown along with the title
- `"summary"`, for the article summary
- `"published_at"`, for the timestamp in unix seconds (stringified) of the first time the article was published

Editability

These articles are meant to be editable, so they should include a `d` tag with an identifier for the article. Clients should take care to only publish and read these events from relays that implement that. If they don't do that they should also take care to hide old versions of the same article they may receive.

Linking

The article may be linked to using the [NIP-19](19.md) `naddr` code along with the `a` tag.

References

References to other Nostr notes, articles or profiles must be made according to [NIP-27](27.md), i.e. by using [NIP-21](21.md) `nostr:...` links and optionally adding tags for these (see example below).

Example Event

```json
{
  "kind": 30023,
  "created_at": 1675642635,
  "content": "Lorem [ipsum]
      [nostr:nevent1qqst8cujky046negxgwwm5ynqwn53t8aqjr6afd8g59nfqwxpdhylpcpzamhxue69uhhyetvv9ujuetcv9khqm
      consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut e
      exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in repre
      cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culp
      anim id est laborum.\n\nRead more at
      nostr:naddr1qqzkjurnw4ksz9thwden5te0wfjkccte9ehx7um5wghx7un8qgs2d90kkcq3nk2jry62dyf50k0h36rhpdtd594m

  "tags": [
    ["d", "lorem-ipsum"],
    ["title", "Lorem Ipsum"],
    ["published_at", "1296962229"],
    ["t", "placeholder"],
    ["e", "b3e392b11f5d4f28321cedd09303a748acfd0487aea5a7450b3481c60b6e4f87", "wss://
        relay.example.com"],
    ["a", "30023:a695f6b60119d9521934a691347d9f78e8770b56da16bb255ee286ddf9fda919:ipsum",
        "wss://relay.nostr.org"]
  ],
  "pubkey": "...",
  "id": "..."
}
```

Replies & Comments

Replies to `kind 30023` MUST use [NIP-22](22.md) `kind 1111` comments.

NIP-24

Extra metadata fields and tags

`draft` `optional`

This NIP keeps track of extra optional fields that can added to events which are not defined anywhere else but have become *de facto* standards and other minor implementation possibilities that do not deserve their own NIP and do not have a place in other NIPs.

kind 0

These are extra fields not specified in NIP-01 that may be present in the stringified JSON of metadata events:

- `display_name`: an alternative, bigger name with richer characters than `name`. `name` should always be set regardless of the presence of `display_name` in the metadata.
- `website`: a web URL related in any way to the event author.
- `banner`: an URL to a wide (~1024x768) picture to be optionally displayed in the background of a profile screen.
- `bot`: a boolean to clarify that the content is entirely or partially the result of automation, such as with chatbots or newsfeeds.
- `birthday`: an object representing the author's birth date. The format is { "year": number, "month": number, "day": number }. Each field MAY be omitted.

Deprecated fields

These are fields that should be ignored or removed when found in the wild:

- `displayName`: use `display_name` instead.
- `username`: use `name` instead.

kind 3

These are extra fields not specified in NIP-02 that may be present in the stringified JSON of follow events:

Deprecated fields

- `{<relay-url>: {"read": <true|false>, "write": <true|false>}, ...}`: an object of relays used by a user to read/write. NIP-65 should be used instead.

tags

These tags may be present in multiple event kinds. Whenever a different meaning is not specified by some more specific NIP, they have the following meanings:

- `r`: a web URL the event is referring to in some way.
- `i`: an external id the event is referring to in some way - see NIP-73.
- `title`: name of NIP-51 sets, NIP-52 calendar event, NIP-53 live event or NIP-99 listing.
- `t`: a hashtag. The value MUST be a lowercase string.

NIP-22

Comment

`draft` `optional`

A comment is a threading note always scoped to a root event or an I-tag.

It uses `kind:1111` with plaintext `.content` (no HTML, Markdown, or other formatting).

Comments MUST point to the root scope using uppercase tag names (e.g. `K`, `E`, `A` or `I`) and MUST point to the parent item with lowercase ones (e.g. `k`, `e`, `a` or `i`).

Comments MUST point to the authors when one is available (i.e. tagging a nostr event). `P` for the root scope and `p` for the author of the parent item.

```
{
  "kind": 1111,
  "content": "<comment>",
  "tags": [
    // root scope: event addresses, event ids, or I-tags.
    ["<A, E, I>", "<address, id or I-value>", "<relay or web page hint>",
        "<root event's pubkey, if an E tag>"],
    // the root item kind
    ["K", "<root kind>"],

    // pubkey of the author of the root scope event
    ["P", "<root-pubkey>", "relay-url-hint"],

    // parent item: event addresses, event ids, or i-tags.
    ["<a, e, i>", "<address, id or i-value>", "<relay or web page hint>", "<parent event's
        pubkey, if an e tag>"],
    // parent item kind
    ["k", "<parent comment kind>"],

    // parent item pubkey
    ["p", "<parent-pubkey>", "relay-url-hint"]
  ]
  // other fields
}
```

Tags `K` and `k` MUST be present to define the event kind of the root and the parent items.

`I` and `i` tags create scopes for hashtags, geohashes, URLs, and other external identifiers.

The possible values for `i` tags – and `k` tags, when related to an external identity – are listed on NIP-73. Their uppercase versions use the same type of values but relate to the root item instead of the parent one.

`q` tags MAY be used when citing events in the `.content` with NIP-21.

```
["q", "<event-id> or <event-address>", "<relay-url>", "<pubkey-if-a-regular-event>"]
```

`p` tags SHOULD be used when mentioning pubkeys in the `.content` with NIP-21.

Examples

A comment on a blog post looks like this:

```
{
  "kind": 1111,
  "content": "Great blog post!",
  "tags": [
    // top-level comments scope to event addresses or ids
    ["A", "30023:3c9849383bdea883b0bd16fece1ed36d37e37cdde3ce43b17ea4e9192ec11289:f9347ca7",
        "wss://example.relay"],
    // the root kind
    ["K", "30023"],
    // author of root event
    ["P", "3c9849383bdea883b0bd16fece1ed36d37e37cdde3ce43b17ea4e9192ec11289", "wss://
        example.relay"]

    // the parent event address (same as root for top-level comments)
    ["a", "30023:3c9849383bdea883b0bd16fece1ed36d37e37cdde3ce43b17ea4e9192ec11289:f9347ca7",
        "wss://example.relay"],
    // when the parent event is replaceable or addressable, also include an `e` tag referencing
        its id
    ["e", "5b4fc7fed15672fefe65d2426f67197b71ccc82aa0cc8a9e94f683eb78e07651", "wss://
        example.relay"],
    // the parent event kind
    ["k", "30023"],
    // author of the parent event
    ["p", "3c9849383bdea883b0bd16fece1ed36d37e37cdde3ce43b17ea4e9192ec11289", "wss://
        example.relay"]
  ]
  // other fields
}
```

A comment on a NIP-94 file looks like this:

```
{
  "kind": 1111,
  "content": "Great file!",
  "tags": [
    // top-level comments have the same scope and reply to addresses or ids
    ["E", "768ac8720cdeb59227cf95e98b66560ef03d8bc9a90d721779e76e68fb42f5e6", "wss://
        example.relay", "3721e07b079525289877c366ccab47112bdff3d1b44758ca333feb2dbbbbe5bb"],
    // the root kind
    ["K", "1063"],
    // author of the root event
    ["P", "3721e07b079525289877c366ccab47112bdff3d1b44758ca333feb2dbbbbe5bb"],

    // the parent event id (same as root for top-level comments)
    ["e", "768ac8720cdeb59227cf95e98b66560ef03d8bc9a90d721779e76e68fb42f5e6", "wss://
        example.relay", "3721e07b079525289877c366ccab47112bdff3d1b44758ca333feb2dbbbbe5bb"],
    // the parent kind
    ["k", "1063"],
    ["p", "3721e07b079525289877c366ccab47112bdff3d1b44758ca333feb2dbbbbe5bb"]
  ]
  // other fields
}
```

A reply to a comment looks like this:

```
{
  "kind": 1111,
  "content": "This is a reply to \"Great file!\"",
  "tags": [
    // nip-94 file event id
    ["E", "768ac8720cdeb59227cf95e98b66560ef03d8bc9a90d721779e76e68fb42f5e6", "wss://
        example.relay", "fd913cd6fa9edb8405750cd02a8bbe16e158b8676c0e69fdc27436cc4a54cc9a"],
    // the root kind
    ["K", "1063"],
    ["P", "fd913cd6fa9edb8405750cd02a8bbe16e158b8676c0e69fdc27436cc4a54cc9a"],

    // the parent event
    ["e", "5c83da77af1dec6d7289834998ad7aafbd9e2191396d75ec3cc27f5a77226f36", "wss://
        example.relay", "93ef2ebaaf9554661f33e79949007900bbc535d239a4c801c33a4d67d3e7f546"],
    // the parent kind
    ["k", "1111"],
    ["p", "93ef2ebaaf9554661f33e79949007900bbc535d239a4c801c33a4d67d3e7f546"]
  ]
  // other fields
}
```

A comment on a website's url looks like this:

```
{
  "kind": 1111,
  "content": "Nice article!",
  "tags": [
    // referencing the root url
    ["I", "https://abc.com/articles/1"],
    // the root "kind": for an url
    ["K", "web"],

    // the parent reference (same as root for top-level comments)
    ["i", "https://abc.com/articles/1"],
    // the parent "kind": for an url
    ["k", "web"]
  ]
  // other fields
}
```

A podcast comment example:

```
{
  "id": "80c48d992a38f9c445b943a9c9f1010b396676013443765750431a9004bdac05",
  "pubkey": "252f10c83610ebca1a059c0bae8255eba2f95be4d1d7bcfa89d7248a82d9f111",
  "kind": 1111,
  "content": "This was a great episode!",
  "tags": [
    // podcast episode reference
    ["I", "podcast:item:guid:d98d189b-dc7b-45b1-8720-d4b98690f31f", "https://fountain.fm/
        episode/z1y9TMQRuqXl2awyrQxg"],
    // podcast episode type
    ["K", "podcast:item:guid"],
```

```
    // same value as "I" tag above, because it is a top-level comment (not a reply to a comment)
    ["i", "podcast:item:guid:d98d189b-dc7b-45b1-8720-d4b98690f31f", "https://fountain.fm/
        episode/z1y9TMQRuqXl2awyrQxg"],
    ["k", "podcast:item:guid"]
  ]
  // other fields
}
```

A reply to a podcast comment:

```
{
  "kind": 1111,
  "content": "I'm replying to the above comment.",
  "tags": [
    // podcast episode reference
    ["I", "podcast:item:guid:d98d189b-dc7b-45b1-8720-d4b98690f31f", "https://fountain.fm/
        episode/z1y9TMQRuqXl2awyrQxg"],
    // podcast episode type
    ["K", "podcast:item:guid"],

    // this is a reference to the above comment
    ["e", "80c48d992a38f9c445b943a9c9f1010b396676013443765750431a9004bdac05", "wss://
        example.relay", "252f10c83610ebca1a059c0bae8255eba2f95be4d1d7bcfa89d7248a82d9f111"],
    // the parent comment kind
    ["k", "1111"],
    ["p", "252f10c83610ebca1a059c0bae8255eba2f95be4d1d7bcfa89d7248a82d9f111"]
  ]
  // other fields
}
```

NIP-68

Picture-first feeds

`draft` `optional`

This NIP defines event kind `20` for picture-first clients. Images must be self-contained. They are hosted externally and referenced using `imeta` tags.

The idea is for this type of event to cater to Nostr clients resembling platforms like Instagram, Flickr, Snapchat, or 9GAG, where the picture itself takes center stage in the user experience.

Picture Events

Picture events contain a `title` tag and description in the `.content`.

They may contain multiple images to be displayed as a single post.

```
{
  "id": <32-bytes lowercase hex-encoded SHA-256 of the the serialized event data>,
  "pubkey": <32-bytes lowercase hex-encoded public key of the event creator>,
  "created_at": <Unix timestamp in seconds>,
```

```
"kind": 20,
"content": "<description of post>",
"tags": [
  ["title", "<short title of post>"],

  // Picture Data
  [
    "imeta",
    "url https://nostr.build/i/my-image.jpg",
    "m image/jpeg",
    "thumbhash zOcFFIIKmXdCinaXaHcmcHUFSA==",
    "blurhash eVF$^OI:${M{o#*0-nNFxakD-?xVM}WEWB%iNKxvR-oetmo#R-aen$",
    "dim 3024x4032",
    "alt A scenic photo overlooking the coast of Costa Rica",
    "x <sha256 hash as specified in NIP 94>",
    "fallback https://nostrcheck.me/alt1.jpg",
    "fallback https://void.cat/alt1.jpg"
  ],
  [
    "imeta",
    "url https://nostr.build/i/my-image2.jpg",
    "m image/jpeg",
    "thumbhash zOcFFIIKmXgzmWaXWIcmcFQEGA",
    "dim 3024x4032",
    "alt Another scenic photo overlooking the coast of Costa Rica",
    "x <sha256 hash as specified in NIP 94>",
    "fallback https://nostrcheck.me/alt2.jpg",
    "fallback https://void.cat/alt2.jpg",

    "annotate-user <32-bytes hex of a pubkey>:<posX>:<posY>" // Tag users in specific
      locations in the picture
  ],

  ["content-warning", "<reason>"], // if NSFW

  // Tagged users
  ["p", "<32-bytes hex of a pubkey>", "<optional recommended relay URL>"],
  ["p", "<32-bytes hex of a pubkey>", "<optional recommended relay URL>"],

  // Specify the media type for filters to allow clients to filter by supported kinds
  ["m", "image/jpeg"],

  // Hashes of each image to make them queryable
  ["x", "<sha256>"]

  // Hashtags
  ["t", "<tag>"],
  ["t", "<tag>"],

  // location
  ["location", "<location>"], // city name, state, country
  ["g", "<geohash>"],

  // When text is written in the image, add the tag to represent the language
  ["L", "ISO-639-1"],
```

```
    ["l", "en", "ISO-639-1"]
  ]
}
```

The `imeta` tag `annotate-user` places a user link in the specific position in the image.

Only the following media types are accepted: - `image/apng`: Animated Portable Network Graphics (APNG) - `image/avif`: AV1 Image File Format (AVIF) - `image/gif`: Graphics Interchange Format (GIF) - `image/jpeg`: Joint Photographic Expert Group image (JPEG) - `image/png`: Portable Network Graphics (PNG) - `image/webp`: Web Picture format (WEBP)

Picture events might be used with NIP-71's kind `22` to display short vertical videos in the same feed.

NIP-71

Video Events

`draft` `optional`

This specification defines *video* events representing a dedicated post of externally hosted content.

Unlike a `kind:1` event with a video attached, video events are meant to contain all additional metadata concerning the subject media and to be surfaced in video-specific clients rather than general micro-blogging clients. The thought is for events of this kind to be referenced in a Netflix, YouTube, or TikTok like nostr client where the video itself is at the center of the experience.

Video Events

There are two types of video events represented by different kinds: *normal* and *short* video events. This is meant to allow clients to cater to each as the viewing experience for longer, mostly horizontal (landscape) videos is often different than that of short-form, mostly vertical (portrait), videos ("stories", "reels", "shorts" etc).

Nothing except cavaliership and common sense prevents a *short* video from being long, or a *normal* video from being vertical, and that may or may not be justified, it's mostly a stylistic qualitative difference, not a question of actual raw size.

Format

The format uses a *regular event* kind `21` for *normal* videos and `22` for *short* videos.

Addressable Video Events

For content that may need updates after publication (such as correcting metadata, descriptions, or handling URL migrations), addressable versions are available:

- Kind `34235` for *addressable normal videos*
- Kind `34236` for *addressable short videos*

These addressable events follow the same format as their regular counterparts but include a `d` tag as a unique identifier and can be updated while maintaining the same addressable reference. This is particularly useful for:

- Metadata corrections (descriptions, titles, tags) without republishing
- Preservation of imported content IDs from legacy platforms
- URL migration when hosting changes
- Platform migration tracking

The `.content` of these events is a summary or description on the video content.

The primary source of video information is the `imeta` tags which is defined in [NIP-92](NIP-92)

Each `imeta` tag can be used to specify a variant of the video by the `dim` & `m` properties, as well as multiple audio tracks (languages).

Clients should check if audio tracks are available and prefer them over in-video audio to allow smooth resolution switching without interrupting audio. The `ov` (original version) flags the original language track.

This NIP defines the following additional `imeta` properties aside from those listed in [NIP-92](NIP-92) & [NIP-94](NIP-94):

- `duration` (recommended) the duration of the video/audio in seconds (floating point number)
- `bitrate` (recommended) the average bitrate of the video/audio in bits/sec
- `waveform` (optional, audio only) amplitude values over time, space separated full integers, less than 100 values should be enough to render a nice visual

Example:

```
[
  ["imeta",
    "dim 1920x1080",
    "url https://myvideo.com/1080/12345.mp4",
    "x 3093509d1e0bc604ff60cb9286f4cd7c781553bc8991937befaacfdc28ec5cdc",
    "m video/mp4",
    "image https://myvideo.com/1080/12345.jpg",
    "image https://myotherserver.com/1080/12345.jpg",
    "fallback https://myotherserver.com/1080/12345.mp4",
    "fallback https://andanotherserver.com/1080/12345.mp4",
    "service nip96",
    "bitrate 3000000",
    "duration 29.223"
  ],
  ["imeta",
    "dim 1280x720",
    "url https://myvideo.com/720/12345.mp4",
    "x e1d4f808dae475ed32fb23ce52ef8ac82e3cc760702fca10d62d382d2da3697d",
    "m video/mp4",
    "image https://myvideo.com/720/12345.jpg",
    "image https://myotherserver.com/720/12345.jpg",
    "fallback https://myotherserver.com/720/12345.mp4",
    "fallback https://andanotherserver.com/720/12345.mp4",
    "service nip96",
    "bitrate 2000000",
    "duration 29.24"
  ],
```

```
  ["imeta",
    "dim 1280x720",
    "url https://myvideo.com/720/12345.m3u8",
    "x 704e720af2697f5d6a198ad377789d462054b6e8d790f8a3903afbc1e044014f",
    "m application/x-mpegURL",
    "image https://myvideo.com/720/12345.jpg",
    "image https://myotherserver.com/720/12345.jpg",
    "fallback https://myotherserver.com/720/12345.m3u8",
    "fallback https://andanotherserver.com/720/12345.m3u8",
    "service nip96",
    "duration 29.21"
  ],
  ["imeta",
    "url https://myaudio.com/audio/en/12345.mp3",
    "x b2e0a7a82ac9f3f3a71f1d9a78c381d5be9d1cf19dce258765c17c8a76287c93",
    "m audio/mp3",
    "waveform 0 7 35 8 100 100 49 8 4 16 8 10 7 2 20 10 100 100 100 100 100 100 15 100 100 100
        25 60 5 4 3 1 0 100 100 15 100 29 88 0 33 11 39 100 100 19 4 100 42 35 5 0 1 5 0 0 11
        38 100 94 17 11 44 58 5 100 100 100 55 14 72 100 100 57 6 1 14 2 16 100 100 40 16 100
        100 6 32 14 13 41 36 16 14 6 3 0 1 2 1 6 0",
    "l en ISO-639-1 ov",
    "fallback https://myotherserver.com/audio/en/12345.mp3",
    "fallback https://andanotherserver.com/audio/en/12345.mp3",
    "service nip96",
    "bitrate  320000",
    "duration 29.24"
  ],
]
```

Where `url` is the primary server url and `fallback` are other servers hosting the same file, both `url` and `fallback` should be weighted equally and clients are recommended to use any of the provided video urls.

The `image` tag contains a preview image (at the same resolution). Multiple `image` tags may be used to specify fallback copies in the same way `fallback` is used for `url`.

Additionally `service nip96` may be included to allow clients to search the authors NIP-96 server list to find the file using the hash.

Required tags for addressable events:

- `d` - Unique identifier for this video (user-chosen string, required for kinds 34235, 34236)

Other tags:

- `title` (required) title of the video
- `published_at`, for the timestamp in unix seconds (stringified) of the first time the video was published
- `text-track` (optional, repeated) link to WebVTT file for video, type of supplementary information (captions/subtitles/chapters/metadata), optional language code
- `content-warning` (optional) warning about content of NSFW video
- `alt` (optional) description for accessibility
- `segment` (optional, repeated) start timestamp in format `HH:MM:SS.sss`, end timestamp in format `HH:MM:SS.sss`, chapter/segment title, chapter thumbnail-url
- `t` (optional, repeated) hashtag to categorize video

- `p` (optional, repeated) 32-bytes hex pubkey of a participant in the video, optional recommended relay URL
- `r` (optional, repeated) references / links to web pages

Optional tags for imported content:

- `origin` - Track original platform and ID: `["origin", "<platform>", "<external-id>", "<original-url>", "<optional-metadata>"]`

```json
{
  "id": "<32-bytes lowercase hex-encoded SHA-256 of the the serialized event data>",
  "pubkey": "<32-bytes lowercase hex-encoded public key of the event creator>",
  "created_at": <Unix timestamp in seconds>,
  "kind": 21 | 22,
  "content": "<summary / description of video>",
  "tags": [
    ["title", "<title of video>"],
    ["published_at", "<unix timestamp>"],
    ["alt", "<description>"],

    // video Data
    ["imeta",
      "dim 1920x1080",
      "url https://myvideo.com/1080/12345.mp4",
      "x 3093509d1e0bc604ff60cb9286f4cd7c781553bc8991937befaacfdc28ec5cdc",
      "m video/mp4",
      "image https://myvideo.com/1080/12345.jpg",
      "image https://myotherserver.com/1080/12345.jpg",
      "fallback https://myotherserver.com/1080/12345.mp4",
      "fallback https://andanotherserver.com/1080/12345.mp4",
      "service nip96",
    ],

    ["text-track", "<encoded `kind 6000` event>", "<recommended relay urls>"],
    ["content-warning", "<reason>"],
    ["segment", <start>, <end>, "<title>", "<thumbnail URL>"],

    // participants
    ["p", "<32-bytes hex of a pubkey>", "<optional recommended relay URL>"],
    ["p", "<32-bytes hex of a pubkey>", "<optional recommended relay URL>"],

    // hashtags
    ["t", "<tag>"],
    ["t", "<tag>"],

    // reference links
    ["r", "<url>"],
    ["r", "<url>"]
  ]
}
```

Addressable Event Example

```
{
  "id": <32-bytes lowercase hex-encoded SHA-256 of the the serialized event data>,
  "pubkey": <32-bytes lowercase hex-encoded public key of the event creator>,
  "created_at": <Unix timestamp in seconds>,
  "kind": 34235 | 34236,
  "content": "<summary / description of video>",
  "tags": [
    ["d", "<unique-identifier>"],
    ["title", "<title of video>"],
    ["published_at", "<unix timestamp>"],
    ["alt", "<description for accessibility>"],

    // video data
    ["imeta",
      "url https://example.com/media.mp4",
      "m video/mp4",
      "dim 480x480",
      "blurhash eVF$^OI:${M{%LRjWBoLoLaeR*",
      "image https://example.com/thumb.jpg",
      "x 3093509d1e0bc604ff60cb9286f4cd7c781553bc8991937befaacfdc28ec5cdc"
    ],
    // audio data
    ["imeta",
      "url https://example.com/audio.mp3",
      "x b2e0a7a82ac9f3f3a71f1d9a78c381d5be9d1cf19dce258765c17c8a76287c93",
      "m audio/mp3",
      "l en ISO-639-1 ov"
    ],

    ["duration", <duration in seconds>],
    ["content-warning", "<reason>"],

    // origin tracking for imported content
    ["origin", "<platform>", "<external-id>", "<original-url>", "<optional-metadata>"],

    // participants
    ["p", "<32-bytes hex of a pubkey>", "<optional recommended relay URL>"],

    // hashtags
    ["t", "<tag>"],
    ["t", "<tag>"],

    // reference links
    ["r", "<url>"]
  ]
}
```

Referencing Addressable Events

To reference an addressable video:

```
["a", "34235:<pubkey>:<d-tag-value>", "<relay-url>"]  // for normal videos
["a", "34236:<pubkey>:<d-tag-value>", "<relay-url>"]  // for short videos
```

NIP-A0

Voice Messages

Status: Draft

This NIP defines new events `kind: 1222` for root messages and `kind: 1244` for reply messages to be used for short voice messages, typically up to 60 seconds in length.

Specification

Event Kind `1222` and Kind `1244`

The `kind: 1222` event is defined as follows:

- `content`: MUST be a URL pointing directly to an audio file.
  - The audio file SHOULD be in `audio/mp4` (.m4a) format using AAC or Opus encoding. Clients MAY support other common audio formats like `audio/ogg`, `audio/webm`, or `audio/mpeg` (mp3), but `audio/mp4` is recommended for broad compatibility and efficiency.
  - The audio duration SHOULD be no longer than 60 seconds. Clients publishing `kind: 1222` events SHOULD enforce this limit or provide a clear warning to the user if exceeded.
- `tags`:
  - Tags MAY be included as per other NIPs (e.g., `t` for hashtags, `g` for geohash, etc.).

The `kind: 1244` event is defined as follows:

- To be used for replies, `kind: 1244` events MUST follow the structure of `NIP-22`.
- `content`: MUST be a URL pointing directly to an audio file.
  - The audio file SHOULD be in `audio/mp4` (.m4a) format using AAC or Opus encoding. Clients MAY support other common audio formats like `audio/ogg`, `audio/webm`, or `audio/mpeg` (mp3), but `audio/mp4` is recommended for broad compatibility and efficiency.
  - The audio duration SHOULD be no longer than 60 seconds. Clients publishing `kind: 1222` events SHOULD enforce this limit or provide a clear warning to the user if exceeded.
- `tags`:
  - Tags MAY be included as per other NIPs (e.g., `t` for hashtags, `g` for geohash, etc.).

Visual representation with `imeta` (NIP-92) tag (optional)

The following imeta (NIP-92) tags MAY be included so clients can render a visual preview without having to download the audio file first:

- `waveform`: amplitude values over time, space separated full integers, less than 100 values should be enough to render a nice visual
- `duration`: audio length in seconds

Examples

Root Voice Message Example

```json
{
  "content": "https://blossom.primal.net/
        5fe7df0e46ee6b14b5a8b8b92939e84e3ca5e3950eb630299742325d5ed9891b.mp4",
  "created_at": 1752501052,
  "id": "...",
  "kind": 1222,
  "pubkey": "...",
  "sig": "...",
  "tags": [
    [
      "imeta",
      "url https://blossom.primal.net/
        5fe7df0e46ee6b14b5a8b8b92939e84e3ca5e3950eb630299742325d5ed9891b.mp4",

        "waveform 0 7 35 8 100 100 49 8 4 16 8 10 7 2 20 10 100 100 100 100 100 100 15 100 100
        100 25 60 5 4 3 1 0 100 100 15 100 29 88 0 33 11 39 100 100 19 4 100 42 35 5 0 1 5 0 0
        11 38 100 94 17 11 44 58 5 100 100 100 55 14 72 100 100 57 6 1 14 2 16 100 100 40 16
        100 100 6 32 14 13 41 36 16 14 6 3 0 1 2 1 6 0",
      "duration 8"
    ]
  ]
}

NIP-92
======

Media Attachments Metadata (`imeta`)
------------------------------------

Media attachments (images, videos, and other files) may be added to events by including a URL
        in the event content, along with a matching `imeta` tag.

`imeta` ("inline metadata") tags MAY add information about media URLs in the event's content.
        Each `imeta` tag SHOULD match a URL in the event content. Clients MAY replace imeta
        URLs with rich previews.

The `imeta` tag is variadic, and each entry is a space-delimited key/value pair.
Each `imeta` tag MUST have a `url`, and at least one other field. `imeta` MAY include
any field specified by [NIP 94](./94.md). There SHOULD be only one `imeta` tag per URL.

## Example

```json
{
  "content": "More image metadata tests don’t mind me https://nostr.build/i/my-image.jpg",
  "kind": 1,
  "tags": [
    [
      "imeta",
      "url https://nostr.build/i/my-image.jpg",
      "m image/jpeg",
```
```

```
      "blurhash eVF$^OI:${M{o#*0-nNFxakD-?xVM}WEWB%iNKxvR-oetmo#R-aen$",
      "dim 3024x4032",
      "alt A scenic photo overlooking the coast of Costa Rica",
      "x <sha256 hash as specified in NIP 94>",
      "fallback https://nostrcheck.me/alt1.jpg",
      "fallback https://void.cat/alt1.jpg"
    ]
  ]
}
```

Recommended client behavior

When uploading files during a new post, clients MAY include this metadata after the file is uploaded and included in the post.

When pasting URLs during post composition, the client MAY download the file and add this metadata before the post is sent.

The client MAY ignore `imeta` tags that do not match the URL in the event content.

NIP-37

Draft Wraps

`draft` `optional`

This NIP defines kind `31234` as an encrypted storage for unsigned draft events of any other kind.

The draft is JSON-stringified, NIP44-encrypted to the signer's public key and placed inside the `.content`.

`k` tags identify the kind of the draft.

```
{
  "kind": 31234,
  "tags": [
    ["d", "<identifier>"],
    ["k", "<kind of the draft event>"], // required
    ["expiration", "now + 90 days"] // recommended
  ],
  "content": nip44Encrypt(JSON.stringify(draft_event)),
  // other fields
}
```

A blanked `.content` field signals that the draft has been deleted.

NIP-40 `expiration` tags are recommended.

Clients SHOULD publish kind `31234` events to relays listed on kind `10013` below.

Checkpoints

`kind:1234` defines checkpoints that belong to a parent `kind:31234` event. These can serve to provide a revision history of a parent event.

```
{
  "kind": 1234,
  "tags": [
    ["a", "31234:<pubkey>:<identifier>"]
  ],
  "content": nip44Encrypt(JSON.stringify(draft_event)),
}
```

Relay List for Private Content

Kind `10013` indicates the user's preferred relays to store private events like Draft Wraps.

The event MUST include a list of `relay` URLs in private tags. Private tags are JSON Stringified, [NIP44-encrypted] to the signer's keys and placed inside the .content of the event.

```
{
  "kind": 10013,
  "tags": [],
  "content": nip44Encrypt(
    JSON.stringify(
      [
        ["relay", "wss://myrelay.mydomain.com"]
      ]
    )
  )
  //...other fields
}
```

It's recommended that Private Storage relays SHOULD be NIP-42-authed and only allow downloads of events signed by the authed user.

Clients MUST publish kind `10013` events to the author's NIP-65 `write` relays.

NIP-A4

Public Messages

`draft` `optional`

This NIP defines kind `24` as a simple plaintext message to one or more Nostr users.

The `.content` contains the message. `p` tags identify one or more receivers.

```json
{
  "pubkey": "<sender-pubkey>",
  "kind": 24,
```

```
  "tags": [
    ["p", "<receiver>", "<relay-url>"],
  ],
  "content": "<message-in-plain-text>",
}
```

Messages MUST be sent to the NIP-65 inbox relays of each receiver and the outbox relay of the sender.

Kind `24` is designed to be shown and replied to from notification screens. The goal is to allow clients to support this feature without having to worry about chat history. There are no message chains. The concept of a "thread", a "thread root", or a "chatroom" does not exist in this system, as messages can start and continue without any syntactic connection to each other. `e` tags must not be used.

This kind is not designed to be displayed on feeds, but anyone can see and reply to messages that may not be for them.

Advanced Support

NIP-40 `expiration` tags are recommended. Since there is no concept of a chatroom, it is unlikely that these messages will make sense as time goes on.

NIP-18 quote repost `q` tags MAY be used when citing events in the `.content` with NIP-21.

```
["q", "<event-id> or <event-address>", "<relay-url>", "<pubkey-if-a-regular-event>"]
```

NIP-25 reactions MUST add a `k` tag to `24`.

NIP-57 zaps MUST include the `k` tag to `24`

NIP-21 links that use NIP-19's `nevent1` MUST include a `kind` of `24`. Links that are not `kind:24` are not expected to be rendered natively by the client.

NIP-92 `imeta` tags SHOULD be added for image and video links.

Warnings

There MUST be no expectation of privacy in this kind. It is just a public reply, but without a root note.

Avoid confusing this kind with Kind `14` rumors in NIP-17 DMs. This kind is signed and designed for public consumption.

NIP-17

Private Direct Messages

`draft` `optional` `relay`

This NIP defines an encrypted chat scheme which uses NIP-44 encryption and NIP-59 gift-wrapping.

By convention, `kind 14` direct messages, `kind 15` file messages, and [`kind 7` reactions] may be sent to an encrypted chat. See [Message Rumor Definitions] below.

Chat Rooms

The most common use case is a room with only two peers, but more than one person per room is supported.

The set of `pubkey` + `p` tags defines a chat room. If a new `p` tag is added or a current one is removed, a new room is created with a clean message history.

An optional `subject` tag defines the current name/topic of the conversation. Any member can change the topic by simply submitting a new `subject` to an existing `pubkey` + `p` tags room. There is no need to send `subject` in every message. The newest `subject` in the chat room is the subject of the conversation.

Encrypting

Following [NIP-59], the **unsigned** chat messages must be sealed (`kind:13`) and then gift-wrapped (`kind:1059`) to each receiver and the sender individually.

```
{
  "id": "<usual hash>",
  "pubkey": wrapperPublicKey,
  "created_at": randomTimeUpTo2DaysInThePast,
  "kind": 1059, // gift wrap
  "tags": [
    ["p", receiverPublicKey, "<relay-url>"] // receiver
  ],
  "content": nip44.encrypt(
    {
      "id": "<usual hash>",
      "pubkey": senderPublicKey,
      "created_at": randomTimeUpTo2DaysInThePast,
      "kind": 13, // seal
      "tags": [], // no tags
      "content": nip44.encrypt(
        unsignedMessageRumor,
        nip44.compute_conversation_key(senderPrivateKey, receiverPublicKey)
      ),
      "sig": "<signed by senderPrivateKey>"
    },
    nip44.compute_conversation_key(wrapperPrivateKey, receiverPublicKey)
  ),
  "sig": "<signed by randomPrivateKey>"
}
```

`unsignedMessageRumor` is a rumor (an unsigned event, as per [NIP-59]), usually a `kind:14`, but could also be a different kind, see [Message Rumor Definitions] below.

`wrapperPrivateKey` and `wrappedPublicKey` are a new keypair, generated randomly anew for each message sent.

Clients MUST verify if pubkey of the `kind:13` is the same pubkey as that of the `unsignedMessageRumor`, otherwise any sender can impersonate any other by simply changing the pubkey on the rumor.

Clients SHOULD randomize `created_at` in up to two days in the past in both the seal and the gift wrap to make sure grouping by `created_at` doesn't reveal any metadata.

Publishing

Kind `10050` indicates the user's preferred relays to receive DMs. The event MUST include a list of `relay` tags with relay URIs.

```
{
  "kind": 10050,
  "tags": [
    ["relay", "wss://inbox.nostr.wine"],
    ["relay", "wss://myrelay.nostr1.com"],
  ],
  "content": "",
  // other fields...
}
```

Clients MUST only publish events to the relays listed in the recipient's kind 10050 event. If such a list is not found that indicates the user is not ready to receive messages and clients shouldn't try.

Relays

Relays SHOULD protect message metadata by only serving `kind:1059` events to users p-tagged on the event (enforced using NIP-42 AUTH).

Clients SHOULD guide users to keep `kind:10050` lists small (1-3 relays) and SHOULD spread them to as many relays as viable.

Delete, edit and disappearing messages

In addition to the deletion behavior specified in NIP 59, in which gift wraps can be deleted by the `p`-tagged recipient, clients MAY also allow users to delete messages by wrapping a `kind:5` delete event in a `kind:1059` gift wrap and sending it to the recipient as part of the conversation. Clients SHOULD remove deleted messages from the conversation, or indicate whether a message has been deleted.

Clients MAY implement edit by deleting an event and publishing another one with the same timestamp.

Clients MAY offer disappearing messages by setting an `expiration` tag in the gift wrap of each receiver or by not generating a gift wrap to the sender's public key. This tag SHOULD be included on the `kind:13` seal as well, in case it leaks.

Benefits & Limitations

This NIP offers the following privacy and security features:

1. **No Metadata Leak**: Participant identities, each message's real date and time, event kinds, and other event tags are all hidden from the public. Senders and receivers cannot be linked with public information alone.
2. **No Public Group Identifiers**: There is no public central queue, channel or otherwise converging identifier to correlate or count all messages in the same group.
3. **No Moderation**: There are no group admins: no invitations or bans.
4. **No Shared Secrets**: No secret must be known to all members that can leak or be mistakenly shared
5. **Fully Recoverable**: Messages can be fully recoverable by any client with the user's private key
6. **Optional Forward Secrecy**: Users and clients can opt-in for "disappearing messages".
7. **Uses Public Relays**: Messages can flow through public relays without loss of privacy. Private relays can increase privacy further, but they are not required.
8. **Cold Storage**: Users can unilaterally opt-in to sharing their messages with a separate key that is exclusive for DM backup and recovery.

The main limitation of this approach is having to send a separate encrypted event to each receiver. Group chats with more than 10 participants should find a more suitable messaging scheme.

Spam

Since the wrapper events use random keys, relays cannot apply traditional anti-spam based on pubkey reputation, WoT, etc, therefore a naïve implementation of this NIP is an spam target. The recommended approach for spam mitigation is described in the [Spam Protection section of NIP-59](#).

Message Rumor Definitions

Chat Message

Kind `14` is a chat message. `p` tags identify one or more receivers of the message.

```
{
  "pubkey": "<sender-pubkey>",
  "created_at": "<current-time>",
  "kind": 14,
  "tags": [
    ["p", "<receiver-1-pubkey>", "<relay-url>"],
    ["p", "<receiver-2-pubkey>", "<relay-url>"],
    ["e", "<kind-14-id>", "<relay-url>"] // if this is a reply
    ["subject", "<conversation-title>"],
    // rest of tags...
  ],
  "content": "<message-in-plain-text>",
}
```

`.content` MUST be plain text. Fields `id` and `created_at` are required.

An `e` tag denotes the direct parent message this post is replying to.

`q` tags MAY be used when citing events in the `.content` with NIP-21.

```
["q", "<event-id> or <event-address>", "<relay-url>", "<pubkey-if-a-regular-event>"]
```

File Message

```
{
  "pubkey": "<sender-pubkey>",
  "created_at": "<current-time>",
  "kind": 15,
  "tags": [
    ["p", "<receiver-1-pubkey>", "<relay-url>"],
    ["p", "<receiver-2-pubkey>", "<relay-url>"],
    ["e", "<kind-14-id>", "<relay-url>", "reply"], // if this is a reply
    ["subject", "<conversation-title>"],
    ["file-type", "<file-mime-type>"],
    ["encryption-algorithm", "<encryption-algorithm>"],
    ["decryption-key", "<decryption-key>"],
    ["decryption-nonce", "<decryption-nonce>"],
    ["x", "<the SHA-256 hexencoded string of the file>"],
    // rest of tags...
  ],
  "content": "<file-url>"
}
```

Kind `15` is used for sending encrypted file event messages:

- `file-type`: Specifies the MIME type of the attached file (e.g., `image/jpeg`, `audio/mpeg`, etc.) before encryption.
- `encryption-algorithm`: Indicates the encryption algorithm used for encrypting the file. Supported algorithms: `aes-gcm`.
- `decryption-key`: The decryption key that will be used by the recipient to decrypt the file.
- `decryption-nonce`: The decryption nonce that will be used by the recipient to decrypt the file.
- `content`: The URL of the file (`<file-url>`).
- `x` containing the SHA-256 hexencoded string of the encrypted file.
- `ox` containing the SHA-256 hexencoded string of the file before encryption.
- `size` (optional) size of the encrypted file in bytes
- `dim` (optional) size in pixels in the form `<width>x<height>`
- `thumbhash`(optional) the thumbhash to show while the client is loading the file
- `blurhash`(optional) the blurhash to show while the client is loading the file
- `thumb` (optional) URL of thumbnail with same aspect ratio (encrypted with the same key, nonce)
- `fallback` (optional) zero or more fallback file sources in case `url` fails (encrypted with the same key, nonce)

Examples

This example sends the message `Hola, que tal?` from `nsec1w8udu59ydjvedgs3yv5qccshcj8k05fh3l60k9x57asjrqdpa00qkmr89m` to `nsec12ywtkplvyq5t6twdqwwygavp5lm4fhuang89c943nf2z92eez43szvn4dt`.

The two final GiftWraps, one to the receiver and the other to the sender, respectively, are:


```json
{
   "id":"2886780f7349afc1344047524540ee716f7bdc1b64191699855662330bf235d8",
   "pubkey":"8f8a7ec43b77d25799281207e1a47f7a654755055788f7482653f9c9661c6d51",
   "created_at":1703128320,
   "kind":1059,
   "tags":[
      ["p", "918e2da906df4ccd12c8ac672d8335add131a4cf9d27ce42b3bb3625755f0788"]
   ],
   "content":"AsqzdlMsG304G8h08bE67dhAR1gFTzTckUUyuvndZ8LrGCvwI4pgC3d6hyAK0Wo9gtkLqSr2rT2RyHlE5wRqbCOlQ8WvJE
         45y0rv9aVIw9IWF11u53cf2CP7akACel2WvZdl1htEwFu/v9cFXD06fNVZjfx3OssKM/uHPE9XvZttQboAvP5UoK6lv9o3d+0GM4
         ipnaQP/eFJv/ibNuSfqL6E4BnN/tHJSHYEaTQ/PdrA2i9laG3vJti3kAl5Ih87ct0w/tzYfp4SRPhEF1zzue9G/16eJEMzwmhQ5E
         L0ZY9bFgh6iLDCdB+4iABXyZwT7Ufn762195hrSHcU4Okt0Zns9EeiBOFxnmpXEslYkYBpXw70GmymQfJlFOfoEp93QKCMS2DAEV
         bBUx0fmOXsS3vjqCXW5IjkmSUV7q54GewZqTQBlcx+90xh/LSUxXex7UwZwRnifvyCbZ+zwNTHNb12chYeNjMV7kAIr3cGQv8vlO

   "sig":"a3c6ce632b145c0869423c1afaff4a6d764a9b64dedaf15f170b944ead67227518a72e455567ca1c2a0d187832cecbde7e

}
```

```json
{
   "id":"162b0611a1911cfcb30f8a5502792b346e535a45658b3a31ae5c178465509721",
   "pubkey":"626be2af274b29ea4816ad672ee452b7cf96bbb4836815a55699ae402183f512",
   "created_at":1702711587,
   "kind":1059,
   "tags":[
      ["p", "44900586091b284416a0c001f677f9c49f7639a55c3f1e2ec130a8e1a7998e1b"]
   ],
   "content":"AsTClTzr0gzXXji7uye5UB6LYrx3HDjWGdkNaBS6BAX9CpHa+Vvtt5oI2xJrmWLen+Fo2NBOFazvl285Gb3HSM82gVycrz
         GnG3eGUiUzhiYBoHomj3vztYYxc0QYHOx0WxiHY8dsC6jPsXC7f6k4P+Hv5ZiyTfzvjkSJOckel1lZuE5SfeZ0nduqTlxREGeBJ8
         I0TKJNmqNvSN+IVoUuC7ZOfUV9lOV8Ri0AMfSr2YsdZ9ofV5o82ClZWlWiSWZwy6ypa7CuT1PEGHzywB4CZ5ucpO60Z7hnBQxHLi
         4VLNZRHDcwtfejogjrjdi8p6nfUyqoQRRPARzRGUnnCbh+LqhigT6gQf3sVilnydMRScEc0/YYNLWnaw9nbyBa7wFBAiGbJwO40k
         ioHzNjItVCPaJjDyc6bv+gs1NPCt0qZ69G+JmgHW/PsMMeL4n5bh74g0fJSHqiI9ewEmOG/8bedSREv2XXtKV39STxPweceIOh0k

   "sig":"c94e74533b482aa8eeeb54ae72a5303e0b21f62909ca43c8ef06b0357412d6f8a92f96e1a205102753777fd25321a58fba

}
```


> **Warning** `unrecommended`: deprecated in favor of [NIP-17](17.md)

NIP-04

Encrypted Direct Message

`final` `unrecommended` `optional` `relay`

A special event with kind `4`, meaning "encrypted direct message". It is supposed to have the following attributes:

`content` MUST be equal to the base64-encoded, aes-256-cbc encrypted string of anything a user wants to write, encrypted using a shared cipher generated by combining the recipient's public-key with the sender's private-key; this appended by the base64-encoded initialization vector as if it was a querystring parameter named "iv". The format is the following: `"content": "<encrypted_text>?iv=<initialization_vector>"`.

`tags` MUST contain an entry identifying the receiver of the message (such that relays may naturally forward this event to them), in the form `["p", "<pubkey, as a hex string>"]`.

`tags` MAY contain an entry identifying the previous message in a conversation or a message we are explicitly replying to (such that contextual, more organized conversations may happen), in the form `["e", "<event_id>"]`.

Note: By default in the libsecp256k1 ECDH implementation, the secret is the SHA256 hash of the shared point (both X and Y coordinates). In Nostr, only the X coordinate of the shared point is used as the secret and it is NOT hashed. If using libsecp256k1, a custom function that copies the X coordinate must be passed as the `hashfp` argument in `secp256k1_ecdh`. See here.

Code sample for generating such an event in JavaScript:

```
import crypto from 'crypto'
import * as secp from '@noble/secp256k1'

let sharedPoint = secp.getSharedSecret(ourPrivateKey, '02' + theirPublicKey)
let sharedX = sharedPoint.slice(1, 33)

let iv = crypto.randomFillSync(new Uint8Array(16))
var cipher = crypto.createCipheriv(
  'aes-256-cbc',
  Buffer.from(sharedX),
  iv
)
let encryptedMessage = cipher.update(text, 'utf8', 'base64')
encryptedMessage += cipher.final('base64')
let ivBase64 = Buffer.from(iv.buffer).toString('base64')

let event = {
  pubkey: ourPubKey,
  created_at: Math.floor(Date.now() / 1000),
  kind: 4,
  tags: [['p', theirPublicKey]],
  content: encryptedMessage + '?iv=' + ivBase64
}
```

Security Warning

This standard does not go anywhere near what is considered the state-of-the-art in encrypted communication between peers, and it leaks metadata in the events, therefore it must not be used for anything you really need to keep secret, and only with relays that use `AUTH` to restrict who can fetch your `kind:4` events.

Client Implementation Warning

Clients *should not* search and replace public key or note references from the `.content`. If processed like a regular text note (where `@npub...` is replaced with `#[0]` with a `["p", "..."]` tag) the tags are leaked and the mentioned user will receive the message in their inbox.

> **Warning** `unrecommended`: superseded by the [Marmot Protocol]

NIP-EE

E2EE Messaging using the Messaging Layer Security (MLS) Protocol

`final` `unrecommended` `optional`

This NIP standardizes how to use the [MLS Protocol] with Nostr for efficient and E2EE (end-to-end encrypted) direct and group messaging.

Context

Originally, one-to-one direct messages (DMs) in Nostr happened via the scheme defined in [NIP-04]. This NIP is not recommended because, while it encrypts the content of the message (provides decent confidentiality), it leaks significant amounts of metadata about the parties involved in the conversation (completely lacks privacy).

With the addition of [NIP-44], we have an updated encryption scheme that improves confidentiality guarantees but stops short of defining a new scheme for doing direct messages using this encryption scheme. Hence, makes little to no difference to privacy.

Most recently, [NIP-17] combines [NIP-44] encryption with [NIP-59] gift-wrapping to hide the encrypted direct message inside another set of events to ensure that it's impossible to see who is talking to who and when messages passed between the users. This largely solves the metadata leakage problem; while it's still possible to see that a user is receiving gift-wrapped events, you can't tell from whom and what kind of events are within the gift-wrap outer event. This gives some degree of deniability/repudiation but doesn't solve forward secrecy or post compromise security. That is to say, if a user's private key (or the calculated conversation key shared between two users used to encrypt messages) is compromised, the attacker will have full access to all past and future DMs sent between those users.

In addition, neither [NIP-04] or [NIP-17] attempt to solve the problem of group messages.

Why is this important?

Without proper E2EE, Nostr cannot be used as the protocol for secure messaging clients. While clients like Signal do a fantastic job with E2EE, they still rely on centralized servers and as a result can be shut down by a powerful (i.e. state-level) actor. The goal of Nostr is not only to protect against centralized entities censoring you and your communications, but also protect against the ability of a state-level actor to stop these sorts of services from existing in the first place. By replacing centralized servers with decentralized relays, we make it nearly impossible for a centralized actor to completely stop communications between individual users.

Goals of this NIP

1. Private *and* Confidential DMs and Group messages
    1. **Private** means that an observer cannot tell that Alice and Bob are talking to one another, or that Alice is part of a specific group. This necessarily requires protecting metadata.
    2. **Confidential** means that the contents of conversations can only be viewed by the intended recipients.
2. Forward secrecy and Post-compromise security
    1. **Forward secrecy** means that encrypted content in the past remains encrypted even if a key material is leaked.
    2. **Post compromise security** means that leaking key material doesn't allow an attacker to continue to read messages indefinitely into the future.
3. Scales efficiently for large groups
4. Allows for the use of multiple device/clients in a single conversation/group.

Why MLS?

This scheme adapts the Message Layer Security (MLS) protocol for use with Nostr. You can think of MLS as an evolution of the Signal Protocol. However, it significantly improves the scalability of encryption operations for large group messaging significantly (linear -> log), is built to accommodate federated environments, and also allows for graceful updating of ciphersuites and versions over time. In addition, it's very flexible and agnostic about the message content that is sent.

It's beyond the scope of this NIP to explain the MLS protocol but you can read more about it in it's [Architectural Overview] or the [RFC]. MLS is on track to become an internet standard under the IETF so the protocol itself is extremely well vetted and researched. This also means there is the potential for cross network messaging interoperability in the future as MLS gains more adoption.

Core MLS Concepts

From the [MLS Architectural Overview]:

> MLS provides a way for clients to form groups within which they can communicate securely. For example, a set of users might use clients on their phones or laptops to join a group and communicate with each other. A group may be as small as two clients (e.g., for simple person to person messaging) or as large as hundreds of thousands. A client that is part of a group is a member of that group. As groups change membership and group or member properties, they advance from one epoch to another and the cryptographic state of the group evolves.
>
> The group is represented as a tree, which represents the members as the leaves of a tree. It is used to efficiently encrypt to subsets of the members. Each member has a state called a LeafNode object holding the client's identity, credentials, and capabilities.

The MLS protocol's job is to manage and evolve the cryptographic state of a group. This includes managing the membership of a group, the cryptographic state of a group (ratchet tree, keys, and encryption/decryption/authentication of messages), and managing the evolution of the group over time.

Groups

Groups are created by their first member, who then invites one or more other members. Groups evolve over time in blocks called `Epochs`. New epochs are proposed via one ore more `Proposal` messages and then committed to via a `Commit` message.

Clients

The device/client pair (e.g. Primal on iOS or Coracle on web) with which a user joins the group is represented as a `LeafNode` in the tree. The terms `Client` and `Member` are interchangeable in this regard. It is not possible to share group state across multiple `Clients`. If a user joins a group from 2 separate devices, their state is separate and they will be tracked as 2 separate members of the group.

Messages

There are several different types of messages sent within a group. Some of these are control messages that are used to update the group state over time. These include `Welcome`, `Proposal`, and `Commit` messages. Others are the actual messages that are sent between members in a group. These include `Application` messages.

Messages in MLS are "framed". Meaning that they are wrapped in a data structure that includes information about the sender, the epoch, the message index within the epoch and the message content. This framing makes it possible to authenticate and decrypt messages correctly, even if they arrive out of order.

MLS is agnostic to the "content" of the messages that are sent. This is a key feature of MLS that allows for the use of MLS for a wide variety of applications.

MLS is also agnostic to the transport protocol that is used to send messages. Obviously for us, we'll be using websockets, Nostr events and relays.

The focus of this NIP

This NIP focuses on how to use Nostr to perform the Authentication Service and Delivery Service functions required by the MLS protocol. Most clients will choose to use an MLS implementation to handle keys, ratcheting, group state management, and other aspects of the MLS protocol itself. OpenMLS is the most actively developed library that implements MLS.

This NIP specifies the following:

1. A standardized way that Nostr clients should create MLS groups.
2. The required format of the MLS `Credential` that Nostr clients should use to represent a Nostr user in a group.
3. The structure of KeyPackage Events published to relays that allow Nostr users to be added to a group asynchronously.
4. The structure of Group Events published to relays that represent the evolution of a group's state and the contents of the messages sent in the group.

Security Considerations

This is a concise overview of the security trade-offs and considerations of this NIP in various scenarios. The NIP strives to fully maintain MLS security guarantees.

Forward Secrecy and Post-compromise Security

- As per the MLS spec, keys are deleted as soon as they are used to encrypt or decrypt a message. This is usually handled by the MLS implementation library itself but attention needs to be paid by clients to ensure they're not storing secrets (especially the exporter secret) for longer than absolutely necessary.
- This NIP maintains MLS forward secrecy and post-compromise security guarantees. You can read more about those in the MLS Architectural Overview section on Forward Secrecy and Post-compromise Security.

Leakage of various keys

- This NIP does not depend on a user's Nostr identity key for any aspect of the MLS messaging protocol. Compromise of a user's Nostr identity key does not give access to past or future messages in any MLS-based group.
- For a complete discussion of MLS key leakage, please see the Endpoint Compromise section of the MLS Architectural Overview.

Metadata

- The only group specific metadata published to relays is the Nostr group ID value. This value is used to identify the group in the `h` tag of the Group Message Event (`kind: 445`). These events are published ephemerally and this Nostr group ID value can be updated over the lifetime of the group by group admins. This is a tradeoff to ensure that group participants and group size are obfuscated but still makes it possible to efficiently fan out group messages to all participants. The content field of this event is a value encrypted in two separate ways (using NIP-44 and MLS) with MLS group state/keys. Only group members with up-to-date group state can decrypt and read these messages.
- A user's key package events can be used one or more times to be added to groups. There is a tradeoff inherent here: Reusing key packages (initial signing keys) carries some degree of risk but this risk is mitigated as long as a user rotates their signing key immediately upon joining a group. This step also improves the forward secrecy of the entire group.

Device Compromise

Clients implementing this NIP should take every precaution to ensure that data is stored in a secure way on the device and is protected against unwanted access in the case that a device is compromised (e.g. encryption at rest, biometric authentication, etc.). That said, full device compromise should be viewed as a catastrophic event and any group the compromised device was a part of should be considered compromised until they can remove that member and update their group's state. Some suggestions:

- Clients should support and encourage self-destructing messages (ensuring that full transcript history isn't available on a device forever).
- Clients should regularly suggest to group admins that inactive users be removed.

- Clients should regularly suggest (or automatically) rotate a user's signing key in each of their groups.
- Clients should encrypt group state and keys on the device using a secret value that isn't part of the group state or the user's Nostr identity key.
- Clients should use secure enclave storage where possible.

For a full discussion of the security considerations of MLS, please see the Security Considerations section of the MLS RFC.

Creating groups

MLS Groups are created with a random 32-byte ID value that is effectively permanent. This ID should be treated as private to the group and MUST not be published to relays in any form.

Clients must also ensure that the ciphersuite, capabilities, and extensions they use when creating the group are compatible with those advertised by the users they'd like to invite to the group. They can check this info via the user's published KeyPackage Events.

When creating a new group, the following MLS extensions MUST be used.

- `required_capabilities`
- `ratchet_tree`
- `nostr_group_data`

And the following MLS extension is highly recommended (more here): - `last_resort`

Changes to an MLS group are affected by first creating one or more `Proposal` events and then committing to a set of proposals in a `Commit` event. These are MLS events, not Nostr events. However, for the group state to properly evolve the Commit events (which represent a specific set of proposals - like adding a new user to the group) must be published to relays for the other group members to see. See Group Messages for more information.

MLS Credentials

A `Credential` in MLS is an assertion of who the user is coupled with a signing key. When constructing `Credentials` for MLS, clients MUST use the `BasicCredential` type and set the `identity` value as the 32-byte hex-encoded public key of the user's Nostr identity key. Clients MUST not allow users to change the identity field and MUST validate that all `Proposal` messages do not attempt to change the identity field on any credential in the group.

A `Credential` also has an associated signing key. The initial signing key for a user is included in the KeyPackage event. The signing key MUST be different from the user's Nostr identity key. This signing key SHOULD be rotated over time to provide improved post-compromise security.

Nostr Group Data Extension

As mentioned above, the `nostr_group_data` extension is a required MLS extension used to associate Nostr-specific data with an MLS group in a cryptographically secure and proveable way. This extension MUST be included as a required capability when creating a new group.

The extension stores the following data about the group:

- `nostr_group_id`: A 32-byte ID for the group. This is a different value from the group ID used by MLS and CAN be changed over time. This value is the group ID value used in the `h` tags when sending group message events.
- `name`: The name of the group.
- `description`: A short description of the group.
- `admin_pubkeys`: An array of the hex-encoded public keys of the group admins. The MLS protocol itself does not have a concept of group admins. Clients MUST check the list of `admin_pubkeys` before making any change to the group data (anything in this extension), or before changing group membership (add/remove members), or updating any other aspect of the group itself (e.g. ciphersuite, etc.). Note, all members of the group can send `Proposal` and `Commits` messages for changes to their own credentials (e.g. updating their signing key).
- `relays`: An array of the Nostr relay URLs that the group uses to publish and receive messages.

All of these values can be updated over time using MLS `Proposal` and `Commit` events (by group admins).

KeyPackage Event and Signing Keys

Each user that wishes to be reachable via MLS-based messaging MUST first publish at least one KeyPackage event. The KeyPackage Event is used to authenticate users and create the necessary `Credential` to add members to groups in an asynchronous way. Users can publish multiple KeyPackage Events with different parameters (supporting different ciphersuites or MLS extensions, for example). KeyPackages include a signing key that is used for signing MLS messages within a group. This signing key MUST not be the same as the user's Nostr identity key.

KeyPackage reuse SHOULD be minimized. However, in normal MLS use, KeyPackages are consumed when joining a group. In order to reduce race conditions between invites for multiple groups using the same Key Package, Nostr clients SHOULD use "Last resort" KeyPackages. This requires the inclusion of the `last_resort` extension on the KeyPackage's capabilities (same as with the Group).

It's important that clients immediately rotate a user's signing key after joining a group via a last resort key package to improve post-compromise security. The signing key (the public key included in the KeyPackage Event) is used for signing within the group. Therefore, clients implementing this NIP MUST ensure that they retain access to the private key material of the signing key for each group they are a member of.

In most cases, it's assumed that clients implementing this NIP will manage the creation and rotation of KeyPackage Events.

Example KeyPackage Event

```
{
  "id": <id>,
  "kind": 443,
  "created_at": <unix timestamp in seconds>,
  "pubkey": <main identity pubkey>,
  "content": "",
  "tags": [
      ["mls_protocol_version", "1.0"],
      ["ciphersuite", <MLS CipherSuite ID value e.g. "0x0001">],
```

```
        ["extensions", <An array of MLS Extension ID values e.g. "0x0001, 0x0002">],
        ["client", <client name>, <handler event id>, <optional relay url>],
        ["relays", <array of relay urls>],
        ["-"]
    ],
    "sig": <signed with main identity key>
}
```

- The `content` hex encoded serialized `KeyPackageBundle` from MLS.
- The `mls_protocol_version` tag is required and MUST be the version number of the MLS protocol version being used. For now, this is `1.0`.
- The `ciphersuite` tag is the value of the MLS ciphersuite that this KeyPackage Event supports. Read more about ciphersuites in MLS.
- The `extensions` tag is an array of MLS extension IDs that this KeyPackage Event supports. Read more about MLS extensions.
- (optional) The `client` tag helps other clients manage the user experience when they receive group invites but don't have access to the signing key.
- The `relays` tag identifies each of the relays that the client will attempt to publish this KeyPackage event. This allows for deletion of KeyPackage Events at a later date.
- (optional) The `-` tag can be used to ensure that KeyPackage Events are only published by their authenticated author. Read more in NIP-70

Deleting KeyPackage Events

Clients SHOULD delete the KeyPackage Event on all the listed relays any time they successfully process a group request event for a given KeyPackage Event. Clients MAY also create a new KeyPackage Event at the same time.

If clients cannot process a Welcome message (e.g. because the signing key was generated on another client), clients MUST not delete the KeyPackage Event and SHOULD show a human-understandable error to the user.

Rotating Signing Keys

Clients MUST regularly rotate the user's signing key in each group that they are a part of. The more often the signing key is rotated the stronger the post-compromise security. This rotation is done via `Proposal` and `Commit` events and broadcast to the group via a Group Event. Read more about forward secrecy and post-compromise security inherent in MLS.

KeyPackage Relays List Event

A `kind: 10051` event indicates the relays that a user will publish their KeyPackage Events to. The event MUST include a list of relay tags with relay URIs. These relays SHOULD be readable by anyone the user wants to be able to contact them.

```
{
  "kind": 10051,
  "tags": [
    ["relay", "wss://inbox.nostr.wine"],
    ["relay", "wss://myrelay.nostr1.com"],
  ],
```

```
  "content": "",
  //...other fields
}
```

Welcome Event

When a new user is added to a group via an MLS `Commit` message. The member who sends the `Commit` message to the group is responsible for sending the user being added to the group a Welcome Event. This Welcome Event is sent to the user as a NIP-59 gift-wrapped event. The Welcome Event gives the new member the context they need to join the group and start sending messages.

Clients creating the Welcome Event SHOULD wait until they have received acknowledgement from relays that their Group Event with the `Commit` has been received before publishing the Welcome Event.

```
{
   "id": <id>,
   "kind": 444,
   "created_at": <unix timestamp in seconds>,
   "pubkey": <nostr identity pubkey of sender>,
   "content": <serialized Welcome object>,
   "tags": [
      ["e", <ID of the KeyPackage Event used to add the user to the group>],
      ["relays", <array of relay urls>],
   ],
   "sig": <NOT SIGNED>
}
```

- The `content` field is required and is a serialized MLSMessage object containing the MLS `Welcome` object.
- The `e` tag is required and is the ID of the KeyPackage Event used to add the user to the group.
- The `relays` tag is required and is a list of relays clients should query for Group Events.

Welcome Events are then sealed and gift-wrapped as detailed in NIP-59 before being published. Like all events that are sealed and gift-wrapped, `kind: 444` events MUST never be signed. This ensures that if they were ever leaked they would not be publishable to relays.

Large Groups

For groups above ~150 participants, welcome messages will become larger than the maximum event size allowed by Nostr. There is currently work underway on the MLS protocol to support “light” client welcomes that don’t require the full Ratchet Tree state to be sent to the new member. This section will be updated with recommendations for how to handle large groups.

Group Events

Group Events are all the messages that are sent within a group. This includes all “control” events that update the shared group state over time (`Proposal`, `Commit`) and messages sent between members of the group (`Application` messages).

Group Events are published using an ephemeral Nostr keypair to obfuscate the number and identity of group participants. Clients MUST use a new Nostr keypair for each Group Event they publish.

```json
{
  "id": <id>,
  "kind": 445,
  "created_at": <unix timestamp in seconds>,
  "pubkey": <ephemeral sender pubkey>,
  "content": <NIP-44 encrypted serialized MLSMessage object>,
  "tags": [
    ["h", <group id>]
  ],
  "sig": <signed with ephemeral sender key>
}
```

- The `content` field is a [tls-style]() serialized [`MLSMessage`]() object which is then encrypted according to [NIP-44](). However, instead of using the sender and receivers keys to derive a `conversation_key`, the NIP-44 encryption is done using a Nostr keypair generated from the MLS [`exporter_secret`]() to calculate the `conversation_key` value. Essentially, you use the hex-encoded `exporter_secret` value as the private key (used as the sender key), calculate the public key for that private key (used as the receiver key), and then proceed with the standard NIP-44 scheme to encrypt and decrypt messages.
- The `exporter_secret` value should be generated with a 32-byte length and labeled `nostr`. This `exporter_secret` value is rotated on each new epoch in the group. Clients should generate a new 32-byte value each time they process a valid `Commit` message.
- The `pubkey` is the hex-encoded public key of the ephemeral sender.
- The `h` tag is the nostr group ID value (from the Nostr Group Data Extension).

Application Messages

Application messages are the messages that are sent within the group by members. These are contained within the `MLSMessage` object. The format of these messages should be unsigned Nostr events of the appropriate kind. For normal DM or group messages, clents SHOULD use `kind: 9` chat message events. If the user reacts to a message, it would be a `kind: 7` event, and so on.

This means that once the application message has been decrypted and deserialized, clients can store those events and treat them as any other Nostr event, effectively creating a private Nostr feed of the group's activity and taking advantage of all the features of Nostr.

These inner unsigned Nostr events MUST use the member's Nostr identity key for the `pubkey` field and clients MUST check that the identity of them member who sent the message matches the pubkey of the inner Nostr event.

These Nostr events MUST remain **unsigned** to ensure that if they were to leak to relays they would not be published publicly. These Nostr events MUST not include any "h" tags or other tags that would identify the group that they belong to.

`Commit` Message race conditions

The MLS protocol is resilient to almost all messages arriving out of order. However, the order of `Commit` messages is important for the group state to move forward from one epoch to the next correctly. Given Nostr's nature as a decentralized network, it is possible for a client to receive 2 or more `Commit` messages all attempting to update to a new epoch at the same time.

Clients sending commit messages MUST wait until they receive acknowledgement from at least one relay that their Group Message Event with the `Commit` has been received before applying the commit to their own group state.

If a client receives 2 or more `Commit` messages attempting to change same epoch, they MUST apply only one of the `Commit` messages they receive, determined by the following:

1. Using the `created_at` timestamp on the kind `445` event. The `Commit` with the lowest value for `created_at` is the message to be applied. The other `Commit` message is discarded.
2. If the `created_at` timestamp is the same for two or more `Commit` messages, the `Commit` message with the lowest value for `id` field is the message to be applied.

Clients SHOULD retain previous group state for a short period of time in order to recover from forked group state.

NIP-40

Expiration Timestamp

`draft` `optional` `relay`

The `expiration` tag enables users to specify a unix timestamp at which the message SHOULD be considered expired (by relays and clients) and SHOULD be deleted by relays.

Spec

```
tag: expiration
values:
 - [UNIX timestamp in seconds]: required
```

Example

```json
{
  "pubkey": "<pub-key>",
  "created_at": 1000000000,
  "kind": 1,
  "tags": [
    ["expiration", "1600000000"]
  ],
  "content": "This message will expire at the specified timestamp and be deleted by relays.\n",
  "id": "<event-id>"
}
```

Note: The timestamp should be in the same format as the created_at timestamp and should be interpreted as the time at which the message should be deleted by relays.

Client Behavior

Clients SHOULD use the `supported_nips` field to learn if a relay supports this NIP. Clients SHOULD NOT send expiration events to relays that do not support this NIP.

Clients SHOULD ignore events that have expired.

Relay Behavior

Relays MAY NOT delete expired messages immediately on expiration and MAY persist them indefinitely. Relays SHOULD NOT send expired events to clients, even if they are stored. Relays SHOULD drop any events that are published to them if they are expired. An expiration timestamp does not affect storage of ephemeral events.

Suggested Use Cases

- Temporary announcements - This tag can be used to make temporary announcements. For example, an event organizer could use this tag to post announcements about an upcoming event.
- Limited-time offers - This tag can be used by businesses to make limited-time offers that expire after a certain amount of time. For example, a business could use this tag to make a special offer that is only available for a limited time.

Warning

The events could be downloaded by third parties as they are publicly accessible all the time on the relays. So don't consider expiring messages as a security feature for your conversations or other uses.

NIP-09

Event Deletion Request

`draft` `optional` `relay`

A special event with kind `5`, meaning "deletion request" is defined as having a list of one or more `e` or `a` tags, each referencing an event the author is requesting to be deleted. Deletion requests SHOULD include a `k` tag for the kind of each event being requested for deletion.

The event's `content` field MAY contain a text note describing the reason for the deletion request.

For example:

```
{
  "kind": 5,
  "pubkey": <32-bytes hex-encoded public key of the event creator>,
  "tags": [
    ["e", "dcd59..464a2"],
    ["e", "968c5..ad7a4"],
```

```
    ["a", "<kind>:<pubkey>:<d-identifier>"],
    ["k", "1"],
    ["k", "30023"]
  ],
  "content": "these posts were published by accident",
  // other fields...
}
```

Relays SHOULD delete or stop publishing any referenced events that have an identical `pubkey` as the deletion request. Clients SHOULD hide or otherwise indicate a deletion request status for referenced events.

Relays SHOULD continue to publish/share the deletion request events indefinitely, as clients may already have the event that's intended to be deleted. Additionally, clients SHOULD broadcast deletion request events to other relays which don't have it.

When an `a` tag is used, relays SHOULD delete all versions of the replaceable event up to the `created_at` timestamp of the deletion request event.

Client Usage

Clients MAY choose to fully hide any events that are referenced by valid deletion request events. This includes text notes, direct messages, or other yet-to-be defined event kinds. Alternatively, they MAY show the event along with an icon or other indication that the author has "disowned" the event. The `content` field MAY also be used to replace the deleted events' own content, although a user interface should clearly indicate that this is a deletion request reason, not the original content.

A client MUST validate that each event `pubkey` referenced in the `e` tag of the deletion request is identical to the deletion request `pubkey`, before hiding or deleting any event. Relays can not, in general, perform this validation and should not be treated as authoritative.

Clients display the deletion request event itself in any way they choose, e.g., not at all, or with a prominent notice.

Clients MAY choose to inform the user that their request for deletion does not guarantee deletion because it is impossible to delete events from all relays and clients.

Relay Usage

Relays MAY validate that a deletion request event only references events that have the same `pubkey` as the deletion request itself, however this is not required since relays may not have knowledge of all referenced events.

Deletion Request of a Deletion Request

Publishing a deletion request event against a deletion request has no effect. Clients and relays are not obliged to support "unrequest deletion" functionality.

Social

A social network on Nostr is just events about people. NIP-02 is the follow list: a kind 3 event that says who you want to hear from. Profiles themselves live in NIP-01 as kind 0, but two NIPs make those profiles easier to find in the wider world. NIP-05 maps a key to a DNS name like `ada@example.com`. NIP-39 links a profile to accounts on other platforms.

Once you can find people, you need ways to respond. NIP-25 is reactions — the protocol's like, emoji, or downvote. Custom emoji (NIP-30) let a community bring its own stickers. Reposts (NIP-18) copy or quote someone else's event into your own feed. NIP-27 is how notes mention other notes, profiles, and relays with `nostr:` links. The older mention syntax in NIP-08 is unrecommended; NIP-27 replaced it.

The rest of the chapter is social context around the feed. User statuses (NIP-38) are a short "what I'm doing" line. Badges (NIP-58) are awards one key gives another. Trusted assertions (NIP-85) let a third party say something about a user or an event — useful for reputation without putting a corporation in the middle. Polls (NIP-88) are a first-class way to ask a question and collect votes. Web bookmarks (NIP-B0) are how you save a URL onto Nostr, next to the people and notes you already follow.

In this chapter

- NIP-02 — Follow list
- NIP-05 — DNS identifiers for keys
- NIP-39 — Linking profiles to other platforms
- NIP-25 — Reactions
- NIP-30 — Custom emoji
- NIP-18 — Reposts
- NIP-27 — Text note references
- NIP-08 — Handling mentions (unrecommended; use NIP-27)
- NIP-38 — User statuses
- NIP-58 — Badges
- NIP-85 — Trusted assertions
- NIP-88 — Polls
- NIP-B0 — Web bookmarks

NIP-02

Follow List

`final` `optional`

A special event with kind `3`, meaning "follow list" is defined as having a list of `p` tags, one for each of the followed/known profiles one is following.

Each tag entry should contain the key for the profile, a relay URL where events from that key can be found (can be set to an empty string if not needed), and a local name (or "petname") for that profile (can also be set to an empty string or not provided), i.e., `["p", <32-bytes hex key>, <main relay URL>, <petname>]`.

The `.content` is not used.

For example:

```
{
  "kind": 3,
  "tags": [
    ["p", "91cf9..4e5ca", "wss://alicerelay.com/", "alice"],
    ["p", "14aeb..8dad4", "wss://bobrelay.com/nostr", "bob"],
    ["p", "612ae..e610f", "ws://carolrelay.com/ws", "carol"]
  ],
  "content": "",
  // other fields...
}
```

Every new following list that gets published overwrites the past ones, so it should contain all entries. Relays and clients SHOULD delete past following lists as soon as they receive a new one.

Whenever new follows are added to an existing list, clients SHOULD append them to the end of the list, so they are stored in chronological order.

Uses

Follow list backup

If one believes a relay will store their events for sufficient time, they can use this kind-3 event to backup their following list and recover on a different device.

Profile discovery and context augmentation

A client may rely on the kind-3 event to display a list of followed people by profiles one is browsing; make lists of suggestions on who to follow based on the follow lists of other people one might be following or browsing; or show the data in other contexts.

Relay sharing

A client may publish a follow list with good relays for each of their follows so other clients may use these to update their internal relay lists if needed, increasing censorship-resistance.

Petname scheme

The data from these follow lists can be used by clients to construct local "petname" tables derived from other people's follow lists. This alleviates the need for global human-readable names. For example:

A user has an internal follow list that says

```
[
  ["p", "21df6d143fb96c2ec9d63726bf9edc71", "", "erin"]
]
```

And receives two follow lists, one from `21df6d143fb96c2ec9d63726bf9edc71` that says

```
[
  ["p", "a8bb3d884d5d90b413d9891fe4c4e46d", "", "david"]
]
```

and another from `a8bb3d884d5d90b413d9891fe4c4e46d` that says

```
[
  ["p", "f57f54057d2a7af0efecc8b0b66f5708", "", "frank"]
]
```

When the user sees `21df6d143fb96c2ec9d63726bf9edc71` the client can show *erin* instead; When the user sees `a8bb3d884d5d90b413d9891fe4c4e46d` the client can show *david.erin* instead; When the user sees `f57f54057d2a7af0efecc8b0b66f5708` the client can show *frank.david.erin* instead.

NIP-05

Mapping Nostr keys to DNS-based internet identifiers

`final` `optional`

On events of kind `0` (`user metadata`) one can specify the key `"nip05"` with an internet identifier (an email-like address) as the value. Although there is a link to a very liberal "internet identifier" specification above, the `<local-part>` part MUST only use characters `a-z0-9-_.`.

Upon seeing that, the client splits the identifier into `<local-part>` and `<domain>` and use these values to make a GET request to `https://<domain>/.well-known/nostr.json?name=<local-part>`.

The result should be a JSON document object with a key `"names"` that should then be a mapping of names to hex formatted public keys, in lowercase. If the public key for the given `<name>` matches the `pubkey` from the `user metadata` event, the client then concludes that the given pubkey can indeed be referenced by its identifier.

Example

If a client sees an event like this:

```
{
  "pubkey": "b0635d6a9851d3aed0cd6c495b282167acf761729078d975fc341b22650b07b9",
  "kind": 0,
  "content": "{\"name\": \"bob\", \"nip05\": \"bob@example.com\"}"
  // other fields...
}
```

It will make a GET request to `https://example.com/.well-known/nostr.json?name=bob` and get back a response that will look like

```
{
  "names": {
    "bob": "b0635d6a9851d3aed0cd6c495b282167acf761729078d975fc341b22650b07b9"
  }
}
```

or with the **recommended** `"relays"` attribute:

```
{
  "names": {
    "bob": "b0635d6a9851d3aed0cd6c495b282167acf761729078d975fc341b22650b07b9"
  },
  "relays": {
    "b0635d6a9851d3aed0cd6c495b282167acf761729078d975fc341b22650b07b9": [ "wss://
        relay.example.com", "wss://relay2.example.com" ]
  }
}
```

If the pubkey matches the one given in `"names"` (as in the example above) that means the association is right and the `"nip05"` identifier is valid and can be displayed.

The recommended `"relays"` attribute may contain an object with public keys as properties and arrays of relay URLs as values. When present, that can be used to help clients learn in which relays the specific user may be found. Web servers which serve `/.well-known/nostr.json` files dynamically based on the query string SHOULD also serve the relays data for any name they serve in the same reply when that is available.

Finding users from their NIP-05 identifier

A client may implement support for finding users' public keys from *internet identifiers*, the flow is the same as above, but reversed: first the client fetches the *well-known* URL and from there it gets the public key of the user, then it tries to fetch the kind `0` event for that user and check if it has a matching `"nip05"`.

Notes

Identification, not verification

The NIP-05 is not intended to *verify* a user, but only to *identify* them, for the purpose of facilitating the exchange of a contact or their search.
Exceptions are people who own (e.g., a company) or are connected (e.g., a project) to a well-known domain, who can exploit NIP-05 as an attestation of their relationship with it, and thus to the organization behind it, thereby gaining an element of trust.

User discovery implementation suggestion

A client can use this to allow users to search other profiles. If a client has a search box or something like that, a user may be able to type "bob@example.com" there and the client would recognize that and do the proper queries to obtain a pubkey and suggest that to the user.

Clients must always follow public keys, not NIP-05 addresses

For example, if after finding that `bob@bob.com` has the public key `abc...def`, the user clicks a button to follow that profile, the client must keep a primary reference to `abc...def`, not `bob@bob.com`. If, for any reason, the address `https://bob.com/.well-known/nostr.json?name=bob` starts returning the public key `1d2...e3f` at any time in the future, the client must not replace `abc...def` in his list of followed profiles for the user (but it should stop displaying "bob@bob.com" for that user, as that will have become an invalid `"nip05"` property).

Public keys must be in hex format

Keys must be returned in hex format, in lowercase. Keys in NIP-19 `npub` format are only meant to be used for display in client UIs, not in this NIP.

Showing just the domain as an identifier

Clients may treat the identifier `_@domain` as the "root" identifier, and choose to display it as just the `<domain>`. For example, if Bob owns `bob.com`, he may not want an identifier like `bob@bob.com` as that is redundant. Instead, Bob can use the identifier `_@bob.com` and expect Nostr clients to show and treat that as just `bob.com` for all purposes.

Reasoning for the `/.well-known/nostr.json?name=<local-part>` format

By adding the `<local-part>` as a query string instead of as part of the path, the protocol can support both dynamic servers that can generate JSON on-demand and static servers with a JSON file in it that may contain multiple names.

Allowing access from JavaScript apps

JavaScript Nostr apps may be restricted by browser [CORS](https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS) policies that prevent them from accessing `/.well-known/nostr.json` on the user's domain. When CORS prevents JS from loading a resource, the JS program sees it as a network failure identical to the resource not existing, so it is not possible for a pure-JS app to tell the user for certain that the failure was caused by a CORS issue. JS Nostr apps that see network failures requesting `/.well-known/nostr.json` files may want to recommend to users that they check the CORS policy of their servers, e.g.:

```
$ curl -sI https://example.com/.well-known/nostr.json?name=bob | grep -i ^Access-Control
Access-Control-Allow-Origin: *
```

Users should ensure that their `/.well-known/nostr.json` is served with the HTTP header `Access-Control-Allow-Origin: *` to ensure it can be validated by pure JS apps running in modern browsers.

Security Constraints

The `/.well-known/nostr.json` endpoint MUST NOT return any HTTP redirects.

Fetchers MUST ignore any HTTP redirects given by the `/.well-known/nostr.json` endpoint.

NIP-39

Linking Profiles to Other Platforms

`draft` `optional`

Abstract

Users can declare their control over one or more online identities such as usernames, profile pages, keypairs in kind `10011` using `i` tags.

```
{
  "kind": 10011,
  "tags": [
    ["i", "github:semisol", "9721ce4ee4fceb91c9711ca2a6c9a5ab"],
    ["i", "twitter:semisol_public", "1619358434134196225"],
    ["i", "mastodon:bitcoinhackers.org/@semisol", "109775066355589974"]
    ["i", "telegram:1087295469", "nostrdirectory/770"]
  ],
  // other fields...
}
```

An `i` tag MUST have two parameters, which are defined as the following: 1. `platform:identity`: This is the platform name (for example `github`) and the identity on that platform (for example `semisol`) joined together with `:`. 2. `proof`: String or object that points to the proof of owning this identity.

Clients SHOULD process any `i` tags with more than 2 values for future extensibility. Identity provider names SHOULD only include `a-z`, `0-9` and the characters `._-/` and MUST NOT include `:`. Identity names SHOULD be normalized if possible by replacing uppercase letters with lowercase letters, and if there are multiple aliases for an entity the primary one should be used.

Claim types

`github`

Identity: A GitHub username.

Proof: A GitHub Gist ID. This Gist should be created by `<identity>` with a single file that has the text `Verifying that I control the following Nostr public key: <npub encoded public key>`. This can be located at `https://gist.github.com/<identity>/<proof>`.

`twitter`

Identity: A Twitter username.

Proof: A Tweet ID. The tweet should be posted by `<identity>` and have the text `Verifying my account on nostr My Public Key: "<npub encoded public key>"`. This can be located at `https://twitter.com/<identity>/status/<proof>`.

`mastodon`

Identity: A Mastodon instance and username in the format `<instance>/@<username>`.

Proof: A Mastodon post ID. This post should be published by `<username>@<instance>` and have the text `Verifying that I control the following Nostr public key: "<npub encoded public key>"`. This can be located at `https://<identity>/<proof>`.

`telegram`

Identity: A Telegram user ID.

Proof: A string in the format `<ref>/<id>` which points to a message published in the public channel or group with name `<ref>` and message ID `<id>`. This message should be sent by user ID `<identity>` and have the text `Verifying that I control the following Nostr public key: "<npub encoded public key>"`. This can be located at `https://t.me/<proof>`.

NIP-25

Reactions

`draft` `optional`

A reaction is a `kind 7` event that is used to indicate user reactions to other events. A reaction's `content` field MUST include user-generated-content indicating the value of the reaction (conventionally `+`, `-`, or an emoji).

A reaction with `content` set to `+` or an empty string MUST be interpreted as a "like" or "upvote". A reaction with `content` set to `-` MUST be interpreted as a "dislike" or "downvote".

A reaction with `content` set to an emoji or [NIP-30] custom emoji SHOULD NOT be interpreted as a "like" or "dislike". Clients MAY instead display this emoji reaction on the post.

Tags

There MUST be always an `e` tag set to the `id` of the event that is being reacted to. The `e` tag SHOULD include a relay hint pointing to a relay where the event being reacted to can be found. If a client decides to include other `e`, which not recommended, the target event `id` should be last of the `e` tags.

There SHOULD be a `p` tag set to the `pubkey` of the event being reacted to. If a client decides to include other `p` tags, which not recommended, the target event `pubkey` should be last the `p` tags.

If the event being reacted to is an addressable event, an `a` SHOULD be included together with the `e` tag, it must be set to the coordinates (`kind:pubkey:d-tag`) of the event being reacted to.

The `e` and `a` tags SHOULD include relay and pubkey hints. The `p` tags SHOULD include relay hints.

The reaction event MAY include a `k` tag with the stringified kind number of the reacted event as its value.

Example code

```
func make_like_event(pubkey: String, privkey: String, liked: NostrEvent, hint: String) ->
        NostrEvent {
    var tags: [[String]] = []
    tags.append(["e", liked.id, hint, liked.pubkey])
    tags.append(["p", liked.pubkey, hint])
    tags.append(["k", String(liked.kind)])
    let ev = NostrEvent(content: "+", pubkey: pubkey, kind: 7, tags: tags)
    ev.calculate_id()
```

```
    ev.sign(privkey: privkey)
    return ev
}
```

External Content Reactions

If the target of a reaction is not a native nostr event, the reaction MUST be a `kind 17` event and MUST include NIP-73 external content `k` + `i` tags to properly reference the content.

Reacting to a website:

```
{
  "kind": 17,
  "content": "⭐",
  "tags": [
    ["k", "web"],
    ["i", "https://example.com"]
  ],
}
```

Reacting to a podcast episode:

```
{
  "kind": 17,
  "content": "+",
  "tags": [
    ["k", "podcast:guid"],
    ["i", "podcast:guid:917393e3-1b1e-5cef-ace4-edaa54e1f810", "https://fountain.fm/show/QRT0l2EfrKXNGDlRrmjL"],
    ["k", "podcast:item:guid"],
    ["i", "podcast:item:guid:PC20-229", "https://fountain.fm/episode/DQqBg5sD3qFGMCZoSuLF"]
  ],
}
```

Custom Emoji Reaction

The client may specify a custom emoji (NIP-30) `:shortcode:` in the reaction content. The client should refer to the emoji tag and render the content as an emoji if shortcode is specified.

```
{
  "kind": 7,
  "content": ":soapbox:",
  "tags": [
    ["emoji", "soapbox", "https://gleasonator.com/emoji/Gleasonator/soapbox.png"]
  ],
  // other fields...
}
```

The content can be set only one `:shortcode:`. And emoji tag should be one.

NIP-30

Custom Emoji

`draft` `optional`

Custom emoji may be added to **kind 0**, **kind 1**, **kind 1111** ([NIP-22](22.md)), **kind 7** ([NIP-25](25.md)) and **kind 30315** ([NIP-38](38.md)) events by including one or more `"emoji"` tags, in the form:

```
["emoji", <shortcode>, <image-url>, <emoji-set-address>]
```

Where:

- `<shortcode>` is a name given for the emoji, which MUST be comprised of only alphanumeric characters, hyphens, and underscores.
- `<image-url>` is a URL to the corresponding image file of the emoji.
- `<emoji-set-address>` is an optional address pointer (`kind:pubkey:d-tag`) to the kind 30030 emoji set ([NIP-51](51.md)) the emoji belongs to.

For each emoji tag, clients should parse emoji shortcodes (aka "emojify") like `:shortcode:` in the event to display custom emoji.

Clients may allow users to add custom emoji to an event by including `:shortcode:` identifier in the event, and adding the relevant `"emoji"` tags.

Kind 0 events

In kind 0 events, the `name` and `about` fields should be emojified.

```json
{
  "kind": 0,
  "content": "{\"name\":\"Alex Gleason :soapbox:\"}",
  "tags": [
    ["emoji", "soapbox", "https://gleasonator.com/emoji/Gleasonator/soapbox.png"]
  ],
  "pubkey": "79c2cae114ea28a981e7559b4fe7854a473521a8d22a66bbab9fa248eb820ff6",
  "created_at": 1682790000
}
```

Kind 1 events

In kind 1 events, the `content` should be emojified.

```json
{
  "kind": 1,
  "content": "Hello :gleasonator: 😂 :ablobcatrainbow: :disputed: yolo",
  "tags": [
    ["emoji", "ablobcatrainbow", "https://gleasonator.com/emoji/blobcat/ablobcatrainbow.png",
        "30030:79c2cae114ea28a981e7559b4fe7854a473521a8d22a66bbab9fa248eb820ff6:blobcats"],
    ["emoji", "disputed", "https://gleasonator.com/emoji/Fun/disputed.png"],
    ["emoji", "gleasonator", "https://gleasonator.com/emoji/Gleasonator/gleasonator.png"]
```

```
  ],
  "pubkey": "79c2cae114ea28a981e7559b4fe7854a473521a8d22a66bbab9fa248eb820ff6",
  "created_at": 1682630000
}
```

Kind 1111 events

In kind 1111 events, the `content` should be emojified.

```json
{
  "kind": 1111,
  "content": "Let's party! :hyves_banaan:",
  "tags": [
    ["emoji", "hyves_banaan", "https://relay.nyves.nl/
        8408b236b11fe5c764d139d37e670d5207b3825fe6396392799b02c4e6bd2ebf.gif",
        "30030:9919981db5d9f59713bd717069212c32a19b55f2280a51e70c49c5b73da306b9:Hyves"]
  ],
  "pubkey": "79c2cae114ea28a981e7559b4fe7854a473521a8d22a66bbab9fa248eb820ff6",
  "created_at": 1682630000
}
```

Kind 7 events

In kind 7 events, the `content` should be emojified.

```json
{
  "kind": 7,
  "content": ":dezh:",
  "tags": [
    ["emoji", "dezh", "https://raw.githubusercontent.com/dezh-tech/brand-assets/main/dezh/logo/
        black-normal.svg"]
  ],
  "pubkey": "79c2cae114ea28a981e7559b4fe7854a473521a8d22a66bbab9fa248eb820ff6",
  "created_at": 1682630000
}
```

NIP-18

Reposts

`draft` `optional`

A repost is a `kind 6` event that is used to signal to followers that a `kind 1` text note is worth reading.

The `content` of a repost event is *the stringified JSON of the reposted note*. It MAY also be empty, but that is not recommended. Reposts of [NIP-70](70.md)-protected events SHOULD always have an empty `content`.

The repost event MUST include an `e` tag with the `id` of the note that is being reposted. That tag MUST include a relay URL as its third entry to indicate where it can be fetched.

The repost SHOULD include a `p` tag with the `pubkey` of the event being reposted.

Quote Reposts

Mentions to NIP-21 entities like `nevent`, `note` and `naddr` on any event must be converted into `q` tags. The `q` tag ensures quote reposts are not pulled and included as replies in threads. It also allows you to easily pull and count all of the quotes for a post. The syntax follows

```
["q", "<event-id> or <event-address>", "<relay-url>", "<pubkey-if-a-regular-event>"]
```

Generic Reposts

Since `kind 6` reposts are reserved for `kind 1` contents, we use `kind 16` as a “generic repost”, that can include any kind of event inside other than `kind 1`.

`kind 16` reposts SHOULD contain a `"k"` tag with the stringified kind number of the reposted event as its value.

When reposting a replaceable event, the repost SHOULD include an `"a"` tag with the event coordinate (`kind:pubkey:d-tag`) of the reposted event.

If the `"a"` tag is not present, it indicates that a specific version of a replaceable event is being reposted, in which case the `content` field must contain the full JSON string of the reposted event.

NIP-27

Text Note References

`draft` `optional`

This document standardizes the treatment given by clients of inline references of other events and profiles inside the `.content` of any event that has readable text in its `.content` (such as kinds 1 and 30023).

When creating an event, clients should include mentions to other profiles and to other events in the middle of the `.content` using NIP-21 codes, such as `nostr:nprofile1qqsw3dy8cpu...6x2argwghx6egsqstvg`.

Including NIP-18’s quote tags (`["q", "<event-id> or <event-address>", "<relay-url>", "<pubkey-if-a-regular-event>"]`) for each reference is optional, clients should do it whenever they want the profile being mentioned to be notified of the mention, or when they want the referenced event to recognize their mention as a reply.

A reader client that receives an event with such `nostr:...` mentions in its `.content` can do any desired context augmentation (for example, linking to the profile or showing a preview of the mentioned event contents) it wants in the process. If turning such mentions into links, they could become internal links, NIP-21 links or direct links to web clients that will handle these references.

Example of a profile mention process

Suppose Bob is writing a note in a client that has search-and-autocomplete functionality for users that is triggered when they write the character `@`.

As Bob types `"hello @mat"` the client will prompt him to autocomplete with [mattn's profile](), showing a picture and name.

Bob presses "enter" and now he sees his typed note as `"hello @mattn"`, `@mattn` is highlighted, indicating that it is a mention. Internally, however, the event looks like this:

```json
{
  "content": "hello
       nostr:nprofile1qqszclxx9f5haga8sfjjrulaxncvkfekj097t6f3pu65f86rvg49ehqj6f9dh",
  "created_at": 1679790774,
  "id": "f39e9b451a73d62abc5016cffdd294b1a904e2f34536a208874fe5e22bbd47cf",
  "kind": 1,
  "pubkey": "79be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798",
  "sig":
       "f8c8bab1b90cc3d2ae1ad999e6af8af449ad8bb4edf64807386493163e29162b5852a796a8f474d6b1001cddbaac0de4392

  "tags": [
    [
      "p",
      "2c7cc62a697ea3a7826521f3fd34f0cb273693cbe5e9310f35449f43622a5cdc"
    ]
  ]
}
```

(Alternatively, the mention could have been a `nostr:npub1...` URL.)

After Bob publishes this event and Carol sees it, her client will initially display the `.content` as it is, but later it will parse the `.content` and see that there is a `nostr:` URL in there, decode it, extract the public key from it (and possibly relay hints), fetch that profile from its internal database or relays, then replace the full URL with the name `@mattn`, with a link to the internal page view for that profile.

Verbose and probably unnecessary considerations

- The example above was very concrete, but it doesn't mean all clients have to implement the same flow. There could be clients that do not support autocomplete at all, so they just allow users to paste raw [NIP-19]() codes into the body of text, then prefix these with `nostr:` before publishing the event.
- The flow for referencing other events is similar: a user could paste a `nevent1...` code and the client will turn that into a `nostr:nevent1...` URL. Then upon reading such references the client may show the referenced note in a preview box or something like that – or nothing at all.
- Other display procedures can be employed: for example, if a client that is designed for dealing with only `kind:1` text notes sees, for example, a [`kind:30023`]() `nostr:naddr1...` URL reference in the `.content`, it can, for example, decide to turn that into a link to some hardcoded webapp capable of displaying such events.
- Clients may give the user the option to include or not include tags for mentioned events or profiles. If someone wants to mention `mattn` without notifying them, but still have a nice augmentable/clickable

link to their profile inside their note, they can instruct their client to *not* create a `["p", ...]` tag for that specific mention.
- In the same way, if someone wants to reference another note but their reference is not meant to show up along other replies to that same note, their client can choose to not include a corresponding `["e", ...]` tag for any given `nostr:nevent1...` URL inside `.content`. Clients may decide to expose these advanced functionalities to users or be more opinionated about things.

Warning `unrecommended`: deprecated in favor of NIP-27

NIP-08

Handling Mentions

`final` `unrecommended` `optional`

This document standardizes the treatment given by clients of inline mentions of other events and pubkeys inside the content of `text_note`s.

Clients that want to allow tagged mentions they MUST show an autocomplete component or something analogous to that whenever the user starts typing a special key (for example, "@") or presses some button to include a mention etc – or these clients can come up with other ways to unambiguously differentiate between mentions and normal text.

Once a mention is identified, for example, the pubkey `27866e9d854c78ae625b867eefdfa9580434bc3e675be08d2acb526610d96fbe`, the client MUST add that pubkey to the `.tags` with the tag `p`, then replace its textual reference (inside `.content`) with the notation `#[index]` in which "index" is equal to the 0-based index of the related tag in the tags array.

The same process applies for mentioning event IDs.

A client that receives a `text_note` event with such `#[index]` mentions in its `.content` CAN do a search-and-replace using the actual contents from the `.tags` array with the actual pubkey or event ID that is mentioned, doing any desired context augmentation (for example, linking to the pubkey or showing a preview of the mentioned event contents) it wants in the process.

Where `#[index]` has an `index` that is outside the range of the tags array or points to a tag that is not an `e` or `p` tag or a tag otherwise declared to support this notation, the client MUST NOT perform such replacement or augmentation, but instead display it as normal text.

NIP-38

User Statuses

`draft` `optional`

Abstract

This NIP enables a way for users to share live statuses such as what music they are listening to, as well as what they are currently doing: work, play, out of office, etc.

Live Statuses

A special event with `kind:30315` "User Status" is defined as an *optionally expiring addressable event*, where the `d` tag represents the status type:

For example:

```
{
  "kind": 30315,
  "content": "Sign up for nostrasia!",
  "tags": [
    ["d", "general"],
    ["r", "https://nostr.world"]
  ],
}
```

```
{
  "kind": 30315,
  "content": "Intergalatic - Beastie Boys",
  "tags": [
    ["d", "music"],
    ["r", "spotify:search:Intergalatic%20-%20Beastie%20Boys"],
    ["expiration", "1692845589"]
  ],
}
```

Two common status types are defined: `general` and `music`. `general` represent general statuses: "Working", "Hiking", etc.

`music` status events are for live streaming what you are currently listening to. The expiry of the `music` status should be when the track will stop playing.

Any other status types can be used but they are not defined by this NIP.

The status MAY include an `r`, `p`, `e` or `a` tag linking to a URL, profile, note, or addressable event.

The `content` MAY include emoji(s), or [NIP-30](#) custom emoji(s). If the `content` is an empty string then the client should clear the status.

Client behavior

Clients MAY display this next to the username on posts or profiles to provide live user status information.

Use Cases

- Calendar nostr apps that update your general status when you're in a meeting

- Nostr Nests that update your general status with a link to the nest when you join
- Nostr music streaming services that update your music status when you're listening
- Podcasting apps that update your music status when you're listening to a podcast, with a link for others to listen as well
- Clients can use the system media player to update playing music status

NIP-58

Badges

`draft` `optional`

Four special events are used to define, award, display and categorize badges in user profiles:

1. A "Badge Definition" event is defined as an addressable event with kind `30009` having a `d` tag with a value that uniquely identifies the badge (e.g. `bravery`) published by the badge issuer. Badge definitions can be updated.

2. A "Badge Award" event is a kind `8` event with a single `a` tag referencing a "Badge Definition" event and one or more `p` tags, one for each pubkey the badge issuer wishes to award. Awarded badges are immutable and non-transferable.

3. A "Profile Badges" event is defined as a *replaceable event* with kind `10008` as a [NIP-51](51.md) standard list. Profile badges contain an ordered list of pairs of `a` and `e` tags referencing a `Badge Definition` and a `Badge Award` for each badge to be displayed. It may also contain `a` tags referencing "Badge Set" events.

4. A "Badge Set" event is defined as an *addressable event* with kind `30008` as a [NIP-51](51.md) set. Badge sets allow users to categorize accepted badges into labeled groups. They contain ordered pairs of `a` and `e` tags referencing a `Badge Definition` and a `Badge Award` for each badge in the set.

Badge Definition event

The following tags MUST be present:

- `d` tag with the unique name of the badge.

The following tags MAY be present:

- A `name` tag with a short name for the badge.
- `image` tag whose value is the URL of a high-resolution image representing the badge. The second value optionally specifies the dimensions of the image as `widthxheight` in pixels. Badge recommended dimensions is 1024x1024 pixels.
- A `description` tag whose value MAY contain a textual representation of the image, the meaning behind the badge, or the reason of its issuance.
- One or more `thumb` tags whose first value is an URL pointing to a thumbnail version of the image referenced in the `image` tag. The second value optionally specifies the dimensions of the thumbnail as `widthxheight` in pixels.

Badge Award event

The following tags MUST be present:

- An `a` tag referencing a kind `30009` Badge Definition event.
- One or more `p` tags referencing each pubkey awarded.

Profile Badges Event

The number of badges a pubkey can be awarded is unbounded. The Profile Badge event allows individual users to accept or reject awarded badges, as well as choose the display order of badges on their profiles.

The following tags MAY be present:

- Zero or more ordered consecutive pairs of `a` and `e` tags referencing a kind `30009` Badge Definition and kind `8` Badge Award, respectively. Clients SHOULD ignore `a` without corresponding `e` tag and viceversa. Badge Awards referenced by the `e` tags should contain the same `a` tag.

Motivation

Users MAY be awarded badges (but not limited to) in recognition, in gratitude, for participation, or in appreciation of a certain goal, task or cause.

Users MAY choose to decorate their profiles with badges for fame, notoriety, recognition, support, etc., from badge issuers they deem reputable.

Recommendations

Clients MAY whitelist badge issuers (pubkeys) for the purpose of ensuring they retain a valuable/special factor for their users.

Badge image recommended aspect ratio is 1:1 with a high-res size of 1024x1024 pixels.

Badge thumbnail image recommended dimensions are: 512x512 (xl), 256x256 (l), 64x64 (m), 32x32 (s) and 16x16 (xs).

Clients MAY choose to render less badges than those specified by users in the Profile Badges event or replace the badge image and thumbnails with ones that fits the theme of the client.

Clients SHOULD attempt to render the most appropriate badge thumbnail according to the number of badges chosen by the user and space available. Clients SHOULD attempt render the high-res version on user action (click, tap, hover).

Example of a Badge Definition event

```json
{
  "pubkey": "alice",
  "kind": 30009,
  "tags": [
    ["d", "bravery"],
    ["name", "Medal of Bravery"],
```

```
    ["description", "Awarded to users demonstrating bravery"],
    ["image", "https://nostr.academy/awards/bravery.png", "1024x1024"],
    ["thumb", "https://nostr.academy/awards/bravery_256x256.png", "256x256"]
  ],
  // other fields...
}
```

Example of Badge Award event

```
{
  "id": "<badge award event id>",
  "kind": 8,
  "pubkey": "alice",
  "tags": [
    ["a", "30009:alice:bravery"],
    ["p", "bob", "wss://relay"],
    ["p", "charlie", "wss://relay"]
  ],
  // other fields...
}
```

Example of a Profile Badges event

Honorable Bob The Brave:

```
{
  "kind": 10008,
  "pubkey": "bob",
  "tags": [
    ["a", "30009:alice:bravery"],
    ["e", "<bravery badge award event id>", "wss://nostr.academy"],
    ["a", "30009:alice:honor"],
    ["e", "<honor badge award event id>", "wss://nostr.academy"]
  ],
  // other fields...
}
```

Deprecated Profile Badges event

An earlier version of this NIP used a kind `30008` addressable event with a `d` tag value of `profile_badges` for the Profile Badges event. Clients should treat these events as equivalent to kind `10008` and migrate to the new format.

NIP-85

Trusted Assertions

`draft` `optional`

Certain Webs of Trust calculations require access to a large volume of events and/or computing power, making it virtually impossible to perform them directly on clients. This NIP allows users to offload such calculations to declared trusted service providers, and for these providers to publish signed “Trusted Assertion” events for the user client’s consumption.

Assertion Events

Trusted Assertions are always addressable (replaceable) events with the d tag pointing to the “subject” of the assertion. This NIP currently recognizes four distinct target “subjects” on which such calculations can be performed: *pubkeys*, *regular events*, *addressable events*, and *nip73 identifiers*. Each subject type is mapped to an event kind:

Subject	Event Kind	d tag value
User	30382	<pubkey>
Event	30383	<event_id>
Addressable Event	30384	<event_address>
NIP-73 Identifier	30385	<i-tag>

Calculation results are saved in pre-defined tags whose syntax and semantics are agreed upon by providers and clients.

Example of ranking a pubkey with a web of trust score of 89:

```
{
  "kind": 30382,
  "pubkey": "<service pubkey>",
  "tags": [
    ["d", "e88a691e98d9987c964521dff60025f60700378a4879180dcbbb4a5027850411"], // target user's public key
    ["rank", "89"],
  ],
  "content": "",
  //...
}
```

Service providers MUST use different service keys for distinct algorithms, including a key per user when the algorithm is personalized to that user’s point of view or settings.

Kind 30382: Users as Subject:

The following result types have been declared:

Result type	Tag name	Tag value format
Follower Count	followers	int
User Rank	rank	int, norm 0-100
First Post Time	first_created_at	unix timestamp
Post Count	post_cnt	int
Reply Count	reply_cnt	int

Result type	Tag name	Tag value format
Reactions Count	`reactions_cnt`	int
Zap Amount Received	`zap_amt_recd`	int, sats
Zap Amount Sent	`zap_amt_sent`	int, sats
Zap Number Received	`zap_cnt_recd`	int
Zap Number Sent	`zap_cnt_sent`	int
Avg Zap Amount/day recd	`zap_avg_amt_day_recd`	int, sats
Avg Zap Amount/day sent	`zap_avg_amt_day_sent`	int, sats
Reports Received	`reports_cnt_recd`	int
Reports Sent	`reports_cnt_sent`	int
Common Topics	`t`	string
Generally active start	`active_hours_start`	int, 0-24, UTC
Generally active end	`active_hours_end`	int, 0-24, UTC

Each provider can offer their own ways to calculate such values. For instance, the Follower Count of one trust provider might remove the user's muted public keys while another provider keeps them. Users can then choose how they want to see this information in their preferred client by picking a provider that aligns with their view.

Kind 30383: Events as Subject

Providers can rate individual events with the following tags:

Result type	Tag name	Tag value format
Event Rank	`rank`	int, norm 0-100
Event Comment Count	`comment_cnt`	int
Event Quote Count	`quote_cnt`	int
Event Repost Count	`repost_cnt`	int
Event Reaction Count	`reaction_cnt`	int
Event Zap Count	`zap_cnt`	int
Event Zap Amount	`zap_amount`	int, sats

Kind 30384: Addressables as Subject

Providers can rate all versions of addressable events using the following tags:

Result type	Tag name	Tag value format
Address Rank	`rank`	int, norm 0-100
Address Comment Count	`comment_cnt`	int

Result type	Tag name	Tag value format
Address Quote Count	`quote_cnt`	int
Address Repost Count	`repost_cnt`	int
Address Reaction Count	`reaction_cnt`	int
Address Zap Count	`zap_cnt`	int
Address Zap Amount	`zap_amount`	int, sats

Kind 30385: External identifier as Subject

Providers can rate books, locations, movies, websites, and hashtags using NIP-73 identifiers.

Result type	Tag name	Tag value format
Rank	`rank`	int, norm 0-100
Comment Count	`comment_cnt`	int
Reaction Count	`reaction_cnt`	int

NIP-73 `k` tags should be added to the event as well.

Declaring Trusted Service Providers

Kind `10040` lists the user's preferred source for each result. Each `kind:tag` is followed by the publisher's `pubkey` and the `relay` where assertion events can be obtained. Service providers may use multiple keys and relays as needed, including a key per user when the algorithm is personalized to that user's point of view or settings. Users can share these settings publicly or keep them private by JSON-stringifying and encrypting the tag list within the `.content` field using NIP-44.

```
{
  "kind": 10040,
  "tags": [
    ["<kind:tag>", "<service key>", "<relay hint>"],

    // examples
    ["30382:rank", "4fd5e210530e4f6b2cb083795834bfe5108324f1ed9f00ab73b9e8fcfe5f12fe", "wss://
        nip85.nostr.band"],
    ["30382:rank", "3d842afecd5e293f28b6627933704a3fb8ce153aa91d790ab11f6a752d44a42d", "wss://
        nostr.wine"],
    ["30382:zap_amt_sent", "4fd5e210530e4f6b2cb083795834bfe5108324f1ed9f00ab73b9e8fcfe5f12fe",
        "wss://nip85.nostr.band"],
  ],
  "content": nip44Encrypt(JSON.stringify([
    ["<kind:tag>", "<service key>", "<relay hint>"],

     // examples
    ["30383:rank", "4fd5e210530e4f6b2cb083795834bfe5108324f1ed9f00ab73b9e8fcfe5f12fe", "wss://
        nip85.nostr.band"],
    ["30384:rank", "4fd5e210530e4f6b2cb083795834bfe5108324f1ed9f00ab73b9e8fcfe5f12fe", "wss://
        nip85.nostr.band"],
  ]),
```

```
  //...
}
```

If the provider offers several algorithms or multiple points of view of an algorithm, the key listed in each tag SHOULD point to the key created for each algorithm or point of view.

Final Considerations

Service providers SHOULD update Trusted Assertions as fast as new information arrives, but only if the contents of each event actually change to avoid re-downloading the same information.

Service providers MAY limit access to the results by using paid relays.

In TAs, `p`, `e`, and `a` tags with the same value as the `d` tag MAY be used to add a relay hint to the home relay of that user or event.

Appendix 1: Service provider discoverability

Service Providers SHOULD sign a kind `0` of each service key that explains who controls the key and what the current version of the algorithm is about.

```
{
  "kind": 0,
  "pubkey": "<service pubkey>",
  "tags": [],
  "content": "{
   \"name\" = \"Vitor's Brainstormer\",
   \"about\" = \"A Web of Trust algorithm from Vitor's point of view that considers Follows and
        Mutes, but no reports, and gives extra score points for anyone around Boston\",
   \"picture\" = \"https://brainstorm.com/logo.png\",
   \"website\" = \"https://brainstorm.com\"
  }",
  // other fields...
}
```

Clients wishing to offer a list of Service Providers to their users SHOULD: 1. Download kind `10040` events of the user's follow list 2. Connect to each of the listed relays and download the kind `0` of the respective service keys 3. Parse the kind `0` and collect the `website` property 4. Load the OpenGraph tags of that website and display them as clickable items

NIP-88

Polls

`draft` `optional`

This NIP defines the event scheme that describe Polls on nostr.

Events

Poll Event

The poll event is defined as a `kind:1068` event.

- **content** key holds the label for the poll.

Major tags in the poll event are:

- **option**: The option tags contain an OptionId(any alphanumeric) field, followed by an option label field.
- **relay**: One or multiple tags that the poll is expecting respondents to respond on.
- **polltype**: can be "singlechoice" or "multiplechoice". Polls that do not have a polltype should be considered a "singlechoice" poll.
- **endsAt**: signifying at which unix timestamp the poll is meant to end.

Example Event

```
{
  "content": "Pineapple on pizza",
  "created_at": 1719888496,
  "id": "9d1b6b9562e66f2ecf35eb0a3c2decc736c47fddb13d6fb8f87185a153ea3634",
  "kind": 1068,
  "pubkey": "dee45a23c4f1d93f3a2043650c5081e4ac14a778e0acbef03de3768e4f81ac7b",
  "sig":
        "7fa93bf3c430eaef784b0dacc217d3cd5eff1c520e7ef5d961381bc0f014dde6286618048d924808e54d1be03f2f2c2f0f8

  "tags": [
    ["option", "qj518h583", "Yay"],
    ["option", "gga6cdnqj", "Nay"],
    ["relay", "<relay url1>"],
    ["relay", "<relay url2>"],
    ["polltype", "singlechoice"],
    ["endsAt", "<unix timestamp in seconds>"]
  ]
}
```

Responses

The response event is a `kind:1018` event. It contains an e tag with the poll event it is referencing, followed by one or more response tags.

- **response** : The tag contains "response" as its first positional argument followed by the option Id selected.

The responses are meant to be published to the relays specified in the poll event.

Example Response Event

```
{
  "content": "",
  "created_at": 1720097117,
  "id": "60a005e32e9596c3f544a841a9bc4e46d3020ca3650d6a739c95c1568e33f6d8",
```

```json
  "kind": 1018,
  "pubkey": "1bc70a0148b3f316da33fe7e89f23e3e71ac4ff998027ec712b905cd24f6a411",
  "sig":
       "30071a633c65db8f3a075c7a8de757fbd8ce65e3607f4ba287fe6d7fbf839a380f94ff4e826fbba593f6faaa13683b7ea91

  "tags": [
    ["e", "1fc80cf813f1af33d5a435862b7ef7fb96b47e68a48f1abcadf8081f5a545550"],
    ["response", "gga6cdnqj"],
    ["response", "m3agjsdq1"]
  ]
}
```

Poll Types

The polltype setting dictates how multiple response tags are handled in the `kind:1018` event.

- **polltype: singlechoice**: The first response tag is to be considered the actual response.
- **polltype: multiplechoice**: The first response tag pointing to each id is considered the actual response, without considering the order of the response tags.

Counting Results

Results can be queried by fetching `kind:1018` events from the relays specified in the poll. The results displayed should only be 1 vote event per pubkey. In case of multiple events for a pubkey, the event with the largest timestamp within the poll limits should be considered.

Example for querying polls.

```ts
const fetchVoteEvents = (filterPubkeys: string[]) => {
  let resultFilter: Filter = {
    "#e": [pollEvent.id],
    kinds: [1018],
  };
  if (filterPubkeys?.length) {
    resultFilter.authors = filterPubkeys;
  }
  if (pollExpiration) {
    resultFilter.until = Number(pollExpiration);
  }
  pool.subscribeMany(relays, [resultFilter], {
    onevent: handleResultEvent,
  });
};
```

Example for maintaining OneVotePerPubkey

```ts
const oneVotePerPubkey = (events: Event[]) => {
  const eventMap = new Map<string, Event>();

  events.forEach((event) => {
    if (
      !eventMap.has(event.pubkey) ||
      event.created_at > eventMap.get(event.pubkey)!.created_at
    ) {
```

```
      eventMap.set(event.pubkey, event);
    }
  });

  return Array.from(eventMap.values());
};
```

Relays

It is advisable for poll authors to use relays that do not allow backdated events and do not honor kind:5 (delete) requests for vote events in order to maintain the integrity of poll results after the poll has ended.

Curation

The clients may configure fetching results by specific people. This can be achieved by creating `kind:30000` follow sets, and fetching results only from the follow set. Clients can also employ other curation algorithms, like Proof Of Work and Web of Trust scores for result curations.

NIP-B0

Web Bookmarking

`draft` `optional`

This NIP defines `kind:39701` for a URI as editable web bookmark.

Format

The format uses `kind:39701`.

The `.content` should be a detailed description of the web bookmark. It can be an empty string.

The `d` tag is just their URI. All characters before the hostname should be omitted if the scheme is `https`.

In this way web bookmarks events can be queried by the `d` tag by clients.

Metadata

Metadata fields can be added as tags to the event as necessary.

- `"t"`, for "tags" / "hashtags" (i.e. topics about which the event might be of relevance)
- `"published_at"`, for the timestamp in unix seconds (stringified) of the first time the bookmark was published
- `"title"`, title about bookmark and can be used as a attribute for the HTML link element

Example

```
{
  "kind": 39701,
```

```
  "id": "d7a92714f81d0f712e715556aee69ea6da6bfb287e6baf794a095d301d603ec7",
  "pubkey": "2729620da105979b22acfdfe9585274a78c282869b493abfa4120d3af2061298",
  "created_at": 1738869705,
  "tags": [
    // Required tags
    ["d", "alice.blog/post"],
    // Optional tags
    ["published_at", "1738863000"],
    ["title", "Blog insights by Alice"],
    ["t", "post"],
    ["t", "insight"]
  ],
  "content": "A marvelous insight by Alice about the nature of blogs and posts.",
  "sig":
      "36d34e6448fe0223e9999361c39c492a208bc423d2fcdfc2a3404e04df7c22dc65bbbd62dbe8a4373c62e4d29aac285b5aa

}
```

Replies & Comments

Replies to `kind:39701` MUST use `kind 1111` events as comments with NIP-22.

Groups

Feeds are one-to-many. Groups are many-to-many: a room, a community, a thread with a title. Nostr has tried a few shapes for this, and they are not all equal.

NIP-28 is the original public chat: channels created as events, with messages that live on ordinary relays. It is unrecommended. The spec is still here because plenty of old events and clients speak it, but new work should look at NIP-29 instead. NIP-29 puts the group on a relay that knows about membership, roles, and moderation. The relay is still replaceable, but it is allowed to be opinionated about who may post.

Two newer NIPs cover lighter rooms. NIP-C7 is a simple chat message (kind 9) meant to be shown as a stream, not a Twitter-style thread. Replies quote a parent rather than nesting forever. NIP-7D is a forum thread: a titled topic (kind 11) with NIP-22 comments as replies. If you are building a Discord-like space, start with NIP-29. If you want a campfire or a forum, C7 and 7D are the smaller tools.

In this chapter

- NIP-28 — Public chat (unrecommended; try NIP-29)
- NIP-29 — Relay-based groups
- NIP-C7 — Chats
- NIP-7D — Forum threads

> **Warning** `unrecommended`: try NIP-29 instead

NIP-28

Public Chat

`draft` `unrecommended` `optional`

This NIP defines new event kinds for public chat channels, channel messages, and basic client-side moderation.

It reserves five event kinds (40-44) for immediate use:

- `40 - channel create`
- `41 - channel metadata`
- `42 - channel message`
- `43 - hide message`
- `44 - mute user`

Client-centric moderation gives client developers discretion over what types of content they want included in their apps, while imposing no additional requirements on relays.

Kind 40: Create channel

Create a public chat channel.

In the channel creation `content` field, Client SHOULD include basic channel metadata (`name`, `about`, `picture` and `relays` as specified in kind 41).

```
{
  "content": "{\"name\": \"Demo Channel\", \"about\": \"A test channel.\", \"picture\": \"https://placekitten.com/200/200\", \"relays\": [\"wss://nos.lol\", \"wss://nostr.mom\"]}",
 // other fields...
}
```

Kind 41: Set channel metadata

Update a channel's public metadata.

Kind 41 is used to update the metadata without modifying the event id for the channel. Only the most recent kind 41 per `e` tag value MAY be available.

Clients SHOULD ignore kind 41s from pubkeys other than the kind 40 pubkey.

Clients SHOULD support basic metadata fields:

- `name` - string - Channel name
- `about` - string - Channel description
- `picture` - string - URL of channel picture
- `relays` - array - List of relays to download and broadcast events to

Clients MAY add additional metadata fields.

Clients SHOULD use NIP-10 marked "e" tags to recommend a relay.

It is also possible to set the category name using the "t" tag. This category name can be searched and filtered.

```
{
  "content": "{\"name\": \"Updated Demo Channel\", \"about\": \"Updating a test channel.\",
        \"picture\": \"https://placekitten.com/201/201\", \"relays\": [\"wss://nos.lol\",
        \"wss://nostr.mom\"]}",
  "tags": [
    ["e", <channel_create_event_id>, <relay-url>, "root"],
    ["t", <category_name-1>],
    ["t", <category_name-2>],
    ["t", <category_name-3>],
  ],
  // other fields...
}
```

Kind 42: Create channel message

Send a text message to a channel.

Clients SHOULD use NIP-10 marked "e" tags to recommend a relay and specify whether it is a reply or root message.

Clients SHOULD append NIP-10 "p" tags to replies.

Root message:

```
{
  "content": <string>,
  "tags": [["e", <kind_40_event_id>, <relay-url>, "root"]],
  // other fields...
}
```

Reply to another message:

```
{
  "content": <string>,
  "tags": [
      ["e", <kind_40_event_id>, <relay-url>, "root"],
      ["e", <kind_42_event_id>, <relay-url>, "reply"],
      ["p", <pubkey>, <relay-url>],
      // rest of tags...
  ],
  // other fields...
}
```

Kind 43: Hide message

User no longer wants to see a certain message.

The `content` may optionally include metadata such as a `reason`.

Clients SHOULD hide event 42s shown to a given user, if there is an event 43 from that user matching the event 42 `id`.

Clients MAY hide event 42s for other users other than the user who sent the event 43.

(For example, if three users 'hide' an event giving a reason that includes the word 'pornography', a Nostr client that is an iOS app may choose to hide that message for all iOS clients.)

```
{
  "content": "{\"reason\": \"Dick pic\"}",
  "tags": [["e", <kind_42_event_id>]],
  // other fields...
}
```

Kind 44: Mute user

User no longer wants to see messages from another user.

The `content` may optionally include metadata such as a `reason`.

Clients SHOULD hide event 42s shown to a given user, if there is an event 44 from that user matching the event 42 `pubkey`.

Clients MAY hide event 42s for users other than the user who sent the event 44.

```
{
  "content": "{\"reason\": \"Posting dick pics\"}",
  "tags": [["p", <pubkey>]],
  // other fields...
}
```

Relay recommendations

Clients SHOULD use the relay URLs of the metadata events.

Clients MAY use any relay URL. For example, if a relay hosting the original kind 40 event for a channel goes offline, clients could instead fetch channel data from a backup relay, or a relay that clients trust more than the original relay.

Motivation

If we're solving censorship-resistant communication for social media, we may as well solve it also for Telegram-style messaging.

We can bring the global conversation out from walled gardens into a true public square open to all.

Additional info

- Chat demo PR with fiatjaf+jb55 comments
- Conversation about NIP16

NIP-29

Relay-based Groups

`draft` `optional` `relay`

This NIP defines a standard for groups that are only writable by a closed set of users. They can be public for reading by external users or not.

Groups are identified by a random string of any length that serves as an *id*.

There is no way to create a group, what happens is just that relays (most likely when asked by users) will create rules around some specific ids so these ids can serve as an actual group, henceforth messages sent to that group will be subject to these rules.

Normally a group will originally belong to one specific relay, but the community may choose to move the group to other relays or even fork the group so it exists in different forms – still using the same *id* – across different relays.

Group identity, migration and forking

A group is held together by two things: the group *id* and the relay that currently enforces access rules over that *id* on behalf of one or more group owners or admins. Because enforcement is tied to a specific relay, the group is only as durable as that relay's willingness and ability to keep hosting it.

Moving a group to another relay

If the relay hosting a group becomes unresponsive, goes offline permanently, or simply becomes untrustworthy, the group can be migrated by copying the existing events to a different relay, as long as that new relay agrees to receive the old group and to continue enforcing the same rules and acknowledging the same admins.

Forking a group

Another situation in which it might be desirable to move a relay to a different group is that of a disagreement between members of the original group.

A dissident faction within a group can take the group's history to a different relay that agrees to *change* the rules of that group and assign admin powers to a different set of users. This is a group *fork*: the new relay keeps the same *id* but enforces a different set of moderation policies and a different admin roster than the original relay.

Forks are not bugs. It's a feature of this NIP that the same *id* can simultaneously identify different communities with different governance, hosted by different relays, each with their own `kind:39000` metadata event, their own `kind:39001` admin list and their own message history.

Detecting migrations and forks

In order to check whether a group has migrated or whether a fork exists, clients SHOULD periodically – and MUST, if their primary relay for a group is offline or unreachable – look at the `kind:10009` event of the group's admins and of trusted friends. The pubkeys of the admins of the groups the user is in SHOULD be cached locally so this check can be performed even when the original relay is down.

If a `kind:10009` entry for a group now points to a different relay hint than the one the client was previously using, the client SHOULD notify the user that the group may have moved or been forked, and offer to fetch the group's events from the new relay and, if the user wishes, to migrate their participation to that relay as well, updating the current user's `kind:10009` in the process.

Replicating a group for resilience

Groups that do not want to lose history in case their primary relay goes down SHOULD set up a live backup/replica of all of the group's events on a second relay. The replica relay does not need to enforce any moderation rules of its own – it exists only to preserve the event history so that, if the primary relay disappears, a new authoritative relay can be chosen and the historical events copied over from the replica.

Relay-generated events

Relays are supposed to generate the events that describe group metadata and group admins. These are *addressable* events signed by the relay keypair directly, with the group *id* as the `d` tag.

Group referencing

A group may be identified by a `naddr1...` code to its `kind:39000` metadata event, with the public key set to the relay `self` public key, the `d` identifier set to the group id, the kind set to `39000` and a relay hint to the relay that is hosting the group.

An invite code may be appended to the group identifier:

```
naddr1...?invite=<code>
```

Clients should use this code in the `code` tag of the `kind:9021` join request. Since the bech32 charset doesn't include `?`, everything before the `?` is still a valid identifier on its own, so clients that don't understand the suffix can just ignore it.

The `h` tag

Events sent by users to groups (chat messages, text notes, moderation events etc) MUST have an `h` tag with the value set to the group *id*.

Timeline references

In order to not be used out of context, events sent to these groups may contain references to previous events seen from the same relay in the `previous` tag. The choice of which previous events to pick belongs to the clients. The references are to be made using the first 8 characters (4 bytes) of any event in the last 50 events seen by the user in the relay, excluding events by themselves. There can be any number of references (including zero), but it's recommended that clients include at least 3 and that relays enforce this.

This is a hack to prevent messages from being broadcasted to external relays that have forks of one group out of context. Relays are expected to reject any events that contain timeline references to events not found in their own database. Clients should also check these to keep relays honest about them.

Example

```
["previous", "eb96c864", "2db75638", "b5d1065f"]
```

Late publication

Relays should prevent late publication (messages published now with a timestamp from days or even hours ago) unless they are open to receive a group forked or moved from another relay.

Group management

Groups can have any number of users with elevated access. These users are identified by role labels which are arbitrarily defined by the relays (see also the description of `kind:39003`). What each role is capable of not defined in this NIP either, it's a relay policy that can vary. Roles can be assigned by other users (as long as they have the capability to add roles) by publishing a `kind:9000` event with that user's pubkey in a `p` tag and the roles afterwards (even if the user is already a group member a `kind:9000` can be issued and the user roles must just be updated).

The roles supported by the group as to having some special privilege assigned to them should be accessible on the event `kind:39003`, but the relay may also accept other role names, arbitrarily defined by clients, and just not do anything with them.

Users with any roles that have any privilege can be considered *admins* in a broad sense and be returned in the `kind:39001` event for a group.

Live audio/video (AV) spaces

Groups may indicate they support live audio and/or video chat by means of a LiveKit server by having the tag `livekit` in the group announcement event. An empty `supported_kinds` tag should be used if the group doesn't support text messages, only live AV chat. This setup allows for both Discord-style independent AV rooms and text rooms and Telegram-style single-roomgroup with support for both text and AV.

Upon seeing a `livekit` tag in a group, clients that want to take part in the AV space should call the token endpoint at the relay path `/.well-known/nip29/livekit/<group-id>` with a NIP-98-style `Authorization` header to get back the LiveKit JWT and the URL of the LiveKit server and proceed with the standard LiveKit flow there.

The `Authorization` event should be of kind `27235` and contain a tag `["u", "https://relay.tld/.well-known/nip29/livekit/<group-id>"]`.

Relays should enforce access control at the LiveKit JWT creation time according to the group settings.

Relays MUST set the `sub` property on the issued JWT (called "identity" in the standard LiveKit libraries) such that the initial 64 characters correspond to the lowercase hex public key of the Nostr user. The same user can be issued multiple JWTs and join the same LiveKit room multiple times, so relays are expected to appending a random identifier to those 64 characters.

In order to inform clients about relay support for AV (so they can display that option for users when creating or editing groups) relays should serve a status code `204` at `https://relay.tld/.well-known/nip29/livekit`.

Event definitions

These are the events expected to be found in NIP-29 groups.

Normal user-created events

Groups may accept any event kind, including chats, threads, long-form articles, calendar, livestreams, market announcements and so on. These should be as defined in their respective NIPs, with the addition of the `h` tag.

User-related group management events

These are events that can be sent by users to manage their situation in a group, they also require the `h` tag.

- *join request* (`kind:9021`)

Any user can send a kind `9021` event to the relay in order to request admission to the group. Relays MUST reject the request if the user has not been added to the group. The accompanying error message SHOULD explain whether the rejection is final, if the request is pending review, or if some other special handling is relevant (e.g. if payment is required). If a user is already a member, the event MUST be rejected with `duplicate:` as the error message prefix.

```
{
  "kind": 9021,
  "content": "optional reason",
  "tags": [
    ["h", "<group-id>"],
    ["code", "<optional-invite-code>"]
  ]
}
```

The optional `code` tag may be used by the relay to preauthorize acceptance, together with the `kind:9009` `create-invite` moderation event.

- *leave request* (`kind:9022`)

Any user can send one of these events to the relay in order to be automatically removed from the group. The relay will automatically issue a `kind:9001` in response removing this user.

```
{
  "kind": 9022,
  "content": "optional reason",
```

```
  "tags": [
    ["h", "<group-id>"]
  ]
}
```

Group state – or moderation

These are events expected to be sent by the relay master key or by group admins – and relays should reject them if they don't come from an authorized admin. They also require the `h` tag.

- *moderation events* (`kinds:9000-9020`) (optional)

Clients can send these events to a relay in order to accomplish a moderation action. Relays must check if the pubkey sending the event is capable of performing the given action based on its role and the relay's internal policy (see also the description of `kind:39003`).

```
{
  "kind": 90xx,
  "content": "optional reason",
  "tags": [
    ["h", "<group-id>"],
    ["previous", /*...*/]
  ]
}
```

Each moderation action uses a different kind and requires different arguments, which are given as tags. These are defined in the following table:

kind	name	tags
9000	`put-user`	`p` with pubkey hex and optional roles
9001	`remove-user`	`p` with pubkey hex
9002	`edit-metadata`	all the fields of *group-metadata*
9005	`delete-event`	`e` with event id hex
9007	`create-group`	
9008	`delete-group`	
9009	`create-invite`	arbitrary `code`
9010	`update-pin-list`	zero or more `e` with event id hex or `a` with event address

It's expected that the group state (of who is an allowed member or not, who is an admin and with which permission or not, what are the group name and picture etc) can be fully reconstructed from the canonical sequence of these events.

Group metadata events

These events contain the group id in a `d` tag instead of the `h` tag. They MUST be created by the relay master key only (as stated by the [NIP-11](NIP-11) `"self"` pubkey) and a single instance of each (or none) should exist at all times for each group. They are merely informative but should reflect the latest group state (as it was changed by moderation events over time).

- *group metadata* (`kind:39000`) (optional)

This event defines the metadata for the group – basically how clients should display it. It must be generated and signed by the relay in which is found. Relays shouldn't accept these events if they're signed by anyone else.

If the group is forked and hosted in multiple relays, there will be multiple versions of this event in each different relay and so on.

```json
{
  "kind": 39000,
  "content": "",
  "tags": [
    ["d", "<group-id>"],
    ["name", "Pizza Lovers"],
    ["picture", "https://pizza.com/pizza.png"],
    ["banner", "https://pizza.com/banner.png"],
    ["about", "a group for people who love pizza"],
    ["private"],
    ["closed"],
    ["supported_kinds", "9", "11"]
  ]
  // other fields...
}
```

- `name`, `picture`, `banner` and `about` are basic metadata for the group for display purposes.
- `private` indicates that only members can *read* group messages. Omitting this tag indicates that anyone can read group messages.
- `restricted` indicates that only members can *write* messages to the group. Omitting this tag indicates that anyone can send messages to the group.
- `hidden` indicates that relays should hide group metadata from non-members. Omitting this tag indicates that anyone can request group metadata events.
- `closed` indicates that join requests are ignored. Omitting this tag indicates that users can expect join requests to be honored.
- `livekit` indicates that a group supports LiveKit-powered media rooms (live audio/video).
- `supported_kinds` is a list of stringified kinds numbers the group supports. When not specified all kinds are assumed to be supported. It can contain no items, in which case no kinds are supported (this is the case for AV-only groups, for example).
- *group admins* (`kind:39001`) (optional)

Each admin is listed along with one or more roles. These roles SHOULD have a correspondence with the roles supported by the relay, as advertised by the `kind:39003` event.

```json
{
  "kind": 39001,
  "content": "list of admins for the pizza lovers group",
  "tags": [
    ["d", "<group-id>"],
    ["p", "<pubkey1-as-hex>", "ceo"],
    ["p", "<pubkey2-as-hex>", "secretary", "gardener"],
    // other pubkeys...
  ],
  // other fields...
}
```

- *group members* (`kind:39002`) (optional)

It's a list of pubkeys that are members of the group. Relays might choose to not to publish this information, to restrict what pubkeys can fetch it or to only display a subset of the members in it.

Clients should not assume this will always be present or that it will contain a full list of members.

```json
{
  "kind": 39002,
  "content": "list of members for the pizza lovers group",
  "tags": [
    ["d", "<group-id>"],
    ["p", "<admin1>"],
    ["p", "<member-pubkey1>"],
    ["p", "<member-pubkey2>"],
    // other pubkeys...
  ],
  // other fields...
}
```

- *group roles* (`kind:39003`) (optional)

This is an event that MAY be published by the relay informing users and clients about what are the roles supported by this relay according to its internal logic.

For example, a relay may choose to support the roles `"admin"` and `"moderator"`, in which the `"admin"` will be allowed to edit the group metadata, delete messages and remove users from the group, while the `"moderator"` can only delete messages (or the relay may choose to call these roles `"ceo"` and `"secretary"` instead, the exact role name is not relevant).

The process through which the relay decides what roles to support and how to handle moderation events internally based on them is specific to each relay and not specified here.

```json
{
  "kind": 39003,
  "content": "list of roles supported by this group",
  "tags": [
    ["d", "<group-id>"],
    ["role", "<role-name>", "<optional-description>"],
    ["role", "<role-name>", "<optional-description>"],
    // other roles...
  ],
```

```
  // other fields...
}
```

- *livekit participants* (`kind:39004`) (optional)

This is an event that MAY be published by the relay informing users and clients about who is live in an AV room.

This should be updated whenever a new user leaves or joins a group, clients are expected to be actively subscribed to it.

The event MUST contain zero or more `participant` tags with the participant's public key as lowercase hex.

```json
{
  "kind": 39004,
  "content": "",
  "tags": [
    ["d", "<group-id>"],
    ["participant", "<pubkey>"],
    // other participants...
  ],
  // other fields...
}
```

- *group pinned events* (`kind:39005`) (optional)

It's a list of events pinned in the group, in the order they should be displayed. The `kind:9010` moderation event carries the full list as `e` tags (regular events) and `a` tags (addressable events), so pinning, unpinning, reordering and clearing pins are all done by submitting a new list. Whenever one is accepted the relay regenerates this event to mirror it. The relay may choose to limit the number of pins.

Clients may display these events at the top of the group view.

```
{
  "kind": 39005,
  "content": "",
  "tags": [
    ["d", "<group-id>"],
    ["e", "<event-id1-as-hex>"],
    ["a", "<kind>:<pubkey>:<d-identifier>"],
    ["e", "<event-id2-as-hex>"],
    // other event references...
  ],
  // other fields...
}
```

Subgroups

Groups MAY be organized hierarchically. A subgroup is an ordinary NIP-29 group (same `kind:39000`, same moderation events, same `h` tag mechanics) whose metadata carries a `parent` tag pointing at its parent's `d` identifier:

```
// root group
{ "kind": 39000, "tags": [["d", "tech"], ["name", "Tech"], ["child", "nostr"]] }

// subgroup
{ "kind": 39000, "tags": [["d", "nostr"], ["name", "Nostr"], ["parent", "tech"], ["child",
"nip29"]] }

// sub-subgroup
{ "kind": 39000, "tags": [["d", "nip29"], ["name", "NIP-29"], ["parent", "nostr"]] }
```

A group without a `parent` tag is a root group. To build the tree, clients fetch `{"kinds":[39000]}` and assemble it locally: events without a `parent` tag are roots; the rest are grouped under the `d` referenced by their `parent` tag. The tree is scoped to a single relay. When aggregating across relays a client MAY see conflicting `parent` values for the same `d`; in that case the `kind:39000` from the relay currently serving the group's events is authoritative.

Relay support detection

A relay that supports subgroups SHOULD advertise it in its NIP-11 relay information document under a `nip29` object:

```json
{
  "supported_nips": [29],
  "nip29": { "subgroups": true }
}
```

Rules

- Parenting and promotion to root are triggered by a `kind:9002` (`edit-metadata`) event from an authorized admin with the desired `parent` value (or no `parent` tag to detach); the relay then updates the group's `kind:39000` accordingly. The relay also updates the parent's `kind:39000` to include the newly added child as a `child` tag (and vice-versa). If a group is switching from one parent to another parent the relay also updates the previous parent accordingly.
- Historical events under the group's `h` tag remain valid across these transitions.
- A `kind:9002` MAY carry at most one `parent` tag. The resulting `kind:39000` therefore also carries at most one `parent` tag.
- Absence of the `parent` tag makes the group a root.
- Relays MUST reject a `kind:9002` whose `parent` value would create a cycle, including self-reference.
- If a group's declared parent does not exist on the relay the `kind:9002` MUST be rejected.
- When a parent is deleted (`kind:9008`), its remaining children automatically become roots.
- Relays MUST reject any `kind:9002` carrying `["parent", "X"]` unless the event author is also an admin of `X` (per `X`'s `kind:39001`).
- A parent group's `kind:39000` MUST include `["child", "<id>"]` tags as an ordered list of its children; the position of each tag in the array is the order. By default relays can just append new children to the list as they are created, but the list can be rearranged by the parent's admin via a `kind:9002` carrying the desired `["child", ...]` tags in order, and each edit replaces the previous list. `kind:9002` metadata edits that do not include all the children as `child` tags MUST be rejected.

Lifecycle example

```
// "social" adopts "nostr" (was under "tech")
{ "kind": 9002, "tags": [["h", "nostr"], ["name", "Nostr"], ["parent", "social"]] }
// Relay emits:
{ "kind": 39000, "tags": [["d", "nostr"], ["name", "Nostr"], ["parent", "social"]] }
{ "kind": 39000, "tags": [["d", "social"], ["child", "nostr"]] }

// Admin promotes "nostr" to root: edit-metadata without a parent tag
{ "kind": 9002, "tags": [["h", "nostr"], ["name", "Nostr"]] }
// Relay emits:
{ "kind": 39000, "tags": [["d", "nostr"], ["name", "Nostr"]] }
{ "kind": 39000, "tags": [["d", "social"]] }
```

Membership

Each subgroup has its own independent membership: users join via `kind:9021` → `kind:9000` like any other group. Nothing is inherited across the parent-child link. Membership does not cascade: being a member of the parent grants no membership in any child, and each group's member list is defined solely by its own `kind:9000`/`kind:9001` events. Admin roles likewise do not inherit; each group's `kind:39001` is authoritative for its own scope, and being an admin of the parent grants no authority over any child unless the child's own `kind:39001` lists the same pubkey.

This spec defines no automatic propagation of membership between parent and child. A relay that wants such behavior MAY emit the corresponding `kind:9000`/`kind:9001` moderation events itself, but clients MUST NOT assume it: the absence of a `kind:9000` in a child means the user is not a member of that child, regardless of parent membership.

Implementation quirks

Checking your own membership in a group

The latest of either `kind:9000` or `kind:9001` events present in a group should tell a user that they are currently members of the group or if they were removed. In case none of these exist the user is assumed to not be a member of the group.

Adding yourself to a group

Anyone can send a `kind:9021` join request to a group. The relay may then generate a `kind:9000` event immediately, or defer that decision to an administrator. If a group is `closed`, join requests are not honored unless they include an invite code.

Storing your list of groups

A definition for `kind:10009` was included in [NIP-51] that allows clients to store the list of groups a user wants to remember being in.

NIP-C7

Chats

`draft` `optional`

A chat message is a `kind 9` event.

```json
{
  "kind": 9,
  "content": "GM",
  "tags": []
}
```

A reply to a `kind 9` is an additional `kind 9` which quotes the parent using a `q` tag.

```json
{
  "kind": 9,
  "content": "nostr:nevent1...\nyes",
  "tags": [
    ["q", <event-id>, <relay-url>, <pubkey>]
  ]
}
```

Clients that render a “chat view” as a stream of ordered events MUST only fetch `kind 9` events in order to prevent missing context across implementations. Other content types MAY be quoted within in a `kind 9` following [NIP 18](18.md).

NIP-7D

Forum Threads

`draft` `optional`

A thread is a `kind 11` event. Threads SHOULD include a `title`.

```json
{
  "kind": 11,
  "content": "Good morning",
  "tags": [
    ["title", "GM"]
  ]
}
```

Replies to `kind 11` MUST use [NIP-22](22.md) `kind 1111` comments. Replies should always be to the root `kind 11` to avoid arbitrarily nested reply hierarchies.

```json
{
  "kind": 1111,
  "content": "Cool beans",
```

```
  "tags": [
    ["K", "11"],
    ["E", <event-id>, <relay-url>, <pubkey>]
  ]
}
```

Moderation

Nostr does not have site-wide moderators. Anyone can post; anyone can ignore. That is the point, and it is also why clients need a shared language for “I don’t want to see this” and “this is harmful.” The NIPs in this chapter are that language. They never force a relay or a client to obey. They give honest software a way to label, list, report, and filter.

NIP-32 is labeling: a generic way to tag events or keys with a namespace and a value, so a community can mark spam, language, or topic without inventing a new kind each time. NIP-51 is lists — mutes, pins, bookmarks, relay sets, interest sets. A mute list is the closest thing Nostr has to a block button, and it lives on your key, not on a platform. NIP-56 is reporting: you tell relays and other clients that something is illegal, spam, impersonation, or abuse. NIP-36 marks sensitive content so clients can hide it behind a warning.

NIP-72 describes moderated communities with approval events. It is unrecommended; NIP-29 groups are the better place for that kind of room. NIP-13 is proof of work. It is not moderation in the human sense. It is a way to attach burned compute to an event so relays can demand a cost from spammers. Some people love it. Some relays ignore it. It belongs here because it is one of the few anti-spam tools that does not require a central bouncer.

In this chapter

- NIP-32 — Labeling
- NIP-51 — Lists
- NIP-56 — Reporting
- NIP-36 — Sensitive content
- NIP-72 — Moderated communities (unrecommended; try NIP-29)
- NIP-13 — Proof of work

NIP-32

Labeling

`draft` `optional`

This NIP defines two new indexable tags to label events and a new event kind (`kind:1985`) to attach those labels to existing events. This supports several use cases, including distributed moderation, collection management, license assignment, and content classification.

New Tags:

- `L` denotes a label namespace
- `l` denotes a label

Label Namespace Tag

An `L` tag can be any string, but publishers SHOULD ensure they are unambiguous by using a well-defined namespace (such as an ISO standard) or reverse domain name notation.

`L` tags are RECOMMENDED in order to support searching by namespace rather than by a specific tag. The special `ugc` ("user generated content") namespace MAY be used when the label content is provided by an end user.

`L` tags starting with `#` indicate that the label target should be associated with the label's value. This is a way of attaching standard nostr tags to events, pubkeys, relays, urls, etc.

Label Tag

An `l` tag's value can be any string. If using an `L` tag, `l` tags MUST include a mark matching an `L` tag value in the same event. If no `L` tag is included, a mark SHOULD still be included. If none is included, `ugc` is implied.

Label Target

The label event MUST include one or more tags representing the object or objects being labeled: `e`, `p`, `a`, `r`, or `t` tags. This allows for labeling of events, people, relays, or topics respectively. As with NIP-01, a relay hint SHOULD be included when using `e` and `p` tags.

Content

Labels should be short, meaningful strings. Longer discussions, such as for an explanation of why something was labeled the way it was, should go in the event's `content` field.

Self-Reporting

`l` and `L` tags MAY be added to other event kinds to support self-reporting. For events with a kind other than 1985, labels refer to the event itself.

Example events

A suggestion that multiple pubkeys be associated with the `permies` topic.

```
{
  "kind": 1985,
  "tags": [
    ["L", "#t"],
    ["l", "permies", "#t"],
    ["p", <pubkey1>, <relay_url>],
    ["p", <pubkey2>, <relay_url>]
  ],
  // other fields...
}
```

A report flagging violence toward a human being as defined by ontology.example.com.

```
{
  "kind": 1985,
  "tags": [
    ["L", "com.example.ontology"],
    ["l", "VI-hum", "com.example.ontology"],
    ["p", <pubkey1>, <relay_url>],
    ["p", <pubkey2>, <relay_url>]
  ],
  // other fields...
}
```

A moderation suggestion for a chat event.

```
{
  "kind": 1985,
  "tags": [
    ["L", "nip28.moderation"],
    ["l", "approve", "nip28.moderation"],
    ["e", <kind40_event_id>, <relay_url>]
  ],
  // other fields...
}
```

Assignment of a license to an event.

```
{
  "kind": 1985,
  "tags": [
    ["L", "license"],
    ["l", "MIT", "license"],
    ["e", <event_id>, <relay_url>]
  ],
  // other fields...
}
```

Publishers can self-label by adding `l` tags to their own non-1985 events. In this case, the kind 1 event's author is labeling their note as being related to Milan, Italy using ISO 3166-2.

```
{
  "kind": 1,
  "tags": [
    ["L", "ISO-3166-2"],
    ["l", "IT-MI", "ISO-3166-2"]
  ],
  "content": "It's beautiful here in Milan!",
  // other fields...
}
```

Author is labeling their note language as English using ISO-639-1.

```
{
  "kind": 1,
  "tags": [
    ["L", "ISO-639-1"],
```

```
    ["l", "en", "ISO-639-1"]
  ],
  "content": "English text",
  // other fields...
}
```

Other Notes

When using this NIP to bulk-label many targets at once, events may be requested for deletion using NIP-09 and a replacement may be published. We have opted not to use addressable/replaceable events for this due to the complexity in coming up with a standard `d` tag. In order to avoid ambiguity when querying, publishers SHOULD limit labeling events to a single namespace.

Before creating a vocabulary, explore how your use case may have already been designed and imitate that design if possible. Reverse domain name notation is encouraged to avoid namespace clashes, but for the sake of interoperability all namespaces should be considered open for public use, and not proprietary. In other words, if there is a namespace that fits your use case, use it even if it points to someone else's domain name.

Vocabularies MAY choose to fully qualify all labels within a namespace (for example, `["l", "com.example.vocabulary:my-label"]`). This may be preferred when defining more formal vocabularies that should not be confused with another namespace when querying without an `L` tag. For these vocabularies, all labels SHOULD include the namespace (rather than mixing qualified and unqualified labels).

A good heuristic for whether a use case fits this NIP is whether labels would ever be unique. For example, many events might be labeled with a particular place, topic, or pubkey, but labels with specific values like "John Doe" or "3.18743" are not labels, they are values, and should be handled in some other way.

Appendix: Known Ontologies

Below is a non-exhaustive list of ontologies currently in widespread use.

- social ontology categories

NIP-51

Lists

`draft` `optional`

This NIP defines lists of things that users can create. Lists can contain references to anything, and these references can be **public** or **private**.

Public items in a list are specified in the event `tags` array, while private items are specified in a JSON array that mimics the structure of the event `tags` array, but stringified and encrypted using the same scheme from NIP-44 (the shared key is computed using the author's public and private key) and stored in the `.content`. An earlier version of this specification used NIP-04 for encryptions. Those are now deprecated. For backward compatibility, Clients can automatically discover if the encryption is NIP-04 or NIP-44 by searching for "iv" in the ciphertext and decrypting accordingly.

When new items are added to an existing list, clients SHOULD append them to the end of the list, so they are stored in chronological order.

Types of lists

Standard lists

Standard lists use normal replaceable events, meaning users may only have a single list of each kind. They have special meaning and clients may rely on them to augment a user's profile or browsing experience.

For example, *mute list* can contain the public keys of spammers and bad actors users don't want to see in their feeds or receive annoying notifications from.

name	kind	description	expected tag items
Follow list	3	microblogging basic follow list, see NIP-02	`"p"` (pubkeys – with optional relay hint and petname)
Mute list	10000	things the user doesn't want to see in their feeds	`"p"` (pubkeys), `"t"` (hashtags), `"word"` (lowercase string), `"e"` (threads)
Pinned notes	10001	events the user intends to showcase in their profile page	`"e"` (kind:1 notes)
Read/write relays	10002	where a user publishes to and where they expect mentions	see NIP-65
Bookmarks	10003	uncategorized, "global" list of things a user wants to save	`"e"` (kind:1 notes), `"a"` (kind:30023 articles)
Communities	10004	NIP-72 communities the user belongs to	`"a"` (kind:34550 community definitions)
Public chats	10005	NIP-28 chat channels the user is in	`"e"` (kind:40 channel definitions)
Blocked relays	10006	relays clients should never connect to	`"relay"` (relay URLs)
Search relays	10007	relays clients should use when performing search queries	`"relay"` (relay URLs)
Profile badges	10008	NIP-58 badges a user has accepted and chosen to display	`"a"` (kind:30009 badge definitions) and `"e"` (kind:8 badge awards),

name	kind	description	expected tag items
			and "a" (kind:30008 badge set)
Simple groups	10009	NIP-29 groups the user is in	"group" (NIP-29 group id + relay URL + optional group name), "r" for each relay in use
Favorite follow sets	10011	user favorite follow sets	"a" (kind:30000 follow set)
Relay feeds	10012	user favorite browsable relays (and relay sets)	"relay" (relay URLs) and "a" (kind:30002 relay set)
Private relays	10013	(nip44-encrypted) NIP-37 private relays (drafts)	"relay" (relay URL)
Interests	10015	topics a user may be interested in and pointers	"t" (hashtags) and "a" (kind:30015 interest set)
Media follows	10020	multimedia (photos, short video) follow list	"p" (pubkeys – with optional relay hint and petname)
Emojis	10030	user preferred emojis and pointers to emoji sets	"emoji" (see NIP-30) and "a" (kind:30030 emoji set)
DM relays	10050	Where to receive NIP-17 direct messages	"relay" (see NIP-17)
Blossom servers	10063	Servers where to upload and look for images and other blobs	"server" (see NIP-B7)
Good wiki authors	10101	NIP-54 user recommended wiki authors	"p" (pubkeys)
Good wiki relays	10102	NIP-54 relays deemed to only host useful articles	"relay" (relay URLs)
Git authors	10017	code (people who produce NIP-34 events) follow list	"p" (pubkeys – with optional relay hint and petname)
Git repositories	10018	NIP-34 followed repositories	"a" (kind:30617 repository announcement event)
Favorite podcasts	10054	podcasts a user wants to bookmark as favorites	"p" (NIP-F4 podcast pubkeys), "url" (RSS/ XML podcast URLs)
Authored podcasts	10064	podcasts the user authors	"p" (NIP-F4 podcast pubkeys)

Sets

Sets are lists with well-defined meaning that can enhance the functionality and the UI of clients that rely on them. Unlike standard lists, users are expected to have more than one set of each kind, therefore each of them must be assigned a different `"d"` identifier.

For example, *relay sets* can be displayed in a dropdown UI to give users the option to switch to which relays they will publish an event or from which relays they will read the replies to an event; *curation sets* can be used by apps to showcase curations made by others tagged to different topics.

Aside from their main identifier, the `"d"` tag, sets can optionally have a `"title"`, an `"image"` and a `"description"` tags that can be used to enhance their UI.

name	kind	description	expected tag items
Follow sets	30000	categorized groups of users a client may choose to check out in different circumstances	`"p"` (pubkeys)
Relay sets	30002	user-defined relay groups the user can easily pick and choose from during various operations	`"relay"` (relay URLs)
Bookmark sets	30003	user-defined bookmarks categories , for when bookmarks must be in labeled separate groups	`"e"` (kind:1 notes), `"a"` (kind:30023 articles)
Curation sets	30004	groups of articles picked by users as interesting and/or belonging to the same category	`"a"` (kind:30023 articles), `"e"` (kind:1 notes)
Curation sets	30005	groups of videos picked by users as interesting and/or belonging to the same category	`"e"` (kind:21 videos)
Curation sets	30006	groups of pictures picked by users as interesting and/or belonging to the same category	`"e"` (kind:20 pictures)
Kind mute sets	30007	mute pubkeys by kinds `"d"` tag MUST be the kind string	`"p"` (pubkeys)
Badge sets	30008	NIP-58 categorized groups of badges	`"a"` (kind:30009 badge definitions) and `"e"` (kind:8 badge awards)

name	kind	description	expected tag items
Interest sets	30015	interest topics represented by a bunch of “hashtags”	`"t"` (hashtags)
Emoji sets	30030	categorized emoji groups	`"emoji"` (see NIP-30)
Release artifact sets	30063	group of artifacts of a software release	`"e"` (kind:1063 file metadata events), `"a"` (software application event)
App curation sets	30267	references to multiple software applications	`"a"` (software application event)
Calendar	31924	a set of events categorized in any way	`"a"` (calendar event event)
Starter packs	39089	a named set of profiles to be shared around with the goal of being followed together	`"p"` (pubkeys)
Media starter packs	39092	same as above, but specific to multimedia (photos, short video) clients	`"p"` (pubkeys)

Deprecated standard lists

Some clients have used these lists in the past, but they should work on transitioning to the standard formats above.

kind	“d” tag	use instead
30000	`"mute"`	kind 10000 *mute list*
30001	`"pin"`	kind 10001 *pin list*
30001	`"bookmark"`	kind 10003 *bookmarks list*
30001	`"communities"`	kind 10004 *communities list*

Examples

A *mute list* with some public items and some encrypted items

```
{
  "id": "a92a316b75e44cfdc19986c634049158d4206fcc0b7b9c7ccbcdabe28beebcd0",
  "pubkey": "854043ae8f1f97430ca8c1f1a090bdde6488bd5115c7a45307a2a212750ae4cb",
  "created_at": 1699597889,
  "kind": 10000,
  "tags": [
    ["p", "07caba282f76441955b695551c3c5c742e5b9202a3784780f8086fdcdc1da3a9"],
```

```
    ["p", "a55c15f5e41d5aebd236eca5e0142789c5385703f1a7485aa4b38d94fd18dcc4"]
  ],
  "content": "TJob1dQrf2ndsmdbeGU+05HT5GMnBSx3fx8QdDY/g3NvCa7klfzgaQCmRZuo1d3WQjHDOjzSY1+MgTK5WjewFFumCcOZni
        k4hMAjJ7Bb5Hp9xdmzmCLX9+64+MyeIQQjQAHPj8dkSsRahP7KS3MgMpjaF8nL48Bg5suZMxJayXGVp3BLtgRZx5z5nOk9xyrYk
        YXfTG/RMnoK/bDogRapOV+XToZ+IvsN0BqwKSUDx+ydKpci6htDRF2WDRkU+VQMqwM0CoLzy2H6A2cqyMMMD9SLRRzBg==?iv=S3

  "sig":
        "1173822c53261f8cffe7efbf43ba4a97a9198b3e402c2a1df130f42a8985a2d0d3430f4de350db184141e45ca844ab4e536

}
```

A *curation set* of articles and notes about yaks

```
{
  "id": "567b41fc9060c758c4216fe5f8d3df7c57daad7ae757fa4606f0c39d4dd220ef",
  "pubkey": "d6dc95542e18b8b7aec2f14610f55c335abebec76f3db9e58c254661d0593a0c",
  "created_at": 1695327657,
  "kind": 30004,
  "tags": [
    ["d", "jvdy9i4"],
    ["title", "Yaks"],
    ["image", "https://cdn.britannica.com/40/188540-050-9AC748DE/Yak-Himalayas-Nepal.jpg"],
    ["description", "The domestic yak, also known as the Tartary ox, grunting ox, or hairy
        cattle, is a species of long-haired domesticated cattle found throughout the Himalayan
        region of the Indian subcontinent, the Tibetan Plateau, Gilgit-Baltistan, Tajikistan
        and as far north as Mongolia and Siberia."],
    ["a",
        "30023:26dc95542e18b8b7aec2f14610f55c335abebec76f3db9e58c254661d0593a0c:95ODQzw3ajNoZ8SyMDOzQ"],

    ["a", "30023:54af95542e18b8b7aec2f14610f55c335abebec76f3db9e58c254661d0593a0c:1-
        MYP8dAhramH9J5gJWKx"],
    ["a",
        "30023:f8fe95542e18b8b7aec2f14610f55c335abebec76f3db9e58c254661d0593a0c:D2Tbd38bGrFvU0bIbvSMt"],

    ["e", "d78ba0d5dce22bfff9db0a9e996c9ef27e2c91051de0c4e1da340e0326b4941e"]
  ],
  "content": "",
  "sig":
        "a9a4e2192eede77e6c9d24ddfab95ba3ff7c03fbd07ad011fff245abea431fb4d3787c2d04aad001cb039cb8de91d83ce30

}
```

A *release artifact set* of an Example App

```
{
  "id": "567b41fc9060c758c4216fe5f8d3df7c57daad7ae757fa4606f0c39d4dd220ef",
  "pubkey": "d6dc95542e18b8b7aec2f14610f55c335abebec76f3db9e58c254661d0593a0c",
  "created_at": 1695327657,
  "kind": 30063,
  "content": "Release notes in markdown",
  "tags": [
    ["d", "com.example.app@0.0.1"],
    ["e", "d78ba0d5dce22bfff9db0a9e996c9ef27e2c91051de0c4e1da340e0326b4941e"], // Windows exe
    ["e", "f27e2c91051de0c4e1da0d5dce22bfff9db0a9340e0326b4941ed78bae996c9e"], // MacOS dmg
    ["e", "9d24ddfab95ba3ff7c03fbd07ad011fff245abea431fb4d3787c2d04aad02332"], // Linux AppImage
    ["e", "340e0326b340e0326b4941ed78ba340e0326b4941ed78ba340e0326b49ed78ba"], // PWA
```

```
    ["a",
        "32267:d6dc95542e18b8b7aec2f14610f55c335abebec76f3db9e58c254661d0593a0c:com.example.app"] //
        Reference to parent software application

  ],
  "sig":
        "a9a4e2192eede77e6c9d24ddfab95ba3ff7c03fbd07ad011fff245abea431fb4d3787c2d04aad001cb039cb8de91d83ce30

}
```

An *app curation set*

```
{
    "id": "d8037fa866eb5acd2159960b3ada7284172f7d687b5289cc72a96ca2b431b611",
    "pubkey": "78ce6faa72264387284e647ba6938995735ec8c7d5c5a65737e55130f026307d",
    "sig":
        "c1ce0a04521c020ae7485307cd86285530c1f778766a3fd594d662a73e7c28f307d7cd9a9ab642ae749fce62abbabb3a32f

    "kind": 30267,
    "created_at": 1729302793,
    "content": "My nostr app selection",
    "tags": [
        ["d", "nostr"],
        ["a",
        "32267:7579076d9aff0a4cfdefa7e2045f2486c7e5d8bc63bfc6b45397233e1bbfcb19:com.example.app1"],

        ["a",
        "32267:045f2486c7e5d8bc63bfc6b45397233e1bbfcb197579076d9aff0a4cfdefa7e2:net.example.app2"],

        ["a",
        "32267:264387284e647ba6938995735ec8c7d5c5a6f026307d78ce6faa725737e55130:pl.code.app3"]
    ]
}
```

Encryption process pseudocode

```
val private_items = [
  ["p", "07caba282f76441955b695551c3c5c742e5b9202a3784780f8086fdcdc1da3a9"],
  ["a", "a55c15f5e41d5aebd236eca5e0142789c5385703f1a7485aa4b38d94fd18dcc4"],
]
val base64blob = nip44.encrypt(json.encode_to_string(private_items))
event.content = base64blob
```

NIP-56

Reporting

`optional`

A report is a `kind 1984` event that signals to users and relays that some referenced content is objectionable. The definition of objectionable is obviously subjective and all agents on the network (users, apps, relays, etc.) may consume and take action on them as they see fit.

The `content` MAY contain additional information submitted by the entity reporting the content.

Tags

The report event MUST include a `p` tag referencing the pubkey of the user you are reporting.

If reporting a note, an `e` tag MUST also be included referencing the note id.

A `report type` string MUST be included as the 3rd entry to the `e`, `p` or `x` tag being reported, which consists of the following report types:

- `nudity` - depictions of nudity, porn, etc.
- `malware` - virus, trojan horse, worm, robot, spyware, adware, back door, ransomware, rootkit, kidnapper, etc.
- `profanity` - profanity, hateful speech, etc.
- `illegal` - something which may be illegal in some jurisdiction
- `spam` - spam
- `impersonation` - someone pretending to be someone else
- `other` - for reports that don't fit in the above categories

Some report tags only make sense for profile reports, such as `impersonation`.

- `x` tags SHOULD be info hash of a blob which is intended to be report. when the `x` tag is represented client MUST include an `e` tag which is the id of the event that contains the mentioned blob. also, additionally these events can contain a `server` tag to point to media servers which may contain the mentioned media.

`l` and `L` tags MAY be also be used as defined in NIP-32 to support further qualification and querying.

Example events

```
{
  "kind": 1984,
  "tags": [
    ["p", "<pubkey>", "nudity"],
    ["L", "social.nos.ontology"],
    ["l", "NS-nud", "social.nos.ontology"]
  ],
  "content": "",
  // other fields...
}
```

```
{
  "kind": 1984,
  "tags": [
    ["e", "<eventId>", "illegal"],
    ["p", "<pubkey>"]
  ],
  "content": "He's insulting the king!",
  // other fields...
}
```

```json
{
  "kind": 1984,
  "tags": [
    ["p", "<impersonator pubkey>", "impersonation"]
  ],
  "content": "Profile is impersonating nostr:<victim bech32 pubkey>",
  // other fields...
}
```

```json
{
  "kind": 1984,
  "tags": [
    ["x", "<blob hash>", "malware"],
    ["e", "<event id which contains the blob on x tag>", "malware"],
    ["server", "https://you-may-find-the-blob-here.com/path-to-url.ext"]
  ],
  "content": "This file contains malware software in it.",
  // other fields...
}
```

Client behavior

Clients can use reports from friends to make moderation decisions if they choose to. For instance, if 3+ of your friends report a profile for `nudity`, clients can have an option to automatically blur photos from said account.

Relay behavior

It is not recommended that relays perform automatic moderation using reports, as they can be easily gamed. Admins could use reports from trusted moderators to takedown illegal or explicit content if the relay does not allow such things.

NIP-36

Sensitive Content / Content Warning

`draft` `optional`

The `content-warning` tag enables users to specify if the event's content needs to be approved by readers to be shown. Clients can hide the content until the user acts on it.

`l` and `L` tags MAY be also be used as defined in NIP-32 with the `content-warning` or other namespace to support further qualification and querying.

Spec

```
tag: content-warning
options:
 - [reason]: optional
```

Example

```json
{
  "pubkey": "<pub-key>",
  "created_at": 1000000000,
  "kind": 1,
  "tags": [
    ["t", "hastag"],
    ["L", "content-warning"],
    ["l", "reason", "content-warning"],
    ["L", "social.nos.ontology"],
    ["l", "NS-nud", "social.nos.ontology"],
    ["content-warning", "<optional reason>"]
  ],
  "content": "sensitive content with #hastag\n",
  "id": "<event-id>"
}
```

> **Warning** `unrecommended`: try NIP-29 instead

NIP-72

Moderated Communities (Reddit Style)

`draft` `unrecommended` `optional`

The goal of this NIP is to enable public communities. It defines the replaceable event `kind:34550` to define the community and the current list of moderators/administrators. Users that want to post into the community, simply tag any Nostr event with the community's `a` tag. Moderators may issue an approval event `kind:4550`.

Community Definition

`Kind:34550` SHOULD include any field that helps define the community and the set of moderators. `relay` tags MAY be used to describe the preferred relay to download requests and approvals. A community definition event's `d` tag MAY double as its name, but if a `name` tag is provided, it SHOULD be displayed instead of the `d` tag.

```jsonc
{
  "created_at": <Unix timestamp in seconds>,
  "kind": 34550,
  "tags": [
    ["d", "<community-d-identifier>"],
    ["name", "<Community name>"],
    ["description", "<Community description>"],
    ["image", "<Community image url>", "<Width>x<Height>"],

    //.. other tags relevant to defining the community

    // moderators
    ["p", "<32-bytes hex of a pubkey1>", "<optional recommended relay URL>", "moderator"],
    ["p", "<32-bytes hex of a pubkey2>", "<optional recommended relay URL>", "moderator"],
```

```
    ["p", "<32-bytes hex of a pubkey3>", "<optional recommended relay URL>", "moderator"],

    // relays used by the community (w/optional marker)
    ["relay", "<relay hosting author kind 0>", "author"],
    ["relay", "<relay where to send and receive requests>", "requests"],
    ["relay", "<relay where to send and receive approvals>", "approvals"],
    ["relay", "<relay where to post requests to and fetch approvals from>"]
  ],
  // other fields...
}
```

Posting to a community

NIP-22 kind 1111 events SHOULD be used for text notes posted to a community, with the `A` tag always scoped to the community definition.

Top-level posts

For top-level posts, the uppercase and lowercase NIP-22 tags should both refer to the community definition itself.

```
{
  "kind": 1111,
  "tags": [
    ["A", "34550:<community-author-pubkey>:<community-d-identifier>", "<optional-relay-url>"],
    ["a", "34550:<community-author-pubkey>:<community-d-identifier>", "<optional-relay-url>"],
    ["P", "<community-author-pubkey>", "<optional-relay-url>"],
    ["p", "<community-author-pubkey>", "<optional-relay-url>"],
    ["K", "34550"],
    ["k", "34550"],
  ],
  "content": "Hi everyone. It's great to be here!",
  // other fields...
}
```

Nested replies

For nested replies, the uppercase tags should still refer to the community definition, while the lowercase tags should refer to the parent post or reply.

```
{
  "kind": 1111,
  "tags": [
    // community definition itself
    ["A", "34550:<community-author-pubkey>:<community-d-identifier>", "<optional-relay-url>"],
    ["P", "<community-author-pubkey>", "<optional-relay-url>"],
    ["K", "34550"],

    // parent post or reply
    ["e", "<parent-event-id>", "<optional-relay-url>"],
    ["p", "<parent-event-author-pubkey>", "<optional-relay-url>"],
    ["k", "<parent-event-kind>"] // most likely "1111"
  ],
```

```
  "content": "Agreed! Welcome everyone!",
  // other fields...
}
```

Backwards compatibility note

Previously kind 1 events were used for posts in communities, with an "a" tag pointing to the community. For backwards compatibility, clients MAY still query for kind 1 events, but SHOULD NOT use them for new posts. Instead, clients SHOULD use kind 1111 events with the `A` and `a` tags as described above.

Moderation

Anyone may issue an approval event to express their opinion that a post is appropriate for a community. Clients MAY choose which approval events to honor, but SHOULD at least use ones published by the group's defined moderators.

An approval event MUST include one or more community `a` tags, an `e` or `a` tag pointing to the post, and the `p` tag of the author of the post (for approval notifications). `a` tag prefixes can be used to disambiguate between community and replaceable event pointers (community `a` tags always begin with `34550`).

The event SHOULD also include the JSON-stringified `post request` event inside the `.content`, and a `k` tag with the original post's event kind to allow filtering of approved posts by kind.

Moderators MAY request deletion of their approval of a post at any time using [NIP-09 event deletion requests](#).

```
{
  "pubkey": "<32-bytes lowercase hex-encoded public key of the event creator>",
  "kind": 4550,
  "tags": [
    ["a", "34550:<event-author-pubkey>:<community-d-identifier>", "<optional-relay-url>"],
    ["e", "<post-id>", "<optional-relay-url>"],
    ["p", "<port-author-pubkey>", "<optional-relay-url>"],
    ["k", "<post-request-kind>"]
  ],
  "content": "<the full approved event, JSON-encoded>",
  // other fields...
}
```

It's recommended that multiple moderators approve posts to avoid deleting them from the community when a moderator is removed from the owner's list. In case the full list of moderators must be rotated, the new moderator set must sign new approvals for posts in the past or the community will restart. The owner can also periodically copy and re-sign of each moderator's approval events to make sure posts don't disappear with moderators.

Approvals of replaceable events can be created in three ways:

1. By tagging the replaceable event as an `e` tag if moderators want to approve each individual change to the replaceable event
2. By tagging the replaceable event as an `a` tag if the moderator authorizes the replaceable event author to make changes without additional approvals and

3. By tagging the replaceable event with both its `e` and `a` tag which empowers clients to display the original and updated versions of the event, with appropriate remarks in the UI.

Since relays are instructed to delete old versions of a replaceable event, the `content` of an approval using an `e` tag MUST have the specific version of the event or clients might not be able to find that version of the content anywhere.

Clients SHOULD evaluate any non-`34550:*` `a` tag as posts to be approved for all `34550:*` `a` tags.

Cross-posting

Clients MAY support cross-posting between communities by posting a NIP 18 `kind 6` or `kind 16` repost to one or more communities using `a` tags as described above. The `content` of the repost MUST be the original event, not the approval event.

NIP-13

Proof of Work

`draft` `optional` `relay`

This NIP defines a way to generate and interpret Proof of Work for nostr notes. Proof of Work (PoW) is a way to add a proof of computational work to a note. This is a bearer proof that all relays and clients can universally validate with a small amount of code. This proof can be used as a means of spam deterrence.

`difficulty` is defined to be the number of leading zero bits in the `NIP-01` id. For example, an id of `000000000e9d97a1ab09fc381030b346cdd7a142ad57e6df0b46dc9bef6c7e2d` has a difficulty of `36` with `36` leading 0 bits.

`002f...` is `0000 0000 0010 1111...` in binary, which has 10 leading zeroes. Do not forget to count leading zeroes for hex digits <= `7`.

Mining

To generate PoW for a `NIP-01` note, a `nonce` tag is used:

```
{"content": "It's just me mining my own business", "tags": [["nonce", "1", "21"]]}
```

When mining, the second entry to the nonce tag is updated, and then the id is recalculated (see NIP-01). If the id has the desired number of leading zero bits, the note has been mined. It is recommended to update the `created_at` as well during this process.

The third entry to the nonce tag `SHOULD` contain the target difficulty. This allows clients to protect against situations where bulk spammers targeting a lower difficulty get lucky and match a higher difficulty. For example, if you require 40 bits to reply to your thread and see a committed target of 30, you can safely reject it even if the note has 40 bits difficulty. Without a committed target difficulty you could not reject it. Committing to a target difficulty is something all honest miners should be ok with, and clients `MAY` reject a note matching a target difficulty if it is missing a difficulty commitment.

Example mined note

```json
{
  "id": "000006d8c378af1779d2feebc7603a125d99eca0ccf1085959b307f64e5dd358",
  "pubkey": "a48380f4cfcc1ad5378294fcac36439770f9c878dd880ffa94bb74ea54a6f243",
  "created_at": 1651794653,
  "kind": 1,
  "tags": [
    ["nonce", "776797", "20"]
  ],
  "content": "It's just me mining my own business",
  "sig":
      "284622fc0a3f4f1303455d5175f7ba962a3300d136085b9566801bc2e0699de0c7e31e44c81fb40ad9049173742e904713c

}
```

Validating

Here is some reference C code for calculating the difficulty (aka number of leading zero bits) in a nostr event id:

```c
int zero_bits(unsigned char b)
{
        int n = 0;

        if (b == 0)
                return 8;

        while (b >>= 1)
                n++;

        return 7-n;
}

/* find the number of leading zero bits in a hash */
int count_leading_zero_bits(unsigned char *hash)
{
        int bits, total, i;
        for (i = 0, total = 0; i < 32; i++) {
                bits = zero_bits(hash[i]);
                total += bits;
                if (bits != 8)
                        break;
        }
        return total;
}
```

Here is some JavaScript code for doing the same thing:

```javascript
// hex should be a hexadecimal string (with no 0x prefix)
function countLeadingZeroes(hex) {
  let count = 0;

  for (let i = 0; i < hex.length; i++) {
```

```
    const nibble = parseInt(hex[i], 16);
    if (nibble === 0) {
      count += 4;
    } else {
      count += Math.clz32(nibble) - 28;
      break;
    }
  }

  return count;
}
```

Delegated Proof of Work

Since the `NIP-01` note id does not commit to any signature, PoW can be outsourced to PoW providers, perhaps for a fee. This provides a way for clients to get their messages out to PoW-restricted relays without having to do any work themselves, which is useful for energy-constrained devices like mobile phones.

Relays

Relays are the boring part of Nostr, on purpose. A relay speaks WebSocket, stores events, and answers `REQ`s. Clients choose which relays to use. That only works if relays can describe themselves, if clients can prove who they are, and if the network can find live servers instead of a hardcoded list.

NIP-11 is the relay information document: a JSON blob at the relay URL that names the server, the NIPs it supports, and any limits or fees. NIP-42 is authentication — `AUTH` — so a relay can know which key it is talking to. Protected events (NIP-70) build on that: an event marked – should only be published by its author, which stops other people from copying it onto every relay they can find. NIP-43 is how a relay advertises membership, roles, and join or leave requests, useful when access is not wide open.

Finding and choosing relays is its own problem. NIP-65 is how a user publishes the relays they write to and read from, so other people can find their notes. NIP-66 is relay discovery and liveness monitoring: events that say "this relay exists and I probed it." Search (NIP-50) and counting (NIP-45) are optional extras a relay may offer. NIP-67 lets a relay hint whether an `EOSE` means "that's all I have" or "that's all for now." NIP-77 is negentropy, a set-reconciliation trick so a client and a relay can sync missing events without downloading everything. NIP-86 is an HTTP API for relay operators to manage bans, allow lists, and other admin work.

NIP-48 covers bridged events — notes that originated somewhere else (another protocol, another network) and landed on Nostr with enough metadata to point back at the source.

In this chapter

- NIP-45 — Counting results
- NIP-67 — EOSE completeness hint
- NIP-77 — Negentropy syncing
- NIP-86 — Relay management API
- NIP-48 — Bridged events

NIP-11

Relay Information Document

`draft` `optional` `relay`

Relays may provide server metadata to clients to inform them of capabilities, administrative contacts, and various server attributes. This is made available as a JSON document over HTTP, on the same URI as the relay's websocket.

When a relay receives an HTTP(s) request with an `Accept` header of `application/nostr+json` to a URI supporting WebSocket upgrades, they SHOULD return a document with the following structure.

```
{
  "name": <string identifying relay>,
  "description": <string with detailed information>,
  "banner": <a link to an image (e.g. in .jpg, or .png format)>,
  "icon": <a link to an icon (e.g. in .jpg, or .png format>,
  "pubkey": <administrative contact pubkey>,
  "self": <relay's own pubkey>,
  "contact": <administrative alternate contact>,
  "supported_nips": <a list of NIP numbers supported by the relay>,
  "software": <string identifying relay software URL>,
  "version": <string version identifier>,
  "terms_of_service": <a link to a text file describing the relay's term of service>
}
```

Any field may be omitted, and clients MUST ignore any additional fields they do not understand. Relays MUST accept CORS requests by sending `Access-Control-Allow-Origin`, `Access-Control-Allow-Headers`, and `Access-Control-Allow-Methods` headers.

Field Descriptions

Name

A relay may select a `name` for use in client software. This is a string, and SHOULD be less than 30 characters to avoid client truncation.

Description

Detailed plain-text information about the relay may be contained in the `description` string. It is recommended that this contain no markup, formatting or line breaks for word wrapping, and simply use double newline characters to separate paragraphs. There are no limitations on length.

Banner

To make nostr relay management more user friendly, an effort should be made by relay owners to communicate with non-dev non-technical nostr end users. A banner is a visual representation of the relay. It should aim to visually communicate the brand of the relay, complementing the text `Description`. Here is an example banner mockup as visualized in Damus iOS relay view of the Damus relay.

Icon

Icon is a compact visual representation of the relay for use in UI with limited real estate such as a nostr user's relay list view. Below is an example URL pointing to an image to be used as an icon for the relay. Recommended to be squared in shape.

```json
{
  "icon": "https://nostr.build/i/
       53866b44135a27d624e99c6165cabd76ac8f72797209700acb189fce75021f47.jpg",
  // other fields...
}
```

Pubkey

An administrative contact may be listed with a `pubkey`, in the same format as Nostr events (32-byte hex for a `secp256k1` public key). If a contact is listed, this provides clients with a recommended address to send encrypted direct messages (See NIP-17) to a system administrator. Expected uses of this address are to report abuse or illegal content, file bug reports, or request other technical assistance.

Relay operators have no obligation to respond to direct messages.

Self

A relay MAY maintain an identity independent from its administrator using the `self` field, which MUST be a 32-byte hex public key. This allows relays to respond to requests with events published either in advance or on demand by their own key.

Contact

An alternative contact may be listed under the `contact` field as well, with the same purpose as `pubkey`. Use of a Nostr public key and direct message SHOULD be preferred over this. Contents of this field SHOULD be a URI, using schemes such as `mailto` or `https` to provide users with a means of contact.

Supported NIPs

As the Nostr protocol evolves, some functionality may only be available by relays that implement a specific `NIP`. This field is an array of the integer identifiers of `NIP`s that are implemented in the relay. Examples would include `1`, for `"NIP-01"` and `9`, for `"NIP-09"`. Client-side `NIP`s SHOULD NOT be advertised, and can be ignored by clients.

Software

The relay server implementation MAY be provided in the `software` attribute. If present, this MUST be a URL to the project's homepage.

Version

The relay MAY choose to publish its software version as a string attribute. The string format is defined by the relay implementation. It is recommended this be a version number or commit identifier.

Terms of Service

The relay owner/admin MAY choose to link to a terms of service document.

Extra Fields

Server Limitations

These are limitations imposed by the relay on clients. Your client should expect that requests exceed these *practical* limitations are rejected or fail immediately.

```json
{
  "limitation": {
    "max_message_length": 16384,
    "max_subscriptions": 300,
    "max_limit": 5000,
    "max_subid_length": 100,
    "max_event_tags": 100,
    "max_content_length": 8196,
    "min_pow_difficulty": 30,
    "auth_required": true,
    "payment_required": true,
    "restricted_writes": true,
    "created_at_lower_limit": 31536000,
    "created_at_upper_limit": 3,
    "default_limit": 500
  },
  // other fields...
}
```

- `max_message_length`: the maximum number of bytes for incoming JSON that the relay will attempt to decode and act upon. When you send large subscriptions, you will be limited by this value. It also effectively limits the maximum size of any event. Value is calculated from `[` to `]` after UTF-8 serialization (so some unicode characters will cost 2-3 bytes). It is equal to the maximum size of the WebSocket message frame.

- `max_subscriptions`: total number of subscriptions that may be active on a single websocket connection to this relay. Authenticated clients with a (paid) relationship to the relay may have higher limits.

- `max_subid_length`: maximum length of subscription id as a string.

- `max_limit`: the relay server will clamp each filter's `limit` value to this number. This means the client won't be able to get more than this number of events from a single subscription filter. This clamping is typically done silently by the relay, but with this number, you can know that there are additional results if you narrow your filter's time range or other parameters.

- `max_event_tags`: in any event, this is the maximum number of elements in the `tags` list.

- `max_content_length`: maximum number of characters in the `content` field of any event. This is a count of unicode characters. After serializing into JSON it may be larger (in bytes), and is still subject to the `max_message_length`, if defined.

- `min_pow_difficulty`: new events will require at least this difficulty of PoW, based on [NIP-13](), or they will be rejected by this server.

- `auth_required`: this relay requires [NIP-42]() authentication to happen before a new connection may perform any other action. Even if set to False, authentication may be required for specific actions.

- `payment_required`: this relay requires payment before a new connection may perform any action.

- `restricted_writes`: this relay requires some kind of condition to be fulfilled to accept events (not necessarily, but including `payment_required` and `min_pow_difficulty`). This should only be set to `true` when users are expected to know the relay policy before trying to write to it – like belonging to a special pubkey-based whitelist or writing only events of a specific niche kind or content. Normal anti-spam heuristics, for example, do not qualify.

- `created_at_lower_limit`: 'created_at' lower limit

- `created_at_upper_limit`: 'created_at' upper limit

- `default_limit`: The maximum returned events if you send a filter without a `limit`.

Pay-to-Relay

Relays that require payments may want to expose their fee schedules.

```
{
  "payments_url": "https://my-relay/payments",
  "fees": {
    "admission": [{ "amount": 1000000, "unit": "msats" }],
    "subscription": [{ "amount": 5000000, "unit": "msats", "period": 2592000 }],
    "publication": [{ "kinds": [4], "amount": 100, "unit": "msats" }],
  },
  // other fields...
}
```

Examples

```
~> curl -H "Accept: application/nostr+json" https://nostr.wine | jq
{
  "contact": "wino@nostr.wine",
  "description": "A paid nostr relay for wine enthusiasts and everyone else.",
  "fees": {
```

```
    "admission": [
      {
        "amount": 18888000,
        "unit": "msats"
      }
    ]
  },
  "icon": "https://image.nostr.build/
      30acdce4a81926f386622a07343228ae99fa68d012d54c538c0b2129dffe400c.png",
  "limitation": {
    "auth_required": false,
    "created_at_lower_limit": 94608000,
    "created_at_upper_limit": 300,
    "max_event_tags": 4000,
    "max_limit": 1000,
    "max_message_length": 524288,
    "max_subid_length": 71,
    "max_subscriptions": 50,
    "min_pow_difficulty": 0,
    "payment_required": true,
    "restricted_writes": true
  },
  "name": "nostr.wine",
  "payments_url": "https://nostr.wine/invoices",
  "pubkey": "4918eb332a41b71ba9a74b1dc64276cfff592e55107b93baae38af3520e55975",
  "software": "https://nostr.wine",
  "supported_nips": [ 1, 2, 4, 9, 11, 40, 42, 50, 70, 77 ],
  "terms_of_service": "https://nostr.wine/terms",
  "version": "0.3.3"
}
```


```
~> curl -H "Accept: application/nostr+json" https://nostr.land | jq
```


```
{
  "description": "[   NFDB] nostr.land family of relays (fi-01 [tiger])",
  "name": "[   NFDB] nostr.land",
  "pubkey": "52b4a076bcbbbdc3a1aefa3735816cf74993b1b8db202b01c883c58be7fad8bd",
  "software": "NFDB",
  "icon": "https://i.nostr.build/b3thno790aodH8lE.jpg",
  "supported_nips": [ 1, 2, 4, 8, 9, 10, 11, 13, 14, 15, 16, 17, 18, 19, 21, 22, 23, 24, 25,
      27, 28, 30, 31, 32, 34, 35, 36, 37, 38, 39, 40, 42, 44, 46, 47, 48, 51, 52, 53, 54,
      55, 56, 57, 58, 59, 60, 61, 64, 65, 68, 69, 71, 72, 73, 75, 78, 84, 88, 89, 90, 92,
      99 ],
  "version": "1.0.0",
  "limitation": {
    "payment_required": true,
    "max_message_length": 65535,
    "max_event_tags": 2000,
    "max_subscriptions": 200,
    "auth_required": false
  },
  "payments_url": "https://nostr.land",
  "fees": {
    "subscription": [
      {
        "amount": 4000000,
        "unit": "msats",
```

```
      "period": 2592000
    }
  ]
},
"terms_of_service": "https://nostr.land/terms"
}
```

NIP-42

Authentication of clients to relays

`draft` `optional` `relay`

This NIP defines a way for clients to authenticate to relays by signing an ephemeral event.

Motivation

A relay may want to require clients to authenticate to access restricted resources. For example,

- A relay may request payment or other forms of whitelisting to publish events – this can naïvely be achieved by limiting publication to events signed by the whitelisted key, but with this NIP they may choose to accept any events as long as they are published from an authenticated user;
- A relay may limit access to `kind: 4` DMs to only the parties involved in the chat exchange, and for that it may require authentication before clients can query for that kind.
- A relay may limit subscriptions of any kind to paying users or users whitelisted through any other means, and require authentication.

Definitions

New client-relay protocol messages

This NIP defines a new message, `AUTH`, which relays CAN send when they support authentication and clients can send to relays when they want to authenticate. When sent by relays the message has the following form:

```
["AUTH", <challenge-string>]
```

And, when sent by clients, the following form:

```
["AUTH", <signed-event-json>]
```

Clients MAY provide signed events from multiple pubkeys in a sequence of `AUTH` messages. Relays MUST treat all pubkeys as authenticated accordingly.

`AUTH` messages sent by clients MUST be answered with an `OK` message, like any `EVENT` message.

Canonical authentication event

The signed event is an ephemeral event not meant to be published or queried, it must be of `kind: 22242` and it should have at least two tags, one for the relay URL and one for the challenge string as received from the relay.

Relays MUST exclude `kind: 22242` events from being broadcasted to any client. `created_at` should be the current time. Example:

```json
{
  "kind": 22242,
  "tags": [
    ["relay", "wss://relay.example.com/"],
    ["challenge", "challengestringhere"]
  ],
  // other fields...
}
```

`OK` and `CLOSED` machine-readable prefixes

This NIP defines two new prefixes that can be used in `OK` (in response to event writes by clients) and `CLOSED` (in response to rejected subscriptions by clients):

- `"auth-required: "` - for when a client has not performed `AUTH` and the relay requires that to fulfill the query or write the event.
- `"restricted: "` - for when a client has already performed `AUTH` but the key used to perform it is still not allowed by the relay or is exceeding its authorization.

Protocol flow

At any moment the relay may send an `AUTH` message to the client containing a challenge. The challenge is valid for the duration of the connection or until another challenge is sent by the relay. The client MAY decide to send its `AUTH` event at any point and the authenticated session is valid afterwards for the duration of the connection.

`auth-required` in response to a `REQ` message

Given that a relay is likely to require clients to perform authentication only for certain jobs, like answering a `REQ` or accepting an `EVENT` write, these are some expected common flows:

```
relay: ["AUTH", "<challenge>"]
client: ["REQ", "sub_1", {"kinds": [4]}]
relay: ["CLOSED", "sub_1", "auth-required: we can't serve DMs to unauthenticated users"]
client: ["AUTH", {"id": "abcdef...", ...}]
client: ["AUTH", {"id": "abcde2...", ...}]
relay: ["OK", "abcdef...", true, ""]
relay: ["OK", "abcde2...", true, ""]
client: ["REQ", "sub_1", {"kinds": [4]}]
relay: ["EVENT", "sub_1", {...}]
relay: ["EVENT", "sub_1", {...}]
relay: ["EVENT", "sub_1", {...}]
relay: ["EVENT", "sub_1", {...}]
...
```

In this case, the `AUTH` message from the relay could be sent right as the client connects or it can be sent immediately before the `CLOSED` is sent. The only requirement is that *the client must have a stored challenge associated with that relay* so it can act upon that in response to the `auth-required` `CLOSED` message.

`auth` hint in `EOSE`

Relays MAY use the `"auth"` hint in an `EOSE` message (as defined in NIP-67) to notify the client that more results may be available if they authenticate. In this case, the relay MUST send the `AUTH` challenge before the `EOSE` message.

`auth-required` in response to an `EVENT` message

The same flow is valid for when a client wants to write an `EVENT` to the relay, except now the relay sends back an `OK` message instead of a `CLOSED` message:

```
relay: ["AUTH", "<challenge>"]
client: ["EVENT", {"id": "012345...", ...}]
relay: ["OK", "012345...", false, "auth-required: we only accept events from registered users"]
client: ["AUTH", {"id": "abcdef...", ...}]
relay: ["OK", "abcdef...", true, ""]
client: ["EVENT", {"id": "012345...", ...}]
relay: ["OK", "012345...", true, ""]
```

Signed Event Verification

To verify `AUTH` messages, relays must ensure:

- that the `kind` is `22242`;
- that the event `created_at` is close (e.g. within ~10 minutes) of the current time;
- that the `"challenge"` tag matches the challenge sent before;
- that the `"relay"` tag matches the relay URL:
    - URL normalization techniques can be applied. For most cases just checking if the domain name is correct should be enough.

NIP-70

Protected Events

`draft` `optional` `relay`

When the `"-"` tag is present, that means the event is “protected”.

A protected event is an event that can only be published to relays by its author. This is achieved by relays ensuring that the author is authenticated before publishing their own events or by just rejecting events with `["-"]` outright.

The default behavior of a relay MUST be to reject any event that contains `["-"]`.

Relays that want to accept such events MUST first require that the client perform the NIP-42 `AUTH` flow and then check if the authenticated client has the same pubkey as the event being published and only accept the event in that case.

Reposts of protected events MUST NOT embed the reposted event. If a repost of a protected event embeds the event anyway, relays SHOULD summarily reject it.

The tag

The tag is a simple tag with a single item: `["-"]`. It may be added to any event.

Example flow

- User `79be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798` connects to relay `wss://example.com`:

```
/* client: */
["EVENT",
        {"id":"cb8feca582979d91fe90455867b34dbf4d65e4b86e86b3c68c368ca9f9eef6f2","pubkey":"79be667ef9dcbbac5
        [["-"]],"content":"hello members of the secret
        group","sig":"fa163f5cfb75d77d9b6269011872ee22b34fb48d23251e9879bb1e4ccbdd8aaaf4b6dc5f5084a65ef42c52

/* relay: */
["AUTH", "<challenge>"]
["OK", "cb8feca582979d91fe90455867b34dbf4d65e4b86e86b3c68c368ca9f9eef6f2", false, "auth-
        required: this event may only be published by its author"]
/* client: */
["AUTH", {}]
["EVENT",
        {"id":"cb8feca582979d91fe90455867b34dbf4d65e4b86e86b3c68c368ca9f9eef6f2","pubkey":"79be667ef9dcbbac5
        [["-"]],"content":"hello members of the secret
        group","sig":"fa163f5cfb75d77d9b6269011872ee22b34fb48d23251e9879bb1e4ccbdd8aaaf4b6dc5f5084a65ef42c52

["OK", "cb8feca582979d91fe90455867b34dbf4d65e4b86e86b3c68c368ca9f9eef6f2", true, ""]
```

Why

There are multiple circumstances in which it would be beneficial to prevent the unlimited spreading of an event through all relays imaginable and restrict some to only a certain demographic or to a semi-closed community relay. Even when the information is public it may make sense to keep it compartimentalized across different relays.

It's also possible to create closed access feeds with this when the publisher has some relationship with the relay and trusts the relay to not release their published events to anyone.

Even though it's ultimately impossible to restrict the spread of information on the internet (for example, one of the members of the closed group may want to take an event intended to be restricted and republish it to other relays), most relays would be happy to not facilitate the acts of these so-called "pirates", in respect to the original decision of the author and therefore gladly reject these republish acts if given the means to.

This NIP gives these authors and relays the means to clearly signal when a given event is not intended to be republished by third parties.

NIP-43

Relay Access Metadata and Requests

`draft` `optional` `relay`

This NIP defines a way for relays to advertise membership lists, and for clients to request admission to relays on behalf of users.

Role Definitions

Relays MAY publish a `kind 33534` event which defines a role which may be assigned to members. This event MUST be signed by the pubkey specified in the `self` field of the relay's NIP 11 document.

The following tags are required:

- A NIP 70 – tag
- The event's `d` tag defines the role id.

The following tags are optional:

- `label` - a label for the role.
- `description` - a description of the role.
- `color` - a `hue` value from `0` to `360` for the role.
- `order` - a `order` integer for the role (for client display only).

```
{
  "kind": 33534,
  "pubkey": "<nip11.self>",
  "tags": [
    ["-"],
    ["d", "28b7e50f"],
    ["label", "king"],
    ["description", "ruler of the relay"],
    ["color", "37"],
    ["order", "1"],
  ],
  // ...other fields
}
```

Relays MAY treat members with a given role differently in terms of relay access policies, but no mechanism for introspection is defined at this time.

Membership Lists

Relays MAY publish a `kind 13534` event which indicates pubkeys that have access to a given relay. This event MUST be signed by the pubkey specified in the `self` field of the relay's NIP 11 document.

The following tags are required:

- A NIP 70 - tag
- A `member` tag containing a hex pubkey should be included for each member. Assigned roles may be included as subsequent arguments.

This list should not be considered exhaustive or authoritative. To determine membership, both a `kind 13534` event by the relay, and a `kind 10010` event by the member should be consulted.

Example:

```
{
  "kind": 13534,
  "pubkey": "<nip11.self>",
  "tags": [
    ["-"],
    ["member", "c308e1f882c1f1dff2a43d4294239ddeec04e575f2d1aad1fa21ea7684e61fb5"],
    ["member", "ee1d336e13779e4d4c527b988429d96de16088f958cbf6c074676ac9cfd9c958", "28b7e50f"]
  ],
  // ...other fields
}
```

Add User

Relays MAY publish a `kind 8000` event when a member is added to the relay. This event MUST be signed by the pubkey specified in the `self` field of the relay's NIP 11 document.

The following tags are required:

- A NIP 70 - tag
- A `p` tag indicating the member's hex pubkey

Example:

```
{
  "kind": 8000,
  "pubkey": "<nip11.self>",
  "tags": [
    ["-"],
    ["p", "c308e1f882c1f1dff2a43d4294239ddeec04e575f2d1aad1fa21ea7684e61fb5"]
  ],
  // ...other fields
}
```

Remove User

Relays MAY publish a `kind 8001` event when a member is removed from the relay. This event MUST be signed by the pubkey specified in the `self` field of the relay's NIP 11 document.

The following tags are required:

- A NIP 70 - tag

- A `p` tag indicating the member's hex pubkey

Example:

```
{
  "kind": 8001,
  "pubkey": "<nip11.self>",
  "tags": [
    ["-"],
    ["p", "c308e1f882c1f1dff2a43d4294239ddeec04e575f2d1aad1fa21ea7684e61fb5"]
  ],
  // ...other fields
}
```

Join Request

A user MAY send a `kind 28934` to a relay in order to request admission. It MUST have a `claim` tag containing an invite code. The event's `created_at` MUST be now, plus or minus a few minutes.

```
{
  "kind": 28934,
  "pubkey": "<user pubkey>",
  "tags": [
    ["-"],
    ["claim", "<invite code>"]
  ],
  // ...other fields
}
```

Upon receiving a claim, a relay MUST notify the client as to what the status of the claim is using an `OK` message. Failed claims SHOULD use the same standard `"restricted: "` prefix specified by NIP 42.

Relays SHOULD update their `kind 13534` member list and MAY publish a `kind 8000` "add member" event.

Some examples:

```
["OK", <event-id>, false, "restricted: that invite code is expired."]
["OK", <event-id>, false, "restricted: that is an invalid invite code."]
["OK", <event-id>, true, "duplicate: you are already a member of this relay."]
["OK", <event-id>, true, "info: welcome to wss://relay.bunk.skunk!"]
```

Invite Request

Users may request a claim string from a relay by making a request for `kind 28935` events. This event MUST be signed by the pubkey specified in the `self` field of the relay's [NIP 11]() document.

```
{
  "kind": 28935,
  "pubkey": "<nip11.self>",
  "tags": [
    ["-"],
    ["claim", "<invite code>"],
  ],
```

```
  // ...other fields
}
```

Note that these events are in the `ephemeral` range, which means relays must explicitly opt-in to this behavior by generating claims on the fly when requested. This allows relays to improve security by issuing a different claim for each request, only issuing claims to certain users, or expiring claims.

Leave Request

A user MAY send a `kind 28936` to a relay in order to request that their access be revoked. The event's `created_at` MUST be now, plus or minus a few minutes. This event MUST include a NIP 70 – tag.

```
{
  "kind": 28936,
  "tags": [["-"]],
  // ...other fields
}
```

Relays SHOULD update their `kind 13534` member list and MAY publish a `kind 8001` "remove member" event.

Implementation

Clients MUST only request `kind 28935` events from and send `kind 28934` events to relays which include this NIP in the `supported_nips` section of its NIP 11 relay information document.

NIP-65

Relay List Metadata

`draft` `optional`

Defines a replaceable event using `kind:10002` to advertise relays where the user generally **writes** to and relays where the user generally **reads** mentions.

The event MUST include a list of `r` tags with relay URLs as value and an optional `read` or `write` marker. If the marker is omitted, the relay is both **read** and **write**.

```
{
  "kind": 10002,
  "tags": [
    ["r", "wss://alicerelay.example.com"],
    ["r", "wss://brando-relay.com"],
    ["r", "wss://expensive-relay.example2.com", "write"],
    ["r", "wss://nostr-relay.example.com", "read"]
  ],
  "content": "",
  // other fields...
}
```

When downloading events **from** a user, clients SHOULD use the **write** relays of that user.

When downloading events **about** a user, where the user was tagged (mentioned), clients SHOULD use the user's **read** relays.

When publishing an event, clients SHOULD:

- Send the event to the **write** relays of the author
- Send the event to all **read** relays of each tagged user
- Send the author's `kind:10002` event to all relays the event was published to

Size

Clients SHOULD guide users to keep `kind:10002` lists small (2-4 relays of each category).

Discoverability

Clients SHOULD spread an author's `kind:10002` event to as many relays as viable, paying attention to relays that, at any moment, serve naturally as well-known public indexers for these relay lists (where most other clients and users are connecting to in order to publish and fetch those).

NIP-66

Relay Liveness Monitoring

`draft` `optional` `relay`

This NIP defines events for relay discovery and the announcement of relay monitors.

Relay Discovery Events

`30166` relay discovery events document relay characteristics inferred either from a relay's [NIP 11](11.md) document, or via probing.

Information corresponding to field in a relay's NIP 11 document MAY contradict actual values if monitors find that a different policy is implemented than is advertised.

`content` MAY include the stringified JSON of the relay's NIP-11 informational document.

The only required tag is the `d` tag, which MUST be set to the relay's normalized URL. For relays not accessible via URL, a hex-encoded pubkey MAY be used instead.

Other tags include:

- `rtt-open` - The relay's open round-trip time in milliseconds.
- `rtt-read` - The relay's read round-trip time in milliseconds.
- `rtt-write` - The relay's write round-trip time in milliseconds.
- `n` - The relay's network type. SHOULD be one of `clearnet`, `tor`, `i2p`, `loki`
- `T` - The relay type. Enumerated relay type formatted as `PascalCase`, e.g. `PrivateInbox`
- `N` - NIPs supported by the relay

- `R` - Keys corresponding to requirements per NIP 11's `limitations` array, including `auth`, `writes`, `pow`, and `payment`. False values should be specified using a `!` prefix, for example `!auth`.
- `t` - A topic associated with this relay
- `k` - Accepted and unaccepted kinds (false values prepended by `!`)
- `g` - A NIP-52 geohash

Tags with more than one value should be repeated, rather than putting all values in a single tag, for example `[["t", "cats"], ["t", "dogs"]]`, rather than `[["t", "cats", "dogs"]]`.

Example:

```
{
  "id": "<eventid>",
  "pubkey": "<monitor's pubkey>",
  "created_at": "<created_at  [some recent date ...]>",
  "signature": "<signature>",
  "content": "<optional nip 11 document>",
  "kind": 30166,
  "tags": [
    ["d","wss://some.relay/"],
    ["n", "clearnet"],
    ["N", "40"],
    ["N", "33"],
    ["R", "!payment"],
    ["R", "auth"],
    ["g", "ww8p1r4t8"],
    ["l", "en", "ISO-639-1"],
    ["t", "nsfw" ],
    ["rtt-open", "234" ]
  ]
}
```

Relay Monitor Announcements

Kind `10166` relay monitor announcements advertise the author's intent to publish `30166` events. This event is optional and is intended for monitors who intend to provide monitoring services at a regular and predictable frequency.

Tags include:

- `frequency` - The frequency in seconds at which the monitor publishes events.
- `timeout` (optional) - The timeout values for various checks conducted by a monitor. Index `1` is the monitor's timeout in milliseconds. Index `2` describes what test the timeout is used for. If no index `2` is provided, it is inferred that the timeout provided applies to all tests.
- `c` - a lowercase string describing the checks conducted by a monitor. Examples include `open`, `read`, `write`, `auth`, `nip11`, `dns`, and `geo`.
- `g` - NIP-52 geohash tag

Monitors SHOULD also publish a `kind 0` profile and a `kind 10002` relay selections event.

Example:

```json
{
  "id": "<eventid>",
  "pubkey": "<monitor's pubkey>",
  "created_at": "<created_at  [some recent date ...]>",
  "signature": "<signature>",
  "content": "",
  "tags": [
    [ "timeout", "open", "5000"  ],
    [ "timeout", "read", "3000"  ],
    [ "timeout", "write", "3000" ],
    [ "timeout", "nip11", "3000" ],
    [ "frequency", "3600" ],
    [ "c", "ws" ],
    [ "c", "nip11" ],
    [ "c", "ssl" ],
    [ "c", "dns" ],
    [ "c", "geo" ]
    [ "g", "ww8p1r4t8" ]
  ]
}
```

Risk Mitigation

- Clients MUST NOT require `30166` events to function. Absence of monitoring data MUST NOT prevent relay connections.
- A monitor may publish erroneous `30166` events, either by misconfiguration or malicious intent.
- Clients SHOULD NOT trust a single source. Defenses include: web-of-trust filtering, querying multiple monitors, and discarding filter results if they would remove an unreasonable proportion of relays.

NIP-50

Search Capability

`draft` `optional` `relay`

Abstract

Many Nostr use cases require some form of general search feature, in addition to structured queries by tags or ids. Specifics of the search algorithms will differ between event kinds, this NIP only describes a general extensible framework for performing such queries.

`search` filter field

A new `search` field is introduced for `REQ` messages from clients:

```
{
  // other fields on filter object...
  "search": <string>
}
```

`search` field is a string describing a query in a human-readable form, i.e. "best nostr apps". Relays SHOULD interpret the query to the best of their ability and return events that match it. Relays SHOULD perform matching against `content` event field, and MAY perform matching against other fields if that makes sense in the context of a specific kind.

Results SHOULD be returned in descending order by quality of search result (as defined by the implementation), not by the usual `.created_at`. The `limit` filter SHOULD be applied after sorting by matching score. A query string may contain `key:value` pairs (two words separated by colon), these are extensions, relays SHOULD ignore extensions they don't support.

Clients may specify several search filters, i.e. `["REQ", "", { "search": "orange" }, { "kinds": [1, 2], "search": "purple" }]`. Clients may include `kinds`, `ids` and other filter field to restrict the search results to particular event kinds.

Clients SHOULD use the supported_nips field to learn if a relay supports `search` filter. Clients MAY send `search` filter queries to any relay, if they are prepared to filter out extraneous responses from relays that do not support this NIP.

Clients SHOULD query several relays supporting this NIP to compensate for potentially different implementation details between relays.

Clients MAY verify that events returned by a relay match the specified query in a way that suits the client's use case, and MAY stop querying relays that have low precision.

Relays SHOULD exclude spam from search results by default if they support some form of spam filtering.

Extensions

Relay MAY support these extensions: - `include:spam` - turn off spam filtering, if it was enabled by default - `domain:<domain>` - include only events from users whose valid nip05 domain matches the domain - `language:<two letter ISO 639-1 language code>` - include only events of a specified language - `sentiment:<negative/neutral/positive>` - include only events of a specific sentiment - `nsfw:<true/false>` - include or exclude nsfw events (default: true)

NIP-45

Event Counts

`draft` `optional` `relay`

Relays may support the verb `COUNT`, which provides a mechanism for obtaining event counts.

Motivation

Some queries a client may want to execute against connected relays are prohibitively expensive, for example, in order to retrieve follower counts for a given pubkey, a client must query all kind-3 events referring to a given pubkey only to count them. The result may be cached, either by a client or by a separate indexing server as an alternative, but both options erode the decentralization of the network by creating a second-layer protocol on top of Nostr.

Filters and return values

This NIP defines the verb `COUNT`, which accepts a query id and filters as specified in [NIP 01](01.md) for the verb `REQ`. Multiple filters are OR'd together and aggregated into a single count result.

```
["COUNT", <query_id>, <filters JSON>...]
```

Counts are returned using a `COUNT` response in the form `{"count": <integer>}`. Relays may use probabilistic counts to reduce compute requirements. In case a relay uses probabilistic counts, it MAY indicate it in the response with `approximate` key i.e. `{"count": <integer>, "approximate": <true|false>}`.

```
["COUNT", <query_id>, {"count": <integer>}]
```

Whenever the relay decides to refuse to fulfill the `COUNT` request, it MUST return a `CLOSED` message.

HyperLogLog

Relays may return an HyperLogLog value together with the count, hex-encoded.

```
["COUNT", <subscription_id>, {"count": <integer>, "hll": "<hex>"}]
```

This is so it enables merging results from multiple relays and yielding a reasonable estimate of reaction counts, comment counts and follower counts, while saving many millions of bytes of bandwidth for everybody.

Algorithm

This section describes the steps a relay should take in order to return HLL values to clients.

1. Upon receiving a filter, if it is eligible (see below) for HyperLogLog, compute the deterministic `offset` for that filter (see below);
2. Initialize 256 registers to `0` for the HLL value;
3. For all the events that are to be counted according to the filter, do this:
    1. Read the byte at position `offset` of the event `pubkey`, its value will be the register index `ri`;
    2. Count the number of leading zero bits starting at position `offset+1` of the event `pubkey` and add `1`;
    3. Compare that with the value stored at register `ri`, if the new number is bigger, store it.

That is all that has to be done on the relay side, and therefore the only part needed for interoperability.

On the client side, these HLL values received from different relays can be merged (by simply going through all the registers in HLL values from each relay and picking the highest value for each register, regardless of the relay).

And finally the absolute count can be estimated by running some methods I don't dare to describe here in English, it's better to check some implementation source code (also, there can be different ways of performing the estimation, with different quirks applied on top of the raw registers).

`offset` computation

The `offset` (used in the HLL computation above) is derived deterministically from the filter sent by the client to the relay. The formula for obtaining the `offset` value is as follows:

1. Take the first tag attribute in the filter (with the `#` prefix);
2. From that, take its first item (it will be a string);
3. Obtain a 32-byte hex string from it: - if the string is an event id or pubkey hex, use it as it is; - if the string is an address (`<kind>:<pubkey>:<d-tag>`), use the `<pubkey>` part; - if the string is anything else, hash it with a `sha256()` and take the result as a hex string;
4. From the 64-character hex string obtained before, take the character at position `32`;
5. Read that character as a base-16 number;
6. Add 8 to it: the result is the `offset`.

For cases not covered above (filters without a tag attribute, for example), behavior isn't yet defined. This NIP may be modified later to specify those, but for now there isn't a use case that justifies using HLL in those circumstances.

Rationale

The value of `offset` must be deterministic because that's the only way to allow relays to cache the HLL values so they don't have to count thousands of events from the database on every query. It also allows relays to precompute HLL values for any given target `<id>` or `<pubkey>` without having to store the events themselves directly, which can be handy in case of reactions, for example.

Common filters

Some relays may decide to cache or precompute HLL values for some common canonical queries, and also to refrain from counting events that do not match these specs. These are such queries (this NIP can be modified later if more common useful queries are discovered and start being used):

- **reaction count**: `{"#e": ["<id>"], "kinds": [7]}`
- **repost count**: `{"#e": ["<id>"], "kinds": [6]}`
- **quote count**: `{"#q": ["<id>"], "kinds": [1, 1111]}`
- **reply count**: `{"#e": ["<id>"], "kinds": [1]}`
- **comment count**: `{"#E": ["<id>"], "kinds": [1111]}`
- **follower count**: `{"#p": ["<pubkey>"], "kinds": [3]}`

Notice that these queries only include 1 tag attribute with always a single item in it, which means that implementors don't have to check the order in which these attributes show up in the filter.

Attack vectors

One could mine a pubkey with a certain number of zero bits in the exact place where the HLL algorithm described above would look for them in order to artificially make its reaction or follow "count more" than others. For this to work a different pubkey would have to be created for each different target (event id, followed profile etc). This approach is not very different than creating tons of new pubkeys and using them all to send likes or follow someone in order to inflate their number of followers. The solution is the same in both cases: clients should not fetch these reaction counts from open relays that accept everything, they should base their counts on relays that perform some form of filtering that makes it more likely that only real humans are able to publish there and not bots or artificially-generated pubkeys.

`hll` encoding

The value `hll` value must be the concatenation of the 256 registers, each being a uint8 value (i.e. a byte). Therefore `hll` will be a 512-character hex string.

Client-side usage

This algorithm also allows clients to combine HLL responses received from relays with HLL counts computed locally from raw events. It's recommended that clients keep track of HLL values locally and add to these on each message received from relays. For example:

- a client wants to keep track of the number of reactions an event Z has received over time;
- the client has decided it will read reactions from relays A, B and C (the NIP-65 "read" relays of Z's author);
- of these, only B and C support HLL responses, so the client fetches both and merges them locally;
- then the client fetches all reaction events from A then manually applies each event to the HLL from the previous step, using the same algorithm described above;
- finally, the client reads the estimate count from the HLL and displays that to the user;
- optionally the client may store that HLL value (together with some "last-read-date" for relay A) and repeat the process again later:
    - this time it only needs to fetch the new reactions from A and add those to the HLL
    - and redownload the HLL values from B and C and just reapply them to the local value.

This procedure allows the client to download much less data.

Examples

Count notes and reactions

```
["COUNT", <query_id>, {"kinds": [1, 7], "authors": [<pubkey>]}]
["COUNT", <query_id>, {"count": 5}]
```

Count notes approximately

```
["COUNT", <query_id>, {"kinds": [1]}]
["COUNT", <query_id>, {"count": 93412452, "approximate": true}]
```

Followers count with HyperLogLog

```
["COUNT", <subscription_id>, {"kinds": [3], "#p": [<pubkey>]}]
["COUNT", <subscription_id>, {"count": 16578, "hll":
"0607070505060806050508060707070706090d080b0605090607070b07090606060b0705070709050807080805080407060906080707080507070805060509040a0b06060704060405070706080607050907070b08060808080b080607090a06060805060604070908050607060805050d05060906090809080807050e07050705070609070606060707080800b080707070808070606060908070506060406040907 0a0808050a0506050b0810060a0908070709080b0a07050806060508060607080606080707050806080c0a0707070a080808050608080f070506070706070a0908090c0807080808060905080606060909060 60d07050708080405070708"}]
```

Reaction counts with HyperLogLog

```
["COUNT", <subscription_id>, {"kinds": [7], "#e": [<id>]}]
["COUNT", <subscription_id>, {"count": 2044, "hll":
"01ef070505060806050508060707070706090d080b0605090607070b07090606060b070507070905080708080508040
706090608070708050707080506050904 0a0b0606070406040507070608060705090707 0b08060808080b080607090a0
606080506060407090805060706080505 0d0506090609080908080705 0e070507050706090706060607070808 0b08070
707080807060606090807050606040604 09070a0808050a0506050b0810060a0908070709080b0a07050806060508060
607080606080707050806080c0a0707070a080808050608080f070506070706070a0908090c080708080806090508060
606090906060d070507080804050707 08"}]
```

Relay refuses to count

```
["COUNT", <query_id>, {"kinds": [1059], "#p": [<pubkey>]}]
["CLOSED", <query_id>, "auth-required: cannot count other people's DMs"]
```

NIP-67

EOSE Completeness Hint

`draft` `optional`

This NIP extends the `EOSE` message defined in [NIP-01](01.md) with an optional third element that signals the client that the relay has sent every stored event matching the subscription's filters.

Motivation

`EOSE` as defined in NIP-01 signals the boundary between stored and real-time events. It does not tell the client whether there are more stored events the relay chose not to send. Relays commonly enforce an internal cap (e.g. 500 events per filter) that is independent of the client's `limit`. Clients today guess completeness by comparing the number of events received to the `limit` they requested:

- If fewer than `limit` events arrive, assume completion.
- Otherwise, issue another `REQ` with `until` set to the oldest received event's `created_at`, and repeat.

This heuristic has two well-known problems:

1. When the relay's internal cap is lower than the client's `limit` (e.g. cap 300, client asks for 500), the client receives 300 events and incorrectly concludes the result is complete, silently missing data.
2. When the total number of matching events happens to equal the relay's cap, the client cannot tell and must issue an additional `REQ` just to observe zero results. A relay that caps at 300 events will always be queried at least twice for any subscription that exhausts the cap.

A single extra field on `EOSE` removes this ambiguity.

Specification

A relay that implements this NIP MAY append a third element to the `EOSE` message: an array of hint strings.

```
["EOSE", <subscription_id>, [<hint>, ...]]
```

Currently three hint values are defined:

- `"finish"`: the relay has sent every stored event matching the subscription's filters. The client SHOULD NOT paginate this subscription further.
- `"more"`: the relay holds more matching stored events than it has sent. The client SHOULD paginate to retrieve them. A relay MAY send this hint, but is not required to, since detecting whether more events exist is not cheap on every storage backend.
- `"auth"`: the relay may have more stored events matching the subscription's filters if the user performs `AUTH` (NIP-42). Relays MUST ensure an `AUTH` message (containing a challenge) is sent *before* the `EOSE` containing an `"auth"` hint.

The array MAY carry multiple hints simultaneously, and MAY be empty. Clients MUST ignore unknown hint values without error.

Their presence is definitive; their absence is not. When a relay omits the third element, or sends neither `"finish"` nor `"more"`, the client SHOULD paginate using `until = oldest received event's created_at` as usual.

Clients that do not implement this NIP will ignore the extra element, as JSON arrays with trailing elements are accepted by conforming parsers and existing implementations index `EOSE` by position. Relays that do not implement this NIP continue to send `EOSE` with two elements and clients fall back to the legacy heuristic described above.

The hint refers only to stored events. It has no effect on the real-time delivery of new events, which continues under the regular NIP-01 rules until the client sends `CLOSE` or the relay sends `CLOSED`.

Future revisions of this NIP may define additional hint values. Implementations SHOULD accept unknown hint values without error and treat them as if absent.

Semantics around `created_at` ties

When a relay does not send `"finish"`, multiple events may share the oldest `created_at` value in the response. Clients paginating with `until = oldest_created_at` risk missing events that share that timestamp. Relays SHOULD, when possible, advance the internal cursor so that all events with the boundary `created_at` are included in one response, emitting `"finish"` only when there are no older events remaining.

Relay advertisement

Relays implementing this NIP SHOULD include `67` in the `supported_nips` field of their NIP-11 document.

Examples

Relay has sent every matching stored event:

```
["REQ", "sub2", {"ids": ["abcd..."]}]
["EVENT", "sub2", {...}]
["EOSE", "sub2", ["finish"]]
```

Relay has more matching events:

```
["REQ", "sub2b", {"authors": ["abcd..."], "kinds": [1]}]
["EVENT", "sub2b", {...}]
... (300 events total)
["EOSE", "sub2b", ["more"]]
```

Relay has more matching events that require authentication:

```
["REQ", "sub2b", {"authors": ["abcd..."], "kinds": [1]}]
["EVENT", "sub4", {...}]
["AUTH", "challenge..."]
["EOSE", "sub4", ["auth", "finish"]]
```

NIP-77

Negentropy Syncing

`draft` `optional` `relay`

This document describes a protocol extension for syncing events. It works for both client-relay and relay-relay scenarios. If both sides of the sync have events in common, then this protocol will use less bandwidth than transferring the full set of events (or even just their IDs).

It is a Nostr-friendly wrapper around the [Negentropy] protocol, which uses a technique called [Range-Based Set Reconciliation].

Since Negentropy is a binary protocol, this wrapper hex-encodes its messages. The specification for Negentropy Protocol V1 is attached as an appendix to this NIP below.

High-Level Protocol Description

We're going to call the two sides engaged in the sync the client and the relay (even though the initiator could be another relay instead of a client).

- 1. Client (initiator) chooses a filter, and retrieves the set of events that it has locally that match this filter (or uses a cache), and constructs an initial message.
- (2) Client sends a `NEG-OPEN` message to the relay, which includes the filter and the initial message.
- (3) Relay selects the set of events that it has locally that match the filter (or uses a cache).
- (4) Relay constructs a response and returns it to the client in a `NEG-MSG` message.
- (5) Client parses the message to learn about IDs it has (and relay needs) and IDs it needs (and relay has).
  - If client wishes to continue, then it constructs a new message and sends it to the relay in a `NEG-MSG` message. Goto step 4.
  - If client wishes to stop, then it sends a `NEG-CLOSE` message or disconnects the websocket.

The above protocol only results in the client learning about IDs it has/needs, and does not actually transfer events. Given these IDs, the client can upload events it has with `EVENT`, and/or download events it needs with `REQ`. This can be performed over the same websocket connection in parallel with subsequent `NEG-MSG` messages.

If a client is only interested in determining the number of unique events (ie, reaction counts), it may choose to not download/upload at all.

Nostr Messages

Initial message (client to relay):

```
[
    "NEG-OPEN",
    <subscription ID string>,
    <filter>,
    <initialMessage, hex-encoded>
]
```

- The subscription ID is used by each side to identify which query a message refers to. It only needs to be long enough to distinguish it from any other concurrent subscriptions on this websocket connection (an integer that increments once per `NEG-OPEN` is fine). Subscription IDs are in a separate namespace from `REQ` subscription IDs. If a `NEG-OPEN` is issued for a currently open subscription ID, the existing subscription is first closed.
- The filter is as described in NIP-01.
- `initialMessage` is the initial Negentropy binary message, hex-encoded. See appendix.

Error message (relay to client):

If a request cannot be serviced by the relay, an error is returned to the client:

```
[
    "NEG-ERR",
    <subscription ID string>,
    <reason code string>
]
```

Error reasons are the same format as in NIP-01. They should begin with a machine-readable single-word prefix, followed by a `:` and then a human-readable message with more information.

The current suggested error reasons are

- `blocked`
    - Relays can optionally reject queries that would require them to process too many records, or records that are too old
    - The maximum number of records that can be processed can optionally be returned as the 4th element in the response
    - Example: `blocked: this query is too big`
- `closed`
    - Because the `NEG-OPEN` queries may be stateful, relays may choose to time-out inactive queries to recover memory resources
    - Example: `closed: you took too long to respond!`

After a `NEG-ERR` is issued, the subscription is considered to be closed.

Subsequent messages (bidirectional):

Relay and client alternate sending each other `NEG-MSG`s:

```
[
    "NEG-MSG",
    <subscription ID string>,
    <message, hex-encoded>
]
```

- `message` is a Negentropy binary message, hex-encoded. Both message directions use the same format. See appendix.

Close message (client to relay):

When finished, the client should tell the relay it can release its resources with a `NEG-CLOSE`:

```
[
    "NEG-CLOSE",
    <subscription ID string>
]
```

Appendix: Negentropy Protocol V1

Preparation

There are two protocol participants: Client and server. The client creates an initial message and transmits it to the server, which replies with its own message in response. The client continues querying the server until it is satisfied, and then terminates the protocol. Messages in either direction have the same format.

Each participant has a collection of records. A records consists of a 64-bit numeric timestamp and a 256-bit ID. Each participant starts by sorting their items according to timestamp, ascending. If two timestamps are equal then items are sorted lexically by ID, ascending by first differing byte. Items may not use the max uint64 value (`2**64 - 1`) as a timestamp since this is reserved as a special "infinity" value.

The goal of the protocol is for the client to learn the set of IDs that it has and the server does not, and the set of items that the server has and it does not.

`Varint`

Varints (variable-sized unsigned integers) are represented as base-128 digits, most significant digit first, with as few digits as possible. Bit eight (the high bit) is set on each byte except the last.

```
Varint := <Digit+128>* <Digit>
```

`Id`

IDs are represented as byte-strings of length `32`:

```
Id := Byte{32}
```

`Message`

A reconciliation message is a protocol version byte followed by an ordered list of ranges:

```
Message := <protocolVersion (Byte)> <Range>*
```

The current protocol version is 1, represented by the byte `0x61`. Protocol version 2 will be `0x62`, and so forth. If a server receives a message with a protocol version that it cannot handle, it should reply with a single byte containing the highest protocol version it supports, allowing the client to downgrade and retry its message.

Each Range corresponds to a contiguous section of the timestamp/ID space. The first Range starts at timestamp 0 and an ID of 0 bytes. Ranges are always adjacent (no gaps). If the last Range doesn't end at the special infinity value, an implicit `Skip` to infinity Range is appended. This means that the list of Ranges always covers the full timestamp/ID space.

`Range`

A Range consists of an upper bound, a mode, and a payload:

```
Range := <upperBound (Bound)> <mode (Varint)> <payload (Skip | Fingerprint | IdList)>
```

The contents of the payload is determined by mode:

- If `mode = 0`, then payload is `Skip`, meaning the sender does not wish to process this Range further. This payload is empty:

  ```
  Skip :=
  ```

- If `mode = 1`, then payload is a `Fingerprint`, which is a [digest](#) of all the IDs the sender has within the Range:

  ```
  Fingerprint := Byte{16}
  ```

- If `mode = 2`, the payload is `IdList`, a variable-length list of all IDs the sender has within the Range:

  ```
  IdList := <length (Varint)> <ids (Id)>*
  ```

`Bound`

Each Range is specified by an *inclusive* lower bound and an *exclusive* upper bound. As defined above, each Range only includes an upper bound: the lower bound of a Range is the upper bound of the previous Range, or 0 timestamp/0 ID for the first Range.

A Bound consists of an encoded timestamp and a variable-length disambiguating prefix of an ID (in case multiple items have the same timestamp):

```
Bound := <encodedTimestamp (Varint)> <length (Varint)> <idPrefix (Byte)>*
```

- The timestamp is encoded specially. The infinity timestamp is encoded as `0`. All other values are encoded as `1 + offset`, where offset is the difference between this timestamp and the previously encoded timestamp. The initial offset starts at `0` and resets at the beginning of each message.

Offsets are always non-negative since the upper bound's timestamp is greater than or equal to the lower bound's timestamp, ranges in a message are always encoded in ascending order, and ranges never overlap.

- The size of `idPrefix` is encoded in `length`, and can be between `0` and `32` bytes, inclusive. This allows implementations to use the shortest possible prefix to separate the first record of this Range from the last record of the previous Range. If these records' timestamps differ, then the length should be 0, otherwise it should be the byte-length of their common ID-prefix plus 1.

  If the `idPrefix` length is less than `32` then the omitted trailing bytes are implicitly 0 bytes.

Fingerprint Algorithm

The fingerprint of a Range is computed with the following algorithm:

- Compute the addition mod 2^{256} of the element IDs (interpreted as 32-byte little-endian unsigned integers)
- Concatenate with the number of elements in the Range, encoded as a [Varint]
- Hash with SHA-256
- Take the first 16 bytes

NIP-86

Relay Management API

`draft` `optional`

Relays may provide an API for performing management tasks. This is made available as a JSON-RPC-like request-response protocol over HTTP, on the same URI as the relay's websocket.

When a relay receives an HTTP(s) request with a `Content-Type` header of `application/nostr+json+rpc` to a URI supporting WebSocket upgrades, it should parse the request as a JSON document with the following fields:

```json
{
  "method": "<method-name>",
  "params": ["<array>", "<of>", "<parameters>"]
}
```

Then it should return a response in the format

```json
{
  "result": {"<arbitrary>": "<value>"},
  "error": "<optional error message, if the call has errored>"
}
```

This is the list of **methods** that may be supported:

- `supportedmethods`:
  - params: `[]`

    - result: `["<method-name>", "<method-name>", ...]` (an array with the names of all the other supported methods)
- `banpubkey`:
    - params: `["<32-byte-hex-public-key>", "<optional-reason>"]`
    - result: `true` (a boolean always set to `true`)
- `unbanpubkey`:
    - params: `["<32-byte-hex-public-key>", "<optional-reason>"]`
    - result: `true` (a boolean always set to `true`)
- `listbannedpubkeys`:
    - params: `[]`
    - result: `[{"pubkey": "<32-byte-hex>", "reason": "<optional-reason>"}, ...]`, an array of objects
- `allowpubkey`:
    - params: `["<32-byte-hex-public-key>", "<optional-reason>"]`
    - result: `true` (a boolean always set to `true`)
- `unallowpubkey`:
    - params: `["<32-byte-hex-public-key>", "<optional-reason>"]`
    - result: `true` (a boolean always set to `true`)
- `listallowedpubkeys`:
    - params: `[]`
    - result: `[{"pubkey": "<32-byte-hex>", "reason": "<optional-reason>"}, ...]`, an array of objects
- `createrole`:
    - params: `[id, label, description, color, order]`
    - result: `true` (boolean true)
- `editrole`:
    - params: `[id, label, description, color, order]`
    - result: `true` (boolean true)
- `deleterole`:
    - params: `[id]`
    - result: `true` (boolean true)
- `assignrole`:
    - params: `["<32-byte-hex-public-key>", "<role-id>"]`
    - result: `true` (a boolean always set to `true`)
- `unassignrole`:
    - params: `["<32-byte-hex-public-key>", "<role-id>"]`
    - result: `true` (a boolean always set to `true`)
- `listeventsneedingmoderation`:
    - params: `[]`
    - result: `[{"id": "<32-byte-hex>", "reason": "<optional-reason>"}]`, an array of objects
- `allowevent`:
    - params: `["<32-byte-hex-event-id>", "<optional-reason>"]`
    - result: `true` (a boolean always set to `true`)
- `banevent`:
    - params: `["<32-byte-hex-event-id>", "<optional-reason>"]`
    - result: `true` (a boolean always set to `true`)

- `listbannedevents`:
  - params: `[]`
  - result: `[{"id": "<32-byte hex>", "reason": "<optional-reason>"}, ...]`, an array of objects
- `changerelayname`:
  - params: `["<new-name>"]`
  - result: `true` (a boolean always set to `true`)
- `changerelaydescription`:
  - params: `["<new-description>"]`
  - result: `true` (a boolean always set to `true`)
- `changerelayicon`:
  - params: `["<new-icon-url>"]`
  - result: `true` (a boolean always set to `true`)
- `allowkind`:
  - params: `[<kind-number>]`
  - result: `true` (a boolean always set to `true`)
- `disallowkind`:
  - params: `[<kind-number>]`
  - result: `true` (a boolean always set to `true`)
- `listallowedkinds`:
  - params: `[]`
  - result: `[<kind-number>, ...]`, an array of numbers
- `blockip`:
  - params: `["<ip-address>", "<optional-reason>"]`
  - result: `true` (a boolean always set to `true`)
- `unblockip`:
  - params: `["<ip-address>"]`
  - result: `true` (a boolean always set to `true`)
- `listblockedips`:
  - params: `[]`
  - result: `[{"ip": "<ip-address>", "reason": "<optional-reason>"}, ...]`, an array of objects

Authorization

The request must contain an `Authorization` header with a valid NIP-98 event, except the `payload` tag is required. The `u` tag is the relay URL.

If `Authorization` is not provided or is invalid, the endpoint should return a 401 response.

NIP-48

Bridged events

`draft` `optional`

Nostr events bridged from other protocols such as ActivityPub can link back to the source object by including a `"proxy"` tag, in the form:

```
["proxy", <id>, <protocol>]
```

Where:

- `<id>` is the ID of the source object. The ID format varies depending on the protocol. The ID must be universally unique, regardless of the protocol.
- `<protocol>` is the name of the protocol, e.g. `"activitypub"`.

Clients may use this information to reconcile duplicated content bridged from other protocols, or to display a link to the source object.

Proxy tags may be added to any event kind, and doing so indicates that the event did not originate on the Nostr protocol, and instead originated elsewhere on the web.

Supported protocols

This list may be extended in the future.

Protocol	ID format	Example
`activitypub`	URL	`https://gleasonator.com/objects/9f524868-c1a0-4ee7-ad51-aaa23d68b526`
`atproto`	AT URI	`at://did:plc:zhbjlbmir5dganqhueg7y4i3/app.bsky.feed.post/3jt5hlibeol2i`
`rss`	URL with guid fragment	`https://soapbox.pub/rss/feed.xml#https%3A%2F%2Fsoapbox.pub%2Fblog%2Fmostr-fediverse-nostr-bridge`
`web`	URL	`https://twitter.com/jack/status/20`

Examples

ActivityPub object:

```
{
  "kind": 1,
  "content": "I'm vegan btw",
  "tags": [
    [
      "proxy",
      "https://gleasonator.com/objects/8f6fac53-4f66-4c6e-ac7d-92e5e78c3e79",
      "activitypub"
    ]
  ],
  "pubkey": "79c2cae114ea28a981e7559b4fe7854a473521a8d22a66bbab9fa248eb820ff6",
  "created_at": 1691091365,
  "id": "55920b758b9c7b17854b6e3d44e6a02a83d1cb49e1227e75a30426dea94d4cb2",
  "sig":
      "a72f12c08f18e85d98fb92ae89e2fe63e48b8864c5e10fbdd5335f3c9f936397a6b0a7350efe251f8168b1601d7012d4a6c

}
```

See also

- [FEP-fffd: Proxy Objects]
- [Mostr bridge]

Clients

Clients are where Nostr becomes an app: a feed, a profile, a compose box. The protocol does not care what the UI looks like. It does care that two clients can point at the same event, the same profile, and the same relay without inventing their own URL schemes.

NIP-21 is the `nostr:` URI scheme. NIP-19 is the bech32 encoding behind it — `npub`, `nsec`, `note`, `nevent`, `nprofile`, `naddr`, and friends. Together they are how you copy a key or a note out of one app and open it in another. If you have ever pasted an `npub1…` into a client, you have used these two NIPs.

NIP-03 attaches OpenTimestamps proofs to events, so you can later show that a note existed at a certain time. It is unrecommended: the current scheme is vulnerable to a known attack and needs an update. It is still in the book because attestations exist in the wild and the idea is sound.

NIP-BE describes talking to Nostr over Bluetooth Low Energy. It is unrecommended: it was implemented once, it is unclear whether it works well, and it needs review. Local, radio-based transport is an interesting idea. This particular spec is not something to build on yet.

In this chapter

- NIP-21 — `nostr:` URI scheme
- NIP-19 — bech32-encoded entities
- NIP-03 — OpenTimestamps attestations (unrecommended; needs an update)
- NIP-BE — BLE communications (unrecommended; experimental and unproven)

NIP-21

`nostr:` URI scheme

`draft` `optional`

This NIP standardizes the usage of a common URI scheme for maximum interoperability and openness in the network.

The scheme is `nostr:`.

The identifiers that come after are expected to be the same as those defined in [NIP-19] (except `nsec`).

Examples

- `nostr:npub1sn0wdenkukak0d9dfczzeacvhkrgz92ak56egt7vdgzn8pv2wfqqhrjdv9`
- `nostr:nprofile1qqsrhuxx8l9ex335q7he0f09aej04zpazpl0ne2cgukyawd24mayt8gpp4mhxue69uhhytnc9e3k7mgpz4mhxue`

- `nostr:nevent1qqstna2yrezu5wghjvswqqculvvwxsrcvu7uc0f78gan4xqhvz49d9spr3mhxue69uhkummnw3ez6un9d3shjtn4d`

Linking HTML pages to Nostr entities

`<link>` tags with `rel="alternate"` can be used to associate webpages to Nostr events, in cases where the same content is served via the two mediums (for example, a web server that exposes Markdown articles both as HTML pages and as `kind:30023` events served under itself as a relay or through some other relay). For example:

```
<head>
  <link rel="alternate"
href="nostr:naddr1qqyrzwrxvc6ngvfkqyghwumn8ghj7enfv96x5ctx9e3k7mgzyqalp33lewf5vdq847t6te0wvnags0
gs0mu72kz8938tn24wlfze6qcyqqq823cph95ag" />
</head>
```

Likewise, `<link>` tags with `rel="me"` or `rel="author"` can be used to assign authorship of webpages to Nostr profiles. For example:

```
<head>
  <link rel="me"
href="nostr:nprofile1qyxhwumn8ghj7mn0wvhxcmmvqyd8wumn8ghj7un9d3shjtnhv4ehgetjde38gcewvdhk6qpq80c
vv07tjdrrgpa0j7j7tmnyl2yr6yr7l8j4s3evf6u64th6gkwswpnfsn" />
</head>
```

NIP-19

bech32-encoded entities

`draft` `optional`

This NIP standardizes bech32-formatted strings that can be used to display keys, ids and other information in clients. These formats are not meant to be used anywhere in the core protocol, they are only meant for displaying to users, copy-pasting, sharing, rendering QR codes and inputting data.

It is recommended that ids and keys are stored in either hex or binary format, since these formats are closer to what must actually be used the core protocol.

Bare keys and ids

To prevent confusion and mixing between private keys, public keys and event ids, which are all 32 byte strings. bech32-(not-m) encoding with different prefixes can be used for each of these entities.

These are the possible bech32 prefixes:

- `npub`: public keys
- `nsec`: private keys
- `note`: note ids

Example: the hex public key `3bf0c63fcb93463407af97a5e5ee64fa883d107ef9e558472c4eb9aaaefa459d` translates to `npub180cvv07tjdrrgpa0j7j7tmnyl2yr6yr7l8j4s3evf6u64th6gkwsyjh6w6`.

The bech32 encodings of keys and ids are not meant to be used inside the standard NIP-01 event formats or inside the filters, they're meant for human-friendlier display and input only. Clients should still accept keys in both hex and npub format for now, and convert internally.

Shareable identifiers with extra metadata

When sharing a profile or an event, an app may decide to include relay information and other metadata such that other apps can locate and display these entities more easily.

For these events, the contents are a binary-encoded list of `TLV` (type-length-value), with `T` and `L` being 1 byte each (`uint8`, i.e. a number in the range of 0-255), and `V` being a sequence of bytes of the size indicated by `L`.

These are the possible bech32 prefixes with `TLV`:

- `nprofile`: a nostr profile
- `nevent`: a nostr event
- `naddr`: a nostr *addressable event* coordinate
- `nrelay`: a nostr relay (deprecated)

These possible standardized `TLV` types are indicated here:

- `0`: `special`
  - depends on the bech32 prefix:
    - for `nprofile` it will be the 32 bytes of the profile public key
    - for `nevent` it will be the 32 bytes of the event id
    - for `naddr`, it is the identifier (the `"d"` tag) of the event being referenced. For normal replaceable events use an empty string.
- `1`: `relay`
  - for `nprofile`, `nevent` and `naddr`, *optionally*, a relay in which the entity (profile or event) is more likely to be found, encoded as ascii
  - this may be included multiple times
- `2`: `author`
  - for `naddr`, the 32 bytes of the pubkey of the event
  - for `nevent`, *optionally*, the 32 bytes of the pubkey of the event
- `3`: `kind`
  - for `naddr`, the 32-bit unsigned integer of the kind, big-endian
  - for `nevent`, *optionally*, the 32-bit unsigned integer of the kind, big-endian

Examples

- `npub10elfcs4fr0l0r8af98jlmgdh9c8tcxjvz9qkw038js35mp4dma8qzvjptg` should decode into the public key hex `7e7e9c42a91bfef19fa929e5fda1b72e0ebc1a4c1141673e2794234d86addf4e` and vice-versa
- `nsec1vl029mgpspedva04g90vltkh6fvh240zqtv9k0t9af8935ke9laqsnlfe5` should decode into the private key hex `67dea2ed018072d675f5415ecfaed7d2597555e202d85b3d65ea4e58d2d92ffa` and vice-versa

- `nprofile1qqsrhuxx8l9ex335q7he0f09aej04zpazpl0ne2cgukyawd24mayt8gpp4mhxue69uhhytnc9e3k7mgpz4mhxue69uhkg` should decode into a profile with the following TLV items:
  - pubkey: `3bf0c63fcb93463407af97a5e5ee64fa883d107ef9e558472c4eb9aaaefa459d`
  - relay: `wss://r.x.com`
  - relay: `wss://djbas.sadkb.com`

Notes

- `npub` keys MUST NOT be used in NIP-01 events or in NIP-05 JSON responses, only the hex format is supported there.
- When decoding a bech32-formatted string, TLVs that are not recognized or supported should be ignored, rather than causing an error.
- Bech32-formatted strings SHOULD be limited in size to 5000 characters.

  Warning `unrecommended`: vulnerable to one specific attack, needs update

NIP-03

OpenTimestamps Attestations for Events

`draft` `unrecommended` `optional`

This NIP defines an event with `kind:1040` that can contain an [OpenTimestamps](https://opentimestamps.org/) proof for any other event:

```
{
  "kind": 1040
  "tags": [
    ["e", <target-event-id>, <relay-url>],
    ["k", "<target-event-kind>"]
  ],
  "content": <base64-encoded OTS file data>
}
```

- The OpenTimestamps proof MUST prove the referenced `e` event id as its digest.
- The `content` MUST be the full content of an `.ots` file containing at least one Bitcoin attestation. This file SHOULD contain a **single** Bitcoin attestation (as not more than one valid attestation is necessary and less bytes is better than more) and no reference to “pending” attestations since they are useless in this context.

Example OpenTimestamps proof verification flow

Using [`nak`](https://github.com/fiatjaf/nak), [`jq`](https://jqlang.github.io/jq/) and [`ots`](https://github.com/fiatjaf/ots):

```
~> nak req -i e71c6ea722987debdb60f81f9ea4f604b5ac0664120dd64fb9d23abc4ec7c323 wss://nostr-pub.wellorder.net | jq -r .content | ots verify
> using an esplora server at https://blockstream.info/api
- sequence ending on block 810391 is valid
timestamp validated at block [810391]
```

> **Warning** `unrecommended`: only implemented once and unclear whether it works, requires review

NIP-BE

Nostr BLE Communications Protocol

`draft` `unrecommended` `optional`

This NIP specifies how Nostr apps can use BLE to communicate and synchronize with each other. The BLE protocol follows a client-server pattern, so this NIP emulates the WS structure in a similar way, but with some adaptations to its limitations.

Device advertisement

A device advertises itself with: - Service UUID: `0000180f-0000-1000-8000-00805f9b34fb` - Data: Device UUID in ByteArray format

GATT service

The device exposes a Nordic UART Service with the following characteristics:

1. Write Characteristic
    - UUID: `87654321-0000-1000-8000-00805f9b34fb`
    - Properties: Write
2. Read Characteristic
    - UUID: `12345678-0000-1000-8000-00805f9b34fb`
    - Properties: Notify, Read

Role assignment

When one device initially finds another advertising the service, it will read the service's data to get the device UUID and compare it with its own advertised device UUID. For this communication, the device with the highest ID will take the role of GATT Server (Relay), the other will be considered the GATT Client (Client) and will proceed to establish the connection.

For devices whose purpose will require a single role, its device UUID will always be:

- GATT Server: `FFFFFFFF-FFFF-FFFF-FFFF-FFFFFFFFFFFF`
- GATT Client: `00000000-0000-0000-0000-000000000000`

Messages

All messages will follow [NIP-01](01.md) message structure. For a given message, a compression stream (DEFLATE) is applied to the message to generate a byte array. Depending on the BLE version, the byte array can be too large for a single message (20-23 bytes in BLE 4.2, 256 bytes in BLE > 4.2). In that case, this byte array is split into any number of batches following the structure:

```
[batch index (first 2 bytes)][batch n][is last batch (last byte)]
```

After reception of all batches, the other device can then join them and decompress. To ensure reliability, only 1 message will be read/written at a time. MTU can be negotiated in advance. The maximum size for a message is 64KB; bigger messages will be rejected.

Examples

This example implements a function to split and compress a byte array into chunks, as well as another function to join and decompress them in order to obtain the initial result:

```
fun splitInChunks(message: ByteArray): Array<ByteArray> {
   val chunkSize = 500 // define the chunk size
   var byteArray = compressByteArray(message)
   val numChunks = (byteArray.size + chunkSize - 1) / chunkSize // calculate the number of
      chunks
   var chunkIndex = 0
   val chunks = Array(numChunks) { ByteArray(0) }

   for (i in 0 until numChunks) {
         val start = i * chunkSize
         val end = minOf((i + 1) * chunkSize, byteArray.size)
         val chunk = byteArray.copyOfRange(start, end)

         // add chunk index to the first 2 bytes and last chunk flag to the last byte
         val chunkWithIndex = ByteArray(chunk.size + 2)
         chunkWithIndex[0] = chunkIndex.toByte() // chunk index
         chunk.copyInto(chunkWithIndex, 1)
         chunkWithIndex[chunkWithIndex.size - 1] = numChunks.toByte()

         // store the chunk in the array
         chunks[i] = chunkWithIndex

         chunkIndex++
   }

   return chunks
}

fun joinChunks(chunks: Array<ByteArray>): ByteArray {
   val sortedChunks = chunks.sortedBy { it[0] }
   var reassembledByteArray = ByteArray(0)
   for (chunk in sortedChunks) {
         val chunkData = chunk.copyOfRange(1, chunk.size - 1)
         reassembledByteArray = reassembledByteArray.copyOf(reassembledByteArray.size +
        chunkData.size)
         chunkData.copyInto(reassembledByteArray, reassembledByteArray.size - chunkData.size)
   }

   return decompressByteArray(reassembledByteArray)
}
```

Workflows

Client to relay

- Any message the client wants to send to a relay will be a write message.
- Any message the client receives from a relay will be a read message.

Relay to client

The relay should notify the client about any new event matching subscription's filters by using the Notify action of the Read Characteristic. After that, the client can proceed to read messages from the relay.

Device synchronization

Given the nature of BLE, it is expected that the direct connection between two devices might be extremely intermittent, with gaps of hours or even days. That's why it's crucial to define a synchronization process by following NIP-77 but with an adaptation to the limitations of the technology.

After two devices have successfully connected and established the Client-Server roles, the devices will use half-duplex communication to intermittently send and receive messages.

Half-duplex synchronization

Right after the 2 devices connect, the Client starts the workflow by sending the first message.

1. Client - Writes "NEG-OPEN" message.
2. Server - Sends `write-success`.
3. Client - Sends `read-message`.
4. Server - Responds with "NEG-MSG" message.
5. Client -
    1. If the Client has messages missing on the Server, it writes one `EVENT`.
    2. If the Client doesn't have any messages missing on the Server, it writes `EOSE`. In this case, subsequent messages to the Server will be empty while the Server claims to have more notes for the Client.
6. Server - Sends `write-success`.
7. Client - Sends `read-message`.
8. Server -
    1. If the Server has messages missing on the Client, it responds with one `EVENT`.
    2. If the Client doesn't have any messages missing on the Server, it responds with `EOSE`. In this case, subsequent responses to the Client will be empty.
9. If the Client detects that the devices are not synchronized yet, jump to step 5.
10. After the two devices detect that there are no more missing events on both ends, the workflow will pause at this point.

Half-duplex event spread

While two devices are connected and synchronized, it might happen that one of them receives a new message from another connected peer. Devices MUST keep track of which notes have been sent to its peers while they are connected. If the newly received event is detected as missing in one of the connected and synchronized peers:

1. If the peer is a Server:
   1. Client - It writes the `EVENT`.
   2. Server - Sends `write-success`.
2. If the peer is a Client:
   1. Server - It will send an empty notification to the Client.
   2. Client - Sends `read-message`.
   3. Server - Responds with the `EVENT`.

Payments

Nostr does not have a built-in currency. What it has is events that can request, prove, or discover a payment. Lightning showed up first. Ecash showed up later. The NIPs in this chapter are how clients wire those systems onto keys and notes without turning the protocol into a ledger.

NIP-57 is Lightning zaps: a standardized way to send sats to a note or a profile and publish a receipt. Zap goals (NIP-75) put a fundraising target on an event so a client can show progress. NIP-47, Nostr Wallet Connect, lets an app talk to a wallet it does not hold — you approve payments from a signer you already trust, instead of pasting invoices around.

Cashu is the other half. NIP-60 is a Cashu wallet stored as Nostr events, so your unspent tokens can move with your key. NIP-61 is nutzaps: zaps paid in ecash instead of Lightning. NIP-87 is how Cashu mints and Fedimints announce themselves so wallets can find them. NIP-A3 defines `payto:` payment targets, a way to publish "pay me here" without tying the protocol to one rail.

NIP-69 is peer-to-peer order events: offers to buy or sell bitcoin (and similar) that a coordinator or a counterparty can take. It is closer to a marketplace than to a zap, but it lives here because the point of the event is a trade, not a listing on a wall.

In this chapter

- NIP-57 — Lightning zaps
- NIP-75 — Zap goals
- NIP-47 — Nostr Wallet Connect
- NIP-60 — Cashu wallet
- NIP-61 — Nutzaps
- NIP-87 — Cashu and Fedimint discoverability
- NIP-A3 — `payto:` payment targets
- NIP-69 — Peer-to-peer order events

NIP-57

Lightning Zaps

`draft` `optional`

This NIP defines two new event types for recording lightning payments between users. `9734` is a `zap request`, representing a payer's request to a recipient's lightning wallet for an invoice. `9735` is a `zap receipt`, representing the confirmation by the recipient's lightning wallet that the invoice issued in response to a `zap request` has been paid.

Having lightning receipts on nostr allows clients to display lightning payments from entities on the network. These can be used for fun or for spam deterrence.

Protocol flow

1. Client calculates a recipient's lnurl pay request url from the `zap` tag on the event being zapped (see Appendix G), or by decoding their lud16 field on their profile according to the [lnurl specifications](). The client MUST send a GET request to this url and parse the response. If `allowsNostr` exists and it is `true`, and if `nostrPubkey` exists and is a valid BIP 340 public key in hex, the client should associate this information with the user, along with the response's `callback`, `minSendable`, and `maxSendable` values.
2. Clients may choose to display a lightning zap button on each post or on a user's profile. If the user's lnurl pay request endpoint supports nostr, the client SHOULD use this NIP to request a `zap receipt` rather than a normal lnurl invoice.
3. When a user (the "sender") indicates they want to send a zap to another user (the "recipient"), the client should create a `zap request` event as described in Appendix A of this NIP and sign it.
4. Instead of publishing the `zap request`, the `9734` event should instead be sent to the `callback` url received from the lnurl pay endpoint for the recipient using a GET request. See Appendix B for details and an example.
5. The recipient's lnurl server will receive this `zap request` and validate it. See Appendix C for details on how to properly configure an lnurl server to support zaps, and Appendix D for details on how to validate the `nostr` query parameter.
6. If the `zap request` is valid, the server should fetch a description hash invoice where the description is this `zap request` note and this note only. No additional lnurl metadata is included in the description. This will be returned in the response according to [LUD06]().
7. On receiving the invoice, the client MAY pay it or pass it to an app that can pay the invoice.
8. Once the invoice is paid, the recipient's lnurl server MUST generate a `zap receipt` as described in Appendix E, and publish it to the `relays` specified in the `zap request`.
9. Clients MAY fetch `zap receipts` on posts and profiles, but MUST authorize their validity as described in Appendix F. If the `zap request` note contains a non-empty `content`, it may display a zap comment. Generally clients should show users the `zap request` note, and use the `zap receipt` to show "zap authorized by …" but this is optional.

Reference and examples

Appendix A: Zap Request Event

A `zap request` is an event of kind `9734` that is *not* published to relays, but is instead sent to a recipient's lnurl pay `callback` url. This event's `content` MAY be an optional message to send along with the payment. The event MUST include the following tags:

- `relays` is a list of relays the recipient's wallet should publish its `zap receipt` to. Note that relays should not be nested in an additional list, but should be included as shown in the example below.
- `amount` is the amount in *millisats* the sender intends to pay, formatted as a string. This is recommended, but optional.
- `lnurl` is the lnurl pay url of the recipient, encoded using bech32 with the prefix `lnurl`. This is recommended, but optional.
- `p` is the hex-encoded pubkey of the recipient.

In addition, the event MAY include the following tags:

- `e` is an optional hex-encoded event id. Clients MUST include this if zapping an event rather than a person.
- `a` is an optional event coordinate that allows tipping addressable events such as NIP-23 long-form notes.
- `k` is the stringified kind of the target event.

Example:

```json
{
  "kind": 9734,
  "content": "Zap!",
  "tags": [
    ["relays", "wss://nostr-pub.wellorder.com", "wss://anotherrelay.example.com"],
    ["amount", "21000"],
    ["lnurl",
        "lnurl1dp68gurn8ghj7um5v93kketj9ehx2amn9uh8wetvdskkkmn0wahz7mrww4excup0dajx2mrv92x9xp"],

    ["p", "04c915daefee38317fa734444acee390a8269fe5810b2241e5e6dd343dfbecc9"],
    ["e", "9ae37aa68f48645127299e9453eb5d908a0cbb6058ff340d528ed4d37c8994fb"],
    ["k", "1"]
  ],
  "pubkey": "97c70a44366a6535c145b333f973ea86dfdc2d7a99da618c40c64705ad98e322",
  "created_at": 1679673265,
  "id": "30efed56a035b2549fcaeec0bf2c1595f9a9b3bb4b1a38abaf8ee9041c4b7d93",
  "sig":
      "f2cb581a84ed10e4dc84937bd98e27acac71ab057255f6aa8dfa561808c981fe8870f4a03c1e3666784d82a9c802d3704e1

}
```

Appendix B: Zap Request HTTP Request

A signed `zap request` event is not published, but is instead sent using a HTTP GET request to the recipient's `callback` url, which was provided by the recipient's lnurl pay endpoint. This request should have the following query parameters defined:

- `amount` is the amount in *millisats* the sender intends to pay
- `nostr` is the `9734 zap request` event, JSON encoded then URI encoded
- `lnurl` is the lnurl pay url of the recipient, encoded using bech32 with the prefix `lnurl`

This request should return a JSON response with a `pr` key, which is the invoice the sender must pay to finalize their zap. Here is an example flow in javascript:

```
const senderPubkey // The sender's pubkey
const recipientPubkey = // The recipient's pubkey
const callback = // The callback received from the recipients lnurl pay endpoint
const lnurl = // The recipient's lightning address, encoded as a lnurl
const sats = 21

const amount = sats * 1000
const relays = ['wss://nostr-pub.wellorder.net']
const event = encodeURI(JSON.stringify(await signEvent({
  kind: 9734,
  content: "",
  pubkey: senderPubkey,
  created_at: Math.round(Date.now() / 1000),
  tags: [
    ["relays", ...relays],
    ["amount", amount.toString()],
    ["lnurl", lnurl],
    ["p", recipientPubkey],
  ],
})))

const {pr: invoice} = await fetchJson(`${callback}?amount=${amount}&nostr=${event}&lnurl=$
        {lnurl}`)
```

Appendix C: LNURL Server Configuration

The lnurl server will need some additional pieces of information so that clients can know that zap invoices are supported:

1. Add a `nostrPubkey` to the lnurl-pay static endpoint `/.well-known/lnurlp/<user>`, where `nostrPubkey` is the nostr pubkey your server will use to sign `zap receipt` events. Clients will use this to validate `zap receipts`.
2. Add an `allowsNostr` field and set it to true.

Appendix D: LNURL Server Zap Request Validation

When a client sends a `zap request` event to a server's lnurl-pay callback URL, there will be a `nostr` query parameter whose value is that event which is URI- and JSON-encoded. If present, the `zap request` event must be validated in the following ways:

1. It MUST have a valid nostr signature
2. It MUST have tags
3. It MUST have only one `p` tag
4. It MUST have 0 or 1 `e` tags
5. There should be a `relays` tag with the relays to send the `zap receipt` to.
6. If there is an `amount` tag, it MUST be equal to the `amount` query parameter.
7. If there is an `a` tag, it MUST be a valid event coordinate
8. There MUST be 0 or 1 `P` tags. If there is one, it MUST be equal to the `zap receipt`'s `pubkey`.

The event MUST then be stored for use later, when the invoice is paid.

Appendix E: Zap Receipt Event

A `zap receipt` is created by a lightning node when an invoice generated by a `zap request` is paid. `Zap receipts` are only created when the invoice description (committed to the description hash) contains a `zap request` note.

When receiving a payment, the following steps are executed:

1. Get the description for the invoice. This needs to be saved somewhere during the generation of the description hash invoice. It is saved automatically for you with CLN, which is the reference implementation used here.
2. Parse the bolt11 description as a JSON nostr event. This SHOULD be validated based on the requirements in Appendix D, either when it is received, or before the invoice is paid.
3. Create a nostr event of kind `9735` as described below, and publish it to the `relays` declared in the `zap request`.

The following should be true of the `zap receipt` event:

- The `content` SHOULD be empty.
- The `created_at` date SHOULD be set to the invoice `paid_at` date for idempotency.
- `tags` MUST include the `p` tag (zap recipient) AND optional `e` tag from the `zap request` AND optional `a` tag from the `zap request` AND optional `P` tag from the pubkey of the zap request (zap sender).
- The `zap receipt` MUST have a `bolt11` tag containing the description hash bolt11 invoice.
- The `zap receipt` MUST contain a `description` tag which is the JSON-encoded zap request.
- `SHA256(description)` SHOULD match the description hash in the bolt11 invoice.
- The `zap receipt` MAY contain a `preimage` tag to match against the payment hash of the bolt11 invoice. This isn't really a payment proof, there is no real way to prove that the invoice is real or has been paid. You are trusting the author of the `zap receipt` for the legitimacy of the payment.

The `zap receipt` is not a proof of payment, all it proves is that some nostr user fetched an invoice. The existence of the `zap receipt` implies the invoice as paid, but it could be a lie given a rogue implementation.

A reference implementation for a zap-enabled lnurl server can be found here.

Example `zap receipt`:

```json
{
    "id": "67b48a14fb66c60c8f9070bdeb37afdfcc3d08ad01989460448e4081eddda446",
    "pubkey": "9630f464cca6a5147aa8a35f0bcdd3ce485324e732fd39e09233b1d848238f31",
    "created_at": 1674164545,
    "kind": 9735,
    "tags": [
      ["p", "32e1827635450ebb3c5a7d12c1f8e7b2b514439ac10a67eef3d9fd9c5c68e245"],
      ["P", "97c70a44366a6535c145b333f973ea86dfdc2d7a99da618c40c64705ad98e322"],
      ["e", "3624762a1274dd9636e0c552b53086d70bc88c165bc4dc0f9e836a1eaf86c3b8"],
      ["k", "1"],
      ["bolt11",
        "lnbc10u1p3unwfusp5t9r3yymhpfqculx78u027lxspgxcr2n2987mx2j55nnfs95nxnzqpp5jmrh92pfld78spqs78v9euf238

      ["description", "{\"pubkey\":\"97c70a44366a6535c145b333f973ea86dfdc2d7a99da618c40c64705ad98e322\",\"co
        \"kind\":9734,\"tags\":[[\"e\",\"3624762a1274dd9636e0c552b53086d70bc88c165bc4dc0f9e836a1eaf86c3b8\"]
        brb.io\",\"wss://nostr.bitcoiner.social\",\"ws://monad.jb55.com:8080\",\"wss://relay.snort.social\"]

      ["preimage", "5d006d2cf1e73c7148e7519a4c68adc81642ce0e25a432b2434c99f97344c15f"]
    ],
    "content": "",
  }
```

Appendix F: Validating Zap Receipts

A client can retrieve `zap receipts` on events and pubkeys using a NIP-01 filter, for example `{"kinds": [9735], "#e": [...]}`. Zaps MUST be validated using the following steps:

- The `zap receipt` event's `pubkey` MUST be the same as the recipient's lnurl provider's `nostrPubkey` (retrieved in step 1 of the protocol flow).
- The `invoiceAmount` contained in the `bolt11` tag of the `zap receipt` MUST equal the `amount` tag of the `zap request` (if present).
- The `lnurl` tag of the `zap request` (if present) SHOULD equal the recipient's `lnurl`.

Appendix G: `zap` tag on other events

When an event includes one or more `zap` tags, clients wishing to zap it SHOULD calculate the lnurl pay request based on the tags value instead of the event author's profile field. The tag's second argument is the `hex` string of the receiver's pub key and the third argument is the relay to download the receiver's metadata (Kind-0). An optional fourth parameter specifies the weight (a generalization of a percentage) assigned to the respective receiver. Clients should parse all weights, calculate a sum, and then a percentage to each receiver. If weights are not present, CLIENTS should equally divide the zap amount to all receivers. If weights are only partially present, receivers without a weight should not be zapped (`weight = 0`).

```js
{
    "tags": [
        [ "zap", "82341f882b6eabcd2ba7f1ef90aad961cf074af15b9ef44a09f9d2a8fbfbe6a2", "wss://
        nostr.oxtr.dev", "1" ],  // 25%
        [ "zap", "fa984bd7dbb282f07e16e7ae87b26a2a7b9b90b7246a44771f0cf5ae58018f52", "wss://
        nostr.wine/",    "1" ],  // 25%
```

```
        [ "zap", "460c25e682fda7832b52d1f22d3d22b3176d972f60dcdc3212ed8c92ef85065c", "wss://
        nos.lol/",       "2" ]   // 50%
    ]
}
```

Clients MAY display the zap split configuration in the note.

Future Work

Zaps can be extended to be more private by encrypting `zap request` notes to the target user, but for simplicity it has been left out of this initial draft.

NIP-75

Zap Goals

`draft` `optional`

This NIP defines an event for creating fundraising goals. Users can contribute funds towards the goal by zapping the goal event.

Nostr Event

A `kind:9041` event is used.

The `.content` contains a human-readable description of the goal.

The following tags are defined as REQUIRED.

- `amount` - target amount in milisats.
- `relays` - a list of relays the zaps to this goal will be sent to and tallied from.

Example event:

```
{
  "kind": 9041,
  "tags": [
    ["relays", "wss://alicerelay.example.com", "wss://bobrelay.example.com", /*...*/],
    ["amount", "210000"],
  ],
  "content": "Nostrasia travel expenses",
  // other fields...
}
```

The following tags are OPTIONAL.

- `closed_at` - timestamp for determining which zaps are included in the tally. Zap receipts published after the `closed_at` timestamp SHOULD NOT count towards the goal progress.
- `image` - an image for the goal
- `summary` - a brief description

```
{
  "kind": 9041,
  "tags": [
    ["relays", "wss://alicerelay.example.com", "wss://bobrelay.example.com", /*...*/],
    ["amount", "210000"],
    ["closed_at", "<unix timestamp in seconds>"],
    ["image", "<image URL>"],
    ["summary", "<description of the goal>"],
  ],
  "content": "Nostrasia travel expenses",
  // other fields...
}
```

The goal MAY include an `r` or `a` tag linking to a URL or addressable event.

The goal MAY include multiple beneficiary pubkeys by specifying [zap tags](#).

Addressable events can link to a goal by using a `goal` tag specifying the event id and an optional relay hint.

```
{
  "kind": 3xxxx,
  "tags": [
    ["goal", "<event id>", "<Relay URL (optional)>"],
    // rest of tags...
  ],
  // other fields...
}
```

Client behavior

Clients MAY display funding goals on user profiles.

When zapping a goal event, clients MUST include the relays in the `relays` tag of the goal event in the zap request `relays` tag.

When zapping an addressable event with a `goal` tag, clients SHOULD tag the goal event id in the `e` tag of the zap request.

Use cases

- Fundraising clients
- Adding funding goals to events such as long form posts, badges or live streams

NIP-47

Nostr Wallet Connect (NWC)

`draft` `optional`

Rationale

This NIP describes a way for clients to access a remote lightning wallet through a standardized protocol. Custodians may implement this, or the user may run a bridge that bridges their wallet/node and the Nostr Wallet Connect protocol.

This NIP defines the core Nostr Wallet Connect protocol. More granular authorization models and extensions for advanced wallet functionality may be defined in separate specifications in the NWC repository at `https://github.com/nostr-wallet-connect/nwc`.

Terms

- **client**: Nostr app on any platform that wants to interact with a lightning wallet.
- **user**: The person using the **client**, and wants to connect their wallet to their **client**.
- **wallet service**: Nostr app that typically runs on an always-on computer (eg. in the cloud or on a Raspberry Pi). This app has access to the APIs of the wallets it serves.

Theory of Operation

Fundamentally NWC is communication between a **client** and **wallet service** by the means of E2E-encrypted direct messages over a nostr relay. The relay knows the kinds and tags of notes, but not the content of the encrypted payloads. The **user**'s identity key is not used to avoid linking payment activity to the user. Ideally unique keys are used for each individual connection.

1. **Users** who wish to use this NIP to allow **client(s)** to interact with their wallet must first acquire a special "connection" URI from their NIP-47 compliant wallet application. The wallet application may provide this URI using a QR screen, or a pasteable string, or some other means.

2. The **user** should then copy this URI into their **client(s)** by pasting, or scanning the QR, etc. The **client(s)** should save this URI and use it later whenever the **user** (or the **client** on the user's behalf) wants to interact with the wallet. The **client** should then request an `info` (13194) event from the relay(s) specified in the URI. The **wallet service** will have sent that event to those relays earlier, and the relays will hold it as a replaceable event.

3. When the **user** initiates a payment their nostr **client** create a `pay_invoice` request, encrypts it using a token from the URI, and sends it (kind 23194) to the relay(s) specified in the connection URI. The **wallet service** will be listening on those relays and will decrypt the request and then contact the **user's** wallet application to send the payment. The **wallet service** will know how to talk to the wallet application because the connection URI specified relay(s) that have access to the wallet app API.

4. Once the payment is complete the **wallet service** will send an encrypted `response` (kind 23195) to the **user** over the relay(s) in the URI.

5. Optional extension specifications may define additional wallet-to-client events.

Events

There are three core event kinds:

- `NIP-47 info event`: 13194
- `NIP-47 request`: 23194
- `NIP-47 response`: 23195

Info Event

The info event should be a replaceable event that is published by the **wallet service** on the relay to indicate which capabilities it supports.

The content should be a plaintext string with the supported capabilities space-separated, eg. `pay_invoice get_balance`.

Any additional methods supported through optional NWC extensions advertised in the `extensions` tag SHOULD also be included in the content of the info event.

If the **wallet service** supports optional NWC extensions, the info event SHOULD contain an `extensions` tag with the supported extension identifiers space-separated, eg. `02 03 04`.

It should also contain supported encryption modes as described in the Encryption section. For example:

```
{
    "kind": 13194,
    "tags": [
        ["encryption", "nip44_v2 nip04"], // List of supported encryption schemes as described in the Encryption section.
        ["extensions", "02 03 04"]
        // ...
    ],
    "content": "pay_invoice get_balance make_invoice lookup_invoice get_info",
    // ...
}
```

Request and Response Events

Both the request and response events SHOULD contain one `p` tag, containing the public key of the **wallet service** if this is a request, and the public key of the **client** if this is a response. The response event SHOULD contain an `e` tag with the id of the request event it is responding to. Optionally, a request can have an `expiration` tag that has a unix timestamp in seconds. If the request is received after this timestamp, it should be ignored.

The content of requests and responses is encrypted with NIP44, and is a JSON-RPCish object with a semi-fixed structure.

Important note for backwards-compatibility: The initial version of the protocol used NIP04. If a **wallet service** or client app does not include the `encryption` tag in the `info` or request events, it should be assumed that the connection is using NIP04 for encryption. See the Encryption section for more information.

Example request:

```
{
    "kind" 23194,
    "tags": [
        ["encryption", "nip44_v2"],
        ["p", "03..." ] // public key of the wallet service.
        // ...
    ],
    "content": nip44_encrypt({ // Encryption type corresponds to the `encryption` tag.
        "method": "pay_invoice", // method, string
        "params": { // params, object
            "invoice": "lnbc50n1..." // command-related data
        }
    }),
}
```

Example response:

```
{
    "kind" 23195,
    "tags": [
        ["p", "03..." ] // public key of the requesting client app
        ["e", "1234"] // id of the request event this is responding to
        // ...
    ],
    "content": nip44_encrypt({ // Encrypted using the scheme requested by the client.
        "result_type": "pay_invoice", //indicates the structure of the result field
        "error": { //object, non-null in case of error
            "code": "UNAUTHORIZED", //string error code, see below
            "message": "human readable error message"
        },
        "result": { // result, object. null in case of error.
            "preimage": "0123456789abcdef..." // command-related data
        }
    })
    // ...
}
```

The `result_type` field MUST contain the name of the method that this event is responding to. The `error` field MUST contain a `message` field with a human readable error message and a `code` field with the error code if the command was not successful. If the command was successful, the `error` field must be null.

Error codes

- `RATE_LIMITED`: The client is sending commands too fast. It should retry in a few seconds.
- `NOT_IMPLEMENTED`: The command is not known or is intentionally not implemented.
- `INSUFFICIENT_BALANCE`: The wallet does not have enough funds to cover a fee reserve or the payment amount.
- `QUOTA_EXCEEDED`: The wallet has exceeded its spending quota.
- `RESTRICTED`: This public key is not allowed to do this operation.
- `UNAUTHORIZED`: This public key has no wallet connected.
- `INTERNAL`: An internal error.

- `UNSUPPORTED_ENCRYPTION`: The encryption type of the request is not supported by the wallet service.
- `OTHER`: Other error.

Nostr Wallet Connect URI

Communication between the **client** and **wallet service** requires two keys in order to encrypt and decrypt messages. The connection URI includes the secret key of the **client** and only the public key of the **wallet service**.

The **client** discovers **wallet service** by scanning a QR code, handling a deeplink or pasting in a URI.

The **wallet service** generates this connection URI with protocol `nostr+walletconnect://` and base path its 32-byte hex-encoded `pubkey`, which SHOULD be unique per client connection.

The connection URI contains the following query string parameters:

- `relay` Required. URL of the relay where the **wallet service** is connected and will be listening for events. May be more than one.
- `secret` Required. 32-byte randomly generated hex encoded string. The **client** MUST use this to sign events and encrypt payloads when communicating with the **wallet service**. The **wallet service** MUST use the corresponding public key of this secret to communicate with the **client**.
    - Authorization does not require passing keys back and forth.
    - The user can have different keys for different applications. Keys can be revoked and created at will and have arbitrary constraints (eg. budgets).
    - The key is harder to leak since it is not shown to the user and backed up.
    - It improves privacy because the user's main key would not be linked to their payments.
- `lud16` Recommended. A lightning address that clients can use to automatically setup the `lud16` field on the user's profile if they have none configured.

The **client** should then store this connection and use it when the user wants to perform actions like paying an invoice. Due to this NIP using ephemeral events, it is recommended to pick relays that do not close connections on inactivity to not drop events, and ideally retain the events until they are either consumed or become stale.

- When the **client** sends or receives a message it will use the `secret` from the connection URI and **wallet service**'s `pubkey` to encrypt or decrypt.
- When the **wallet service** sends or receives a message it will use its own secret and the corresponding pubkey of the **client's** `secret` to encrypt or decrypt. The **wallet service** SHOULD NOT store the secret it generates for the client and MUST NOT rely on the knowing the **client** secret for general operation.

Example connection string

```
nostr+walletconnect://b889ff5b1513b641e2a139f661a661364979c5beee91842f8f0ef42ab558e9d4?
        relay=wss%3A%2F%2Frelay.damus.io&secret=71a8c14c1407c113601079c4302dab36460f0ccd0ad506f1f2dc73b5100e
```

Commands

`pay_invoice`

Description: Requests payment of an invoice.

Request:

```
{
    "method": "pay_invoice",
    "params": {
        "invoice": "lnbc50n1...", // bolt11 invoice
        "amount": 123, // invoice amount in msats, optional
        "metadata": {} // generic metadata that can be used to add things like zap/boostagram
        details for a payer name/comment/etc, optional
    }
}
```

Response:

```
{
    "result_type": "pay_invoice",
    "result": {
        "preimage": "0123456789abcdef...", // preimage of the payment
        "fees_paid": 123, // value in msats, optional
    }
}
```

Errors: - `PAYMENT_FAILED`: The payment failed. This may be due to a timeout, exhausting all routes, insufficient capacity or similar.

`make_invoice`

Request:

```
{
    "method": "make_invoice",
    "params": {
        "amount": 123, // value in msats
        "description": "string", // invoice's description, optional
        "description_hash": "string", // invoice's description hash, optional
        "expiry": 213, // expiry in seconds from time invoice is created, optional
        "metadata": {} // generic metadata that can be used to add things like zap/boostagram
        details for a payer name/comment/etc, optional
    }
}
```

Response:

```
{
    "result_type": "make_invoice",
    "result": {
        "type": "incoming", // "incoming" for invoices, "outgoing" for payments
        "state": "pending", // optional
        "invoice": "string", // encoded invoice, optional
        "description": "string", // invoice's description, optional
        "description_hash": "string", // invoice's description hash, optional
        "preimage": "string", // payment's preimage, optional if unpaid
        "payment_hash": "string", // Payment hash for the payment
        "amount": 123, // value in msats
        "fees_paid": 123, // value in msats
```

```
        "created_at": unixtimestamp, // invoice/payment creation time
        "expires_at": unixtimestamp, // invoice expiration time, optional if not applicable
        "metadata": {} // generic metadata that can be used to add things like zap/boostagram
        details for a payer name/comment/etc.
    }
}
```

`lookup_invoice`

Request:

```
{
    "method": "lookup_invoice",
    "params": {
        "payment_hash": "31afdf1..", // payment hash of the invoice, one of payment_hash or
        invoice is required
        "invoice": "lnbc50n1..." // invoice to lookup
    }
}
```

Response:

```
{
    "result_type": "lookup_invoice",
    "result": {
        "type": "incoming", // "incoming" for invoices, "outgoing" for payments
        "state": "pending", // can be "pending", "settled", "accepted" (for hold invoices),
        "expired" (for invoices) or "failed" (for payments), optional
        "invoice": "string", // encoded invoice, optional
        "description": "string", // invoice's description, optional
        "description_hash": "string", // invoice's description hash, optional
        "preimage": "string", // payment's preimage, optional if unpaid
        "payment_hash": "string", // Payment hash for the payment
        "amount": 123, // value in msats
        "fees_paid": 123, // value in msats
        "created_at": unixtimestamp, // invoice/payment creation time
        "expires_at": unixtimestamp, // invoice expiration time, optional if not applicable
        "settled_at": unixtimestamp, // invoice/payment settlement time, optional if unpaid
        "metadata": {} // generic metadata that can be used to add things like zap/boostagram
        details for a payer name/comment/etc.
    }
}
```

Errors: - `NOT_FOUND`: The invoice could not be found by the given parameters.

`get_balance`

Request:

```
{
    "method": "get_balance",
    "params": {}
}
```

Response:

```
{
    "result_type": "get_balance",
    "result": {
        "balance": 10000, // user's balance in msats
    }
}
```

`get_info`

Request:

```
{
    "method": "get_info",
    "params": {}
}
```

Response:

```
{
    "result_type": "get_info",
    "result": {
            "alias": "string",
            "color": "hex string",
            "pubkey": "hex string",
            "network": "string", // mainnet, testnet, signet, or regtest
            "block_height": 1,
            "block_hash": "hex string",
            "methods": ["pay_invoice", "get_balance", "make_invoice", "lookup_invoice",
        "get_info"], // list of supported methods for this connection
            "extensions": ["02", "03", "04", "06", "07", "08"], // list of supported optional
        NWC extension specs for this connection, optional
    }
}
```

Extensions

This NIP defines the core protocol and a small common command set. Additional methods, metadata conventions, pairing flows, and more granular authorization behavior MAY be defined by optional NWC extension specifications.

Additional optional NWC specifications are maintained in the dedicated NWC repository:

- `https://github.com/nostr-wallet-connect/nwc`

That repository reserves `01.md` for the NWC core specification corresponding to this simplified NIP-47, and defines optional features in separate specs starting with `02.md`.

Example pay invoice flow

1. The user scans the QR code generated by the **wallet service** with their **client** application, they follow a `nostr+walletconnect://` deeplink or configure the connection details manually.

2. **client** sends an event to the **wallet service** with kind `23194`. The content is a `pay_invoice` request. The private key is the secret from the connection string above.
3. **wallet service** verifies that the author's key is authorized to perform the payment, decrypts the payload and sends the payment.
4. **wallet service** responds to the event by sending an event with kind `23195` and content being a response either containing an error message or a preimage.

Encryption

The initial version of NWC used [NIP-04](NIP-04) for encryption which has been deprecated and replaced by [NIP-44](NIP-44). NIP-44 should always be preferred for encryption, but there may be legacy cases where the **wallet service** or **client** has not yet migrated to NIP-44. The **wallet service** and **client** should negotiate the encryption method to use based on the `encryption` tag in the `info` event.

The encryption tag can contain either `nip44_v2` or `nip04`. The absence of this tag implies that the wallet only supports `nip04`.

Encryption code	Use	Notes
`nip44_v2`	NIP-44	Required
`nip04`	NIP-04	Deprecated and only here for backward compatibility
<not present>	NIP-04	Deprecated and only here for backward compatibility

The negotiation works as follows.

1. The **wallet service** includes an `encryption` tag in the `info` event. This tag contains a space-separated list of encryption schemes that the **wallet service** supports (eg. `nip44_v2 nip04`)
2. The **client application** includes an `encryption` tag in each request event. This tag contains the encryption scheme which should be used for the request. The **client application** should always prefer nip44 if supported by the **wallet service**.

When a **client application** establishes a connection, it should read the info event and look for the `encryption` tag.

Absence of this tag implies that the wallet only supports nip04.

If the `encryption` tag is present, the **client application** will choose optimal encryption supported by both itself, and the **wallet service**, which should always be nip44 if possible.

Request events

When a **client application** sends a request event, it should include a `encryption` tag with the encryption scheme it is using. The scheme MUST be supported by the **wallet service** as indicated by the info event. For example, if the client application supports nip44, the request event might look like:

```
{
    "kind": 23194,
    "tags": [
        ["encryption", "nip44_v2"],
        // ...
```

```
  ],
  // ...
}
```

If the **wallet service** does not support the specified encryption scheme, it will return an `UNSUPPORTED_ENCRYPTION` error. Absence of the `encryption` tag indicates use of nip04 for encryption.

Using a dedicated relay

This NIP does not specify any requirements on the type of relays used. However, if the user is using a custodial service it might make sense to use a relay that is hosted by the custodial service. The relay may then enforce authentication to prevent metadata leaks. Not depending on a 3rd party relay would also improve reliability in this case.

Appendix

Example NIP-47 info event

```json
{
  "id": "df467db0a9f9ec77ffe6f561811714ccaa2e26051c20f58f33c3d66d6c2b4d1c",
  "pubkey": "c04ccd5c82fc1ea3499b9c6a5c0a7ab627fbe00a0116110d4c750faeaecba1e2",
  "created_at": 1713883677,
  "kind": 13194,
  "tags": [
    [ "encryption", "nip44_v2 nip04" ],
    [ "extensions", "02 03 04 06 07 08" ]
  ],
  "content": "pay_invoice get_balance get_info make_invoice lookup_invoice",
  "sig":
      "31f57b369459b5306a5353aa9e03be7fbde169bc881c3233625605dd12f53548179def16b9fe1137e6465d7e4d5bb27ce81

}
```

NIP-60

Cashu Wallets

`draft` `optional`

This NIP defines the operations of a cashu-based wallet.

A cashu wallet is a wallet which information is stored in relays to make it accessible across applications.

The purpose of this NIP is: * ease-of-use: new users immediately are able to receive funds without creating accounts with other services. * interoperability: users' wallets follows them across applications.

This NIP doesn't deal with users' *receiving* money from someone else, it's just to keep state of the user's wallet.

High-level flow

1. A user has a `kind:17375` event that represents a wallet.
2. A user has `kind:7375` events that represent the unspent proofs of the wallet. – The proofs are encrypted with the user's private key.
3. A user has `kind:7376` events that represent the spending history of the wallet – This history is for informational purposes only and is completely optional.

Wallet Event

```
{
    "kind": 17375,
    "content": nip44_encrypt([
        [ "privkey", "hexkey" ],
        [ "mint", "https://mint1" ],
        [ "mint", "https://mint2" ]
    ]),
    "tags": []
}
```

The wallet event is an replaceable event `kind:17375`.

Tags: * `mint` - Mint(s) this wallet uses – there MUST be one or more mint tags. * `privkey` - Private key used to unlock P2PK ecash. MUST be stored encrypted in the `.content` field. **This is a different private key exclusively used for the wallet, not associated in any way to the user's Nostr private key** – This is only used for receiving NIP-61 nutzaps.

Token Event

Token events are used to record unspent proofs.

There can be multiple `kind:7375` events for the same mint, and multiple proofs inside each `kind:7375` event.

```
{
    "kind": 7375,
    "content": nip44_encrypt({
        "mint": "https://stablenut.umint.cash",
        "unit": "sat",
        "proofs": [
            // one or more proofs in the default cashu format
            {
                "id": "005c2502034d4f12",
                "amount": 1,
                "secret": "z+zyxAVLRqN9lEjxuNPSyRJzEstbl69Jc1vtimvtkPg=",
                "C": "0241d98a8197ef238a192d47edf191a9de78b657308937b4f7dd0aa53beae72c46"
            }
        ],
        // tokens that were destroyed in the creation of this token (helps on wallet state
        transitions)
        "del": [ "token-event-id-1", "token-event-id-2" ]
    }),
```

```
    "tags": []
}
```

- `.content` is a NIP-44 encrypted payload:
  - `mint`: The mint the proofs belong to.
  - `proofs`: unencoded proofs
  - `unit` the base unit the proofs are denominated in (eg: `sat`, `usd`, `eur`). Default: `sat` if omitted.
  - `del`: token-ids that were destroyed by the creation of this token. This assists with state transitions.

When one or more proofs of a token are spent, the token event should be NIP-09-deleted and, if some proofs are unspent from the same token event, a new token event should be created rolling over the unspent proofs and adding any change outputs to the new token event (the change output should include a `del` field).

The `kind:5` *delete event* created in the NIP-09 process MUST have a tag `["k", "7375"]` to allow easy filtering by clients interested in state transitions.

Spending History Event

Clients SHOULD publish `kind:7376` events to create a transaction history when their balance changes.

```
{
    "kind": 7376,
    "content": nip44_encrypt([
        [ "direction", "in" ], // in = received, out = sent
        [ "amount", "1" ],
        [ "unit", "sat" ],
        [ "e", "<event-id-of-created-token>", "", "created" ]
    ]),
    "tags": [
        [ "e", "<event-id-of-created-token>", "", "redeemed" ]
    ]
}
```

- `direction` - The direction of the transaction; `in` for received funds, `out` for sent funds.
- `unit` the base unit of the amount (eg: `sat`, `usd`, `eur`). Default: `sat` if omitted.

Clients MUST add `e` tags to create references of destroyed and created token events along with the marker of the meaning of the tag: * `created` - A new token event was created. * `destroyed` - A token event was destroyed. * `redeemed` - A NIP-61 nutzap was redeemed.

All tags can be NIP-44 encrypted. Clients SHOULD leave `e` tags with a `redeemed` marker unencrypted.

Multiple `e` tags can be added, and should be encrypted, except for tags with the `redeemed` marker.

Flow

A client that wants to check for user's wallets information starts by fetching `kind:10019` events from the user's relays, if no event is found, it should fall back to using the user's NIP-65 relays.

Fetch wallet and token list

From those relays, the client should fetch wallet and token events.

```
"kinds": [17375, 7375], "authors": ["<my-pubkey>"]
```

Fetch proofs

Spending token

If Alice spends 4 sats from this token event

```
{
    "kind": 7375,
    "id": "event-id-1",
    "content": nip44_encrypt({
        "mint": "https://stablenut.umint.cash",
        "unit": "sat",
        "proofs": [
            { "id": "1", "amount": 1 },
            { "id": "2", "amount": 2 },
            { "id": "3", "amount": 4 },
            { "id": "4", "amount": 8 },
        ]
    }),
    "tags": []
}
```

Her client: * MUST roll over the unspent proofs:

```
{
    "kind": 7375,
    "id": "event-id-2",
    "content": nip44_encrypt({
        "mint": "https://stablenut.umint.cash",
        "unit": "sat",
        "proofs": [
            { "id": "1", "amount": 1 },
            { "id": "2", "amount": 2 },
            { "id": "4", "amount": 8 },
        ],
        "del": [ "event-id-1" ]
    }),
    "tags": []
}
```

- MUST delete event `event-id-1`
- SHOULD add the `event-id-1` to the `del` array of deleted token-ids.
- SHOULD create a `kind:7376` event to record the spend

```
{
    "kind": 7376,
    "content": nip44_encrypt([
        [ "direction", "out" ],
```

```
        [ "amount", "4" ],
        [ "unit", "sat" ],
        [ "e", "<event-id-1>", "", "destroyed" ],
        [ "e", "<event-id-2>", "", "created" ],
    ]),
    "tags": []
}
```

Redeeming a quote (optional)

When creating a quote at a mint, an event can be used to keep the state of the quote ID, which will be used to check when the quote has been paid. These events should be created with an expiration tag NIP-40 of 2 weeks (which is around the maximum amount of time a Lightning payment may be in-flight).

However, application developers SHOULD use local state when possible and only publish this event when it makes sense in the context of their application.

```
{
    "kind": 7374,
    "content": nip44_encrypt("quote-id"),
    "tags": [
        [ "expiration", "<expiration-timestamp>" ],
        [ "mint", "<mint-url>" ]
    ]
}
```

Appendix 1: Validating proofs

Clients can optionally validate proofs to make sure they are not working from an old state; this logic is left up to particular implementations to decide when and why to do it, but if some proofs are checked and deemed to have been spent, the client should delete the token and roll over any unspent proof.

NIP-61

Nutzaps

`draft` `optional`

A Nutzap is a P2PK Cashu token in which the payment itself is the receipt.

High-level flow

Alice wants to nutzap 1 sat to Bob because of an event `event-id-1` she liked.

Alice nutzaps Bob

1. Alice fetches event `kind:10019` from Bob to see the mints Bob trusts.
2. She mints a token at that mint (or swaps some tokens she already had in that mint) P2PK-locked to the pubkey Bob has listed in his `kind:10019`.

3. She publishes a `kind:9321` event to the relays Bob indicated with the proofs she minted.

Bob receives the nutzap

1. At some point, Bob's client fetches `kind:9321` events p-tagging him from his relays.
2. Bob's client swaps the token into his wallet.

Nutzap informational event

```
{
    "kind": 10019,
    "tags": [
        [ "relay", "wss://relay1" ],
        [ "relay", "wss://relay2" ],
        [ "mint", "https://mint1", "usd", "sat" ],
        [ "mint", "https://mint2", "sat" ],
        [ "pubkey", "<p2pk-pubkey>" ]
    ]
}
```

- `kind:10019` is an event that is useful for others to know how to send money to the user.
- `relay`: relays where the user will be reading token events from. If a user wants to send money to the user, they should write to these relays.
- `mint`: mints the user is explicitly agreeing to use to receive funds on. Clients SHOULD not send money on mints not listed here or risk burning their money. Additional markers can be used to list the supported base units of the mint.
- `pubkey`: Public key that MUST be used to P2PK-lock receiving nutzaps – implementations MUST NOT use the target user's main Nostr public key. This public key corresponds to the `privkey` field encrypted in a user's [nip-60](nip-60) *wallet event*.

Nutzap event

Event `kind:9321` is a nutzap event published by the sender, p-tagging the recipient. The outputs are P2PK-locked to the public key the recipient indicated in their `kind:10019` event.

Clients MUST prefix the public key they P2PK-lock with `"02"` (for nostr<>cashu compatibility).

```
{
    "kind": 9321,
    "content": "Thanks for this great idea.",
    "pubkey": "<sender-pubkey>",
    "tags": [
        [ "proof", "{\"amount\":1,\"C\":\"02277c66191736eb72fce9d975d08e3191f8f96afb73ab1eec37e4465683066d3f
        \"secret\":\"[\\\"P2PK\\\",{\\\"nonce\\\":\\\"b00bdd0467b0090a25bdf2d2f0d45ac4e355c482c1418350f273a0
        \"02eaee8939e3565e48cc62967e2fde9d8e2a4b3ec0081f29eceff5c64ef10ac1ed\\\"}]\"}" ],

        [ "unit", "sat" ],
        [ "u", "https://stablenut.umint.cash" ],
        [ "e", "<nutzapped-event-id>", "<relay-hint>" ],
        [ "k", "<nutzapped-kind>"],
        [ "p", "e9fbced3a42dcf551486650cc752ab354347dd413b307484e4fd1818ab53f991" ], //
        recipient of nutzap
```

```
    ]
}
```

- `.content` is an optional comment for the nutzap
- `.tags`:
    - `proof` is one or more proofs P2PK-locked to the public key the recipient specified in their `kind:10019` event and including a DLEQ proof.
    - `unit` the base unit the proofs are denominated in (eg: `sat`, `usd`, `eur`). Default: `sat` if omitted.
    - `u` is the mint the URL of the mint EXACTLY as specified by the recipient's `kind:10019`.
    - `p` is the Nostr identity public key of nutzap recipient.
    - `e` is the event that is being nutzapped, if any.

Sending a nutzap

- The sender fetches the recipient's `kind:10019`.
- The sender mints/swaps ecash on one of the recipient's listed mints.
- The sender P2PK-locks to the recipient's specified public key in their `kind:10019`

Receiving nutzaps

Clients should REQ for nutzaps: * Filtering with `#u` for mints they expect to receive ecash from. * this is to prevent even interacting with mints the user hasn't explicitly signaled. * Filtering with `since` of the most recent `kind:7376` event the same user has created. * this can be used as a marker of the nutzaps that have already been swapped by the user – clients might choose to use other kinds of markers, including internal state – this is just a guidance of one possible approach.

`{ "kinds": [9321], "#p": ["my-pubkey"], "#u": ["<mint-1>", "<mint-2>"], "since": <latest-created_at-of-kind-7376> }`.

Upon receiving a new nutzap, the client should swap the tokens into a wallet the user controls, either a [NIP-60](NIP-60) wallet, their own LN wallet or anything else.

Updating nutzap-redemption history

When claiming a token the client SHOULD create a `kind:7376` event and `e` tag the original nutzap event. This is to record that this token has already been claimed (and shouldn't be attempted again) and as signaling to the recipient that the ecash has been redeemed.

Multiple `kind:9321` events can be tagged in the same `kind:7376` event.

```
{
    "kind": 7376,
    "content": nip44_encrypt([
        [ "direction", "in" ], // in = received, out = sent
        [ "amount", "1" ],
        [ "unit", "sat" ],
        [ "e", "<7375-event-id>", "<relay-hint>", "created" ] // new token event that was
        created
    ]),
    "tags": [
```

```
        [ "e", "<9321-event-id>", "<relay-hint>", "redeemed" ], // nutzap event that has been
        redeemed
        [ "p", "<sender-pubkey>" ] // pubkey of the author of the 9321 event (nutzap sender)
    ]
}
```

Events that redeem a nutzap SHOULD be published to the sender's NIP-65 "read" relays.

Verifying a Cashu Zap

When listing or counting zaps received by any given event, observer clients SHOULD:

- check that the receiving user has issued a `kind:10019` tagging the mint where the cashu has been minted.
- check that the token is locked to the pubkey the user has listed in their `kind:10019`.
- look at the `u` tag and check that the token is issued in one of the mints listed in the `kind:10019`.
- locally verify the DLEQ proof of the tokens being sent.

All these checks can be done offline (as long as the observer has the receiver mints' keyset and their `kind:10019` event), so the process should be reasonably fast.

Final Considerations

1. Clients SHOULD guide their users to use NUT-11 (P2PK) and NUT-12 (DLEQ proofs) compatible-mints in their `kind:10019` event to avoid receiving nutzaps anyone can spend.
2. Clients SHOULD normalize and deduplicate mint URLs as described in NIP-65.
3. A nutzap event MUST include proofs in one of the mints the recipient has listed in their `kind:10019` and published to the NIP-65 relays of the recipient, failure to do so may result in the recipient donating the tokens to the mint since the recipient might never see the event.

NIP-87

Cashu and Fedimint Discoverability

`draft` `optional`

This NIP describes `kind:38173`, `kind:38172` and `kind:38000`: a way to discover ecash mints, their capabilities, and people who recommend them.

Rationale

Nostr's discoverability and transparent event interaction is one of its most interesting/novel mechanics. This NIP provides a simple way for users to discover ecash mints recommended by other users and to interact with them.

Parties involved

There are three actors to this workflow:

- An ecash mint operator, announces their mint and its capabilities.
  - Publishes `kind:38173` or `kind:38172`, detailing how to connect to it and its capabilities.
- user A, who recommends an ecash mint
  - Publishes `kind:38000`
- user B, who seeks a recommendation for an ecash mint
  - Queries for `kind:38000` and, based on results, queries for `kind:38173`/`kind:38172`

Events

Recommendation event

```json
{
  "kind": 38000,
  "pubkey": <recommender-user-pubkey>,
  "tags": [
    ["k", "38173"],
    ["d", "<d-identifier>"],
    ["u", <recommended-fedimint-invite-code>],
    ["a", "38173:fedimint-pubkey:<d-identifier>", "wss://relay1"]
  ],
  "content": "I trust this mint with my life"
}
```

The recommendation event is a parameterized-replaceable event so that a user can change edit their recommendation without creating a new event.

The `d` tag in `kind:38000` is the `kind:38173`/`kind:38172` event identifier this event is recommending, if no event exists, the `d` tag can still be calculated from the mint's pubkey/id. The `k` tag is the kind number that corresponds to the event kind that the user is recommending, in this case `kind:38173` for fedimints and `kind:38172` for cashu mints.

Optional `u` tags can be added to give a recommend way to connect to the mint. The value of the tag is the URL or invite code of the ecash mint. Multiple `u` tags can appear on the same `kind:38000`.

`a` tags are used to point to the `kind:38173`/`kind:38172` event of the ecash mint. The first value of the tag is the `kind:38173`/`kind:38172` event identifier, the second value of the tag is a relay hint. This is used to correctly point to the mint's `kind:38173`/`kind:38172` event in case there are duplicates claiming to be the same mint.

The content can be used to give a review.

Mint Information

Cashu mints SHOULD publish `kind:38172` events to announce their capabilities and how to connect to them.

For cashu mints, the `u` tag SHOULD be the URL to the cashu mint and it should list the nuts that the cashu mint supports.

The `d` tag SHOULD be the mint's pubkey (found when querying `/v1/info`), this way users can query by pubkey and find the mint announcement.

An `n` tag SHOULD be added to signify the network the cashu mint is on (either `mainnet`, `testnet`, `signet`, or `regtest`)

```
{
  "kind": 38172,
  "pubkey": "<application-pubkey>",
  "content": "<optional-kind:0-style-metadata>",
  "tags": [
    ["d", <cashu mint pubkey>],
    ["u", "https://cashu.example.com"],
    ["nuts", "1,2,3,4,5,6,7"],
    ["n", "mainnet"]
  ]
}
```

Fedimints SHOULD publish `kind:38173` events to announce their capabilities and how to connect to them.

For fedimints, it should list all known fedimint invite codes in `u` tags and it should list the modules it supports.

The `d` tag SHOULD be the federation id, this way users can query by federation id and find the fedimint announcement.

An `n` tag SHOULD be added to signify the network the fedimint is on (either `mainnet`, `testnet`, `signet`, or `regtest`)

```
{
  "kind": 38173,
  "pubkey": "<application-pubkey>",
  "content": "<optional-kind:0-style-metadata>",
  "tags": [
    ["d", <federation-id>],
    ["u", "fed11abc.."],
    ["u", "fed11xyz.."],
    ["modules", "lightning,wallet,mint"],
    ["n", "signet"]
  ]
}
```

- `content` is an optional `metadata`-like stringified JSON object, as described in NIP-01. This content is useful when the pubkey creating the `kind:38173` is not a normal user. If `content` is empty, the `kind:0` of the pubkey should be used to display mint information (e.g. name, picture, web, LUD16, etc.)

Example

User A recommends some mints

User A might be a user of a cashu mint. Using a client, user A publishes an event recommending the cashu mint they use.

```json
{
  "kind": 38000,
  "tags": [
    ["u", "fed11abc..", "fedimint"],
    ["u", "https://cashu.example.com", "cashu"],
    ["a", "38173:fedimint-pubkey:<d-identifier>", "wss://relay1", "fedimint"],
    ["a", "38172:cashu-mint-pubkey:<d-identifier>", "wss://relay2", "cashu"]
  ],
  ...
}
```

User B finds a mint

User B wants to use an ecash wallet, they need to find a mint.

User B's wallet client queries for `kind:38000` events, looking for recommendations for ecash mints.

```
["REQ", <id>, [{ "kinds": [38000], "authors": [<user>, <users-contact-list>], "#k":
        ["38173"] }]]
```

User B, who follows User A, sees that `kind:38000` event and tries to connect to the corresponding mints.

Alternative query bypassing `kind:38000`

Alternatively, users might choose to query directly for `kind:38173` for an event kind. Clients SHOULD be careful doing this and use spam-prevention mechanisms or querying high-quality restricted relays to avoid directing users to malicious handlers.

```
["REQ", <id>, [{ "kinds": [38173], "authors": [...] }]]
```

NIP-A3

payto: Payment Targets

`draft` `optional`

This NIP standardizes how payment addresses are declared across any network or platform.

Tag Format

Payment targets are specified using `payto` tags with the following structure:

```
["payto", "<type>", "<address>"]
```

Where: - The first element is always the literal string `"payto"` - The second element is the payment `type` (e.g., `"bitcoin"`, `"lightning"`), always lowercase. - The third element is the `address` (e.g., address, username)

Clients may perform additional network- or platform-specific validation for recognized types.

Example

```
{
  "pubkey": "afc93622eb4d79c0fb75e56e0c14553f7214b0a466abeba14cb38968c6755e6a",
  "kind": 10133,
  "content": "",
  "tags": [
    ["payto", "bitcoin", "bc1qxq66e0t8d7ugdecwnmv58e90tpry23nc84pg9k"],
    ["payto", "nano", "nano_1dctqbmqxfppo9pswbm6kg9d4s4mbraqn8i4m7ob9gnzz91aurmuho48jx3c"],
    ["payto", "unknowntype", "l7tbta5b9xze6ckkfc99uohzxd009b0r"]
  ],
  ...
}
```

For each `payto` tag in kind `10133` events, clients can render the payment address as a button or link. Clients may use a specific URI scheme when one is available, such as `bitcoin:<address>` or `ethereum:<address>`. Otherwise fall back to the `payto` URI scheme defined by [RFC-8905](): `payto://<type>/<address>`.

Possible rendered URIs: - `bitcoin:bc1qxq66e0t8d7ugdecwnmv58e90tpry23nc84pg9k` - `payto://nano/nano_1dctqbmqxfppo9pswbm6kg9d4s4mbraqn8i4m7ob9gnzz91aurmuho48jx3c` - `payto://unknowntype/l7tbta5b9xze6ckkfc99uohzxd009b0r`

Commonly Used Tags

- `bip352` (silent payments)
- `bip353` (DNS addresses)
- `bitcoin`
- `cashme` (Cash App $-prefixed cashtag)
- `ethereum`
- `lightning` (lightning address)
- `litecoin`
- `monero`
- `nano`
- `paypal`
- `revolut`
- `solana`
- `venmo`
- `zcash`

New widely deployed formats can be added to this list later.

NIP-69

Peer-to-peer Order events

`draft` `optional`

Abstract

Peer-to-peer (P2P) platforms have seen an upturn in recent years, while having more and more options is positive, in the specific case of p2p, having several options contributes to the liquidity split, meaning sometimes there's not enough assets available for trading. If we combine all these individual solutions into one big pool of orders, it will make them much more competitive compared to centralized systems, where a single authority controls the liquidity.

This NIP defines a simple standard for peer-to-peer order events, which enables the creation of a big liquidity pool for all p2p platforms participating.

The event

Events are addressable events and use `38383` as event kind, a p2p event look like this:

```json
{
  "id": "84fad0d29cb3529d789faeff2033e88fe157a48e071c6a5d1619928289420e31",
  "pubkey": "dbe0b1be7aafd3cfba92d7463edbd4e33b2969f61bd554d37ac56f032e13355a",
  "created_at": 1702548701,
  "kind": 38383,
  "tags": [
    ["d", "ede61c96-4c13-4519-bf3a-dcf7f1e9d842"],
    ["k", "sell"],
    ["f", "VES"],
    ["s", "pending"],
    ["amt", "0"],
    ["fa", "100"],
    ["pm", "face to face", "bank transfer"],
    ["premium", "1"],
    [
      "rating",
      "{\"total_reviews\":1,\"total_rating\":3.0,\"last_rating\":3,\"max_rate\":5,\"min_rate\":1}"

    ],
    ["source", "https://t.me/p2plightning/xxxxxxx"],
    ["network", "mainnet"],
    ["layer", "lightning"],
    ["name", "Nakamoto"],
    ["g", "<geohash>"],
    ["bond", "0"],
    ["expires_at", "1719391096"],
    ["expiration", "1719995896"],
    ["y", "lnp2pbot"],
    ["z", "order"]
  ],
  "content": "",
  "sig":
      "7e8fe1eb644f33ff51d8805c02a0e1a6d034e6234eac50ef7a7e0dac68a0414f7910366204fa8217086f90eddaa37ded71e

}
```

Tags

- `d` < Order ID >: A unique identifier for the order.
- `k` < Order type >: `sell` or `buy`.
- `f` < Currency >: The asset being traded, using the [ISO 4217] standard.
- `s` < Status >: `pending`, `canceled`, `in-progress`, `success`, `expired`.
- `amt` < Amount >: The amount of Bitcoin to be traded, the amount is defined in satoshis, if `0` means that the amount of satoshis will be obtained from a public API after the taker accepts the order.
- `fa` < Fiat amount >: The fiat amount being traded, for range orders two values are expected, the minimum and maximum amount.
- `pm` < Payment method >: The payment method used for the trade, if the order has multiple payment methods, they should be separated by a comma.
- `premium` < Premium >: The percentage of the premium the maker is willing to pay.
- `source` [Source]: The source of the order, it can be a URL that redirects to the order.
- `rating` [Rating]: The rating of the maker, this document does not define how the rating is calculated, it's up to the platform to define it.
- `network` < Network >: The network used for the trade, it can be `mainnet`, `testnet`, `signet`, etc.
- `layer` < Layer >: The layer used for the trade, it can be `onchain`, `lightning`, `liquid`, etc.
- `name` [Name]: The name of the maker.
- `g` [Geohash]: The geohash of the operation, it can be useful in a face to face trade.
- `bond` [Bond]: The bond amount, the bond is a security deposit that both parties must pay.
- `expires_at` < Expires At>: The expiration date of the event being published in `pending` status, after this time the event status SHOULD be changed to `expired`.
- `expiration` < Expiration>: The expiration date of the event, after this time the relay SHOULD delete it ([NIP-40]).
- `y` < Platform >: The platform that created the order.
- `z` < Document >: `order`.

Mandatory tags are enclosed with `<tag>`, optional tags are enclosed with `[tag]`.

Implementations

Currently implemented on the following platforms:

- [Mostro]
- [@lnp2pBot]
- [Robosats]
- [Peach Bitcoin]

References

- [Mostro protocol specification]
- [Messages specification for peer 2 peer NIP proposal]
- [n3xB]

Third Parties

Sometimes you want someone — or some program — to act near your key without holding the key. These NIPs are the protocol's ways to wrap, delegate, and outsource. They are powerful and easy to get wrong. Read them with a slightly suspicious eye.

NIP-59 is gift wrap: put an event inside a sealed, addressed wrapper so relays and bystanders cannot tell who wrote the inner note or who it is for. Private DMs (NIP-17) depend on this. The wrapper is its own event; the rumor inside is what you actually meant to say.

NIP-46 is remote signing, often called Nostr Connect. A bunker or a signer holds the key; an app asks it to sign. That is how you can use a web client without pasting an `nsec` into the browser. NIP-26 is an older idea, delegated event signing, where a token lets another key sign as you for a while. It is unrecommended: the extra burden is real and the gain is small compared to a remote signer.

NIP-90 is data vending machines: a general job market where you pay a service to produce an event (transcription, translation, image generation, and so on). It is unrecommended. The spec grew until it was trying to be every API at once. Prefer a small, use-case-specific convention over a universal machine.

In this chapter

- NIP-59 — Gift wrap
- NIP-46 — Nostr remote signing
- NIP-26 — Delegated event signing (unrecommended; little gain for the complexity)
- NIP-90 — Data vending machines (unrecommended; prefer smaller, specific standards)

NIP-59

Gift Wrap

`optional` `relay`

This NIP defines a protocol for encapsulating any nostr event. This makes it possible to obscure most metadata for a given event, perform collaborative signing, and more.

This NIP *does not* define any messaging protocol. Applications of this NIP should be defined separately.

This NIP relies on [NIP-44](#)'s versioned encryption algorithms.

Overview

This protocol uses three main concepts to protect the transmission of a target event: `rumors`, `seals`, and `gift wraps`.

- A `rumor` is a regular nostr event, but is **not signed**. This means that if it is leaked, it cannot be verified.

- A `rumor` is serialized to JSON, encrypted, and placed in the `content` field of a `seal`. The `seal` is then signed by the author of the note. The only information publicly available on a `seal` is who signed it, but not what was said.
- A `seal` is serialized to JSON, encrypted, and placed in the `content` field of a `gift wrap`.

This allows the isolation of concerns across layers:

- A rumor carries the content but is unsigned, which means if leaked it will be rejected by relays and clients, and can't be authenticated. This provides a measure of deniability.
- A seal identifies the author without revealing the content or the recipient.
- A gift wrap can add metadata (recipient, tags, a different author) without revealing the true author.

Protocol Description

1. The Rumor Event Kind

A `rumor` is the same thing as an unsigned event. Any event kind can be made a `rumor` by removing the signature.

2. The Seal Event Kind

A `seal` is a `kind:13` event that wraps a `rumor` with the sender's regular key. The `seal` is **always** encrypted to a receiver's pubkey but there is no `p` tag pointing to the receiver. There is no way to know who the rumor is for without the receiver's or the sender's private key. The only public information in this event is who is signing it.

```json
{
  "id": "<id>",
  "pubkey": "<real author's pubkey>",
  "content": "<encrypted rumor>",
  "kind": 13,
  "created_at": 1686840217,
  "tags": [],
  "sig": "<real author's pubkey signature>"
}
```

Tags MUST always be empty in a `kind:13`. The inner event MUST always be unsigned.

3. Gift Wrap Event Kind

A `gift wrap` event is a `kind:1059` event that wraps any other event. `tags` SHOULD include any information needed to route the event to its intended recipient, including the recipient's `p` tag or [NIP-13](NIP-13) proof of work.

```json
{
  "id": "<id>",
  "pubkey": "<random, one-time-use pubkey>",
  "content": "<encrypted kind 13>",
  "kind": 1059,
  "created_at": 1686840217,
  "tags": [["p", "<recipient pubkey>"]],
  "sig": "<random, one-time-use pubkey signature>"
}
```

4. Ephemeral Gift Wrap Event Kind

An ephemeral gift wrap is a `kind:21059` event. It has the same structure as `kind:1059` but follows ephemeral event semantics (relays MUST NOT store it).

```
{
  "id": "<id>",
  "pubkey": "<random, one-time-use pubkey>",
  "content": "<encrypted kind 13>",
  "kind": 21059,
  "created_at": 1686840217,
  "tags": [["p", "<recipient pubkey>"]],
  "sig": "<random, one-time-use pubkey signature>"
}
```

`kind:1059` is intended for regular direct messages and asynchronous communication where message persistence and offline delivery are expected, such as in NIP-17. `kind:21059` is intended for real-time applications that do not require asynchronous operations and where messages are only relevant to currently connected recipients, such as live chat or real-time gaming.

Encrypting Payloads

Encryption is done following NIP-44 on the JSON-encoded event. Place the encryption payload in the `.content` of the wrapper event (either a `seal` or a `gift wrap`).

Spam Protection

Gift wrap events are signed by random one-time keys, so relays cannot rely on pubkey-based anti-spam or anti-sybil measures (e.g. reputation, trust scores, rate limits by key). To mitigate this, a relay may enforce a NIP-42 `auth-required` policy and authenticate clients before accepting gift wraps. Therefore clients SHOULD support authenticating to such relays using the user's real identity. Because NIP-42 authentication is ephemeral (it is not recorded in the event signature), it does not fully undermine the privacy or deniability properties of gift wrapping.

Other Considerations

If a `rumor` is intended for more than one party, or if the author wants to retain an encrypted copy, a single `rumor` may be wrapped and addressed for each recipient individually.

The canonical `created_at` time belongs to the `rumor`. All other timestamps SHOULD be tweaked to thwart time-analysis attacks. Note that some relays don't serve events dated in the future, so all timestamps SHOULD be in the past.

Relays may choose not to store gift wrapped events due to them not being publicly useful. Clients MAY choose to attach a certain amount of proof-of-work to the wrapper event per NIP-13 in a bid to demonstrate that the event is not spam or a denial-of-service attack.

To protect recipient metadata, relays SHOULD guard access to `kind 1059` events based on user AUTH. When possible, clients should only send wrapped events to relays that offer this protection.

To protect recipient metadata, relays SHOULD only serve `kind 1059` events intended for the marked recipient. When possible, clients should only send wrapped events to `read` relays for the recipient that implement AUTH, and refuse to serve wrapped events to non-recipients.

When adding expiration tags to both `seal` and `gift wrap` layers, implementations SHOULD use independent random timestamps for each layer. Using different `created_at` values increases timing variance and helps protect against metadata correlation attacks.

Since signing keys are random, relays SHOULD delete `kind:1059` events whose p-tag matches the signer of NIP-09 deletions or NIP-62 vanish requests.

An Example

Let's send a wrapped `kind 1` message between two parties asking "Are you going to the party tonight?"

- Author private key: `0beebd062ec8735f4243466049d7747ef5d6594ee838de147f8aab842b15e273`
- Recipient private key: `e108399bd8424357a710b606ae0c13166d853d327e47a6e5e038197346bdbf45`
- Ephemeral wrapper key: `4f02eac59266002db5801adc5270700ca69d5b8f761d8732fab2fbf233c90cbd`

Note that this messaging protocol should not be used in practice, this is just an example. Refer to other NIPs for concrete messaging protocols that depend on gift wraps.

1. Create an event

Create a `kind 1` event with the message, the receivers, and any other tags you want, signed by the author. Do not sign the event.

```
{
  "created_at": 1691518405,
  "content": "Are you going to the party tonight?",
  "tags": [],
  "kind": 1,
  "pubkey": "611df01bfcf85c26ae65453b772d8f1dfd25c264621c0277e1fc1518686faef9",
  "id": "9dd003c6d3b73b74a85a9ab099469ce251653a7af76f523671ab828acd2a0ef9"
}
```

2. Seal the rumor

Encrypt the JSON-encoded `rumor` with a conversation key derived using the author's private key and the recipient's public key. Place the result in the `content` field of a `kind 13` `seal` event. Sign it with the author's key.

```
{
  "content": "AqBCdwoS7/tPK+QGkPCadJTn8FxGkd24iApo3BR9/M0uw6n4RFAFSPAKKMgkzVMoRyR3ZS/aqATDFvoZJOkE9cPG/TAzmy
        WUIS8kLmuI1dCA+itFF6+ULZqbkWS0YcVU0j6UDvMBvVlGTzHz+UHzWYJLUq2LnlynJtFap5k8560+tBGtxi9Gx2NIycKgbOUv0g
        eVEO9aJJnqagomFSkqCscttf/o6VeT2+A9JhcSxLmjcKFG3FEK3Try/WkarJa1jM3lMRQqVOZrzHAaLFW/5sXano6DqqC5ERD6Cc
        v5Wsy31+BwZA8cUlfAZ0f5EYRo9/vKSd8TV0wRb9DQ=",

  "kind": 13,
  "created_at": 1703015180,
  "pubkey": "611df01bfcf85c26ae65453b772d8f1dfd25c264621c0277e1fc1518686faef9",
  "tags": [],
```

```
  "id": "28a87d7c074d94a58e9e89bb3e9e4e813e2189f285d797b1c56069d36f59eaa7",
  "sig":
        "02fc3facf6621196c32912b1ef53bac8f8bfe9db51c0e7102c073103586b0d29c3f39bdaa1e62856c20e90b6c7cc5dc34ca

}
```

3. Wrap the seal

Encrypt the JSON-encoded `kind 13` event with your ephemeral, single-use random key. Place the result in the `content` field of a `kind 1059`. Add a single `p` tag containing the recipient's public key. Sign the `gift wrap` using the random key generated in the previous step.

```
{
  "content": "AhC3Qj/QsKJFWuf6xroiYip+2yK95qPwJjVvFujhzSguJWb/6TlPpBW0CGFwfufCs2Zyb0JeuLmZhNlnqecAAalC4ZCugB
        DAPmzmMVx0SR63frRTCz6Cuth40D+VzluKu1/Fg2Q1LSst65DE7o2efTtZ4Z9j15rQAOZfE9jwMCQZt27rBBK3yVwqVEriFpg2mH
        o+FS3gG1dG7gJHu3KkGK5UXpmgyFKt+421m5o++RMD/BylS3iazS1S93IzTLeGfMCk+7IKxuSCO06k1+DaasJJe8RE4/rmismUvw
        HDutZWkvOAhd4z4khZo7bJLtiCzZCZ74lZcjOB4CYtuAX2ZGpc4I1iOKkvwTuQy9BWYpkzGg3ZoSWRD6ty7U+KN+fTTmIS4CelhE
        ODnI4hpwj8didtk7IMBI3Ra3uUP7ya6vptkd9TwQkd/7cOFaSJmU+BIsLpOXbirJACMn+URoDXhuEtiO6xirNtrPN8jYqpwvMUm5
        2VUUoKCGe0kfCcX/EsHbJLUUtlHXmTqaOJpmQnW1tZ/siPwKRl6oEsIJWTUYxPQmrM2fUpYZCuAo/29lTLHiHMlTbarFOd6J/ybI
        LFSryty3Cnf6aae+A9uizFBUdCwTwffc3vCBae802+R92OL78bbqHKPbSZOXNC+6ybqziezwG+OPWHx1Qk39RYaF0aFsM4uZWrFi
        laWKmS3e7wFzBsG8+qwqwm09aVbDVMhOmeUXRMkxcj4QreQkHxLkCx97euZpC7xhvYnCHarHTDeD6nVK+xzbPNtzeGzNpYoiMqxZ
        IZ1Qa3lY08Nqm9MAGxZ2Ou6R0/Z5z30ha/Q71q6meAs3uHQcpSuRaQeV29IASmye2A2Nif+lmbhV7w8hjFYoaLCRsdchiVyNjOEM

  "kind": 1059,
  "created_at": 1703021488,
  "pubkey": "18b1a75918f1f2c90c23da616bce317d36e348bcf5f7ba55e75949319210c87c",
  "id": "5c005f3ccf01950aa8d131203248544fb1e41a0d698e846bd419cec3890903ac",
  "sig":
        "35fabdae4634eb630880a1896a886e40fd6ea8a60958e30b89b33a93e6235df750097b04f9e13053764251b8bc5dd7e8e07

  "tags": [["p", "166bf3765ebd1fc55decfe395beff2ea3b2a4e0a8946e7eb578512b555737c99"]],
}
```

4. Broadcast Selectively

Broadcast the `kind:1059` or `kind:21059` event to the recipient's relays only. Delete all the other events.

Code Samples

JavaScript

```javascript
import {bytesToHex} from "@noble/hashes/utils"
import type {EventTemplate, UnsignedEvent, Event} from "nostr-tools"
import {getPublicKey, getEventHash, nip19, nip44, finalizeEvent, generateSecretKey} from "nostr-
        tools"

type Rumor = UnsignedEvent & {id: string}

const TWO_DAYS = 2 * 24 * 60 * 60

const now = () => Math.round(Date.now() / 1000)
const randomNow = () => Math.round(now() - (Math.random() * TWO_DAYS))

const nip44ConversationKey = (privateKey: Uint8Array, publicKey: string) =>
  nip44.v2.utils.getConversationKey(bytesToHex(privateKey), publicKey)
```

```
const nip44Encrypt = (data: EventTemplate, privateKey: Uint8Array, publicKey: string) =>
  nip44.v2.encrypt(JSON.stringify(data), nip44ConversationKey(privateKey, publicKey))

const nip44Decrypt = (data: Event, privateKey: Uint8Array) =>
  JSON.parse(nip44.v2.decrypt(data.content, nip44ConversationKey(privateKey, data.pubkey)))

const createRumor = (event: Partial<UnsignedEvent>, privateKey: Uint8Array) => {
  const rumor = {
    created_at: now(),
    content: "",
    tags: [],
    ...event,
    pubkey: getPublicKey(privateKey),
  } as any

  rumor.id = getEventHash(rumor)

  return rumor as Rumor
}

const createSeal = (rumor: Rumor, privateKey: Uint8Array, recipientPublicKey: string) => {
  return finalizeEvent(
    {
      kind: 13,
      content: nip44Encrypt(rumor, privateKey, recipientPublicKey),
      created_at: randomNow(),
      tags: [],
    },
    privateKey
  ) as Event
}

const createWrap = (event: Event, recipientPublicKey: string) => {
  const randomKey = generateSecretKey()

  return finalizeEvent(
    {
      kind: 1059,
      content: nip44Encrypt(event, randomKey, recipientPublicKey),
      created_at: randomNow(),
      tags: [["p", recipientPublicKey]],
    },
    randomKey
  ) as Event
}

const createEphemeralWrap = (event: Event, recipientPublicKey: string) => {
  const randomKey = generateSecretKey()

  return finalizeEvent(
    {
      kind: 21059,
      content: nip44Encrypt(event, randomKey, recipientPublicKey),
      created_at: randomNow(),
```

```
      tags: [["p", recipientPublicKey]],
    },
    randomKey
  ) as Event
}

// Test case using the above example
const senderPrivateKey =
        nip19.decode(`nsec1p0ht6p3wepe47sjrgesyn4m50m6avk2waqudu9rl324cg2c4ufesyp6rdg`).data
const recipientPrivateKey =
        nip19.decode(`nsec1uyyrnx7cgfp40fcskcr2urqnzekc20fj0er6de0q8qvhx34ahazsvs9p36`).data
const recipientPublicKey = getPublicKey(recipientPrivateKey)

const rumor = createRumor(
  {
    kind: 1,
    content: "Are you going to the party tonight?",
  },
  senderPrivateKey
)

const seal = createSeal(rumor, senderPrivateKey, recipientPublicKey)
const wrap = createWrap(seal, recipientPublicKey)

// Recipient unwraps with their private key.

const unwrappedSeal = nip44Decrypt(wrap, recipientPrivateKey)
const unsealedRumor = nip44Decrypt(unwrappedSeal, recipientPrivateKey)
```

NIP-46

Nostr Remote Signing

Changes

`remote-signer-key` is introduced, passed in bunker url, clients must differentiate between `remote-signer-pubkey` and `user-pubkey`, must call `get_public_key` after connect, nip05 login is removed, create_account moved to another NIP.

Rationale

Private keys should be exposed to as few systems - apps, operating systems, devices - as possible as each system adds to the attack surface.

This NIP describes a method for 2-way communication between a remote signer and a Nostr client. The remote signer could be, for example, a hardware device dedicated to signing Nostr events, while the client is a normal Nostr client.

Terminology

- **user**: A person that is trying to use Nostr.
- **client**: A user-facing application that *user* is looking at and clicking buttons in. This application will send requests to *remote-signer*.
- **remote-signer**: A daemon or server running somewhere that will answer requests from *client*, also known as "bunker".
- **client-keypair/pubkey**: The keys generated by *client*. Used to encrypt content and communicate with *remote-signer*.
- **remote-signer-keypair/pubkey**: The keys used by *remote-signer* to encrypt content and communicate with *client*. This keypair MAY be same as *user-keypair*, but not necessarily.
- **user-keypair/pubkey**: The actual keys representing *user* (that will be used to sign events in response to `sign_event` requests, for example). The *remote-signer* generally has control over these keys.

All pubkeys specified in this NIP are in hex format.

Overview

1. *client* generates `client-keypair`. This keypair doesn't need to be communicated to *user* since it's largely disposable. *client* might choose to store it locally and they should delete it on logout;
2. A connection is established (see below), *remote-signer* learns `client-pubkey`, *client* learns `remote-signer-pubkey`.
3. *client* uses `client-keypair` to send requests to *remote-signer* by `p`-tagging and encrypting to `remote-signer-pubkey`;
4. *remote-signer* responds to *client* by `p`-tagging and encrypting to the `client-pubkey`.
5. *client* requests `get_public_key` to learn `user-pubkey`.

Initiating a connection

There are two ways to initiate a connection:

Direct connection initiated by *remote-signer*

remote-signer provides connection token in the form:

```
bunker://<remote-signer-pubkey>?relay=<wss://relay-to-connect-on>&relay=<wss://another-relay-to-connect-on>&secret=<optional-secret-value>
```

user passes this token to *client*, which then sends `connect` request to *remote-signer* via the specified relays. When connecting this way *client* SHOULD include `optional_client_metadata` in the `connect` request (see [Client metadata]), since unlike `nostrconnect://` this token carries no information identifying *client*. Optional secret can be used for single successfully established connection only, *remote-signer* SHOULD ignore new attempts to establish connection with old secret.

Direct connection initiated by the *client*

client provides a connection token using `nostrconnect://` as the protocol, and `client-pubkey` as the origin. Additional information should be passed as query parameters:

- `relay` (required) - one or more relay urls on which the *client* is listening for responses from the *remote-signer*.
- `secret` (required) - a short random string that the *remote-signer* should return as the `result` field of its response.
- `perms` (optional) - a comma-separated list of permissions the *client* is requesting be approved by the *remote-signer*
- `name` (optional) - the name of the *client* application
- `url` (optional) - the canonical url of the *client* application
- `image` (optional) - a small image representing the *client* application

Here's an example:

```
nostrconnect://83f3b2ae6aa368e8275397b9c26cf550101d63ebaab900d19dd4a4429f5ad8f5?relay=wss%3A%2F%2Frelay1.example.com&perms=nip44_encrypt%2Cnip44_decrypt%2Csign_event%3A13%2Csign_event%3A14%2Csign_event%3A1059&name=My+Client&secret=0s8j2djs&relay=wss%3A%2F%2Frelay2.example2.com
```

user passes this token to *remote-signer*, which then sends `connect` *response* event to the `client-pubkey` via the specified relays. Client discovers `remote-signer-pubkey` from connect response author. `secret` value MUST be provided to avoid connection spoofing, *client* MUST validate the `secret` returned by `connect` response.

Request Events `kind: 24133`

```json
{
    "kind": 24133,
    "pubkey": <local_keypair_pubkey>,
    "content": <nip44(<request>)>,
    "tags": [["p", <remote-signer-pubkey>]],
}
```

The `content` field is a JSON-RPC-like message that is [NIP-44](NIP-44) encrypted and has the following structure:

```json
{
    "id": <random_string>,
    "method": <method_name>,
    "params": [array_of_strings]
}
```

- `id` is a random string that is a request ID. This same ID will be sent back in the response payload.
- `method` is the name of the method/command (detailed below).
- `params` is a positional array of string parameters.

Methods/Commands

Each of the following are methods that the *client* sends to the *remote-signer*.

Command	Params	Result
connect	[<remote-signer-pubkey>, <optional_secret>, <optional_requested_perms>, <optional_client_metadata>]	"ack" OR <required-secret-value>
sign_event	[<{kind, content, tags, created_at}>]	json_stringified(<signed_event>)
ping	[]	"pong"
get_public_key	[]	<user-pubkey>
nip04_encrypt	[<third_party_pubkey>, <plaintext_to_encrypt>]	<nip04_ciphertext>
nip04_decrypt	[<third_party_pubkey>, <nip04_ciphertext_to_decrypt>]	<plaintext>
nip44_encrypt	[<third_party_pubkey>, <plaintext_to_encrypt>]	<nip44_ciphertext>
nip44_decrypt	[<third_party_pubkey>, <nip44_ciphertext_to_decrypt>]	<plaintext>
switch_relays	[]	["<relay-url>", "<relay-url>", ...] OR null
logout	[]	"ack"

Requested permissions

The `connect` method may be provided with `optional_requested_perms` for user convenience. The permissions are a comma-separated list of `method[:params]`, i.e. `nip44_encrypt,sign_event:4` meaning permissions to call `nip44_encrypt` and to call `sign_event` with `kind:4`. Optional parameter for `sign_event` is the kind number, parameters for other methods are to be defined later. Same permission format may be used for `perms` field of `metadata` in `nostrconnect://` string.

Client metadata

The `connect` method may be provided with `optional_client_metadata` so that *remote-signer* can identify and label the connection. It is a JSON-stringified object whose fields mirror the `nostrconnect://` query parameters, all optional:

- `name` - the name of the *client* application
- `url` - the canonical url of the *client* application
- `image` - a small image representing the *client* application

This is most useful for a connection initiated by *remote-signer* (the `bunker://` flow), where — unlike `nostrconnect://` — *remote-signer* has no other source for this information. To send metadata without requesting permissions, an empty string MUST be passed for `optional_requested_perms` so that the metadata occupies the fourth position. The metadata is *client*-supplied and unauthenticated; *remote-signer* SHOULD treat it as a display hint only and MUST NOT use it for authorization decisions.

Switching relays

At all times, the *remote-signer* should be in control of what relays are being used for the connection between it and the *client*. Therefore it should be possible for it to evolve its set of relays over time as old relays go out of operation and new ones appear. Even more importantly, in the case of the connection initiated by the *client* the client may pick relays completely foreign to the *remote-signer*'s preferences, so it must be possible for it to switch those immediately.

Therefore, compliant clients should send a `switch_relays` request immediately upon establishing a connection (always, or at reasonable intervals). Upon receiving such requests, the *remote-signer* should reply with its updated list of relays, or `null` if there is nothing to be changed. Immediately upon receiving an updated relay list, the *client* should update its local state and send further requests on the new relays. The `remote-signer` should then be free to disconnect from the previous relays if that is desired.

Ending a session

When the *user* logs out of the *client*, the *client* MAY send a `logout` request to inform the *remote-signer* that the session is no longer needed. Upon receiving such a request, the *remote-signer* SHOULD remove the session associated with the requesting `client-pubkey` and reject any further requests from it (a new `connect` would be required to re-establish a session). The *remote-signer* should reply with `"ack"` before removing the session.

The `logout` request is a courtesy hint and is not a security boundary: the *remote-signer* MUST NOT rely on it being sent, and the *client* MUST still delete its locally stored `client-keypair` on logout regardless of whether the request was acknowledged.

Response Events `kind:24133`

```
{
    "id": <id>,
    "kind": 24133,
    "pubkey": <remote-signer-pubkey>,
    "content": <nip44(<response>)>,
    "tags": [["p", <client-pubkey>]],
    "created_at": <unix timestamp in seconds>
}
```

The `content` field is a JSON-RPC-like message that is [NIP-44] encrypted and has the following structure:

```
{
    "id": <request_id>,
    "result": <results_string>,
    "error": <optional_error_string>
}
```

- `id` is the request ID that this response is for.
- `results` is a string of the result of the call (this can be either a string or a JSON stringified object)
- `error`, *optionally*, it is an error in string form, if any. Its presence indicates an error with the request.

Requests made with unknown or unsupported methods MUST be replied with an error.

Example flow for signing an event

- `remote-signer-pubkey` is `fa984bd7dbb282f07e16e7ae87b26a2a7b9b90b7246a44771f0cf5ae58018f52`
- `user-pubkey` is also `fa984bd7dbb282f07e16e7ae87b26a2a7b9b90b7246a44771f0cf5ae58018f52`
- `client-pubkey` is `eff37350d839ce3707332348af4549a96051bd695d3223af4aabce4993531d86`

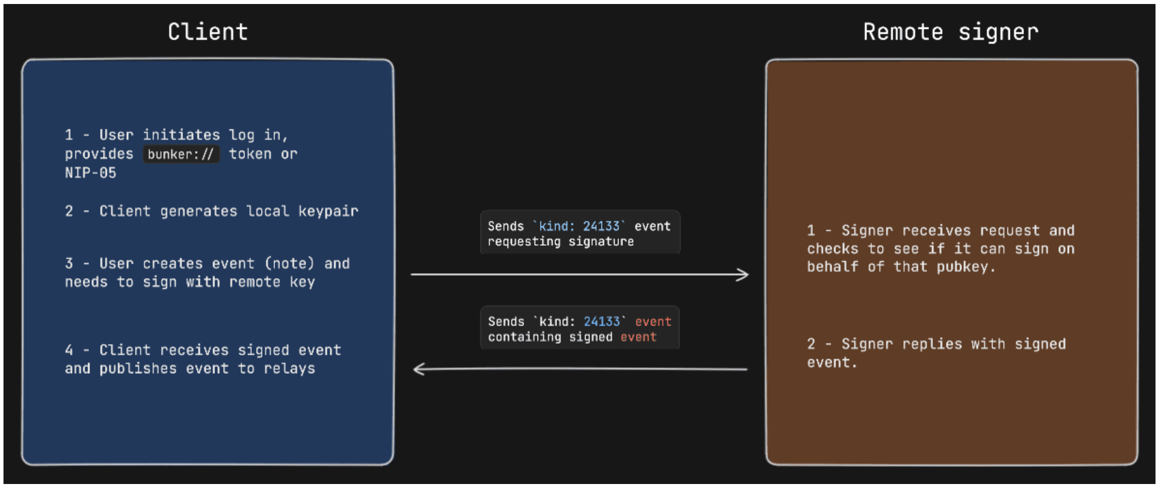
Signature request

```
{
    "kind": 24133,
    "pubkey": "eff37350d839ce3707332348af4549a96051bd695d3223af4aabce4993531d86",
    "content": nip44({
        "id": <random_string>,
        "method": "sign_event",
        "params": [json_stringified(<{
            content: "Hello, I'm signing remotely",
            kind: 1,
            tags: [],
            created_at: 1714078911
        }>)]
    }),
    "tags": [["p", "fa984bd7dbb282f07e16e7ae87b26a2a7b9b90b7246a44771f0cf5ae58018f52"]], // p-
        tags the remote-signer-pubkey
}
```

Response event

```
{
    "kind": 24133,
    "pubkey": "fa984bd7dbb282f07e16e7ae87b26a2a7b9b90b7246a44771f0cf5ae58018f52",
    "content": nip44({
        "id": <random_string>,
        "result": json_stringified(<signed-event>)
    }),
    "tags": [["p", "eff37350d839ce3707332348af4549a96051bd695d3223af4aabce4993531d86"]], // p-
        tags the client-pubkey
}
```

Diagram



signing-example

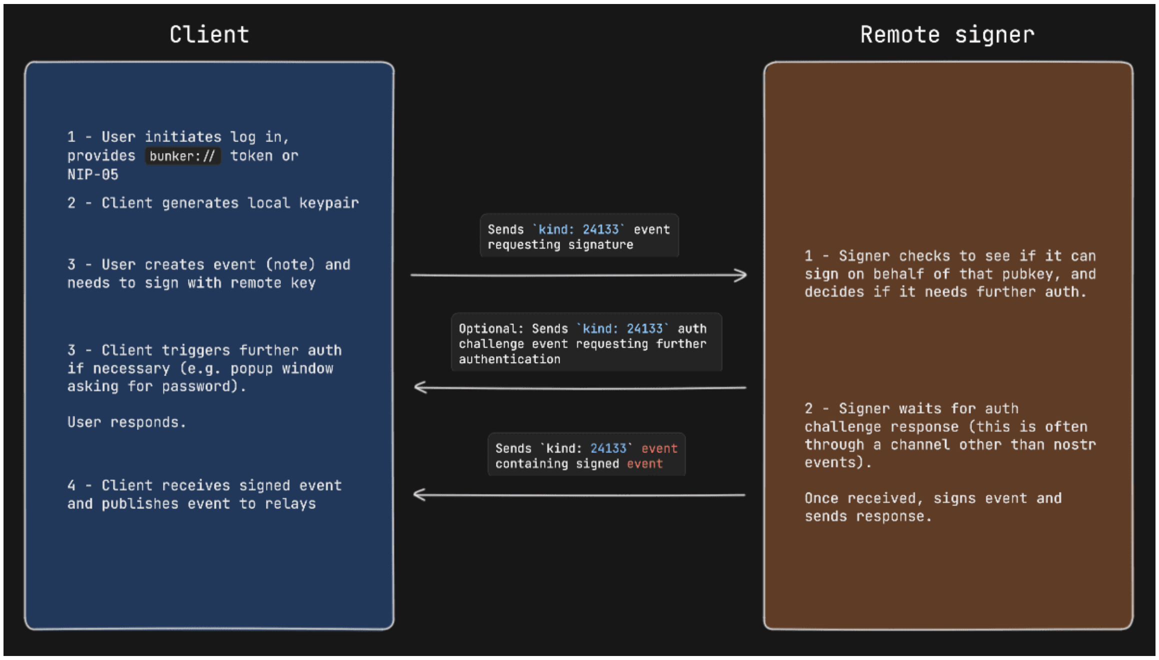
Auth Challenges

An Auth Challenge is a response that a *remote-signer* can send back when it needs the *user* to authenticate via other means. The response `content` object will take the following form:

```
{
    "id": <request_id>,
    "result": "auth_url",
    "error": <URL_to_display_to_end_user>
}
```

client should display (in a popup or new tab) the URL from the `error` field and then subscribe/listen for another response from the *remote-signer* (reusing the same request ID). This event will be sent once the user authenticates in the other window (or will never arrive if the user doesn't authenticate).

Example event signing request with auth challenge



signing-example-with-auth-challenge

Appendix

Announcing *remote-signer* metadata

remote-signer MAY publish its metadata by using [NIP-05](NIP-05) and [NIP-89](NIP-89). With NIP-05, a request to `<remote-signer>/.well-known/nostr.json?name=_` MAY return this:

```
{
    "names":{
        "_": <remote-signer-app-pubkey>,
    },
    "nip46": {
        "relays": ["wss://relay1","wss://relay2"...],
        "nostrconnect_url": "https://remote-signer-domain.example/<nostrconnect>"
    }
}
```

The `<remote-signer-app-pubkey>` MAY be used to verify the domain from *remote-signer*'s NIP-89 event (see below). `relays` SHOULD be used to construct a more precise `nostrconnect://` string for the specific `remote-signer`. `nostrconnect_url` template MAY be used to redirect users to *remote-signer*'s connection flow by replacing `<nostrconnect>` placeholder with an actual `nostrconnect://` string.

Remote signer discovery via NIP-89

remote-signer MAY publish a NIP-89 `kind: 31990` event with `k` tag of `24133`, which MAY also include one or more `relay` tags and MAY include `nostrconnect_url` tag. The semantics of `relay` and `nostrconnect_url` tags are the same as in the section above.

client MAY improve UX by discovering *remote-signers* using their `kind: 31990` events. *client* MAY then pre-generate `nostrconnect://` strings for the *remote-signers*, and SHOULD in that case verify that `kind: 31990` event's author is mentioned in signer's `nostr.json?name=_` file as `<remote-signer-app-pubkey>`.

> **Warning** `unrecommended`: adds unnecessary burden for little gain

NIP-26

Delegated Event Signing

`draft` `unrecommended` `optional` `relay`

This NIP defines how events can be delegated so that they can be signed by other keypairs.

Another application of this proposal is to abstract away the use of the 'root' keypairs when interacting with clients. For example, a user could generate new keypairs for each client they wish to use and authorize those keypairs to generate events on behalf of their root pubkey, where the root keypair is stored in cold storage.

Introducing the 'delegation' tag

This NIP introduces a new tag: `delegation` which is formatted as follows:

```
[
  "delegation",
  <pubkey of the delegator>,
  <conditions query string>,
  <delegation token: 64-byte Schnorr signature of the sha256 hash of the delegation string>
]
```

Delegation Token

The **delegation token** should be a 64-byte Schnorr signature of the sha256 hash of the following string:

```
nostr:delegation:<pubkey of publisher (delegatee)>:<conditions query string>
```

Conditions Query String

The following fields and operators are supported in the above query string:

Fields: 1. `kind` - *Operators*: - `=${KIND_NUMBER}` - delegatee may only sign events of this kind 2. `created_at` - *Operators*: - `<${TIMESTAMP}` - delegatee may only sign events created ***before*** the specified timestamp - `>${TIMESTAMP}` - delegatee may only sign events created ***after*** the specified timestamp

In order to create a single condition, you must use a supported field and operator. Multiple conditions can be used in a single query string, including on the same field. Conditions must be combined with &.

For example, the following condition strings are valid:

- `kind=1&created_at<1675721813`
- `kind=0&kind=1&created_at>1675721813`

- `kind=1&created_at>1674777689&created_at<1675721813`

For the vast majority of use-cases, it is advisable that: 1. Query strings should include a `created_at` ***after*** condition reflecting the current time, to prevent the delegatee from publishing historic notes on the delegator's behalf. 2. Query strings should include a `created_at` ***before*** condition that is not empty and is not some extremely distant time in the future. If delegations are not limited in time scope, they expose similar security risks to simply using the root key for authentication.

Example

```
# Delegator:
privkey: ee35e8bb71131c02c1d7e73231daa48e9953d329a4b701f7133c8f46dd21139c
pubkey:  8e0d3d3eb2881ec137a11debe736a9086715a8c8beeeda615780064d68bc25dd

# Delegatee:
privkey: 777e4f60b4aa87937e13acc84f7abcc3c93cc035cb4c1e9f7a9086dd78fffce1
pubkey:  477318cfb5427b9cfc66a9fa376150c1ddbc62115ae27cef72417eb959691396
```

Delegation string to grant note publishing authorization to the delegatee (477318cf) from now, for the next 30 days, given the current timestamp is `1674834236`.

```
nostr:delegation:477318cfb5427b9cfc66a9fa376150c1ddbc62115ae27cef72417eb959691396:kind=1&created_at>16748342
```

The delegator (8e0d3d3e) then signs a SHA256 hash of the above delegation string, the result of which is the delegation token:

```
6f44d7fe4f1c09f3954640fb58bd12bae8bb8ff4120853c4693106c82e920e2b898f1f9ba9bd65449a987c39c0423426
ab7b53910c0c6abfb41b30bc16e5f524
```

The delegatee (477318cf) can now construct an event on behalf of the delegator (8e0d3d3e). The delegatee then signs the event with its own private key and publishes.

```json
{
  "id": "e93c6095c3db1c31d15ac771f8fc5fb672f6e52cd25505099f62cd055523224f",
  "pubkey": "477318cfb5427b9cfc66a9fa376150c1ddbc62115ae27cef72417eb959691396",
  "created_at": 1677426298,
  "kind": 1,
  "tags": [
    [
      "delegation",
      "8e0d3d3eb2881ec137a11debe736a9086715a8c8beeeda615780064d68bc25dd",
      "kind=1&created_at>1674834236&created_at<1677426236",
      "6f44d7fe4f1c09f3954640fb58bd12bae8bb8ff4120853c4693106c82e920e2b898f1f9ba9bd65449a987c39c0423426ab7b5

    ]
  ],
  "content": "Hello, world!",
  "sig":
        "633db60e2e7082c13a47a6b19d663d45b2a2ebdeaf0b4c35ef83be2738030c54fc7fd56d139652937cdca875ee61b51904a

}
```

The event should be considered a valid delegation if the conditions are satisfied (`kind=1`, `created_at>1674834236` and `created_at<1677426236` in this example) and, upon validation of the delegation token, are found to be unchanged from the conditions in the original delegation string.

Clients should display the delegated note as if it was published directly by the delegator (8e0d3d3e).

Relay & Client Support

Relays should answer requests such as `["REQ", "", {"authors": ["A"]}]` by querying both the `pubkey` and delegation tags `[1]` value.

Relays SHOULD allow the delegator (8e0d3d3e) to delete the events published by the delegatee (477318cf).

> **Warning** `unrecommended`: this got totally out of control, prefer use-case-specific microstandards

NIP-90

Data Vending Machine

`draft` `unrecommended` `optional`

This NIP defines the interaction between customers and Service Providers for performing on-demand computation.

Money in, data out.

Kinds

This NIP reserves the range `5000-7000` for data vending machine use.

Kind	Description
5000-5999	Job request kinds
6000-6999	Job result
7000	Job feedback

Job results always use a kind number that is `1000` higher than the job request kind. (e.g. request: `kind:5001` gets a result: `kind:6001`).

Job request types are defined [separately](https://github.com/nostr-protocol/data-vending-machines/tree/master/kinds).

Rationale

Nostr can act as a marketplace for data processing, where users request jobs to be processed in certain ways (e.g., “speech-to-text”, “summarization”, etc.), but they don’t necessarily care about “who” processes the data.

This NIP is not to be confused with a 1:1 marketplace; instead, it describes a flow where a user announces a desired output, willingness to pay, and service providers compete to fulfill the job requirement in the best way possible.

Actors

There are two actors in the workflow described in this NIP: * Customers (npubs who request a job) * Service providers (npubs who fulfill jobs)

Job request (`kind:5000-5999`)

A request to process data, published by a customer. This event signals that a customer is interested in receiving the result of some kind of compute.

```
{
    "kind": 5xxx, // kind in 5000-5999 range
    "content": "",
    "tags": [
        [ "i", "<data>", "<input-type>", "<relay>", "<marker>" ],
        [ "output", "<mime-type>" ],
        [ "relays", "wss://..." ],
        [ "bid", "<msat-amount>" ],
        [ "t", "bitcoin" ]
    ],
    // other fields...
}
```

All tags are optional.

- `i` tag: Input data for the job (zero or more inputs)
  - `<data>`: The argument for the input
  - `<input-type>`: The way this argument should be interpreted. MUST be one of:
    - `url`: A URL to be fetched of the data that should be processed.
    - `event`: A Nostr event ID.
    - `job`: The output of a previous job with the specified event ID. The determination of which output to build upon is up to the service provider to decide (e.g. waiting for a signaling from the customer, waiting for a payment, etc.)
    - `text`: `<data>` is the value of the input, no resolution is needed
  - `<relay>`: If `event` or `job` input-type, the relay where the event/job was published, otherwise optional or empty string
  - `<marker>`: An optional field indicating how this input should be used within the context of the job
- `output`: Expected output format. Different job request `kind` defines this more precisely.
- `param`: Optional parameters for the job as key (first argument)/value (second argument). Different job request `kind` defines this more precisely. (e.g. `[ "param", "lang", "es" ]`)
- `bid`: Customer MAY specify a maximum amount (in millisats) they are willing to pay
- `relays`: List of relays where Service Providers SHOULD publish responses to
- `p`: Service Providers the customer is interested in. Other SPs MIGHT still choose to process the job

Encrypted Params

If the user wants to keep the input parameters a secret, they can encrypt the `i` and `param` tags with the service provider's 'p' tag and add it to the content field. Add a tag `encrypted` as tags. Encryption for private tags will use NIP-04 - Encrypted Direct Message encryption, using the user's private and service provider's public key for the shared secret

```
[
  ["i", "what is the capital of France? ", "text"],
  ["param", "model", "LLaMA-2"],
  ["param", "max_tokens", "512"],
  ["param", "temperature", "0.5"],
  ["param", "top-k", "50"],
  ["param", "top-p", "0.7"],
  ["param", "frequency_penalty", "1"]
]
```

This param data will be encrypted and added to the `content` field and `p` tag should be present

```
{
  "content":
        "BE2Y4xvS6HIY7TozIgbEl3sAHkdZoXyLRRkZv4fLPh3R7LtviLKAJM5qpkC7D6VtMbgIt4iNcMpLtpo...",
  "tags": [
    ["p", "04f74530a6ede6b24731b976b8e78fb449ea61f40ff10e3d869a3030c4edc91f"],
    ["encrypted"]
  ],
  // other fields...
}
```

Job result (`kind:6000-6999`)

Service providers publish job results, providing the output of the job result. They should tag the original job request event id as well as the customer's pubkey.

```
{
  "pubkey": "<service-provider pubkey>",
  "content": "<payload>",
  "kind": 6xxx,
  "tags": [
    ["request", "<job-request>"],
    ["e", "<job-request-id>", "<relay-hint>"],
    ["i", "<input-data>"],
    ["p", "<customer's-pubkey>"],
    ["amount", "requested-payment-amount", "<optional-bolt11>"]
  ],
  // other fields...
}
```

- `request`: The job request event stringified-JSON.
- `amount`: millisats that the Service Provider is requesting to be paid. An optional third value can be a bolt11 invoice.
- `i`: The original input(s) specified in the request.

Encrypted Output

If the request has encrypted params, then output should be encrypted and placed in `content` field. If the output is encrypted, then avoid including `i` tag with input-data as clear text. Add a tag encrypted to mark the output content as `encrypted`

```json
{
  "pubkey": "<service-provider pubkey>",
  "content": "<encrypted payload>",
  "kind": 6xxx,
  "tags": [
    ["request", "<job-request>"],
    ["e", "<job-request-id>", "<relay-hint>"],
    ["p", "<customer's-pubkey>"],
    ["amount", "requested-payment-amount", "<optional-bolt11>"],
    ["encrypted"]
  ],
  // other fields...
}
```

Job feedback

Service providers can give feedback about a job back to the customer.

```json
{
  "kind": 7000,
  "content": "<empty-or-payload>",
  "tags": [
    ["status", "<status>", "<extra-info>"],
    ["amount", "requested-payment-amount", "<bolt11>"],
    ["e", "<job-request-id>", "<relay-hint>"],
    ["p", "<customer's-pubkey>"],
  ],
  // other fields...
}
```

- `content`: Either empty or a job-result (e.g. for partial-result samples)
- `amount` tag: as defined in the Job Result section.
- `status` tag: Service Providers SHOULD indicate what this feedback status refers to. Job Feedback Status defines status. Extra human-readable information can be added as an extra argument.
- NOTE: If the input params requires input to be encrypted, then `content` field will have encrypted payload with `p` tag as key.

Job feedback status

status	description
`payment-required`	Service Provider requires payment before continuing.
`processing`	Service Provider is processing the job.

status	description
`error`	Service Provider was unable to process the job.
`success`	Service Provider successfully processed the job.
`partial`	Service Provider partially processed the job. The `.content` might include a sample of the partial results.

Any job feedback event MIGHT include results in the `.content` field, as described in the Job Result section. This is useful for service providers to provide a sample of the results that have been processed so far.

Protocol Flow

- Customer publishes a job request (e.g. `kind:5000` speech-to-text).
- Service Providers MAY submit `kind:7000` job-feedback events (e.g. `payment-required`, `processing`, `error`, etc.).
- Upon completion, the service provider publishes the result of the job with a `kind:6000` job-result event.
- At any point, if there is an `amount` pending to be paid as instructed by the service provider, the user can pay the included `bolt11` or zap the job result event the service provider has sent to the user.

Job feedback (`kind:7000`) and Job Results (`kind:6000-6999`) events MAY include an `amount` tag, this can be interpreted as a suggestion to pay. Service Providers MUST use the `payment-required` feedback event to signal that a payment is required and no further actions will be performed until the payment is sent.

Customers can always either pay the included `bolt11` invoice or zap the event requesting the payment and service providers should monitor for both if they choose to include a bolt11 invoice.

Notes about the protocol flow

The flow is deliberately ambiguous, allowing vast flexibility for the interaction between customers and service providers so that service providers can model their behavior based on their own decisions/perceptions of risk.

Some service providers might choose to submit a `payment-required` as the first reaction before sending a `processing` or before delivering results, some might choose to serve partial results for the job (e.g. a sample), send a `payment-required` to deliver the rest of the results, and some service providers might choose to assess likelihood of payment based on an npub's past behavior and thus serve the job results before requesting payment for the best possible UX.

It's not up to this NIP to define how individual vending machines should choose to run their business.

Cancellation

A job request might be canceled by publishing a `kind:5` delete request event tagging the job request event.

Appendix 1: Job chaining

A Customer MAY request multiple jobs to be processed as a chain, where the output of a job is the input of another job. (e.g. podcast transcription -> summarization of the transcription). This is done by specifying as input an event id of a different job with the `job` type.

Service Providers MAY begin processing a subsequent job the moment they see the prior job's result, but they will likely wait for a zap to be published first. This introduces a risk that Service Provider of job #1 might delay publishing the zap event in order to have an advantage. This risk is up to Service Providers to mitigate or to decide whether the service provider of job #1 tends to have good-enough results so as to not wait for an explicit zap to assume the job was accepted.

This gives a higher level of flexibility to service providers (which sophisticated service providers would take anyway).

Appendix 2: Service provider discoverability

Service Providers MAY use NIP-89 announcements to advertise their support for job kinds:

```
{
  "kind": 31990,
  "pubkey": "<pubkey>",
  "content": "{
    \"name\": \"Translating DVM\",
    \"about\": \"I'm a DVM specialized in translating Bitcoin content.\"
  }",
  "tags": [
    ["k", "5005"], // e.g. translation
    ["t",
        "bitcoin"] // e.g. optionally advertises it specializes in bitcoin audio transcription
        that won't confuse "Drivechains" with "Ridechains"
  ],
  // other fields...
}
```

Customers can use NIP-89 to see what service providers their follows use.

Application Features

Nostr's event shape is boring on purpose. Almost any app can be "a kind, some tags, and some content." This chapter is the long tail of those apps: calendars, live rooms, wikis, files, git, games. You do not need them to speak the protocol. You need them if you are building that particular thing, or if you want your client to open it when someone else did.

Time and performance come first. NIP-52 is calendar events and RSVPs. NIP-53 is live streaming and audio spaces, including live chat. NIP-F4 is podcasts — episodes and metadata that a Nostr client can treat as a show, not just a file link.

Then publishing. Highlights (NIP-84) let you quote a passage and point back at the source. NIP-54 is a wiki. NIP-C0 is code snippets. NIP-73 is how to name something that lives off-Nostr — an ISBN, a DOI, a URL — so different clients can talk about the same book or paper. NIP-34 is git: repositories, patches, issues, and status, so you can host a workflow on relays instead of only on GitHub. NIP-5A is static websites (nsites), a way to publish a site as Nostr events.

Commerce and files sit together because both are about objects you might want to find later. NIP-15 is a full marketplace. It is unrecommended: too complicated; try classified listings (NIP-99) instead. File metadata (NIP-94) describes a blob. NIP-96 is HTTP file storage integration; it is unrecommended and replaced by Blossom (NIP-B7), which is how most new clients move images and video. NIP-35 is torrents.

Two playful NIPs show how far a kind can stretch. NIP-64 is chess games recorded as PGN. NIP-CC is geocaching. NIP-78 is the escape hatch: application-specific data, a place for an app to stash its own state without pretending the rest of the network understands it.

In this chapter

NIP-52

Calendar Events

`draft` `optional`

This specification defines calendar events representing an occurrence at a specific moment or between moments. These calendar events are *addressable* and deletable per [NIP-09](NIP-09).

Unlike the term `calendar event` specific to this NIP, the term `event` is used broadly in all the NIPs to describe any Nostr event. The distinction is being made here to discern between the two terms.

Calendar Events

There are two types of calendar events represented by different kinds: *date-based* and *time-based* calendar events.

These tags are common to both types of calendar events:

- `d` (required) a short unique string identifier. Generated by the client creating the calendar event.
- `title` (required) title of the calendar event
- `summary` (optional) brief description of the calendar event
- `image` (optional) url of an image to use for the event
- `location` (optional, repeated) location of the calendar event. e.g. address, GPS coordinates, meeting room name, link to video call
- `g` (optional) geohash to associate calendar event with a searchable physical location
- `p` (optional, repeated) 32-bytes hex pubkey of a participant, optional recommended relay URL, and participant's role in the meeting
- `t` (optional, repeated) hashtag to categorize calendar event
- `r` (optional, repeated) references / links to web pages, documents, video calls, recorded videos, etc.
- `a` (repeated) reference tag to kind `31924` calendar event requesting to be included in Calendar

The following tags are deprecated:

- `name` name of the calendar event. Use only if `title` is not available.

Calendar events are *not* required to be part of a calendar.

Collaborative Calendar Event Requests

Calendar events can include an `a` tag referencing a calendar (kind 31924) to request addition to that calendar. When a calendar event includes such a reference, clients should interpret this as a request to add the event to the referenced calendar by referencing it with an `a` tag.

This enables collaborative calendar management where multiple users can contribute events to calendars they do not own, subject to the calendar owner's approval.

Date-Based Calendar Event

This kind of calendar event starts on a date and ends before a different date in the future. Its use is appropriate for all-day or multi-day events where time and time zone hold no significance. e.g., anniversary, public holidays, vacation days.

It's an *addressable event* of `kind:31922`.

The `.content` of these events SHOULD be a description of the calendar event.

Aside from the common tags, this also takes the following tags:

- `start` (required) inclusive start date in ISO 8601 format (YYYY-MM-DD). Must be less than `end`, if it exists.

- `end` (optional) exclusive end date in ISO 8601 format (YYYY-MM-DD). If omitted, the calendar event ends on the same date as `start`.

Example:

```
{
  "id": <32-bytes lowercase hex-encoded SHA-256 of the serialized event data>,
  "pubkey": <32-bytes lowercase hex-encoded public key of the event creator>,
  "created_at": <unix timestamp in seconds>,
  "kind": 31922,
  "content": "<description of calendar event>",
  "tags": [
    ["d", "<random-identifier>"],

    ["title", "<title of calendar event>"],

    // dates
    ["start", "<YYYY-MM-DD>"],
    ["end", "<YYYY-MM-DD>"],

    // location
    ["location", "<location>"],
    ["g", "<geohash>"],

    // participants
    ["p", "<32-bytes hex of a pubkey>", "<optional recommended relay URL>", "<role>"],
    ["p", "<32-bytes hex of a pubkey>", "<optional recommended relay URL>", "<role>"],
  ]
}
```

Time-Based Calendar Event

This kind of calendar event spans between a start time and end time.

It's an *addressable event* of `kind:31923`.

The `.content` of these events should be a description of the calendar event. It is required but can be an empty string.

Aside from the common tags, this also takes the following tags:

- `start` (required) inclusive start Unix timestamp in seconds. Must be less than `end`, if it exists.
- `end` (optional) exclusive end Unix timestamp in seconds. If omitted, the calendar event ends instantaneously.
- `start_tzid` (optional) time zone of the start timestamp, as defined by the IANA Time Zone Database. e.g., `America/Costa_Rica`
- `end_tzid` (optional) time zone of the end timestamp, as defined by the IANA Time Zone Database. e.g., `America/Costa_Rica`. If omitted and `start_tzid` is provided, the time zone of the end timestamp is the same as the start timestamp.
- `D` (required) the day-granularity unix timestamp on which the event takes place, calculated as `floor(unix_seconds() / seconds_in_one_day)`. Multiple tags SHOULD be included to cover the event's timeframe.

```
{
  "id": <32-bytes lowercase hex-encoded SHA-256 of the the serialized event data>,
  "pubkey": <32-bytes lowercase hex-encoded public key of the event creator>,
  "created_at": <unix timestamp in seconds>,
  "kind": 31923,
  "content": "<description of calendar event>",
  "tags": [
    ["d", "<random-identifier>"],

    ["title", "<title of calendar event>"],
    ["summary", "<brief description of the calendar event>"],
    ["image", "<string with image URI>"],

    // timestamps
    ["start", "<unix timestamp in seconds>"],
    ["end", "<unix timestamp in seconds>"],
    ["D", "82549"],

    ["start_tzid", "<IANA Time Zone Database identifier>"],
    ["end_tzid", "<IANA Time Zone Database identifier>"],

    // location
    ["location", "<location>"],
    ["g", "<geohash>"],

    // participants
    ["p", "<32-bytes hex of a pubkey>", "<optional recommended relay URL>", "<role>"],
    ["p", "<32-bytes hex of a pubkey>", "<optional recommended relay URL>", "<role>"],
  ]
}
```

Calendar

A calendar is a collection of calendar events, represented as a custom *addressable list* event using kind `31924`. A user can have multiple calendars. One may create a calendar to segment calendar events for specific purposes. e.g., personal, work, travel, meetups, and conferences.

Calendars can accept event requests from other users. When calendar events reference a calendar via an `a` tag, this represents a request for inclusion.

The `.content` of these events should be a detailed description of the calendar. It is required but can be an empty string.

- `d` (required) universally unique identifier. Generated by the client creating the calendar.
- `title` (required) calendar title
- `a` (repeated) reference tag to kind `31922` or `31923` calendar event being responded to

```
{
  "id": <32-bytes lowercase hex-encoded SHA-256 of the the serialized event data>,
  "pubkey": <32-bytes lowercase hex-encoded public key of the event creator>,
  "created_at": <unix timestamp in seconds>,
  "kind": 31924,
  "content": "<description of calendar>",
```

```
  "tags": [
    ["d", "<random-identifier>"],
    ["title", "<calendar title>"],
    ["a", "<31922 or 31923>:<calendar event author pubkey>:<d-identifier of calendar event>",
        "<optional relay url>"],
    ["a", "<31922 or 31923>:<calendar event author pubkey>:<d-identifier of calendar event>",
        "<optional relay url>"]
  ]
}
```

Calendar Event RSVP

A calendar event RSVP is a response to a calendar event to indicate a user's attendance intention.

If a calendar event tags a pubkey, that can be interpreted as the calendar event creator inviting that user to attend. Clients MAY choose to prompt the user to RSVP for the calendar event.

Any user may RSVP, even if they were not tagged on the calendar event. Clients MAY choose to prompt the calendar event creator to invite the user who RSVP'd. Clients also MAY choose to ignore these RSVPs.

This NIP is intentionally not defining who is authorized to attend a calendar event if the user who RSVP'd has not been tagged. It is up to the calendar event creator to determine the semantics.

This NIP is also intentionally not defining what happens if a calendar event changes after an RSVP is submitted.

The RSVP MUST have an `a` tag of the event coordinates to the calendar event, and optionally an `e` tag of the id of the specific calendar event revision. If an `e` tag is present, clients SHOULD interpret it as an indication that the RSVP is a response to that revision of the calendar event, and MAY interpret it to not necessarily apply to other revisions of the calendar event.

The RSVP MAY tag the author of the calendar event it is in response to using a `p` tag so that clients can easily query all RSVPs that pertain to the author.

The RSVP is an *addressable event* of `kind:31925`.

The `.content` of these events is optional and should be a free-form note that adds more context to this calendar event response.

The list of tags is as follows:

- `a` (required) coordinates to a kind `31922` or `31923` calendar event being responded to.
- `e` (optional) event id of a kind `31922` or `31923` calendar event being responded to.
- `d` (required) universally unique identifier. Generated by the client creating the calendar event RSVP.
- `status` (required) `accepted`, `declined`, or `tentative`. Determines attendance status to the referenced calendar event.
- `fb` (optional) `free` or `busy`. Determines if the user would be free or busy for the duration of the calendar event. This tag must be omitted or ignored if the `status` label is set to `declined`.
- `p` (optional) pubkey of the author of the calendar event being responded to.

```
{
  "id": <32-bytes lowercase hex-encoded SHA-256 of the the serialized event data>,
```

```json
  "pubkey": <32-bytes lowercase hex-encoded public key of the event creator>,
  "created_at": <unix timestamp in seconds>,
  "kind": 31925,
  "content": "<note>",
  "tags": [
    ["e", "<kind 31922 or 31923 event id", "<optional recommended relay URL>"]
    ["a", "<31922 or 31923>:<calendar event author pubkey>:<d-identifier of calendar event>",
        "<optional recommended relay URL>"],
    ["d", "<random-identifier>"],
    ["status", "<accepted/declined/tentative>"],
    ["fb", "<free/busy>"],
    ["p", "<hex pubkey of kind 31922 or 31923 event>", "<optional recommended relay URL>"]
  ]
}
```

Intentionally Unsupported Scenarios

Recurring Calendar Events

Recurring calendar events come with a lot of complexity, making it difficult for software and humans to deal with. This complexity includes time zone differences between invitees, daylight savings, leap years, multiple calendar systems, one-off changes in schedule or other metadata, etc.

This NIP intentionally omits support for recurring calendar events and pushes that complexity up to clients to manually implement if they desire. i.e., individual calendar events with duplicated metadata represent recurring calendar events.

NIP-53

Live Streaming and Spaces

`draft` `optional`

This NIP introduces event kinds to advertise live spaces and the participation of pubkeys in them.

Live Streaming

A special event with `kind:30311` “Live Streaming Event” is defined as an *addressable event* whose tags advertise the content and participants of a live stream.

Each `p` tag SHOULD have a **displayable** marker name for the current role (e.g. `Host`, `Speaker`, `Participant`) of the user in the event and the relay information MAY be empty. This event will be constantly updated as participants join and leave the activity.

For example:

```json
{
  "kind": 30311,
  "tags": [
```

```
    ["d", "<unique identifier>"],
    ["title", "<name of the event>"],
    ["summary", "<description>"],
    ["image", "<preview image url>"],
    ["t", "hashtag"]
    ["streaming", "<url>"],
    ["recording", "<url>"], // used to place the edited video once the activity is over
    ["starts", "<unix timestamp in seconds>"],
    ["ends", "<unix timestamp in seconds>"],
    ["status", "<planned, live, ended>"],
    ["current_participants", "<number>"],
    ["total_participants", "<number>"],
    ["p", "91cf9..4e5ca", "wss://provider1.com/", "Host", "<proof>"],
    ["p", "14aeb..8dad4", "wss://provider2.com/nostr", "Speaker"],
    ["p", "612ae..e610f", "ws://provider3.com/ws", "Participant"],
    ["relays", "wss://one.com", "wss://two.com", /*...*/],
    ["pinned", "<event id of pinned live chat message>"],
  ],
  "content": "",
  // other fields...
}
```

A distinct `d` tag should be used for each activity. All other tags are optional.

Providers SHOULD keep the participant list small (e.g. under 1000 users) and, when limits are reached, Providers SHOULD select which participants get named in the event. Clients should not expect a comprehensive list. Once the activity ends, the event can be deleted or updated to summarize the activity and provide async content (e.g. recording of the event).

Clients are expected to subscribe to `kind:30311` events in general or for given follow lists and statuses. Clients MAY display participants' roles in activities as well as access points to join the activity.

Live Activity management clients are expected to constantly update `kind:30311` during the event. Clients MAY choose to consider `status=live` events after 1hr without any update as `ended`. The `starts` and `ends` timestamp SHOULD be updated when the status changes to and from `live`

The activity MUST be linked to using the NIP-19 `naddr` code along with the `a` tag.

Proof of Agreement to Participate

Event owners can add proof as the 5th term in each `p` tag to clarify the participant's agreement in joining the event. The proof is a signed SHA256 of the complete `a` Tag of the event (`kind:pubkey:dTag`) by each `p`'s private key, encoded in hex.

Clients MAY only display participants if the proof is available or MAY display participants as "invited" if the proof is not available.

This feature is important to avoid malicious event owners adding large account holders to the event, without their knowledge, to lure their followers into the malicious owner's trap.

Live Chat Message

Event `kind:1311` is live chat's channel message. Clients MUST include the `a` tag of the activity. An `e` tag denotes the direct parent message this post is replying to.

```json
{
  "kind": 1311,
  "tags": [
    ["a", "30311:<Community event author pubkey>:<d-identifier of the community>", "<Optional
        relay url>"],
  ],
  "content": "Zaps to live streams is beautiful.",
  // other fields...
}
```

`q` tags MAY be used when citing events in the `.content` with NIP-21.

```json
["q", "<event-id> or <event-address>", "<relay-url>", "<pubkey-if-a-regular-event>"]
```

Hosts may choose to pin one or more live chat messages by updating the `pinned` tags in the live event kind `30311`.

Examples

Live Streaming

```json
{
  "id": "57f28dbc264990e2c61e80a883862f7c114019804208b14da0bff81371e484d2",
  "pubkey": "1597246ac22f7d1375041054f2a4986bd971d8d196d7997e48973263ac9879ec",
  "created_at": 1687182672,
  "kind": 30311,
  "tags": [
    ["d", "demo-cf-stream"],
    ["title", "Adult Swim Metalocalypse"],
    ["summary", "Live stream from IPTV-ORG collection"],
    ["streaming", "https://adultswim-vodlive.cdn.turner.com/live/metalocalypse/stream.m3u8"],
    ["starts", "1687182672"],
    ["status", "live"],
    ["t", "animation"],
    ["t", "iptv"],
    ["image", "https://i.imgur.com/CaKq6Mt.png"]
  ],
  "content": "",
  "sig":
        "5bc7a60f5688effa5287244a24768cbe0dcd854436090abc3bef172f7f5db1410af4277508dbafc4f70a754a891c90ce3b9

}
```

Live Streaming chat message

```json
{
  "id": "97aa81798ee6c5637f7b21a411f89e10244e195aa91cb341bf49f718e36c8188",
  "pubkey": "3f770d65d3a764a9c5cb503ae123e62ec7598ad035d836e2a810f3877a745b24",
  "created_at": 1687286726,
```

```
  "kind": 1311,
  "tags": [
    ["a", "30311:1597246ac22f7d1375041054f2a4986bd971d8d196d7997e48973263ac9879ec:demo-cf-
        stream", "", "root"]
  ],
  "content": "Zaps to live streams is beautiful.",
  "sig":
        "997f62ddfc0827c121043074d50cfce7a528e978c575722748629a4137c45b75bdbc84170bedc723ef0a5a4c3daebf1fef2

}
```

Meeting Spaces

Meeting spaces contain one or more video/audio rooms where users can join and participate in the streaming.

Meeting Space Event (kind:30312)

A special event with `kind:30312` "Space Host" defines the configuration and properties of a virtual interactive space. Each space has a unique identifier and can host multiple events/meetings.

```
{
  "kind": 30312,
  "tags": [
    ["d", "<unique identifier>"],              // Required: Room identifier
    ["room", "<name of the room>"],            // Required: Display name
    ["summary", "<description>"],              // Optional: Room description
    ["image", "<preview image url>"],          // Optional: Room image
    ["status", "<open, private, closed>"],     // Required: Room accessibility
    ["service", "<url>"],                      // Required: URL to access the room
    ["endpoint", "<url>"],                     // Optional: API endpoint for status/info
    ["t", "<hashtag>"],                        // Optional: Multiple hashtags allowed
    ["p", "<pubkey>", "<url>", "<role>", "<proof>"],  // Required: At least one provider
    ["relays", "<url>", "<url>", /*...*/]     // Optional: Preferred relays
  ],
  "content": ""  // Usually empty, may contain additional metadata
}
```

Space properties: * MUST be either open, private or closed. Closed means the room is not in operation. * MAY specify access control policy for private rooms (e.g. invite-only, payment required) * MAY persist when not in use * MUST have at least one provider with "Host" role * MAY have multiple providers with different roles

Provider roles (p tags): * Host: Full room management capabilities * Moderator: Room moderation capabilities * Speaker: Allowed to present/speak * Optional proof field for role verification

Meeting Room Events (kind:30313)

A special event with kind:30313 represents a scheduled or ongoing meeting within a room. It MUST reference its parent room using the d tag.

```
{
  "kind": 30313,
  "tags": [
    ["d", "<event-unique-identifier>"],        // Required: Event identifier
```

```
    ["a", "30312:<pubkey>:<room-id>", "wss://nostr.example.com"], // Required: Reference to
        parent space, 'd' from 30312
    ["title", "<meeting-title>"],              // Required: Meeting title
    ["summary", "<description>"],              // Optional: Meeting description
    ["image", "<preview image url>"],          // Optional: Meeting image
    ["starts", "<unix timestamp>"],            // Required: Start time
    ["ends", "<unix timestamp>"],              // Optional: End time
    ["status", "<planned, live, ended>"],      // Required: Meeting status
    ["total_participants", "<number>"],        // Optional: Total registered
    ["current_participants", "<number>"],      // Optional: Currently active
    ["p", "<pubkey>", "<url>", "<role>"],     // Optional: Participant with role
  ],
  "content": ""  // Usually empty, may contain additional metadata
}
```

Event properties: * MUST reference parent room via d tag * MUST have a status (planned/live/ended) * MUST have a start time * MAY track participant counts * MAY include participant roles specific to the event

Event management: * Clients SHOULD update event status regularly when live * Events without updates for 1 hour MAY be considered ended * starts and ends timestamps SHOULD be updated when status changes

Examples

Meeting Space (kind:30312)

```
{
  "kind": 30312,
  "tags": [
    ["d", "main-conference-room"],
    ["room", "Main Conference Hall"],
    ["summary", "Our primary conference space"],
    ["image", "https://example.com/room.jpg"],
    ["status", "open"],
    ["service", "https://meet.example.com/room"],
    ["endpoint", "https://api.example.com/room"],
    ["t", "conference"],
    ["p", "f7234bd4c1394dda46d09f35bd384dd30cc552ad5541990f98844fb06676e9ca", "wss://
        nostr.example.com/", "Owner"],
    ["p", "14aeb..8dad4", "wss://provider2.com/", "Moderator"],
    ["relays", "wss://relay1.com", "wss://relay2.com"]
  ],
  "content": ""
}
```

Meeting room (kind:30313)

```
{
  "kind": 30313,
  "tags": [
    ["d", "annual-meeting-2025"],
    ["a", "30312:f7234bd4c1394dda46d09f35bd384dd30cc552ad5541990f98844fb06676e9ca:main-
        conference-room", "wss://nostr.example.com"]
    ["title", "Annual Company Meeting 2025"],
    ["summary", "Yearly company-wide meeting"],
```

```
    ["image", "https://example.com/meeting.jpg"],
    ["starts", "1676262123"],
    ["ends", "1676269323"],
    ["status", "live"],
    ["total_participants", "180"],
    ["current_participants", "175"],
    ["p", "91cf9..4e5ca", "wss://provider1.com/", "Speaker"],
  ],
  "content": ""
}
```

Room Presence

New `kind: 10312` provides an event which signals presence of a listener.

The presence event SHOULD be updated at regular intervals and clients SHOULD filter presence events older than a given time window.

This kind `10312` is a regular replaceable event, as such presence can only be indicated in one room at a time.

```
{
  "kind": 10312,
  "tags": [
    ["a" , "<room-a-tag>", "<relay-hint>", "root"],
    ["hand", "1"] // hand raised flag
  ]
}
```

NIP-F4

Podcasts

`draft` `optional`

This NIP defines how podcast episodes can be fetched from relays. It's intended to fit easily into existing podcast players.

Rationale

RSS feeds are great, but they have some problems that are solved by moving feeds from RSS URLs to multiple Nostr events, one for each episode:

- they depend on a URL and that is hard for most people, so podcasters tend to use service providers that can
  - charge money – which isn't a bad thing per se but it turns out that with a Nostr-native podcast feed protocol it would be much simpler to host feeds and infinitely cheaper to host the media, so the ecosystem could be more decentralized
  - seal them into their walled gardens (like Spotify has been doing), slowly turning a previously open ecosystem into a centralized system captured by big corporations

        - censor podcasters and prevent them from migrating (on Nostr this wouldn't be a problem even if a relay banned someone as moving to other relays would be trivial as long as the podcaster held their key and podcast clients could easily discover the new relay URLs)
- they can only be loaded in full, with no pagination or filtering of any kind, which means that
    - the sync process in normal RSS clients is slow and cumbersome (which also nudges people into using centralized solutions)
        - this has also led to the creation of broken schemes like Podping, which wouldn't be necessary in a Nostr-native podcast feed
    - It's impossible to reference a single episode directly (since it only exists as a member in the full RSS list of episodes), this means there is no way to share a podcast episode with friends or on social media, so people tend to share links to centralized platforms or to YouTube videos of the same podcast content, which is an antipattern
- they cannot be interacted with in any way: listeners cannot "like" or comment or signal that they have listened to the episode, which is also another factor in the push towards centralized closed platforms: better analytics and insights and possibilities of engagement with the public

Concept

The idea is that each podcast is its own Nostr keypair. This is simple and allows podcasts to combine their *podcasting* presence with a normal `kind:0`/`kind:1` microblogging presence.

It also allows podcasts to have shared ownership (via MuSig2 or just common agreements between friends) and to change that ownership over time (again, based on human trust over handling of key material).

Event definitions

Podcast Metadata

Podcast metadata is stored in `kind:10154` *replaceable* events. This event contains the show-level information that podcast clients need to display the podcast.

Similar information may exist in the `kind:0` profile for the podcast, but podcast-specific clients should be able to read directly from `kind:10154` and ignore `kind:0` entirely.

```
{
  "id": "...",
  "pubkey": "<podcast-pubkey>",
  "kind": 10154,
  "created_at": 1700682555,
  "tags": [
    ["title", "<podcast-title>"],
    ["image", "<podcast-cover-image-URL>"],
    ["description", "<podcast-description>"],
    ["website", "<podcast-website-URL>"], // optional, multiple
    ["p", "<podcast-author-pubkey>", "<role>"] // optional, multiple
  ],
  "content": "",
  "sig": "..."
}
```

`<role>` is optional, can be `"host"`, `"cohost"` or `"editor"`.

Authored Podcasts

The `p` tag in `kind:10154` shouldn't be blindly trusted as implying authorship, since any podcast can falsely claim anyone to be their author, so before it is displayed as information by clients it must be checked against an equivalent counter-claim by the podcast authors themselves.

This is the `kind:10164` event. Here podcast authors can list the public keys of the podcasts they author.

```
{
  "id": "...",
  "pubkey": "<user-pubkey>",
  "kind": 10064,
  "created_at": 1700682555,
  "tags": [
    ["p", "<podcast-pubkey-1>"],
    ["p", "<podcast-pubkey-2>"]
  ],
  "content": "",
  "sig": "..."
}
```

Clients can also use this to discover all podcasts authored by a given user.

Podcast Episodes

Podcast episodes are `kind:54` events authored by the podcast public key directly.

```
{
  "id": "...",
  "pubkey": "<podcast-pubkey>",
  "kind": 54,
  "created_at": 1700682555,
  "tags": [
    ["title", "<episode title>"],
    ["image", "<optional episode image>"],
    ["description", "<a brief description>"],
    ["audio", "<audio-url>", "<optional_media_type>"], // can be specified multiple times
    // ...other important fields to be specified here later after further discovery
  ],
  "content": "<markdown content>",
  "sig": "...",
}
```

Favorite podcasts

See [NIP-51] for the description of a `kind:10054` event in which any user can list the podcasts they publicly advertise to listening to, as a soft recommendation.

NIP-84

Highlights

`draft` `optional`

This NIP defines `kind:9802`, a “highlight” event, to signal content a user finds valuable.

Format

The `.content` of these events is the highlighted portion of the text.

`.content` might be empty for highlights of non-text based media (e.g. NIP-94 audio/video).

References

Events SHOULD tag the source of the highlight, whether nostr-native or not. The following tags may be used to indicate different types of content:

- `a` and/or `e` tags for nostr events
- `i` tags for structured sources per NIP 73
- `r` tags for anything else (may contain a URL or text)

Attribution

Clients MAY include one or more `p` tags, tagging the original authors of the material being highlighted; this is particularly useful when highlighting non-nostr content for which the client might be able to get a nostr pubkey somehow (e.g. prompting the user or reading a `<link rel="me" href="nostr:nprofile1..." />` tag on the document). A role MAY be included as the last value of the tag.

```
{
  "tags": [
    ["p", "<pubkey-hex>", "<relay-url>", "author"],
    ["p", "<pubkey-hex>", "<relay-url>", "author"],
    ["p", "<pubkey-hex>", "<relay-url>", "editor"]
  ],
  // other fields...
}
```

Context

Clients MAY include a `context` tag, useful when the highlight is a subset of a paragraph and displaying the surrounding content might be beneficial to give context to the highlight.

Quote Highlights

A `comment` tag may be added to create a quote highlight. This SHOULD be rendered like a quote repost with the highlight as the quoted note.

This is to prevent the creation of multiple notes (highlight + kind 1) for a single highlight action, which looks bad in micro-blogging clients where these notes may appear in succession.

p-tag mentions MUST have a `mention` attribute to distinguish it from authors and editors.

r-tag urls from the comment MUST have a `mention` attribute to distinguish from the highlighted source url. The source url MUST have the `source` attribute.

NIP-54

Wiki

`draft` `optional`

This NIP defines `kind:30818` (an *addressable event*) for descriptions (or encyclopedia entries) of particular subjects, and it's expected that multiple people will write articles about the exact same subjects, with either small variations or completely independent content.

Articles are identified by lowercase, normalized `d` tags.

Articles

```
{
  "content": "A wiki is a hypertext publication collaboratively edited and managed by its own
       audience.",
  "tags": [
    ["d", "wiki"],
    ["title", "Wiki"]
  ]
}
```

`d` tag normalization rules

- All letters with uppercase/lowercase variants MUST be converted to lowercase.
- Whitespace MUST be converted to `-`.
- Punctuation and symbols SHOULD be removed.
- Multiple consecutive `-` SHOULD be collapsed to a single `-`.
- Leading and trailing `-` SHOULD be removed.
- Non-ASCII letters (e.g., Japanese, Chinese, Arabic, Cyrillic) MUST be preserved as UTF-8.
- Numbers MUST be preserved.

For example: - `"Wiki Article"` → `"wiki-article"` - `"What's Up?"` → `"whats-up"` - `" Hello World "` → `"hello-world"` - `"Article 1"` → `"article-1"` - `"ウィキペディア"` → `"ウィキペディア"` (Japanese, no case change) - `"Ñoño"` → `"ñoño"` (Spanish, lowercased) - `"Москва"` → `"москва"` (Russian, lowercased) - `"日本語 Article"` → `"日本語-article"` (mixed scripts)

Content

The `content` should be Djot with two special functionalities:

1. Links can have target URIs in NIP-21 format, like `[Bob](nostr:npub1...)`.
2. When a reference can't be found for a reference-style link, it should link to the wiki article with that name instead (wikilink behavior). The target is normalized following the `d` tag normalization rules above.

For example:

```
Bitcoin is a [cryptocurrency][] invented by [Satoshi Nakamoto][].

See also: [proof of work][] and [lightning network][Lightning Network].

[Satoshi Nakamoto]: nostr:npub1satoshi...
```

In the article above: - `[cryptocurrency][]` links to the wiki article with `d` tag `"cryptocurrency"` (no reference defined, so it becomes a wikilink) - `[Satoshi Nakamoto][]` links to the npub (reference is defined) - `[proof of work][]` links to the article with `d` tag `"proof-of-work"` - `[lightning network][Lightning Network]` links to `"lightning-network"`, displays as “lightning network”

Wikilinks also work with non-Latin scripts following the same normalization rules: - `[ビットコイン][]` → links to article with `d` tag `"ビットコイン"` - `[Japanese Article][日本語 記事]` → links to `"日本語-記事"`, displays as “Japanese Article” - `[Биткойн][]` → links to article with `d` tag `"биткойн"` (Cyrillic, lowercased)

NIP-21 `nostr:` links can also be used to link to profiles or arbitrary Nostr events. It is not recommended to link to specific versions of articles — the wikilink syntax should be preferred instead, since it should be left to the reader and their client to decide what version of any given article they want to read.

Optional extra tags

- `title`: for when the display title should be different from the `d` tag.
- `summary`: for display in lists.
- `a` and `e`: for referencing the original event a wiki article was forked from.

Merge Requests

Event `kind:818` represents a request to merge from a forked article into the source. It is directed to a pubkey and references the original article and the modified event.

```
{
  "content": "I added information about the block size limit",
  "kind": 818,
  "tags": [
    ["a", "30818:<destination-pubkey>:bitcoin", "<relay-url>"],
    ["e", "<version-against-which-the-modification-was-made>", "<relay-url>"],
    ["p", "<destination-pubkey>"],
    ["e", "<version-to-be-merged>", "<relay-url>", "source"]
```

```
  ]
}
```

- `.content`: an optional explanation detailing why this merge is being requested.
- `a` tag: tag of the article which should be modified (i.e. the target of this merge request).
- `e` tag: optional version of the article on which this modification is based.
- `e` tag with `source` marker: the ID of the event that should be merged. This event id MUST be of a `kind:30818` as defined in this NIP.

The destination pubkey can create NIP-25 reactions that tag the `kind:818` event with `+` or `-` to accept or reject the merge request.

Redirects

Event `kind:30819` is also defined to stand for "wiki redirects", i.e. if one thinks "BTC" should redirect to "Bitcoin" they can issue one of these events instead of replicating the content. These events can be used for automatically redirecting between articles on a client, but also for generating crowdsourced "disambiguation" pages (common in Wikipedia).

```
{
  "kind": 30819,
  "tags": [
    ["d", "btc"],
    ["a", "30818:<pubkey>:bitcoin", "<relay-url>"]
  ],
  "content": ""
}
```

How to decide what article to display

As there could be many articles for each given name, some kind of prioritization must be done by clients. Criteria for this should vary between users and clients, but some means that can be used are described below:

Reactions

NIP-25 reactions are very simple and can be used to create a simple web-of-trust between wiki article writers and their content. While just counting a raw number of "likes" is unproductive, reacting to any wiki article event with a `+` can be interpreted as a recommendation for that article specifically and a partial recommendation of the author of that article. When 2 or 3-level deep recommendations are followed, suddenly a big part of all the articles may have some form of tagging.

Relays

NIP-51 lists of relays can be created with the kind 10102 and then used by wiki clients in order to determine where to query articles first and to rank these differently in relation to other events fetched from other relays.

Contact lists

NIP-02 contact lists can form the basis of a recommendation system that is then expanded with relay lists and reaction lists through nested queries. These lists form a good starting point only because they are so widespread.

Wiki-related contact lists

NIP-51 lists can also be used to create a list of users that are trusted only in the context of wiki authorship or wiki curationship.

Forks

Wiki-events can tag other wiki-events with a `fork` marker to specify that this event came from a different version. Both `a` and `e` tags SHOULD be used and have the `fork` marker applied, to identify the exact version it was forked from.

Deference

Wiki-events can tag other wiki-events with a `defer` marker to indicate that it considers someone else's entry as a "better" version of itself. If using a `defer` marker both `a` and `e` tags SHOULD be used.

This is a stronger signal of trust than a `+` reaction.

This marker is useful when a user edits someone else's entry; if the original author includes the editor's changes and the editor doesn't want to keep/maintain an independent version, the `defer` tag could effectively be considered a "deletion" of the editor's version and putting that pubkey's WoT weight behind the original author's version.

Why Djot?

Djot is a markup language created by John MacFarlane (author of Pandoc and co-author of CommonMark). It was chosen for the following reasons:

- **Well-defined spec**: Unlike Markdown (many incompatible dialects) or Asciidoc (spec tied to Ruby implementation), Djot has a clear, standalone specification.
- **Native implementations**: Available in JavaScript, Lua, Rust, Go, and other languages without transpilation.
- **Rich features**: Supports superscript, subscript, footnotes, tables, definition lists, and math — features useful for encyclopedic content.
- **Familiar syntax**: Similar to basic Markdown, making it easy to learn.
- **Fast parsing**: Designed for efficient linear-time parsing.

NIP-C0

Code Snippets

`draft` `optional`

Abstract

This NIP defines a new event kind for sharing and storing code snippets. Unlike regular text notes (`kind:1`), code snippets have specialized metadata like language, extension, and other code-specific attributes that enhance discoverability, syntax highlighting, and improved user experience.

Event Kind

This NIP defines `kind:1337` as a code snippet event.

The `.content` field contains the actual code snippet text.

Optional Tags

- `l` - Programming language name (lowercase). Examples: "javascript", "python", "rust"
- `name` - Name of the code snippet, commonly a filename. Examples: "hello-world.js", "quick-sort.py"
- `extension` - File extension (without the dot). Examples: "js", "py", "rs"
- `description` - Brief description of what the code does
- `runtime` - Runtime or environment specification (e.g., "node v18.15.0", "python 3.11")
- `license` - License under which the code (along with any related data contained within the event, when available, such as the description) is shared. This MUST be a standard SPDX short identifier (e.g., "MIT", "GPL-3.0-or-later", "Apache-2.0") when available. An additional parameter containing a reference to the actual text of the license MAY be provided. This tag can be repeated, to indicate multi-licensing, allowing recipients to use the code under any license of choosing among the referenced ones
- `dep` - Dependency required for the code to run (can be repeated)
- `repo` - Reference to a repository where this code originates. This MUST be either a standard URL or, alternatively, the address of a NIP-34 Git repository announcement event in the form `"30617:<32-bytes hex a pubkey>:<d tag value>"`. If a repository announcement is referenced, a recommended relay URL where to find the event should be provided as an additional parameter

Format

```
{
  "id": "<32-bytes lowercase hex-encoded SHA-256 of the the serialized event data>",
  "pubkey": "<32-bytes lowercase hex-encoded public key of the event creator>",
  "created_at": <Unix timestamp in seconds>,
  "kind": 1337,
  "content": "function helloWorld() {\n  console.log('Hello, Nostr!');\n}\n\nhelloWorld();",
  "tags": [
    ["l", "javascript"],
```

```
    ["extension", "js"],
    ["name", "hello-world.js"],
    ["description", "A basic JavaScript function that prints 'Hello, Nostr!' to the console"],
    ["runtime", "node v18.15.0"],
    ["license", "MIT"],
    ["repo", "https://github.com/nostr-protocol/nostr"]
  ],
  "sig": "<64-bytes signature of the id>"
}
```

Client Behavior

Clients that support this NIP SHOULD:

1. Display code snippets with proper syntax highlighting based on the language.
2. Allow copying the full code snippet with a single action.
3. Render the code with appropriate formatting, preserving whitespace and indentation.
4. Display the language and extension prominently.
5. Provide "run" functionality for supported languages when possible.
6. Display the description (if available) as part of the snippet presentation.

Clients MAY provide additional functionality such as:

1. Code editing capabilities
2. Forking/modifying snippets
3. Creating executable environments based on the runtime/dependencies
4. Downloading the snippet as a file using the provided extension
5. Sharing the snippet with attribution

NIP-73

External Content IDs

`draft` `optional`

There are certain established global content identifiers such as Book ISBNs, Podcast GUIDs, and Movie ISANs that are useful to reference in nostr events so that clients can query all the events associated with these ids.

`i` tags are used for referencing these external content ids, with `k` tags representing the external content id kind so that clients can query all the events for a specific kind.

Supported IDs

Type	`i` tag	`k` tag
URLs	"`<URL, normalized, no fragment>`"	"web"
Books	"isbn:`<id, without hyphens>`"	"isbn"

Type	i tag	k tag
Geohashes	"geo:<geohash, lowercase>"	"geo"
Countries	"iso3166:<code, uppercase>"	"iso3166"
Movies	"isan:<id, without version part>"	"isan"
Papers	"doi:<id, lowercase>"	"doi"
Hashtags	"#<topic, lowercase>"	"#"
Podcast Feeds	"podcast:guid:<guid>"	"podcast:guid"
Podcast Episodes	"podcast:item:guid:<guid>"	"podcast:item:guid"
Podcast Publishers	"podcast:publisher:guid:<guid>"	"podcast:publisher:guid"
Blockchain Transaction	"<blockchain>:[<chainId>:]tx:<txid, hex, lowercase>"	"<blockchain>:tx"
Blockchain Address	"<blockchain>:[<chainId>:]address:<address>"	"<blockchain>:address"

Examples

Webpages

For the webpage "https://myblog.example.com/post/2012-03-27/hello-world" the "i" and "k" tags are:

```
[
  ["i", "https://myblog.example.com/post/2012-03-27/hello-world"],
  ["k", "web"]
]
```

Geohashes:

- Geohash: `["i", "geo:ezs42e44yx96"]` - https://www.movable-type.co.uk/scripts/geohash.html

Geohashes are a geocoding system that encodes geographic locations into short strings of letters and digits. They MUST be lowercase.

Countries:

ISO 3166 codes can reference countries (ISO 3166-1 alpha-2) or subdivisions like states/provinces (ISO 3166-2).

- Country (Venezuela): `["i", "iso3166:VE"]`
- Subdivision (California, USA): `["i", "iso3166:US-CA"]`

ISO 3166 codes MUST be uppercase. More info: https://en.wikipedia.org/wiki/ISO_3166

Books:

- Book ISBN: `["i", "isbn:9780765382030"]` - https://isbnsearch.org/isbn/9780765382030

Book ISBNs MUST be referenced ***without hyphens*** as many book search APIs return the ISBNs without hyphens. Removing hyphens from ISBNs is trivial, whereas adding the hyphens back in is non-trivial requiring a library.

Podcasts:

- Podcast RSS Feed GUID: `["i", "podcast:guid:c90e609a-df1e-596a-bd5e-57bcc8aad6cc"]` - https://podcastindex.org/podcast/c90e609a-df1e-596a-bd5e-57bcc8aad6cc
- Podcast RSS Item GUID: `["i", "podcast:item:guid:d98d189b-dc7b-45b1-8720-d4b98690f31f"]`
- Podcast RSS Publisher GUID: `["i", "podcast:publisher:guid:18bcbf10-6701-4ffb-b255-bc057390d738"]`

Movies:

- Movie ISAN: `["i", "isan:0000-0000-401A-0000-7"]` - https://web.isan.org/public/en/isan/0000-0000-401A-0000-7

Movie ISANs SHOULD be referenced ***without the version part*** as the versions / edits of movies are not relevant. More info on ISAN parts here - https://support.isan.org/hc/en-us/articles/360002783131-Records-relations-and-hierarchies-in-the-ISAN-Registry

Blockchain

<blockchain> can be any layer 1 chain (`bitcoin`, `ethereum`, `solana`, ...). If necessary (e.g. for ethereum), you can specify a <chainId>.

Bitcoin

```
bitcoin:address:<bech32, lowercase | base58, case sensitive>
bitcoin:tx:<txid, hex, lowercase>
```

E.g. https://blockstream.info/tx/a1075db55d416d3ca199f55b6084e2115b9345e16c5cf302fc80e9d5fbf5d48d

```
[
  ["i", "bitcoin:tx:a1075db55d416d3ca199f55b6084e2115b9345e16c5cf302fc80e9d5fbf5d48d"],
  ["k", "bitcoin:tx"]
]
```

E.g. https://blockstream.info/address/1HQ3Go3ggs8pFnXuHVHRytPCq5fGG8Hbhx

```
[
  ["i", "bitcoin:address:1HQ3Go3ggs8pFnXuHVHRytPCq5fGG8Hbhx"],
  ["k", "bitcoin:address"]
]
```

Ethereum (and other EVM chains)

```
ethereum:<chainId, integer>:tx:<txHash, hex, lowercase>
ethereum:<chainId, integer>:address:<hex, lowercase>
```

E.g. https://etherscan.io/address/0xd8dA6BF26964aF9D7eEd9e03E53415D37aA96045

```
[
  ["i", "ethereum:1:address:0xd8da6bf26964af9d7eed9e03e53415d37aa96045"],
  ["k", "ethereum:address"]
]
```

E.g. https://gnosisscan.io/tx/0x98f7812be496f97f80e2e98d66358d1fc733cf34176a8356d171ea7fbbe97ccd

```
[
  ["i", "ethereum:100:tx:0x98f7812be496f97f80e2e98d66358d1fc733cf34176a8356d171ea7fbbe97ccd"],
  ["k", "ethereum:tx"]
]
```

Optional URL Hints

Each `i` tag MAY have a url hint as the second argument to redirect people to a website if the client isn't opinionated about how to interpret the id:

`["i", "podcast:item:guid:d98d189b-dc7b-45b1-8720-d4b98690f31f", https://fountain.fm/episode/z1y9TMQRuqXl2awyrQxg]`

`["i", "isan:0000-0000-401A-0000-7", https://www.imdb.com/title/tt0120737]`

NIP-34

`git` stuff

`draft` `optional`

This NIP defines all the ways code collaboration using and adjacent to [git](https://git-scm.com/) can be done using Nostr.

Repository announcements

Git repositories are hosted in Git-enabled servers, but their existence can be announced using Nostr events. By doing so the author asserts themselves as a maintainer of the primary project, unless a `u` tag is included.

```
{
  "kind": 30617,
  "content": "",
  "tags": [
    ["d", "<repo-id>"], // usually kebab-case short name
    ["name", "<human-readable project name>"],
    ["description", "brief human-readable project description>"],
```

```
    ["web", "<url for browsing>", ...], // a webpage url, if the git server being used provides
        such a thing
    ["clone", "<url for git-cloning>", ...], // a url to be given to `git clone` so anyone can
        clone it
    ["relays", "<relay-url>", ...], // relays that this repository will monitor for patches and
        issues
    ["r", "<earliest-unique-commit-id>", "euc"],
    ["maintainers", "<other-recognized-maintainer>", ...],
    ["u","30617:<pubkey>:<identifer>|<git-url-https-prefered>","<relay-hint>","<author-
        pubkey>"], // optionally indicate repository is a subordinate fork
    ["t", "<arbitrary string>"], // hashtags labelling the repository
  ]
}
```

The tags `web`, `clone`, `relays`, `maintainers` can have multiple values.

The `r` tag annotated with the `"euc"` marker should be the commit ID of the earliest unique commit of this repo, made to identify it among forks and group it with other repositories hosted elsewhere that may represent essentially the same project. In most cases it will be the root commit of a repository. In case of a permanent fork between two projects, then the first commit after the fork should be used.

Except `d`, all tags are optional.

Nostr Clone URL format

A `nostr://` URL can be used to reference a repository announcement in a way that is compatible with `git clone` when a [git-remote-nostr] helper is installed:

```
nostr://<naddr>
nostr://<npub|nip05>/<identifier>
nostr://<npub|nip05>/<relay-hint>/<identifier>
```

Both `<relay-hint>` and `<identifier>` MUST be percent-encoded per [RFC 3986 §2.1]. `<relay-hint>` is a relay URL whose `wss://` scheme may be omitted for brevity. `<identifier>` is the `d` tag value of the kind `30617` event.

Examples:

```
nostr://npub15qydau2hjma6ngxkl2cyar74wzyjshvl65za5k5rl69264ar2exs5cyejr/relay.ngit.dev/ngit
nostr://npub15qydau2hjma6ngxkl2cyar74wzyjshvl65za5k5rl69264ar2exs5cyejr/my%20%F0%9F%9A%80%20repo
nostr://danconwaydev.com/ws%3A%2F%2Flocalhost%3A7334/my-local-only-repo
nostr://danconwaydev.com/relay.ngit.dev/ngit
```

Repository state announcements

An optional source of truth for the state of branches and tags in a repository.

```
{
  "kind": 30618,
  "content": "",
  "tags": [
    ["d", "<repo-id>"], // matches the identifier in the corresponding repository announcement
    ["refs/<heads|tags>/<branch-or-tag-name>","<commit-id>"]
    ["HEAD", "ref: refs/heads/<branch-name>"]
```

```
  ]
}
```

The `refs` tag may appear multiple times, or none.

Patches and Pull Requests (PRs)

Patches and PRs can be sent by anyone to any repository. Patches and PRs to a specific repository SHOULD be sent to the relays specified in that repository's announcement event's `"relays"` tag. Patch and PR events SHOULD include an `a` tag pointing to that repository's announcement address.

Patches SHOULD be used if each event is under 60kb, otherwise PRs SHOULD be used.

Patches

Patches in a patch set SHOULD include a NIP-10 `e reply` tag pointing to the previous patch.

The first patch revision in a patch revision SHOULD include a NIP-10 `e reply` to the original root patch.

```
{
  "kind": 1617,
  "content": "<patch>", // contents of <git format-patch>
  "tags": [
    ["a", "30617:<base-repo-owner-pubkey>:<base-repo-id>"],
    ["r", "<earliest-unique-commit-id-of-
        repo>"] // so clients can subscribe to all patches sent to a local git repo
    ["p", "<repository-owner>"],
    ["p", "<other-user>"], // optionally send the patch to another user to bring it to their
        attention

    ["t", "root"], // omitted for additional patches in a series
    // for the first patch in a revision
    ["t", "root-revision"],

    // optional tags for when it is desirable that the merged patch has a stable commit id
    // these fields are necessary for ensuring that the commit resulting from applying a patch
    // has the same id as it had in the proposer's machine -- all these tags can be omitted
    // if the maintainer doesn't care about these things
    ["commit", "<current-commit-id>"],
    ["r", "<current-commit-id>"] // so clients can find existing patches for a specific commit
    ["parent-commit", "<parent-commit-id>"],
    ["commit-pgp-sig", "-----BEGIN PGP SIGNATURE-----..."], // empty string for unsigned commit
    ["committer", "<name>", "<email>", "<timestamp>", "<timezone offset in minutes>"],
  ]
}
```

The first patch in a series MAY be a cover letter in the format produced by `git format-patch`.

Pull Requests

A pull request is an event that points to proposed changes in a git repository.

```
{
  "kind": 1618,
```

```
  "content": "<markdown text>",
  "tags": [
    ["a", "30617:<base-repo-owner-pubkey>:<base-repo-id>"],
    ["r", "<earliest-unique-commit-id-of-repo>"] // so clients can subscribe to all PRs sent to
        a local git repo
    ["p", "<repository-owner>"],
    ["p", "<other-user>"], // optionally send the PR to another user to bring it to their
        attention

    ["subject", "<PR-subject>"],
    ["t", "<PR-label>"], // optional
    ["t", "<another-PR-label>"], // optional

    ["c", "<current-commit-id>"], // tip of the PR branch
    ["clone", "<clone-url>", ...], // at least one git clone url where commit can be downloaded
    ["branch-name", "<branch-name>"], // optional recommended branch name

    ["e", "<root-patch-event-id>"], // optionally indicate PR is a revision of an existing
        patch, which should be closed
    ["merge-base", "<commit-id>"], // optional: the most recent common ancestor with the target
        branch
  ]
}
```

Pull Request Updates

A PR Update changes the tip of a referenced PR event.

```
{
  "kind": 1619,
  "content": "",
  "tags": [
    ["a", "30617:<base-repo-owner-pubkey>:<base-repo-id>"],
    ["r", "<earliest-unique-commit-id-of-repo>"] // so clients can subscribe to all PRs sent to
        a local git repo
    ["p", "<repository-owner>"],
    ["p", "<other-user>"], // optionally send the PR to another user to bring it to their
        attention

    // NIP-22 tags
    ["E", "<pull-request-event-id>"],
    ["P", "<pull-request-author>"],

    ["c", "<current-commit-id>"], // updated tip of PR
    ["clone", "<clone-url>", ...], // at least one git clone url where commit can be downloaded
    ["merge-base", "<commit-id>"], // optional: the most recent common ancestor with the target
        branch
  ]
}
```

Issues

Issues are Markdown text that is just human-readable conversational threads related to the repository: bug reports, feature requests, questions or comments of any kind. Like patches, these SHOULD be sent to the relays specified in that repository's announcement event's `"relays"` tag.

Issues may have a `subject` tag, which clients can utilize to display a header. Additionally, one or more `t` tags may be included to provide labels for the issue.

```json
{
  "kind": 1621,
  "content": "<markdown text>",
  "tags": [
    ["a", "30617:<base-repo-owner-pubkey>:<base-repo-id>"],
    ["p", "<repository-owner>"]
    ["subject", "<issue-subject>"]
    ["t", "<issue-label>"]
    ["t", "<another-issue-label>"]
  ]
}
```

Replies

Replies to either a `kind:1621` (*issue*), `kind:1617` (*patch*) or `kind:1618` (*pull request*) event should follow [NIP-22 comment](#).

Status

Root Patches, PRs and Issues have a Status that defaults to 'Open' and can be set by issuing Status events.

```
{
  "kind": 1630, // Open
  "kind": 1631, // Applied / Merged for Patches; Resolved for Issues
  "kind": 1632, // Closed
  "kind": 1633, // Draft
  "content": "<markdown text>",
  "tags": [
    ["e", "<issue-or-PR-or-original-root-patch-id-hex>", "", "root"],
    ["e", "<accepted-revision-root-id-hex>", "", "reply"], // for when revisions applied
    ["p", "<repository-owner>"],
    ["p", "<root-event-author>"],
    ["p", "<revision-author>"],

    // optional for improved subscription filter efficiency
    ["a", "30617:<base-repo-owner-pubkey>:<base-repo-id>", "<relay-url>"],
    ["r", "<earliest-unique-commit-id-of-repo>"]

    // optional for `1631` status
    ["q", "<applied-or-merged-patch-event-id>", "<relay-url>", "<pubkey>"], // for each
    // when merged
    ["merge-commit", "<merge-commit-id>"]
    ["r", "<merge-commit-id>"]
    // when applied
    ["applied-as-commits", "<commit-id-in-master-branch>", ...]
    ["r", "<applied-commit-id>"] // for each
  ]
}
```

The most recent Status event (by `created_at` date) from either the issue/patch author or a maintainer is considered valid.

The Status of a patch-revision is to either that of the root-patch, or `1632` (*Closed*) if the root-patch's Status is `1631` (*Applied/Merged*) and the patch-revision isn't tagged in the `1631` (*Applied/Merged*) event.

User grasp list

List of grasp servers the user generally wishes to use for NIP-34 related activity. It is similar in function to the NIP-65 relay list and NIP-B7 blossom list.

The event SHOULD include a list of `g` tags with grasp service websocket URLs in order of preference.

```
{
  "kind": 10317,
  "content": "",
  "tags": [
    ["g", "<grasp-service-websocket-url>"], // zero or more grasp sever urls
  ],
}
```

Possible things to be added later

- inline file comments kind (we probably need one for patches and a different one for merged files)

NIP-5A

Static Websites (nsites)

`draft` `optional`

This NIP describes a method by which static websites can be hosted from Blossom assets.

Site Manifests

A site manifest event MUST be a regular, replaceable, or an addressable event as defined in NIP-01.

There are two types of nsite manifest event kinds:

- **Root site**: Uses kind `15128` and MUST NOT include a `d` tag. This is a single replaceable event per pubkey and serves as the root site for the pubkey.
- **Named sites**: Uses kind `35128`. These can be smaller websites under a pubkey and can be thought of as subdomains.

Named Sites

A named site uses kind `35128` and MUST include a `d` tag containing the site identifier.

The `d` tag identifies the named site and is used as the site's identifier within the author's namespace.

For canonical named-site URLs, the d tag value MUST match ^[a-z0-9-]{1,13}$ and MUST NOT end with -.

Because DNS labels are limited to 63 characters and pubkeyB36 uses 50 of them, the d tag value MUST be 1-13 characters.

Nsite Event Tags

Tag	Role	Required
path	Maps an absolute path to the sha256 hash of the file served at that path.	Yes, one or more
x	Provides the aggregate hash of the full manifest so the site version is indexable and discoverable.	Optional, recommended
d	Short identifier for the named site. This tag is used only for kind 35128 events.	Required for named sites only
a	References the immediate parent nsite from which this copied site was made.	Required for copied sites only
A	References the origin nsite of a copied site's lineage.	Required for copied sites only
server	Hints which blossom servers can be used to retrieve the blobs.	No
title	Provides a human-readable site title.	No
description	Provides a short human-readable description of the site.	No
source	Points to the site's source code repository or source archive.	No

The event MUST include one or more path tags that map absolute paths to sha256 hashes. Each path tag MUST have the format ["path", "/absolute/path", "sha256hash"] where:

- The first element is the literal string "path"
- The second element is an absolute path ending with a filename and extension
- The third element is the sha256 hash of the file that will be served under this path

The event SHOULD include an x tag containing the site aggregate hash as defined below. The x tag MUST have the format ["x", "<sha256-hex>", "aggregate"], where <sha256-hex> is the lowercase hexadecimal aggregate hash.

The event MAY include server tags that hint at which blossom servers can be used to find the blobs associated with the hashes.

The event MAY include title and description tags that provide simple site information.

The event MAY include a source tag that links to the site's source code repository or source archive. The source tag MUST have the format ["source", "<url>"]. For source code repositories, <url> MAY be a nostr:// git URL as defined by NIP-34, or an absolute https:// git URL. For source archives, <url> MUST be an absolute https:// URL.

Manifest Snapshots

A manifest snapshot uses kind `5128` and is a regular event. Its purpose is to capture the state of a root site or named site at a particular point in time.

When publishing root sites or named sites, authors MAY also publish manifest snapshot events alongside them as additional events in order to preserve version history and make historical versions addressable by snapshot event id. For a given site, the snapshot event's `created_at` timestamp is the version timestamp of that snapshot.

For snapshotting an existing root site or named site, the manifest snapshot event MUST copy the source manifest's `path` tags, MUST include exactly one `x` tag whose value exactly matches the source manifest's aggregate `x` tag, and MUST include exactly one `a` tag referencing the source root site or named site. If the source manifest includes an `A` tag, the snapshot event MUST copy that `A` tag unchanged. If the source manifest does not include an `A` tag, the snapshot event SHOULD omit it. The referenced site identifies the snapshotted nsite, while the snapshot event's own `created_at` identifies the version. Other valid nsite tags such as `title`, `description`, `source`, `app`, `server`, or relay hints MAY also be copied if the author intends to preserve them, but they are optional.

Aggregate Hash

The site aggregate hash is a deterministic hash of the site's `path` tags. It identifies a specific site version and can be used to identify equivalent site manifests published by different pubkeys.

The aggregate hash MUST be computed using only `path` tags. It MUST NOT depend on the order of tags in the manifest, and it MUST ignore all other tags and event fields, including `app` tags.

Two site manifest events are equivalent if and only if they produce the same aggregate hash.

A site manifest SHOULD include this value in an `x` tag with the format `["x", "<sha256-hex>", "aggregate"]` so the aggregate hash is indexable for discoverability and lookup. Manifest snapshot events of kind `5128` MUST include exactly one such tag, and it MUST exactly match the parent nsite's aggregate `x` tag.

To compute the aggregate hash:

1. Collect every `path` tag.
2. For each tag, produce a line in the exact format `<sha256hash> <absolute-path>\n`.
3. Sort all lines in ascending lexicographic order.
4. Concatenate the sorted lines as UTF-8 bytes.
5. Compute the SHA-256 hash of the concatenated bytes.

The resulting digest MUST be encoded as lowercase hexadecimal.

Example inputs:

```
186ea5fd14e88fd1ac49351759e7ab906fa94892002b60bf7f5a428f28ca1c99 /index.html
fedcba0987654321fedcba0987654321fedcba0987654321fedcba0987654321 /favicon.ico
```

Example calculations:

```
nak req -k 15128 -a <pubkey> --limit 1 wss://relay.example.com | jq -r '.tags[] | select(.[0] == "path") | "\(.[2]) \(.[1])"' | sort | sha256sum
```

```
const aggregateHash = sha256(event.tags.filter(t => t[0] === "path").map(([, path, hash]) => `${hash} ${path}\n`).sort().join(""))
```

Upstream App Descriptors

An nsite MAY include one or more `app` tags that reference upstream app descriptor events that the manifest is part of. The `app` tag MUST have the format `["app", "<kind>:<pubkey>:<d-tag>", "<relay>"]`, where the second element is an addressable event reference to an app descriptor and the third element is a relay hint for locating that event.

This linkage is optional. It is intended to make room for application-level discovery and packaging flows without requiring app creators to extend nsite manifests themselves or overload them with non-manifest concerns.

The referenced app descriptor kind is intentionally generic so nsites can reference NIP-89 applications and other present or future forms of app descriptors. Discoverability, app store metadata, runtime compatibility, and similar application-level concerns belong in those app descriptor events, while the nsite manifest remains focused on describing the site's files.

Copying nsites

An nsite MAY be created by copying another root site or named site in order to pin that site under a different author's namespace at a specific state.

The purpose of copying is to preserve and republish an existing nsite while allowing the new author to maintain it independently. For this reason, authors creating a copied nsite SHOULD copy all tags from the source nsite whenever they remain applicable, including `path`, `x`, `app`, `server`, `title`, `description`, and `source`.

Copying an nsite does not require preserving the original site's kind or identifier. A copied root site MAY become a named site, a copied named site MAY become a root site, and a copied named site MAY use a different `d` tag than its parent.

When a site is copied from another nsite, the new manifest MUST include exactly one lowercase `a` tag referencing the immediate parent nsite from which it was copied.

Copied sites MUST also include exactly one uppercase `A` tag referencing the origin nsite of the copy lineage. If a site is copied directly from an original nsite, the `a` and `A` tags MAY reference the same nsite. If a site is copied from another copied site, the `A` tag MUST be copied unchanged from the parent site.

Because copied sites may change kind or identifier, the `a` and `A` tags are the stable, indexable references that preserve the lineage of the copied site.

Sites that were not copied from another nsite SHOULD NOT include `a` or `A` tags.

Host server implementation

A host server is a HTTP server that is responsible for serving pubkey static websites

Address Formats

For interoperability, host servers SHOULD use the following canonical URL formats:

- Root site: <npub>.nsite-host.com
- Snapshot: v<snapshotIdB36>.nsite-host.com
- Named site: <pubkeyB36><dTag>.nsite-host.com

pubkeyB36 is the author's raw 32-byte pubkey encoded with base36 (lowercase, digits 0-9 then letters a-z, no padding) and is always exactly 50 characters.

snapshotIdB36 is the raw 32-byte snapshot event id encoded with base36 (lowercase, digits 0-9 then letters a-z, no padding) and is always exactly 50 characters.

dTag is the site identifier (d tag value) as plain text. It is appended directly after pubkeyB36 with no separator.

This single-label format avoids wildcard certificate limitations with multi-level subdomains.

If the host server is using subdomain routing it MAY serve anything at its own root domain nsite-host.com (a landing page for example).

Example subdomains:

- Root site: npub10phxfsms72rhafrklqdyhempujs9h67nye0p67qe424dyvcx0dkqgvap0e.nsite-host.com
- Snapshot: v<50-char-snapshotIdB36>.nsite-host.com
- Named site: <50-char-pubkeyB36><dTag>.nsite-host.com

For canonical subdomain formats, the host server MUST parse the left-most DNS label as follows:

1. If the label is a valid npub, decode it and resolve the root site manifest.
2. Otherwise, if the label matches ^v[0-9a-z]{50}$, treat it as a snapshot label where:

- snapshotIdB36 is the final 50 characters after the leading v
- decode snapshotIdB36 to the 32-byte event id
- resolve the manifest snapshot for that event id

1. Otherwise, if the label matches ^[0-9a-z]{50}[a-z0-9-]{1,13}$ and does not end with -, treat it as a named-site label where:

- pubkeyB36 is the first 50 characters
- dTag is the remaining 1-13 characters
- decode pubkeyB36 to a 32-byte pubkey
- use dTag as the identifier (d tag value)
- resolve the named site manifest for that pubkey and identifier

If parsing fails, the host server MUST treat the site as not found.

Resolving Paths

Once the site manifest event is found, the host server MUST extract the path-to-hash mappings from the `path` tags in the manifest. The host server SHOULD look for a `path` tag where the second element matches the requested path.

The host server SHOULD verify that the sha256 hash of any served blob matches the hash in the corresponding `path` tag. If the blob hash does not match, the host server SHOULD treat the site as not found.

Index Pages

If the request path does not end with a filename, the host server MUST fall back to using the `index.html` filename.

For example: `/` -> `/index.html` or `/blog/` -> `/blog/index.html`

Once the host server has found the site manifest event and located the matching `path` tag for the requested path, it should use the sha256 hash defined in the third element of the `path` tag to retrieve the file.

The host server SHOULD prioritize using `server` tags from the site manifest event as hints for which blossom servers to query. If the manifest includes `server` tags, the host server SHOULD attempt to retrieve the file from those servers first.

If the pubkey has a `10063` [BUD-03 user servers] event, the server MUST attempt to retrieve the file from the listed servers using the path defined in [BUD-01].

If a pubkey does not have a `10063` event and no `server` tags are found in the manifest, the host server MUST respond with a status code 404.

The host server MUST forward the `Content-Type` and `Content-Length` headers from the Blossom server. If neither is defined, the host server MAY set `Content-Type` from the file extension in the requested path.

Not Found

If a host server is unable to find a site manifest event or a matching `path` tag for the requested path, it MUST use `/404.html` as a fallback path.

Examples

Root Site Manifest

```
{
  "content": "",
  "created_at": 1727373475,
  "id": "5324d695ed7abf7cdd2a48deb881c93b7f4e43de702989bbfb55a1b97b35a3de",
  "kind": 15128,
  "pubkey": "266815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd408a5",
  "sig":
      "f4e4a9e785f70e9fcaa855d769438fea10781e84cd889e3fcb823774f83d094cf2c05d5a3ac4aebc1227a4ebc3d5686728

  "tags": [
```

```
    ["path", "/index.html", "186ea5fd14e88fd1ac49351759e7ab906fa94892002b60bf7f5a428f28ca1c99"],
    ["path", "/about.html", "a1b2c3d4e5f67890123456789012345678901234567890abcdef1234567890abcdef123456"],
    ["path", "/favicon.ico",
        "fedcba0987654321fedcba0987654321fedcba0987654321fedcba0987654321"],
    ["x", "<site-aggregate-sha256>", "aggregate"],
    ["app", "31990:266815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd408a5:my-app",
        "wss://relay.example.com"],
    ["app",
        "32267:266815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd408a5:com.example.webapp",
        "wss://relay.example.com"],

    ["server", "https://blossom.example.com"],
    ["title", "My Nostr Site"],
    ["description", "A static website hosted on Nostr"],
    ["source", "https://github.com/example/my-nostr-site"]
  ]
}
```

Named Site Manifest

```
{
  "content": "",
  "created_at": 1727373475,
  "id": "a1b2c3d4e5f67890123456789012345678901234567890abcdef1234567890abcdef123456",
  "kind": 35128,
  "pubkey": "266815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd408a5",
  "sig":
        "f4e4a9e785f70e9fcaa855d769438fea10781e84cd889e3fcb823774f83d094cf2c05d5a3ac4aebc1227a4ebc3d56867286

  "tags": [
    ["d", "blog"],
    ["path", "/index.html", "186ea5fd14e88fd1ac49351759e7ab906fa94892002b60bf7f5a428f28ca1c99"],
    ["path", "/post.html", "a1b2c3d4e5f67890123456789012345678901234567890abcdef1234567890abcdef123456"],
    ["x", "<site-aggregate-sha256>", "aggregate"],
    ["app", "31990:266815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd408a5:my-blog-
        app", "wss://relay.example.com"],
    ["app",
        "32267:266815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd408a5:com.example.blog",
        "wss://relay.example.com"],

    ["server", "https://blossom.example.com"],
    ["title", "My Blog"],
    ["description", "A blog hosted on Nostr"],
    ["source", "https://github.com/example/my-nostr-blog"]
  ]
}
```

Copied Named Site Manifest

```
{
  "content": "",
  "created_at": 1727373475,
  "id": "b1b2c3d4e5f67890123456789012345678901234567890abcdef1234567890abcdef123456",
  "kind": 35128,
  "pubkey": "9f6815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd4000ff",
  "sig":
```

```
"f4e4a9e785f70e9fcaa855d769438fea10781e84cd889e3fcb823774f83d094cf2c05d5a3ac4aebc1227a4ebc3d5686
7286c15a6df92d55045658bb428fd5fb5",
  "tags": [
    ["d", "blog"],
    // copied directly from the original named site
    ["a", "35128:266815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd408a5:blog"],
    // origin of this copy lineage; same as `a` for a direct copy
    ["A", "35128:266815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd408a5:blog"],
    ["path", "/index.html", "186ea5fd14e88fd1ac49351759e7ab906fa94892002b60bf7f5a428f28ca1c99"],
    ["path", "/post.html", "b1b2c3d4e5f67890123456789012345678901234567890abcdef1234567890abcdef123456"],
    ["x", "<site-aggregate-sha256>", "aggregate"],
    ["title", "My Copied Blog"],
    ["description", "A copy of another named nsite"]
  ]
}
```

Copied Site Derived From Another Copied Site

```
{
  "content": "",
  "created_at": 1727374475,
  "id": "c1b2c3d4e5f67890123456789012345678901234567890abcdef1234567890abcdef123456",
  "kind": 35128,
  "pubkey": "7a6815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd4bbbb",
  "sig":
"f4e4a9e785f70e9fcaa855d769438fea10781e84cd889e3fcb823774f83d094cf2c05d5a3ac4aebc1227a4ebc3d5686
7286c15a6df92d55045658bb428fd5fb5",
  "tags": [
    ["d", "blog"],
    // copied from another copied site, not directly from the origin
    ["a", "35128:9f6815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd4000ff:blog"],
    // still points at the original site where this lineage began
    ["A", "35128:266815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd408a5:blog"],
    ["path", "/index.html", "186ea5fd14e88fd1ac49351759e7ab906fa94892002b60bf7f5a428f28ca1c99"],
    ["path", "/post.html", "c1b2c3d4e5f67890123456789012345678901234567890abcdef1234567890abcdef123456"],
    ["x", "<site-aggregate-sha256>", "aggregate"],
    ["title", "My Derived Blog"],
    ["description", "A copy of a copied nsite that preserves the original lineage"]
  ]
}
```

Manifest Snapshot

```
{
  "content": "",
  "created_at": 1727373475,
  "id": "9f8e7d6c5b4a392817161514131211110ffeeddccbbaa99887766554433221100",
  "kind": 5128,
  "pubkey": "266815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd408a5",
  "sig":
"f4e4a9e785f70e9fcaa855d769438fea10781e84cd889e3fcb823774f83d094cf2c05d5a3ac4aebc1227a4ebc3d5686
7286c15a6df92d55045658bb428fd5fb5",
  "tags": [
    ["a", "35128:266815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd408a5:blog"],
```

```
    ["path", "/index.html", "186ea5fd14e88fd1ac49351759e7ab906fa94892002b60bf7f5a428f28ca1c99"],
    ["path", "/post.html", "a1b2c3d4e5f6789012345678901234567890abcdef1234567890abcdef123456"],
    ["x", "<site-aggregate-sha256>", "aggregate"],
    ["server", "https://blossom.example.com"],
    ["title", "My Blog v1"],
    ["description", "An immutable snapshot of a blog hosted on Nostr"],
    ["source", "https://github.com/example/my-nostr-blog"]
  ]
}
```

Manifest Snapshot Of A Copied Site

```
{
  "content": "",
  "created_at": 1727375475,
  "id": "8f8e7d6c5b4a39281716151413121110ffeeddccbbaa99887766554433221100",
  "kind": 5128,
  "pubkey": "9f6815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd4000ff",
  "sig":
"f4e4a9e785f70e9fcaa855d769438fea10781e84cd889e3fcb823774f83d094cf2c05d5a3ac4aebc1227a4ebc3d5686
7286c15a6df92d55045658bb428fd5fb5",
  "tags": [
    // the site manifest this snapshot captures
    ["a", "35128:9f6815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd4000ff:blog"],
    // copied unchanged from the snapshotted site's origin lineage
    ["A", "35128:266815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd408a5:blog"],
    ["path", "/index.html", "186ea5fd14e88fd1ac49351759e7ab906fa94892002b60bf7f5a428f28ca1c99"],
    ["path", "/post.html", "b1b2c3d4e5f6789012345678901234567890abcdef1234567890abcdef123456"],
    ["x", "<site-aggregate-sha256>", "aggregate"],
    ["app", "31990:266815e0c9210dfa324c6cba3573b14bee49da4209a9456f9484e5106cd408a5:my-blog-
app", "wss://relay.example.com"],
    ["title", "My Copied Blog v1"],
    ["description", "An immutable snapshot of a copied nsite"]
  ]
}
```

> **Warning** `unrecommended`: too complicated, try NIP-99 instead

NIP-15

Nostr Marketplace

`draft` `unrecommended` `optional`

Based on Diagon-Alley.

Implemented in NostrMarket and Plebeian Market.

Terms

- `merchant` - seller of products with NOSTR key-pair

- `customer` - buyer of products with NOSTR key-pair
- `product` - item for sale by the `merchant`
- `stall` - list of products controlled by `merchant` (a `merchant` can have multiple stalls)
- `marketplace` - clientside software for searching `stalls` and purchasing `products`

Nostr Marketplace Clients

Merchant admin

Where the `merchant` creates, updates and deletes `stalls` and `products`, as well as where they manage sales, payments and communication with `customers`.

The `merchant` admin software can be purely clientside, but for `convenience` and uptime, implementations will likely have a server client listening for NOSTR events.

Marketplace

`Marketplace` software should be entirely clientside, either as a stand-alone app, or as a purely frontend webpage. A `customer` subscribes to different merchant NOSTR public keys, and those `merchants` `stalls` and `products` become listed and searchable. The marketplace client is like any other ecommerce site, with basket and checkout. `Marketplaces` may also wish to include a `customer` support area for direct message communication with `merchants`.

`Merchant` publishing/updating products (event)

A merchant can publish these events:

Kind		Description
`0`	`set_meta`	The merchant description (similar with any `nostr` public key).
`30017`	`set_stall`	Create or update a stall.
`30018`	`set_product`	Create or update a product.
`4`	`direct_message`	Communicate with the customer. The messages can be plain-text or JSON.
`5`	`delete`	Delete a product or a stall.

Event `30017`: Create or update a stall.

Event Content

```
{
  "id": <string, id generated by the merchant. Sequential IDs (`0`, `1`, `2`...) are
        discouraged>,
  "name": <string, stall name>,
  "description": <string (optional), stall description>,
  "currency": <string, currency used>,
  "shipping": [
    {
      "id": <string, id of the shipping zone, generated by the merchant>,
      "name": <string (optional), zone name>,
      "cost": <float, base cost for shipping. The currency is defined at the stall level>,
      "regions": [<string, regions included in this zone>]
    }
  ]
}
```

Fields that are not self-explanatory: - `shipping`: - an array with possible shipping zones for this stall. - the customer MUST choose exactly one of those shipping zones. - shipping to different zones can have different costs. For some goods (digital for example) the cost can be zero. - the `id` is an internal value used by the merchant. This value must be sent back as the customer selection. - each shipping zone contains the base cost for orders made to that shipping zone, but a specific shipping cost per product can also be specified if the shipping cost for that product is higher than what's specified by the base cost.

Event Tags

```
{
  "tags": [["d", <string, id of stall]],
  // other fields...
}
```

- the `d` tag is required, its value MUST be the same as the stall `id`.

Event `30018`: Create or update a product

Event Content

```
{
  "id": <string, id generated by the merchant (sequential ids are discouraged)>,
  "stall_id": <string, id of the stall to which this product belong to>,
  "name": <string, product name>,
  "description": <string (optional), product description>,
  "images": <[string], array of image URLs, optional>,
  "currency": <string, currency used>,
  "price": <float, cost of product>,
  "quantity": <int or null, available items>,
  "specs": [
    [<string, spec key>, <string, spec value>]
  ],
  "shipping": [
    {
      "id": <string, id of the shipping zone (must match one of the zones defined for the
        stall)>,
      "cost": <float, extra cost for shipping. The currency is defined at the stall level>
    }
  ]
}
```

Fields that are not self-explanatory: - `quantity` can be null in the case of items with unlimited availability, like digital items, or services - `specs`: - an optional array of key pair values. It allows for the Customer UI to present product specifications in a structure mode. It also allows comparison between products - eg: `[["operating_system", "Android 12.0"], ["screen_size", "6.4 inches"], ["connector_type", "USB Type C"]]`

```
_Open_: better to move `spec` in the `tags` section of the event?
```

- `shipping`:
    - an *optional* array of extra costs to be used per shipping zone, only for products that require special shipping costs to be added to the base shipping cost defined in the stall
    - the `id` should match the id of the shipping zone, as defined in the `shipping` field of the stall

- to calculate the total cost of shipping for an order, the user will choose a shipping option during checkout, and then the client must consider this costs:
    - the `base cost from the stall` for the chosen shipping option
    - the result of multiplying the product units by the `shipping costs specified in the product`, if any.

Event Tags

```
  "tags": [
    ["d", <string, id of product],
    ["t", <string (optional), product category],
    ["t", <string (optional), product category],
    // other fields...
  ],
  ...
```

- the `d` tag is required, its value MUST be the same as the product `id`.
- the `t` tag is as searchable tag, it represents different categories that the product can be part of (`food`, `fruits`). Multiple `t` tags can be present.

Checkout events

All checkout events are sent as JSON strings using NIP-04.

The `merchant` and the `customer` can exchange JSON messages that represent different actions. Each `JSON` message `MUST` have a `type` field indicating the what the JSON represents. Possible types:

Message Type	Sent By	Description
0	Customer	New Order
1	Merchant	Payment Request
2	Merchant	Order Status Update

Step 1: `customer` order (event)

The below JSON goes in content of NIP-04.

```
{
  "id": <string, id generated by the customer>,
  "type": 0,
  "name": <string (optional), ???>,
  "address": <string (optional), for physical goods an address should be provided>,
  "message": <string (optional), message for merchant>,
  "contact": {
    "nostr": <32-bytes hex of a pubkey>,
    "phone": <string (optional), if the customer wants to be contacted by phone>,
    "email": <string (optional), if the customer wants to be contacted by email>
  },
  "items": [
    {
      "product_id": <string, id of the product>,
```

```
      "quantity": <int, how many products the customer is ordering>
    }
  ],
  "shipping_id": <string, id of the shipping zone>
}
```

Open: is `contact.nostr` required?

Step 2: `merchant` request payment (event)

Sent back from the merchant for payment. Any payment option is valid that the merchant can check.

The below JSON goes in `content` of NIP-04.

`payment_options`/`type` include:

- `url` URL to a payment page, stripe, paypal, btcpayserver, etc
- `btc` onchain bitcoin address
- `ln` bitcoin lightning invoice
- `lnurl` bitcoin lnurl-pay

```
{
  "id": <string, id of the order>,
  "type": 1,
  "message": <string, message to customer, optional>,
  "payment_options": [
    {
      "type": <string, option type>,
      "link": <string, url, btc address, ln invoice, etc>
    },
    {
      "type": <string, option type>,
      "link": <string, url, btc address, ln invoice, etc>
    },
    {
      "type": <string, option type>,
      "link": <string, url, btc address, ln invoice, etc>
    }
  ]
}
```

Step 3: `merchant` verify payment/shipped (event)

Once payment has been received and processed.

The below JSON goes in `content` of NIP-04.

```
{
  "id": <string, id of the order>,
  "type": 2,
  "message": <string, message to customer>,
  "paid": <bool: has received payment>,
  "shipped": <bool: has been shipped>,
}
```

Customize Marketplace

Create a customized user experience using the `naddr` from [NIP-19](). The use of `naddr` enables easy sharing of marketplace events while incorporating a rich set of metadata. This metadata can include relays, merchant profiles, and more. Subsequently, it allows merchants to be grouped into a market, empowering the market creator to configure the marketplace's user interface and user experience, and share that marketplace. This customization can encompass elements such as market name, description, logo, banner, themes, and even color schemes, offering a tailored and unique marketplace experience.

Event `30019`: Create or update marketplace UI/UX

Event Content

```
{
  "name": <string (optional), market name>,
  "about": <string (optional), market description>,
  "ui": {
    "picture": <string (optional), market logo image URL>,
    "banner": <string (optional), market logo banner URL>,
    "theme": <string (optional), market theme>,
    "darkMode": <bool, true/false>
  },
  "merchants": [array of pubkeys (optional)],
  // other fields...
}
```

This event leverages naddr to enable comprehensive customization and sharing of marketplace configurations, fostering a unique and engaging marketplace environment.

Auctions

Event `30020`: Create or update a product sold as an auction

Event Content:

```
{
    "id": <String, UUID generated by the merchant. Sequential IDs (`0`, `1`, `2`...) are
        discouraged>,
    "stall_id": <String, UUID of the stall to which this product belong to>,
    "name": <String, product name>,
    "description": <String (optional), product description>,
    "images": <[String], array of image URLs, optional>,
    "starting_bid": <int>,
    "start_date": <int (optional) UNIX timestamp, date the auction started / will start>,
    "duration": <int, number of seconds the auction will run for, excluding eventual time
        extensions that might happen>,
    "specs": [
        [<String, spec key>, <String, spec value>]
    ],
    "shipping": [
        {
```

```
            "id": <String, UUID of the shipping zone. Must match one of the zones defined for
        the stall>,
            "cost": <float, extra cost for shipping. The currency is defined at the stall level>
        }
    ]
}
```

> [!NOTE] Items sold as an auction are very similar in structure to fixed-price items, with some important differences worth noting.

- The `start_date` can be set to a date in the future if the auction is scheduled to start on that date, or can be omitted if the start date is unknown/hidden. If the start date is not specified, the auction will have to be edited later to set an actual date.
- The auction runs for an initial number of seconds after the `start_date`, specified by `duration`.

Event `1021`: Bid

```
{
    "content": <int, amount of sats>,
    "tags": [["e", <event ID of the auction to bid on>]],
    // other fields...
}
```

Bids are simply events of kind `1021` with a `content` field specifying the amount, in the currency of the auction. Bids must reference an auction.

> [!NOTE] Auctions can be edited as many times as desired (they are "addressable events") by the author - even after the start_date, but they cannot be edited after they have received the first bid! This is enforced by the fact that bids reference the event ID of the auction (rather than the product UUID), which changes with every new version of the auctioned product. So a bid is always attached to one "version". Editing the auction after a bid would result in the new product losing the bid!

Event `1022`: Bid confirmation

Event Content:

```
{
    "status": <String, "accepted" | "rejected" | "pending" | "winner">,
    "message": <String (optional)>,
    "duration_extended": <int (optional), number of seconds>
}
```

Event Tags:

```
  "tags": [["e" <event ID of the bid being confirmed>], ["e", <event ID of the auction>]],
```

Bids should be confirmed by the merchant before being considered as valid by other clients. So clients should subscribe to *bid confirmation* events (kind `1022`) for every auction that they follow, in addition to the actual bids and should check that the pubkey of the bid confirmation matches the pubkey of the merchant (in addition to checking the signature).

The `content` field is a JSON which includes *at least* a `status`. `winner` is how the *winning bid* is replied to after the auction ends and the winning bid is picked by the merchant.

The reasons for which a bid can be marked as `rejected` or `pending` are up to the merchant's implementation and configuration - they could be anything from basic validation errors (amount too low) to the bidder being blacklisted or to the bidder lacking sufficient *trust*, which could lead to the bid being marked as `pending` until sufficient verification is performed. The difference between the two is that `pending` bids *might* get approved after additional steps are taken by the bidder, whereas `rejected` bids can not be later approved.

An additional `message` field can appear in the `content` JSON to give further context as of why a bid is `rejected` or `pending`.

Another thing that can happen is - if bids happen very close to the end date of the auction - for the merchant to decide to extend the auction duration for a few more minutes. This is done by passing a `duration_extended` field as part of a bid confirmation, which would contain a number of seconds by which the initial duration is extended. So the actual end date of an auction is always `start_date + duration + (SUM(c.duration_extended) FOR c in all confirmations`.

Customer support events

Customer support is handled over whatever communication method was specified. If communicating via nostr, NIP-04 is used.

Additional

Standard data models can be found here

NIP-99

Classified Listings

`draft` `optional`

This NIP defines `kind:30402`: an addressable event to describe classified listings that list any arbitrary product, service, or other thing for sale or offer and includes enough structured metadata to make them useful.

The specification supports a broad range of use cases physical goods, services, work opportunities, rentals, free giveaways, personals, etc. To promote interoperability between clients implementing NIP-99 for e-commerce, you can find the extension proposal here which standardizes the e-commerce use case while maintaining the specification's lightweight and flexible nature. While NIP-15 provides a strictly structured marketplace specification, NIP-99 has emerged as a simpler and more flexible alternative.

The structure of these events is very similar to NIP-23 long-form content events.

Draft / Inactive Listings

`kind:30403` has the same structure as `kind:30402` and is used to save draft or inactive classified listings.

Content

The `.content` field should be a description of what is being offered and by whom. These events should be a string in Markdown syntax.

Author

The `.pubkey` field of these events are treated as the party creating the listing.

Metadata

- For "tags"/"hashtags" (i.e. categories or keywords of relevance for the listing) the `"t"` event tag should be used.
- For images, whether included in the markdown content or not, clients SHOULD use `image` tags as described in NIP-58. This allows clients to display images in carousel format more easily.

The following tags, used for structured metadata, are standardized and SHOULD be included. Other tags may be added as necessary.

- `"title"`, a title for the listing
- `"summary"`, for short tagline or summary for the listing
- `"published_at"`, for the timestamp (in unix seconds – converted to string) of the first time the listing was published.
- `"location"`, for the location.
- `"price"`, for the price of the thing being listed. This is an array in the format `[ "price", "<number>", "<currency>", "<frequency>" ]`.
  - `"price"` is the name of the tag
  - `"<number>"` is the amount in numeric format (but included in the tag as a string)
  - `"<currency>"` is the currency unit in 3-character ISO 4217 format or ISO 4217-like currency code (e.g. `"btc"`, `"eth"`).
  - `"<frequency>"` is optional and can be used to describe recurring payments. SHOULD be in noun format (hour, day, week, month, year, etc.)
- `"status"` (optional), the status of the listing. SHOULD be either "active" or "sold".

`price` examples

- $50 one-time payment `["price", "50", "USD"]`
- €15 per month `["price", "15", "EUR", "month"]`
- £50,000 per year `["price", "50000", "GBP", "year"]`

Other common tags that might be useful.

- `"g"`, a geohash for more precise location

Example Event

```json
{
  "kind": 30402,
  "created_at": 1675642635,
```

```
  // Markdown content
  "content": "Lorem [ipsum]
        [nostr:nevent1qqst8cujky046negxgwwm5ynqwn53t8aqjr6afd8g59nfqwxpdhylpcpzamhxue69uhhyetvv9ujuetcv9khqm
        consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut e
        exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in repre
        cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culp
        anim id est laborum.\n\nRead more at
        nostr:naddr1qqzkjurnw4ksz9thwden5te0wfjkccte9ehx7um5wghx7un8qgs2d90kkcq3nk2jry62dyf50k0h36rhpdtd594m

  "tags": [
    ["d", "lorem-ipsum"],
    ["title", "Lorem Ipsum"],
    ["published_at", "1296962229"],
    ["t", "electronics"],
    ["image", "https://url.to.img", "256x256"],
    ["summary", "More lorem ipsum that is a little more than the title"],
    ["location", "NYC"],
    ["price", "100", "USD"],
    [
      "e",
      "b3e392b11f5d4f28321cedd09303a748acfd0487aea5a7450b3481c60b6e4f87",
      "wss://relay.example.com"
    ],
    [
      "a",
      "30023:a695f6b60119d9521934a691347d9f78e8770b56da16bb255ee286ddf9fda919:ipsum",
      "wss://relay.nostr.org"
    ]
  ],
  "pubkey": "...",
  "id": "..."
}
```

NIP-94

File Metadata

`draft` `optional`

The purpose of this NIP is to allow an organization and classification of shared files. So that relays can filter and organize in any way that is of interest. With that, multiple types of filesharing clients can be created. NIP-94 support is not expected to be implemented by "social" clients that deal with `kind:1` notes or by longform clients that deal with `kind:30023` articles.

Event format

This NIP specifies the use of the `1063` event kind, having in `content` a description of the file content, and a list of tags described below:

- `url` the url to download the file

- `m` a string indicating the data type of the file. The [MIME types](#) format must be used, and they should be lowercase.
- `x` containing the SHA-256 hexencoded string of the file.
- `ox` containing the SHA-256 hexencoded string of the original file, before any transformations done by the upload server
- `size` (optional) size of file in bytes
- `dim` (optional) size of file in pixels in the form `<width>x<height>`
- `magnet` (optional) URI to magnet file
- `i` (optional) torrent infohash
- `blurhash`(optional) the [blurhash](#) to show while the file is being loaded by the client
- `thumb` (optional) url of thumbnail with same aspect ratio
- `image` (optional) url of preview image with same dimensions
- `summary` (optional) text excerpt
- `alt` (optional) description for accessibility
- `fallback` (optional) zero or more fallback file sources in case `url` fails
- `service` (optional) service type which is serving the file (eg. [NIP-96](#))

```
{
  "kind": 1063,
  "tags": [
    ["url",<string with URI of file>],
    ["m", <MIME type>],
    ["x", <Hash SHA-256>],
    ["ox", <Hash SHA-256>],
    ["size", <size of file in bytes>],
    ["dim", <size of file in pixels>],
    ["magnet", <magnet URI> ],
    ["i", <torrent infohash>],
    ["blurhash", <value>],
    ["thumb", <string with thumbnail URI>, <Hash SHA-256>],
    ["image", <string with preview URI>, <Hash SHA-256>],
    ["summary", <excerpt>],
    ["alt", <description>]
  ],
  "content": "<caption>",
  // other fields...
}
```

Suggested use cases

- A relay for indexing shared files. For example, to promote torrents.
- A pinterest-like client where people can share their portfolio and inspire others.
- A simple way to distribute configurations and software updates.

> **Warning** `unrecommended`: deprecated in favor of [NIP-B7](#)

NIP-96

HTTP File Storage Integration

`draft` `unrecommended` `optional`

Introduction

This NIP defines a REST API for HTTP file storage servers intended to be used in conjunction with the nostr network. The API will enable nostr users to upload files and later reference them by url on nostr notes.

The spec DOES NOT use regular nostr events through websockets for storing, requesting nor retrieving data because, for simplicity, the server will not have to learn anything about nostr relays.

Server Adaptation

File storage servers wishing to be accessible by nostr users should opt-in by making available an https route at `/.well-known/nostr/nip96.json` with `api_url`:

```
{
  // Required
  // File upload and deletion are served from this url
  // Also downloads if "download_url" field is absent or empty string
  "api_url": "https://your-file-server.example/custom-api-path",
  // Optional
  // If absent, downloads are served from the api_url
  "download_url": "https://a-cdn.example/a-path",
  // Optional
  // Note: This field is not meant to be set by HTTP Servers.
  // Use this if you are a nostr relay using your /.well-known/nostr/nip96.json
  // just to redirect to someone else's http file storage server's /.well-known/nostr/nip96.json
  // In this case, "api_url" field must be an empty string
  "delegated_to_url": "https://your-file-server.example",
  // Optional
  "supported_nips": [60],
  // Optional
  "tos_url": "https://your-file-server.example/terms-of-service",
  // Optional
  "content_types": ["image/jpeg", "video/webm", "audio/*"],
  // Optional
  "plans": {
    // "free" is the only standardized plan key and
    // clients may use its presence to learn if server offers free storage
    "free": {
      "name": "Free Tier",
      // Default is true
      // All plans MUST support NIP-98 uploads
      // but some plans may also allow uploads without it
      "is_nip98_required": true,
      "url": "https://...", // plan's landing page if there is one
```

```
      "max_byte_size": 10485760,
      // Range in days / 0 for no expiration
      // [7, 0] means it may vary from 7 days to unlimited persistence,
      // [0, 0] means it has no expiration
      // early expiration may be due to low traffic or any other factor
      "file_expiration": [14, 90],
      "media_transformations": {
        "image": [
          "resizing"
        ]
      }
    }
  }
}
```

Relay Hints

Note: This section is not meant to be used by HTTP Servers.

A nostr relay MAY redirect to someone else's HTTP file storage server by adding a `/.well-known/nostr/nip96.json` with "delegated_to_url" field pointing to the url where the server hosts its own `/.well-known/nostr/nip96.json`. In this case, the "api_url" field must be an empty string and all other fields must be absent.

If the nostr relay is also an HTTP file storage server, it must use the "api_url" field instead.

List of Supporting File Storage Servers

See https://github.com/aljazceru/awesome-nostr#nip-96-file-storage-servers.

Auth

When indicated, `clients` must add an [NIP-98](https://github.com/nostr-protocol/nips/blob/master/98.md) `Authorization` header (**optionally** with the encoded `payload` tag set to the base64-encoded 256-bit SHA-256 hash of the file - not the hash of the whole request body).

Upload

`POST $api_url` as `multipart/form-data`.

AUTH required

List of form fields:

- `file`: **REQUIRED** the file to upload
- `caption`: **RECOMMENDED** loose description;
- `expiration`: UNIX timestamp in seconds. Empty string if file should be stored forever. The server isn't required to honor this.
- `size`: File byte size. This is just a value the server can use to reject early if the file size exceeds the server limits.
- `alt`: **RECOMMENDED** strict description text for visibility-impaired users.

- `media_type`: "avatar" or "banner". Informs the server if the file will be used as an avatar or banner. If absent, the server will interpret it as a normal upload, without special treatment.
- `content_type`: mime type such as "image/jpeg". This is just a value the server can use to reject early if the mime type isn't supported.
- `no_transform`: "true" asks server not to transform the file and serve the uploaded file as is, may be rejected.

Others custom form data fields may be used depending on specific `server` support. The `server` isn't required to store any metadata sent by `clients`.

The `filename` embedded in the file may not be honored by the `server`, which could internally store just the SHA-256 hash value as the file name, ignoring extra metadata. The hash is enough to uniquely identify a file, that's why it will be used on the `download` and `delete` routes.

The `server` MUST link the user's `pubkey` string as the owner of the file so to later allow them to delete the file.

`no_transform` can be used to replicate a file to multiple servers for redundancy, clients can use the server list to find alternative servers which might contain the same file. When uploading a file and requesting `no_transform` clients should check that the hash matches in the response in order to detect if the file was modified.

Response codes

- `200 OK`: File upload exists, but is successful (Existing hash)
- `201 Created`: File upload successful (New hash)
- `202 Accepted`: File upload is awaiting processing, see Delayed Processing section
- `413 Payload Too Large`: File size exceeds limit
- `400 Bad Request`: Form data is invalid or not supported.
- `403 Forbidden`: User is not allowed to upload or the uploaded file hash didnt match the hash included in the `Authorization` header `payload` tag.
- `402 Payment Required`: Payment is required by the server, **this flow is undefined**.

The upload response is a json object as follows:

```
{
  // "success" if successful or "error" if not
  "status": "success",
  // Free text success, failure or info message
  "message": "Upload successful.",
  // Optional. See "Delayed Processing" section
  "processing_url": "...",
  // This uses the NIP-94 event format but DO NOT need
  // to fill some fields like "id", "pubkey", "created_at" and "sig"
  //
  // This holds the download url ("url"),
  // the ORIGINAL file hash before server transformations ("ox")
  // and, optionally, all file metadata the server wants to make available
  //
  // nip94_event field is absent if unsuccessful upload
  "nip94_event": {
    // Required tags: "url" and "ox"
    "tags": [
      // Can be same from /.well-known/nostr/nip96.json's "download_url" field
```

```
      // (or "api_url" field if "download_url" is absent or empty) with appended
      // original file hash.
      //
      // Note we appended .png file extension to the `ox` value
      // (it is optional but extremely recommended to add the extension as it will help nostr
        clients
      // with detecting the file type by using regular expression)
      //
      // Could also be any url to download the file
      // (using or not using the /.well-known/nostr/nip96.json's "download_url" prefix),
      // for load balancing purposes for example.
      ["url", "https://your-file-server.example/custom-api-path/
        719171db19525d9d08dd69cb716a18158a249b7b3b3ec4bbdec5698dca104b7b.png"],
      // SHA-256 hash of the ORIGINAL file, before transformations.
      // The server MUST store it even though it represents the ORIGINAL file because
      // users may try to download/delete the transformed file using this value
      ["ox", "719171db19525d9d08dd69cb716a18158a249b7b3b3ec4bbdec5698dca104b7b"],
      // Optional. SHA-256 hash of the saved file after any server transformations.
      // The server can but does not need to store this value.
      ["x", "543244319525d9d08dd69cb716a18158a249b7b3b3ec4bbde5435543acb34443"],
      // Optional. Recommended for helping clients to easily know file type before downloading
        it.
      ["m", "image/png"],
      // Optional. Recommended for helping clients to reserve an adequate UI space to show the
        file before downloading it.
      ["dim", "800x600"]
      // ... other optional NIP-94 tags
    ],
    "content": ""
  },
  // ... other custom fields (please consider adding them to this NIP or to NIP-94 tags)
}
```

Note that if the server didn't apply any transformation to the received file, both `nip94_event.tags.*.ox` and `nip94_event.tags.*.x` fields will have the same value. The server MUST link the saved file to the SHA-256 hash of the **original** file before any server transformations (the `nip94_event.tags.*.ox` tag value). The **original** file's SHA-256 hash will be used to identify the saved file when downloading or deleting it.

`clients` may upload the same file to one or many `servers`. After successful upload, the `client` may optionally generate and send to any set of nostr `relays` a NIP-94 event by including the missing fields.

Alternatively, instead of using NIP-94, the `client` can share or embed on a nostr note just the above url.

`clients` may also use the tags from the `nip94_event` to construct an `imeta` tag

Delayed Processing

Sometimes the server may want to place the uploaded file in a processing queue for deferred file processing.

In that case, the server MUST serve the original file while the processing isn't done, then swap the original file for the processed one when the processing is over. The upload response is the same as usual but some optional metadata like `nip94_event.tags.*.x` and `nip94_event.tags.*.size` won't be available.

The expected resulting metadata that is known in advance should be returned on the response. For example, if the file processing would change a file from "jpg" to "webp", use ".webp" extension on the `nip94_event.tags.*.url` field value and set "image/webp" to the `nip94_event.tags.*.m` field. If some metadata are unknown before processing ends, omit them from the response.

The upload response MAY include a `processing_url` field informing a temporary url that may be used by clients to check if the file processing is done.

If the processing isn't done, the server should reply at the `processing_url` url with **200 OK** and the following JSON:

```
{
  // It should be "processing". If "error" it would mean the processing failed.
  "status": "processing",
  "message": "Processing. Please check again later for updated status.",
  "percentage": 15 // Processing percentage. An integer between 0 and 100.
}
```

When the processing is over, the server replies at the `processing_url` url with **201 Created** status and a regular successful JSON response already mentioned before (now **without** a `processing_url` field), possibly including optional metadata at `nip94_event.tags.*` fields that weren't available before processing.

File compression

File compression and other transformations like metadata stripping can be applied by the server. However, for all file actions, such as download and deletion, the **original** file SHA-256 hash is what identifies the file in the url string.

Download

```
GET $api_url/<sha256-hash>(.ext)
```

The primary file download url informed at the upload's response field `nip94_event.tags.*.url` can be that or not (it can be any non-standard url the server wants). If not, the server still MUST also respond to downloads at the standard url mentioned on the previous paragraph, to make it possible for a client to try downloading a file on any NIP-96 compatible server by knowing just the SHA-256 file hash.

Note that the "<sha256-hash>" part is from the **original** file, **not** from the **transformed** file if the uploaded file went through any server transformation.

Supporting ".ext", meaning "file extension", is required for `servers`. It is optional, although recommended, for `clients` to append it to the path. When present it may be used by `servers` to know which `Content-Type` header to send (e.g.: "Content-Type": "image/png" for ".png" extension). The file extension may be absent because the hash is the only needed string to uniquely identify a file.

Example: `$api_url/719171db19525d9d08dd69cb716a18158a249b7b3b3ec4bbdec5698dca104b7b.png`

Media Transformations

servers may respond to some media transformation query parameters and ignore those they don't support by serving the original media file without transformations.

Image Transformations

Resizing

Upon upload, servers may create resized image variants, such as thumbnails, respecting the original aspect ratio. clients may use the w query parameter to request an image version with the desired pixel width. servers can then serve the variant with the closest width to the parameter value or an image variant generated on the fly.

Example: $api_url/<sha256-hash>.png?w=32

Deletion

```
DELETE $api_url/<sha256-hash>(.ext)
```

AUTH required

Note that the /<sha256-hash> part is from the **original** file, **not** from the **transformed** file if the uploaded file went through any server transformation.

The extension is optional as the file hash is the only needed file identification.

The server should reject deletes from users other than the original uploader with the appropriate http response code (403 Forbidden).

It should be noted that more than one user may have uploaded the same file (with the same hash). In this case, a delete must not really delete the file but just remove the user's pubkey from the file owners list (considering the server keeps just one copy of the same file, because multiple uploads of the same file results in the same file hash).

The successful response is a 200 OK one with just basic JSON fields:

```json
{
  "status": "success",
  "message": "File deleted."
}
```

Listing files

```
GET $api_url?page=x&count=y
```

AUTH required

Returns a list of files linked to the authenticated users pubkey.

Example Response:

```
{
  "count": 1, // server page size, eg. max(1, min(server_max_page_size, arg_count))
  "total": 1, // total number of files
  "page": 0, // the current page number
  "files": [
    {
      "tags": [
        ["ox", "719171db19525d9d08dd69cb716a18158a249b7b3b3ec4bbdec5698dca104b7b"],
        ["x", "5d2899290e0e69bcd809949ee516a4a1597205390878f780c098707a7f18e3df"],
        ["size", "123456"],
        ["alt", "a meme that makes you laugh"],
        ["expiration",  "1715691139"],
        // ...other metadata
      ],
      "content": "haha funny meme", // caption
      "created_at": 1715691130 // upload timestamp
    }
  ]
}
```

`files` contains an array of NIP-94 events

Query args

- `page` page number (`offset=page*count`)
- `count` number of items per page

Selecting a Server

Note: HTTP File Storage Server developers may skip this section. This is meant for client developers.

A File Server Preference event is a kind 10096 replaceable event meant to select one or more servers the user wants to upload files to. Servers are listed as `server` tags:

```
{
  "kind": 10096,
  "content": "",
  "tags": [
    ["server", "https://file.server.one"],
    ["server", "https://file.server.two"]
  ],
  // other fields...
}
```

NIP-B7

Blossom media

`draft` `optional`

This NIP specifies how Nostr clients can use Blossom for handling media.

Blossom is a set of standards (called BUDs) for dealing with servers that store files addressable by their SHA-256 sums. Nostr clients may make use of all the BUDs for allowing users to upload files, manage their own files and so on, but most importantly Nostr clients SHOULD make use of [BUD-03]() to fetch `kind:10063` lists of servers for each user:

```
{
  "id": "e4bee088334cb5d38cff1616e964369c37b6081be997962ab289d6c671975d71",
  "pubkey": "781208004e09102d7da3b7345e64fd193cd1bc3fce8fdae6008d77f9cabcd036",
  "content": "",
  "kind": 10063,
  "created_at": 1708774162,
  "tags": [
    ["server", "https://blossom.self.hosted"],
    ["server", "https://cdn.blossom.cloud"]
  ],
  "sig":
      "cc5efa74f59e80622c77cacf4dd62076bcb7581b45e9acff471e7963a1f4d8b3406adab5ee1ac9673487480e57d20e52342

}
```

Whenever a Nostr client finds a URL in an event published by a given user and that URL ends a 64-character hex string (with or without an ending file extension) and that URL is not available anymore, that means that string is likely a representation of a sha256 and that the user may have a `kind:10063` list of Blossom servers published.

Given that, the client SHOULD look into the `kind:10063` list for other Blossom servers and lookup for the same 64-character hex string in them, by just using the hex string as a path (optionally with the file extension at the end), producing a URL like `https://blossom.self.hosted/<hex-string>.png`.

When downloading such files Nostr clients SHOULD verify that the sha256-hash of its contents matches the 64-character hex string.

More information can be found at [BUD-03]().

More complex interactions

Clients may use other facilities exposed by Blossom servers (for example, for checking if a file exists in a Blossom server, instead of actually downloading it) which are better documented in the [BUDs]().

NIP-35

Torrents

`draft` `optional`

This NIP defined a new `kind 2003` which is a Torrent.

`kind 2003` is a simple torrent index where there is enough information to search for content and construct the magnet link. No torrent files exist on nostr.

Tags

- `x`: V1 BitTorrent Info Hash, as seen in the [magnet link](#) `magnet:?xt=urn:btih:HASH`
- `file`: A file entry inside the torrent, including the full path ie. `info/example.txt`
- `tracker`: (Optional) A tracker to use for this torrent

In order to make torrents searchable by general category, you SHOULD include a few tags like `movie`, `tv`, `HD`, `UHD` etc.

Tag prefixes

Tag prefixes are used to label the content with references, ie. `["i", "imdb:1234"]`

- `tcat`: A comma separated text category path, ie. `["i", "tcat:video,movie,4k"]`, this should also match the `newznab` category in a best effort approach.
- `newznab`: The category ID from [newznab](#)
- `tmdb`: [The movie database](#) id.
- `ttvdb`: [TV database](#) id.
- `imdb`: [IMDB](#) id.
- `mal`: [MyAnimeList](#) id.
- `anilist`: [AniList](#) id.

A second level prefix should be included where the database supports multiple media types. - `tmdb:movie:693134` maps to `themoviedb.org/movie/693134` - `ttvdb:movie:290272` maps to `thetvdb.com/movies/dune-part-two` - `mal:anime:9253` maps to `myanimelist.net/anime/9253` - `mal:manga:17517` maps to `myanimelist.net/manga/17517`

In some cases the url mapping isnt direct, mapping the url in general is out of scope for this NIP, the section above is only a guide so that implementers have enough information to succsesfully map the url if they wish.

```json
{
  "kind": 2003,
  "content": "<long-description-pre-formatted>",
  "tags": [
    ["title", "<torrent-title>"],
    ["x", "<bittorrent-info-hash>"],
    ["file", "<file-name>", "<file-size-in-bytes>"],
    ["file", "<file-name>", "<file-size-in-bytes>"],
    ["tracker", "udp://mytacker.com:1337"],
    ["tracker", "http://1337-tracker.net/announce"],
    ["i", "tcat:video,movie,4k"],
    ["i", "newznab:2045"],
    ["i", "imdb:tt15239678"],
    ["i", "tmdb:movie:693134"],
    ["i", "ttvdb:movie:290272"],
    ["t", "movie"],
    ["t", "4k"],
  ]
}
```

Torrent Comments

A torrent comment is a `kind 2004` event which is used to reply to a torrent event.

This event works exactly like a `kind 1` and should follow `NIP-10` for tagging.

Implementations

1. dtan.xyz
2. nostrudel.ninja

NIP-64

Chess (Portable Game Notation)

`draft` `optional`

This NIP defines `kind:64` notes representing chess games in PGN format, which can be read by humans and is also supported by most chess software.

Note

Content

The `.content` of these notes is a string representing a PGN-database.

Notes

```json
{
  "kind": 64,
  "content": "1. e4 *",
  // other fields...
}
```

```json
{
  "kind": 64,
  "tags": [
    ["alt", "Fischer vs. Spassky in Belgrade on 1992-11-04 (F/S Return Match, Round 29)"],
    // rest of tags...
  ],
```

```
  "content": "[Event \"F/S Return Match\"]\n[Site \"Belgrade, Serbia JUG\"]\n[Date
        \"1992.11.04\"]\n[Round \"29\"]\n[White \"Fischer, Robert J.\"]\n[Black \"Spassky,
        Boris V.\"]\n[Result \"1/2-1/2\"]
        \n\n1. e4 e5 2. Nf3 Nc6 3. Bb5 {This opening is called the Ruy Lopez.} 3... a6\n4. Ba4
        Nf6 5. O-O Be7 6. Re1 b5 7. Bb3 d6 8. c3 O-O 9. h3 Nb8 10. d4 Nbd7\n11. c4 c6 12. cxb5
        axb5 13. Nc3 Bb7 14. Bg5 b4 15. Nb1 h6 16. Bh4 c5 17.
        dxe5\nNxe4 18. Bxe7 Qxe7 19. exd6 Qf6 20. Nbd2 Nxd6 21. Nc4 Nxc4 22. Bxc4 Nb6\n23. Ne5
        Rae8 24. Bxf7+ Rxf7 25. Nxf7 Rxe1+ 26. Qxe1 Kxf7 27. Qe3 Qg5 28. Qxg5\nhxg5 29. b3 Ke6
        30. a3 Kd6 31. axb4 cxb4 32. Ra5 Nd5 33. f3 Bc8 34. Kf2 Bf5\n35. Ra7 g6 36. Ra6+ Kc5
        37. Ke1 Nf4 38. g3 Nxh3 39. Kd2 Kb5 40. Rd6 Kc5 41. Ra6\nNf2 42. g4 Bd3 43. Re6
        1/2-1/2",
  // other fields...
}
```

Client Behavior

Clients SHOULD display the content represented as chessboard.

Clients SHOULD publish PGN notes in ["export format"] ("strict mode", i.e. created by machines) but expect incoming notes to be in ["import format"] ("lax mode", i.e. created by humans).

Clients SHOULD check whether the formatting is valid and all moves comply with chess rules.

Clients MAY include additional tags (e.g. like `"alt"`) in order to represent the note to users of non-supporting clients.

Relay Behavior

Relays MAY validate PGN contents and reject invalid notes.

Examples

```
// A game where nothing is known. Game still in progress, game abandoned, or result otherwise
unknown.
// Maybe players died before a move has been made.
*
```

```
1. e4 *
```

```
[White "Fischer, Robert J."]
[Black "Spassky, Boris V."]

1. e4 e5 2. Nf3 Nc6 3. Bb5 {This opening is called the Ruy Lopez.} *
```

```
[Event "F/S Return Match"]
[Site "Belgrade, Serbia JUG"]
[Date "1992.11.04"]
[Round "29"]
[White "Fischer, Robert J."]
[Black "Spassky, Boris V."]
[Result "1/2-1/2"]

1. e4 e5 2. Nf3 Nc6 3. Bb5 {This opening is called the Ruy Lopez.} 3... a6
4. Ba4 Nf6 5. O-O Be7 6. Re1 b5 7. Bb3 d6 8. c3 O-O 9. h3 Nb8 10. d4 Nbd7
```

```
11. c4 c6 12. cxb5 axb5 13. Nc3 Bb7 14. Bg5 b4 15. Nb1 h6 16. Bh4 c5 17. dxe5
Nxe4 18. Bxe7 Qxe7 19. exd6 Qf6 20. Nbd2 Nxd6 21. Nc4 Nxc4 22. Bxc4 Nb6
23. Ne5 Rae8 24. Bxf7+ Rxf7 25. Nxf7 Rxe1+ 26. Qxe1 Kxf7 27. Qe3 Qg5 28. Qxg5
hxg5 29. b3 Ke6 30. a3 Kd6 31. axb4 cxb4 32. Ra5 Nd5 33. f3 Bc8 34. Kf2 Bf5
35. Ra7 g6 36. Ra6+ Kc5 37. Ke1 Nf4 38. g3 Nxh3 39. Kd2 Kb5 40. Rd6 Kc5 41. Ra6
Nf2 42. g4 Bd3 43. Re6 1/2-1/2
```

```
[Event "Hourly HyperBullet Arena"]
[Site "https://lichess.org/wxx4GldJ"]
[Date "2017.04.01"]
[White "T_LUKE"]
[Black "decidement"]
[Result "1-0"]
[UTCDate "2017.04.01"]
[UTCTime "11:56:14"]
[WhiteElo "2047"]
[BlackElo "1984"]
[WhiteRatingDiff "+10"]
[BlackRatingDiff "-7"]
[Variant "Standard"]
[TimeControl "30+0"]
[ECO "B00"]
[Termination "Abandoned"]

1. e4 1-0


[Event "Hourly HyperBullet Arena"]
[Site "https://lichess.org/rospUdSk"]
[Date "2017.04.01"]
[White "Bastel"]
[Black "oslochess"]
[Result "1-0"]
[UTCDate "2017.04.01"]
[UTCTime "11:55:56"]
[WhiteElo "2212"]
[BlackElo "2000"]
[WhiteRatingDiff "+6"]
[BlackRatingDiff "-4"]
[Variant "Standard"]
[TimeControl "30+0"]
[ECO "A01"]
[Termination "Normal"]

1. b3 d5 2. Bb2 c6 3. Nc3 Bf5 4. d4 Nf6 5. e3 Nbd7 6. f4 Bg6 7. Nf3 Bh5 8. Bd3 e6 9. O-O Be7 10.
Qe1 O-O 11. Ne5 Bg6 12. Nxg6 hxg6 13. e4 dxe4 14. Nxe4 Nxe4 15. Bxe4 Nf6 16. c4 Bd6 17. Bc2 Qc7
18. f5 Be7 19. fxe6 fxe6 20. Qxe6+ Kh8 21. Qh3+ Kg8 22. Bxg6 Qd7 23. Qe3 Bd6 24. Bf5 Qe7 25.
Be6+ Kh8 26. Qh3+ Nh7 27. Bf5 Rf6 28. Qxh7# 1-0
```

Resources

- PGN Specification: PGN (Portable Game Notation) specification
- PGN Specification Supplement: Addition for adding graphical elements, clock values, eval, …
- PGN Formal Syntax

- PGN Seven Tag Roster
- PGN Import Format
- PGN Export Format
- lichess / pgn-viewer (GitHub): PGN viewer widget, designed to be embedded in content pages

NIP-CC

Geocaching Events

`draft` `optional`

This NIP defines event kinds for geocaching on Nostr. These events allow users to create, share, and log geocaches in a decentralized manner.

Geocache Listing Event (Kind 37516)

Geocache listing events are addressable events of kind `37516` with the following structure:

```json
{
  "kind": 37516,
  "content": "<cache description>",
  "tags": [
    ["d", "<cache-identifier>"],
    ["name", "<cache-name>"],
    ["g", "<geohash>"],
    ["D", "<1-5>"],
    ["T", "<1-5>"],
    ["S", "<size>"],
    ["t", "<type>"],
    ["n", "<type-modifier>"],
    ["hint", "<plaintext hint>"],
    ["mission", "<key quest mission>"],
    ["image", "<image-url>"],
    ["r", "<relay-url>"],
    ["verification", "<verification-pubkey-hex>"]
  ]
}
```

Listing events require all information about the cache and information relevant to finding the cache. These include the `name`, location (`g`), difficulty (`D`) and terrain (`T`) scores, and size (`S`). The type of cache (`t`) is optional and defaults to `traditional` if not specified.

Cache types are determined by individual clients, with common types including `traditional`, `multi`, and `mystery`. Clients should decide which cache types they support based on their implementation needs.

These requirements are well-known and follow existing standards, such as those outlined on geocaching.com.

These events are assumed to be owned by the submitter of the cache, and core details should be maintained by that submitter. However, community logs should also provide context on the current state and validity of the cache.

Content

The content field contains the cache description and any additional information about the cache.

Tags

- `d` (required) - unique identifier for the cache
- `name` (required) - human-readable name for the cache
- `g` (required) - geohash of cache location. To allow for a proximity search, include multiple geohash tags at different precision levels (3-9 characters)
- `D` (required) - integer 1-5 indicating puzzle/finding difficulty (indexed)
- `T` (required) - integer 1-5 indicating terrain difficulty (indexed)
- `S` (required) - one of: `micro`, `small`, `regular`, `large`, `other` (indexed)
- `t` (optional) - cache type, with common values including: `traditional`, `multi`, `mystery`. Defaults to `traditional` if not specified
- `n` (optional) - type modifier(s) that affect lifecycle, claim semantics, or prize nature. See Type Modifiers. Multiple `n` tags MAY be present, but at most one per modifier category
- `hint` (optional) - plaintext hint to help find the cache
- `mission` (optional) - plaintext "Key Quest" mission that finders are expected to complete to legitimately claim the cache (e.g. a passphrase, riddle answer, or item to bring). A treasure MUST NOT include more than one `mission` tag; if multiple are present, clients SHOULD use the first and ignore the rest. When present, clients SHOULD restrict found-log submission to finders who have proof of physical presence (typically the verification key from the cache location). Completions of the mission MAY be recorded as NIP-GD Good Deed events whose `a` tag references the cache
- `image` (optional) - image URLs related to the cache
- `r` (optional) - preferred relay URLs for logs
- `verification` (optional) - hex-encoded public key for verifying finds at this cache
- `F` (optional) - locks in the first-to-find winner. Only valid when the treasure carries the `first-to-find` `n` modifier. Format: `["F", "<winner-pubkey-hex>"]`. See Type Modifiers › `first-to-find`. At most one `F` tag SHOULD be present

Found Log Event (Kind 7516)

Found log events record successful visits to geocaches:

```json
{
  "kind": 7516,
  "content": "<log message>",
  "tags": [
    ["a", "37516:<pubkey>:<d-tag>"]
  ]
}
```

Tags

- `a` (required) - reference to the geocache being logged
- `image` (optional) - photos from the visit
- `verification` (optional) - embedded verification event (see Verified Finds section)

Comment Log Events (Kind 1111)

Non-found logs use comment events (kind `1111`) following NIP-22 comment structure:

```
{
  "kind": 1111,
  "content": "<log message>",
  "tags": [
    ["A", "37516:<pubkey>:<d-tag>"],
    ["K", "37516"],
    ["P", "<cache-owner-pubkey>"],
    ["a", "37516:<pubkey>:<d-tag>"],
    ["k", "37516"],
    ["p", "<cache-owner-pubkey>"],
    ["t", "<log-type>"]
  ]
}
```

These events capture failures, notes, and status-related information about the cache via human reporting, following the NIP-22 comment threading model where the geocache listing is both the root and parent content.

Comment log types include `dnf` (did not find), `note` (helpful or neutral context), and `maintenance` (cache needs attention). If no `t` tag is present, the comment is assumed to be a general note.

Owners of the cache can officially retire caches using an `archived` tag value in the tag `t`, thus allowing the cache's history to be preserved without fully deleting it.

Tags

The `A`/`K`/`P` (root) and `a`/`k`/`p` (parent) tags follow NIP-22. For these top-level comments, the root and parent are identical: they reference the geocache listing (`37516:<pubkey>:<d-tag>`), kind `37516`, and the cache owner's pubkey.

- `t` (optional) - log type: `dnf`, `note`, `maintenance`, `archived`. If omitted, assumed to be `note`
- `image` (optional) - photos from the visit

Geocache Verification Event (Kind 7517)

Verification events provide cryptographic proof that someone physically located a geocache. These events are signed by the cache's verification private key.

```
{
  "kind": 7517,
  "content": "Geocache verification for <finder-npub>",
  "tags": [
    ["a", "<finder-pubkey-hex>:<geocache-naddr>"]
  ]
}
```

Content

The content field must follow the static format: `"Geocache verification for <finder-npub>"` where `<finder-npub>` is the NIP-19 encoded public key (npub) of the person who found the cache.

Tags

- `a` (required) - composite identifier containing the finder's pubkey in hex format and the geocache naddr being verified

Usage

Verification events are created when a finder obtains access to the cache's verification private key (typically via QR code at the cache location). The event must be signed by the cache's verification private key and can be:

1. Embedded in the `verification` tag as a JSON string in the Verified Found Log event (kind 7516).
2. Published to relays as standalone events
3. Both embedded and published for redundancy

Verified Finds

Geocaches with verification enabled can provide cryptographic proof that a finder physically located the cache. This is accomplished through a verification event (kind 7517) signed by the cache's verification key.

Verification Process

When a cache has a `verification` tag containing a public key, finders can create a verified log by:

1. Obtaining the cache's verification private key (typically via QR code at the cache location)
2. Creating a verification event (kind 7517) signed by this key
3. Embedding the verification event in their log entry

Verification Validation

To validate a verified find:

1. Check that the verification event is signed by the expected verification public key
2. Verify that the finder pubkey in the `a` tag matches the log author
3. Confirm the geocache naddr in the `a` tag correctly references the target cache
4. Validate the event signature using standard Nostr verification

Type Modifiers

Geocache listings MAY include one or more `n` tags that classify the geocache with additional type modifiers. Unlike the `t` cache-type tag (which describes *how* a cache is found), `n` modifiers describe *how the cache behaves* once published — its lifecycle, claim semantics, or prize nature.

The `mission` tag is also a type modifier in the broader sense, but it uses its own dedicated tag because it carries a payload (the mission text). Clients SHOULD treat `mission` and `n` modifiers consistently for display purposes (badges, filters, etc.).

Rules

1. Each modifier value belongs to exactly one **category** (see below).
2. A geocache SHOULD include at most one `n` tag per category. If multiple values from the same category are present, clients SHOULD use the first occurrence and ignore the rest.
3. Modifiers from different categories compose freely. Any combination is valid unless a specific modifier's definition states otherwise.
4. Clients SHOULD ignore `n` values they do not recognize, allowing forward compatibility as new modifiers are defined.

Categories and Modifiers

Claim semantics

Modifiers that affect how claims on the treasure are interpreted.

- `first-to-find` — Single-claim geocache listing. The first verified found log (kind 7516 with valid embedded kind 7517) constitutes the exclusive claim. Subsequent verified found logs remain valid records of physical presence at the location but do not constitute additional claims. Clients SHOULD render the geocache listing as effectively archived once any valid verified found log exists, hiding find-submission affordances and displaying the winning finder prominently. Requires a `verification` tag on the geocache listing event.

  Determining the winning log (provisional, before lock-in):

  - The winning log is the verified found log with the earliest `created_at` value.
  - Ties on `created_at` are broken by ascending lexicographic comparison of the event `id`.
  - All verified logs are evidence of physical presence (QR access was required to produce them); the exclusive claim is attributed only to the earliest one.

  Locking in the winner (`F` tag):

  - Once the geocache listing creator has confirmed the claim, the creator SHOULD publish a new revision of the geocache listing event that BOTH archives the listing (adds `["t", "archived"]`) AND locks the winner in by appending an `F` tag: `["F", "<winner-pubkey-hex>"]`
  - The value is the winning finder's pubkey (lowercase hex). The specific winning verified found log is recoverable by querying for verified found logs whose `a` tag references this treasure and whose author matches the `F` pubkey.
  - At most one `F` tag SHOULD be present. If multiple are present, clients SHOULD use the first.
  - When an `F` tag is present, clients MUST attribute the exclusive claim to the pubkey in the `F` tag, regardless of which verified found log currently appears earliest. Because `created_at` is author-supplied and forgeable, this protects the locked-in claim from being displaced by a later log carrying a forged earlier timestamp.

Prize nature

Modifiers that describe what the physical treasure IS.

- `art` — The geocache itself is a physical work of art (a print, sculpture, sticker, zine, painted object, mural, installation, etc.). Whether the work is takeable, viewable in place, photographable only, or otherwise interacted with is determined by other modifiers and by the treasure's `content` description.

Forward Compatibility

New modifiers MAY be defined in future revisions of this NIP or in supplementary NIPs. New categories MAY also be introduced. Clients implementing this NIP SHOULD ignore unknown `n` values rather than rejecting the event.

Geocache Curation List Event (Kind 37517)

Curation list events are addressable events of kind `37517` that group geocaches into curated collections. Clients may present these as adventures, trails, treasure hunts, or any other themed experience.

```json
{
  "kind": 37517,
  "content": "<full description>",
  "tags": [
    ["d", "<list-identifier>"],
    ["title", "<list-title>"],
    ["description", "<short summary>"],
    ["image", "<banner-image-url>"],
    ["g", "<geohash>"],
    ["theme", "<page-theme>"],
    ["map", "<map-style>"],
    ["a", "37516:<pubkey>:<d-tag>"],
    ["a", "37516:<pubkey>:<d-tag>"]
  ]
}
```

Content

The content field contains the full description of the curation list — rules, tips, narrative, or any other context the creator wants to provide.

Tags

- `d` (required) - unique identifier for the list
- `title` (required) - human-readable name for the list
- `a` (required, 1+) - references to geocache listing events (kind 37516 or 37515). Order is preserved and meaningful
- `description` (optional) - short summary shown in browse/card views
- `image` (optional) - banner image URL
- `g` (optional) - geohash of list center location. Include multiple precision levels (3-6 characters) for discovery

- `theme` (optional) - default page theme for the list. Clients should apply this theme when displaying the list, unless the user has explicitly chosen a different theme. Supported values: `adventure`, `mojave`
- `map` (optional) - default map style for the list. Clients should use this as the initial map style when displaying the list, but allow the user to change it. Supported values: `original`, `dark`, `satellite`, `adventure`

Cross-Author References

Curation lists can reference any public geocache regardless of author. The `a` tags use standard Nostr addressable event coordinates (`<kind>:<pubkey>:<d-tag>`), allowing a single list to span geocaches from multiple creators.

Clients

For the best Geocaching experience, clients implementing geocaching support should:

- Support hint encoding, such as ROT13, to prevent spoilers.
- Determine cache status from recent log patterns. Multiple DNF entries and/or maintenance notes would indicate an issue with the cache.
- Publish logs to relays specified in the cache's `r` tags when available.
- Validate geohash precision meets minimum requirements (8+ characters, 9+ for micro caches) before accepting cache submissions.

Examples

Basic Cache

```
{
  "kind": 37516,
  "content": "The first Nostr treasure, left in the aftermath of Oslo Freedom Forum!",
  "tags": [
    ["d", "first-treasure-1748619568668"],
    ["name", "First Treasure"],
    ["g", "u4x"],
    ["g", "u4xs"],
    ["g", "u4xsu"],
    ["g", "u4xsu6"],
    ["g", "u4xsu6r"],
    ["g", "u4xsu6ry"],
    ["g", "u4xsu6ryb"],
    ["D", "1"],
    ["T", "1"],
    ["S", "small"],
    ["t", "traditional"],
    ["hint", "In the branches"],
    ["image", "https://blossom.primal.net/
        74efe01a767b27dead71b8a9bb8278a108360438e78e55194ed9ab14a9382dd3.jpg"]
  ]
}
```

Verified Cache

```json
{
  "kind": 37516,
  "content": "High-security treasure requiring physical verification!",
  "tags": [
    ["d", "verified-treasure-1748619568669"],
    ["name", "Verified Treasure"],
    ["g", "u4xsu6ry"],
    ["D", "3"],
    ["T", "2"],
    ["S", "small"],
    ["t", "traditional"],
    ["hint", "Look for the secret code"],
    ["verification", "6805d4e5c0df48b4f76e2fdcb67a2acb1d97567b01c6fe17a236dc32f34f1c07"]
  ]
}
```

Cache with Key Quest

```json
{
  "kind": 37516,
  "content": "Solve the riddle to claim your prize.",
  "tags": [
    ["d", "key-quest-treasure-1748619568670"],
    ["name", "Riddle of the Old Oak"],
    ["g", "u4xsu6ry"],
    ["D", "4"],
    ["T", "2"],
    ["S", "small"],
    ["t", "mystery"],
    ["hint", "Count the rings on the fallen log"],
    ["mission", "Bring a token of nature you found along the way"],
    ["verification", "6805d4e5c0df48b4f76e2fdcb67a2acb1d97567b01c6fe17a236dc32f34f1c07"]
  ]
}
```

First-to-Find Art

A single-claim treasure where the cache itself is a physical artwork. The first verified finder is the exclusive claimant; how the work is fulfilled (taken home, photographed, etc.) is described in the cache content.

```json
{
  "kind": 37516,
  "content": "Hand-pulled linocut, edition of 1, signed on the back next to the QR. Whoever
       finds it keeps it.",
  "tags": [
    ["d", "linocut-aftermath-1748619568671"],
    ["name", "Aftermath (Linocut #1)"],
    ["g", "u4xsu6ry"],
    ["D", "2"],
    ["T", "2"],
    ["S", "small"],
    ["t", "traditional"],
```

```
    ["n", "first-to-find"],
    ["n", "art"],
    ["hint", "Behind glass, but not in a frame"],
    ["image", "https://blossom.primal.net/example-linocut.jpg"],
    ["verification", "6805d4e5c0df48b4f76e2fdcb67a2acb1d97567b01c6fe17a236dc32f34f1c07"]
  ]
}
```

Found Log

```json
{
  "kind": 7516,
  "content": "Found it! Great hiding spot.",
  "tags": [
    ["a", "37516:0461fcbecc4c3374439932d6b8f11269ccdb7cc973ad7a50ae362db135a474dd:first-treasure-1748619568668"]
  ]
}
```

DNF Log

```json
{
  "kind": 1111,
  "content": "Searched for 30 minutes but couldn't find it. Maybe it's missing?",
  "tags": [
    ["A", "37516:0461fcbecc4c3374439932d6b8f11269ccdb7cc973ad7a50ae362db135a474dd:first-treasure-1748619568668"],
    ["K", "37516"],
    ["P", "0461fcbecc4c3374439932d6b8f11269ccdb7cc973ad7a50ae362db135a474dd"],
    ["a", "37516:0461fcbecc4c3374439932d6b8f11269ccdb7cc973ad7a50ae362db135a474dd:first-treasure-1748619568668"],
    ["k", "37516"],
    ["p", "0461fcbecc4c3374439932d6b8f11269ccdb7cc973ad7a50ae362db135a474dd"],
    ["t", "dnf"]
  ]
}
```

Note Log

```json
{
  "kind": 1111,
  "content": "Lots of muggles around during the day. Best to visit in the evening.",
  "tags": [
    ["A", "37516:0461fcbecc4c3374439932d6b8f11269ccdb7cc973ad7a50ae362db135a474dd:first-treasure-1748619568668"],
    ["K", "37516"],
    ["P", "0461fcbecc4c3374439932d6b8f11269ccdb7cc973ad7a50ae362db135a474dd"],
    ["a", "37516:0461fcbecc4c3374439932d6b8f11269ccdb7cc973ad7a50ae362db135a474dd:first-treasure-1748619568668"],
    ["k", "37516"],
    ["p", "0461fcbecc4c3374439932d6b8f11269ccdb7cc973ad7a50ae362db135a474dd"],
    ["t", "note"]
  ]
}
```

Verification Event


```
{
  "kind": 7517,
  "content": "Geocache verification for
        npub1qc0lc5lxnhxnfxlw2lxkv4x4vp6xsf4d5qwvlhfx6qmz6x4nfhqd8h2z3",
  "tags": [
    ["a",
        "0461fcbecc4c3374439932d6b8f11269ccdb7cc973ad7a50ae362db135a474dd:naddr1qqxnzd3e8q6n2dfk8qcnjve48qmr

  ],
  "pubkey": "6805d4e5c0df48b4f76e2fdcb67a2acb1d97567b01c6fe17a236dc32f34f1c07",
  "created_at": 1672531200,
  "sig":
        "a1b2c3d4e5f67890123456789012345678901234567890abcdef1234567890abcdef1234567890123456789012345678901234567890abcdef12345

}
```


Verified Found Log


```
{
  "kind": 7516,
  "content": "Found it! Great hiding spot.",
  "tags": [
    ["a", "37516:0461fcbecc4c3374439932d6b8f11269ccdb7cc973ad7a50ae362db135a474dd:verified-
        treasure-1748619568669"],
    ["verification", "{\"kind\":7517,\"content\":\"Geocache verification for npub1qc0lc5lxnhxnfxlw2lxkv4x4vp
        \"sig\":\"a1b2c3d4e5f67890123456789012345678901234567890abcdef1234567890abcdef1234567890123456789012345678901234567890ab

  ]
}
```


Geocache Curation List


```
{
  "kind": 37517,
  "content": "Explore the festival grounds and find all hidden treasures before the jousting
        tournament!",
  "tags": [
    ["d", "ren-fest-hunt-1748619568670"],
    ["title", "Texas Ren Fest Treasure Hunt"],
    ["description", "Find all the hidden treasures at the festival!"],
    ["image", "https://blossom.primal.net/banner-example.jpg"],
    ["g", "9vk"],
    ["g", "9vk5"],
    ["g", "9vk5b"],
    ["g", "9vk5b7"],
    ["theme", "adventure"],
    ["map", "adventure"],
    ["a", "37516:0461fcbecc4c3374439932d6b8f11269ccdb7cc973ad7a50ae362db135a474dd:first-
        treasure-1748619568668"],
    ["a", "37516:0461fcbecc4c3374439932d6b8f11269ccdb7cc973ad7a50ae362db135a474dd:verified-
        treasure-1748619568669"]
  ]
}
```

NIP-78

Arbitrary custom app data

`draft` `optional`

The goal of this NIP is to enable remoteStorage-like capabilities for custom applications that do not care about interoperability.

Even though interoperability is great, some apps do not want or do not need interoperability, and it wouldn't make sense for them. Yet Nostr can still serve as a generalized data storage for these apps in a "bring your own database" way, for example: a user would open an app and somehow input their preferred relay for storage, which would then enable these apps to store application-specific data there.

Nostr event

This NIP specifies the use of event kinds `30078` (an *addressable* event) with a `d` tag containing some reference to the app name and context – or any other arbitrary string. `content` and other `tags` can be anything or in any format.

It also defines the kind `78` (a *normal* event) for when the app needs to store and query multiple events of the same type. Apps should use some unique tag (even a `d` tag) for grouping these events together.

Some use cases

- User personal settings on Nostr clients (and other apps unrelated to Nostr)
- A way for client developers to propagate dynamic parameters to users without these having to update
- Personal private data generated by apps that have nothing to do with Nostr, but allow users to use Nostr relays as their personal database

Security

Nostr's security model is short: whoever holds the private key *is* the user. There is no password reset, no support desk, and no company that can freeze an account. These NIPs are about keeping that key usable, encrypting payloads so relays cannot read them, and asking the network to forget you if you need to leave.

NIP-44 is versioned payload encryption. It is the crypto underneath modern private messages (NIP-17) and a better default than the scheme in NIP-04. You will rarely implement it as a user-facing feature. You will implement it if you handle secrets on Nostr at all.

Key handling is the rest of the personal side. NIP-06 derives keys from a mnemonic seed phrase. It is unrecommended; the current advice is to treat a single `nsec` as the identity rather than a BIP-39 tree of them. NIP-49 is `ncryptsec`, a way to encrypt a private key with a password so you can back it up without leaving the secret in plaintext. NIP-98 is HTTP authentication: sign a small event to prove you own a key when talking to an ordinary web API.

NIP-62 is a request to vanish. You ask relays to delete everything from your key, including deletion events, and not to take the notes back. Some jurisdictions treat that kind of request as binding. Relays that support it should actually delete. This is as close as Nostr gets to "I want out."

In this chapter

- NIP-44 — Encrypted payloads (versioned)
- NIP-06 — Key derivation from a mnemonic (unrecommended; prefer a single `nsec`)
- NIP-49 — Private key encryption (`ncryptsec`)
- NIP-98 — HTTP auth
- NIP-62 — Request to vanish

NIP-44

Encrypted Payloads (Versioned)

`optional`

The NIP introduces a new data format for keypair-based encryption. This NIP is versioned to allow multiple algorithm choices to exist simultaneously. This format may be used for many things, but MUST be used in the context of a signed event as described in NIP-01.

Note: this format DOES NOT define any `kinds` related to a new direct messaging standard, only the encryption required to define one. It SHOULD NOT be used as a drop-in replacement for NIP-04 payloads.

Versions

Currently defined encryption algorithms:

- `0x00` - Reserved
- `0x01` - Deprecated and undefined
- `0x02` - secp256k1 ECDH, HKDF, padding, ChaCha20, HMAC-SHA256, base64

Limitations

Every nostr user has their own public key, which solves key distribution problems present in other solutions. However, nostr's relay-based architecture makes it difficult to implement more robust private messaging protocols with things like metadata hiding, forward secrecy, and post compromise secrecy.

The goal of this NIP is to have a *simple* way to encrypt payloads used in the context of a signed event. When applying this NIP to any use case, it's important to keep in mind your users' threat model and this NIP's limitations. For high-risk situations, users should chat in specialized E2EE messaging software and limit use of nostr to exchanging contacts.

On its own, messages sent using this scheme have a number of important shortcomings:

- No deniability: it is possible to prove an event was signed by a particular key
- No forward secrecy: when a key is compromised, it is possible to decrypt all previous conversations

- No post-compromise security: when a key is compromised, it is possible to decrypt all future conversations
- No post-quantum security: a powerful quantum computer would be able to decrypt the messages
- IP address leak: user IP may be seen by relays and all intermediaries between user and relay
- Date leak: `created_at` is public, since it is a part of NIP-01 event
- Limited message size leak: padding only partially obscures true message length
- No attachments: they are not supported

Lack of forward secrecy may be partially mitigated by only sending messages to trusted relays, and asking relays to delete stored messages after a certain duration has elapsed.

Version 2

NIP-44 version 2 has the following design characteristics:

- Payloads are authenticated using a MAC before signing rather than afterwards because events are assumed to be signed as specified in NIP-01. The outer signature serves to authenticate the full payload, and MUST be validated before decrypting.
- ChaCha is used instead of AES because it's faster and has better security against multi-key attacks.
- ChaCha is used instead of XChaCha because XChaCha has not been standardized. Also, xChaCha's improved collision resistance of nonces isn't necessary since every message has a new (key, nonce) pair.
- HMAC-SHA256 is used instead of Poly1305 because polynomial MACs are much easier to forge.
- SHA256 is used instead of SHA3 or BLAKE because it is already used in nostr. Also BLAKE's speed advantage is smaller in non-parallel environments.
- A custom padding scheme is used instead of padmé because it provides better leakage reduction for small messages.
- Base64 encoding is used instead of another encoding algorithm because it is widely available, and is already used in nostr.

Encryption

1. Calculate a conversation key
    - Execute ECDH (scalar multiplication) of public key B by private key A Output `shared_x` must be unhashed, 32-byte encoded x coordinate of the shared point
    - Use HKDF-extract with sha256, `IKM=shared_x` and `salt=utf8_encode('nip44-v2')`
    - HKDF output will be a `conversation_key` between two users.
    - It is always the same, when key roles are swapped: `conv(a, B) == conv(b, A)`
2. Generate a random 32-byte nonce
    - Always use CSPRNG
    - Don't generate a nonce from message content
    - Don't re-use the same nonce between messages: doing so would make them decryptable, but won't leak the long-term key
3. Calculate message keys
    - The keys are generated from `conversation_key` and `nonce`. Validate that both are 32 bytes long
    - Use HKDF-expand, with sha256, `PRK=conversation_key`, `info=nonce` and `L=76`
    - Slice 76-byte HKDF output into: `chacha_key` (bytes 0..32), `chacha_nonce` (bytes 32..44), `hmac_key` (bytes 44..76)

4. Add padding
    - Content must be encoded from UTF-8 into byte array
    - Validate plaintext length. Minimum is 1 byte, maximum is 4,294,967,295 bytes
    - Padding algorithm is related to powers-of-two, with min padded msg size of 32 bytes
    - Plaintext length prefix is encoded in big-endian:
        - If length is less than 65536: prefix is 2 bytes (`u16`), format is `[plaintext_length: u16][plaintext][zero_bytes]`
        - If length is 65536 or greater: prefix is 6 bytes (2 zero bytes + `u32`), format is `[0x00, 0x00][plaintext_length: u32][plaintext][zero_bytes]`
    - A zero value in the first 2 bytes signals the extended format; since valid plaintext is at least 1 byte, a u16 length of 0 is otherwise invalid
5. Encrypt padded content
    - Use ChaCha20, with key and nonce from step 3
6. Calculate MAC (message authentication code)
    - AAD (additional authenticated data) is used - instead of calculating MAC on ciphertext, it's calculated over a concatenation of `nonce` and `ciphertext`
    - Validate that AAD (nonce) is 32 bytes
7. Base64-encode (with padding) params using `concat(version, nonce, ciphertext, mac)`

Encrypted payloads MUST be included in an event's payload, hashed, and signed as defined in NIP 01, using schnorr signature scheme over secp256k1.

Decryption

Before decryption, the event's pubkey and signature MUST be validated as defined in NIP 01. The public key MUST be a valid non-zero secp256k1 curve point, and the signature must be valid secp256k1 schnorr signature. For exact validation rules, refer to BIP-340.

1. Check if first payload's character is `#`
    - `#` is an optional future-proof flag that means non-base64 encoding is used
    - The `#` is not present in base64 alphabet, but, instead of throwing `base64 is invalid`, implementations MUST indicate that the encryption version is not yet supported
2. Decode base64
    - Base64 is decoded into `version, nonce, ciphertext, mac`
    - If the version is unknown, implementations must indicate that the encryption version is not supported
    - Validate minimum length of base64 message to prevent DoS on base64 decoder: it must be at least 132 chars
    - Validate minimum length of decoded message to verify output of the decoder: it must be at least 99 bytes
3. Calculate conversation key
    - See step 1 of [encryption](#encryption)
4. Calculate message keys
    - See step 3 of [encryption](#encryption)
5. Calculate MAC (message authentication code) with AAD and compare
    - Stop and throw an error if MAC doesn't match the decoded one from step 2
    - Use constant-time comparison algorithm

6. Decrypt ciphertext
   - Use ChaCha20 with key and nonce from step 3
7. Remove padding
   - Read the first 2 bytes as a big-endian u16
     - If zero, read the next 4 bytes as a big-endian u32 plaintext length (6-byte prefix total)
     - Otherwise, use those 2 bytes as the u16 plaintext length (2-byte prefix total)
   - Verify that the length of sliced plaintext matches the decoded length
   - Verify that calculated padding from step 3 of the encryption process matches the actual padding

Implementation considerations

The theoretical maximum plaintext size is 2^32 - 1 bytes (~4 GB). Implementations SHOULD enforce their own maximum payload size based on platform and resource constraints, rejecting oversized payloads early in `decode_payload` (before base64 decoding) to prevent denial-of-service. Decryption may require several times the payload size in working memory due to base64 decoding, byte array slicing, and padding operations. For reference, JVM-based systems are limited to ~2 GB contiguous arrays, and mobile devices may have significantly less available memory. Note that `calc_padded_len` can return values up to 2^32, which exceeds the range of unsigned 32-bit integers; implementations must use 64-bit (or larger) arithmetic for padding calculations.

Details

- Cryptographic methods
  - `secure_random_bytes(length)` fetches randomness from CSPRNG.
  - `hkdf(IKM, salt, info, L)` represents HKDF (RFC 5869) with SHA256 hash function comprised of methods `hkdf_extract(IKM, salt)` and `hkdf_expand(OKM, info, L)`.
  - `chacha20(key, nonce, data)` is ChaCha20 (RFC 8439) with starting counter set to 0.
  - `hmac_sha256(key, message)` is HMAC (RFC 2104).
  - `secp256k1_ecdh(priv_a, pub_b)` is multiplication of point B by scalar a (`a ⋅ B`), defined in BIP340. The operation produces a shared point, and we encode the shared point's 32-byte x coordinate, using method `bytes(P)` from BIP340. Private and public keys must be validated as per BIP340: pubkey must be a valid, on-curve point, and private key must be a scalar in range `[1, secp256k1_order - 1]`. NIP44 doesn't do hashing of the output: keep this in mind, because some libraries hash it using sha256. As an example, in libsecp256k1, unhashed version is available in `secp256k1_ec_pubkey_tweak_mul`
- Operators
  - `x[i:j]`, where `x` is a byte array and `i, j <= 0` returns a `(j - i)`-byte array with a copy of the `i`-th byte (inclusive) to the `j`-th byte (exclusive) of `x`.
- Constants `c`:
  - `min_plaintext_size` is 1. 1 byte msg is padded to 32 bytes.
  - `max_plaintext_size` is 4294967295 (2^32 - 1).
  - `extended_prefix_threshold` is 65536. Lengths below this use a 2-byte u16 prefix; lengths at or above use a 6-byte prefix (2 zero bytes + u32).
- Functions
  - `base64_encode(string)` and `base64_decode(bytes)` are Base64 (RFC 4648, with padding)
  - `concat` refers to byte array concatenation
  - `is_equal_ct(a, b)` is constant-time equality check of 2 byte arrays

- `utf8_encode(string)` and `utf8_decode(bytes)` transform string to byte array and back
- `write_u8(number)` restricts number to values 0..255 and encodes into Big-Endian uint8 byte array
- `write_u16_be(number)` restricts number to values 0..65535 and encodes into Big-Endian uint16 byte array
- `write_u32_be(number)` restricts number to values 0..4294967295 and encodes into Big-Endian uint32 byte array
- `read_uint16_be(bytes)` reads 2 bytes as a Big-Endian unsigned 16-bit integer
- `read_uint32_be(bytes)` reads 4 bytes as a Big-Endian unsigned 32-bit integer
- `zeros(length)` creates byte array of length `length >= 0`, filled with zeros
- `floor(number)` and `log2(number)` are well-known mathematical methods

Implementation pseudocode

The following is a collection of python-like pseudocode functions which implement the above primitives, intended to guide implementers. A collection of implementations in different languages is available at https://github.com/paulmillr/nip44.

```
# Calculates length of the padded byte array.
def calc_padded_len(unpadded_len):
  next_power = 1 << (floor(log2(unpadded_len - 1))) + 1
  if next_power <= 256:
    chunk = 32
  else:
    chunk = next_power / 8
  if unpadded_len <= 32:
    return 32
  else:
    return chunk * (floor((unpadded_len - 1) / chunk) + 1)

# Converts unpadded plaintext to padded bytearray
def pad(plaintext):
  unpadded = utf8_encode(plaintext)
  unpadded_len = len(unpadded)
  if (unpadded_len < c.min_plaintext_size or
      unpadded_len > c.max_plaintext_size): raise Exception('invalid plaintext length')
  if unpadded_len >= c.extended_prefix_threshold:
    prefix = concat([0, 0], write_u32_be(unpadded_len))  # 6 bytes
  else:
    prefix = write_u16_be(unpadded_len)                   # 2 bytes
  suffix = zeros(calc_padded_len(unpadded_len) - unpadded_len)
  return concat(prefix, unpadded, suffix)

# Converts padded bytearray to unpadded plaintext
def unpad(padded):
  first_two = read_uint16_be(padded[0:2])
  if first_two == 0:
    unpadded_len = read_uint32_be(padded[2:6])
    if unpadded_len < c.extended_prefix_threshold: raise Exception('invalid padding')
    prefix_len = 6
  else:
    unpadded_len = first_two
    prefix_len = 2
  unpadded = padded[prefix_len:prefix_len+unpadded_len]
```

```
  if (unpadded_len == 0 or
      len(unpadded) != unpadded_len or
      len(padded) != prefix_len + calc_padded_len(unpadded_len)): raise Exception('invalid
        padding')
  return utf8_decode(unpadded)

# metadata: always 65b (version: 1b, nonce: 32b, mac: 32b)
# plaintext: 1b to 0xffffffff
# padded plaintext (small, <65536): 32b to 0x10000, with 2b prefix -> 34b to 0x10000+2
# padded plaintext (large, >=65536): 0x10000 to 0x100000000, with 6b prefix -> 0x10006 to
        0x100000000+6
# ciphertext: same as padded plaintext (chacha20 doesn't change length)
# raw payload (small): 99 (65+34) to 65603 (65+0x10000+2)
# raw payload (large): 65607 (65+0x10006) to 4294967367 (65+0x100000000+6)
def decode_payload(payload):
  plen = len(payload)
  if plen == 0 or payload[0] == '#': raise Exception('unknown version')
  if plen < 132: raise Exception('invalid payload size')
  data = base64_decode(payload)
  dlen = len(data)
  if dlen < 99: raise Exception('invalid data size');
  vers = data[0]
  if vers != 2: raise Exception('unknown version ' + vers)
  nonce = data[1:33]
  ciphertext = data[33:dlen - 32]
  mac = data[dlen - 32:dlen]
  return (nonce, ciphertext, mac)

def hmac_aad(key, message, aad):
  if len(aad) != 32: raise Exception('AAD associated data must be 32 bytes');
  return hmac(sha256, key, concat(aad, message));

# Calculates long-term key between users A and B: `get_key(Apriv, Bpub) == get_key(Bpriv, Apub)`
def get_conversation_key(private_key_a, public_key_b):
  shared_x = secp256k1_ecdh(private_key_a, public_key_b)
  return hkdf_extract(IKM=shared_x, salt=utf8_encode('nip44-v2'))

# Calculates unique per-message key
def get_message_keys(conversation_key, nonce):
  if len(conversation_key) != 32: raise Exception('invalid conversation_key length')
  if len(nonce) != 32: raise Exception('invalid nonce length')
  keys = hkdf_expand(OKM=conversation_key, info=nonce, L=76)
  chacha_key = keys[0:32]
  chacha_nonce = keys[32:44]
  hmac_key = keys[44:76]
  return (chacha_key, chacha_nonce, hmac_key)

def encrypt(plaintext, conversation_key, nonce):
  (chacha_key, chacha_nonce, hmac_key) = get_message_keys(conversation_key, nonce)
  padded = pad(plaintext)
  ciphertext = chacha20(key=chacha_key, nonce=chacha_nonce, data=padded)
  mac = hmac_aad(key=hmac_key, message=ciphertext, aad=nonce)
  return base64_encode(concat(write_u8(2), nonce, ciphertext, mac))

def decrypt(payload, conversation_key):
```

```
  (nonce, ciphertext, mac) = decode_payload(payload)
  (chacha_key, chacha_nonce, hmac_key) = get_message_keys(conversation_key, nonce)
  calculated_mac = hmac_aad(key=hmac_key, message=ciphertext, aad=nonce)
  if not is_equal_ct(calculated_mac, mac): raise Exception('invalid MAC')
  padded_plaintext = chacha20(key=chacha_key, nonce=chacha_nonce, data=ciphertext)
  return unpad(padded_plaintext)

# Usage:
#   conversation_key = get_conversation_key(sender_privkey, recipient_pubkey)
#   nonce = secure_random_bytes(32)
#   payload = encrypt('hello world', conversation_key, nonce)
#   'hello world' == decrypt(payload, conversation_key)
```

Audit

The v2 of the standard was audited by Cure53 in December 2023. Check out audit-2023.12.pdf and auditor's website.

Tests and code

A collection of implementations in different languages is available at https://github.com/paulmillr/nip44.

We publish extensive test vectors. Instead of having it in the document directly, a sha256 checksum of vectors is provided:

```
269ed0f69e4c192512cc779e78c555090cebc7c785b609e338a62afc3ce25040  nip44.vectors.json
```

Example of a test vector from the file:


```
{
  "sec1": "0000000000000000000000000000000000000000000000000000000000000001",
  "sec2": "0000000000000000000000000000000000000000000000000000000000000002",
  "conversation_key": "c41c775356fd92eadc63ff5a0dc1da211b268cbea22316767095b2871ea1412d",
  "nonce": "0000000000000000000000000000000000000000000000000000000000000001",
  "plaintext": "a",
  "payload":
        "AgAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABee0G5VSK0/9YypIObAtDKfYEAjD35uVkHyB0F4DwrcNaCXlCWZKaArs

}
```


The file also contains intermediate values. A quick guidance with regards to its usage:

- `valid.get_conversation_key`: calculate conversation_key from secret key sec1 and public key pub2
- `valid.get_message_keys`: calculate chacha_key, chacha_nonce, hmac_key from conversation_key and nonce
- `valid.calc_padded_len`: take unpadded length (first value), calculate padded length (second value)
- `valid.encrypt_decrypt`: emulate real conversation. Calculate pub2 from sec2, verify conversation_key from (sec1, pub2), encrypt, verify payload, then calculate pub1 from sec1, verify conversation_key from (sec2, pub1), decrypt, verify plaintext.
- `valid.encrypt_decrypt_long_msg`: same as previous step, but instead of a full plaintext and payload, their checksum is provided.
- `invalid.encrypt_msg_lengths`
- `invalid.get_conversation_key`: calculating conversation_key must throw an error

- `invalid.decrypt`: decrypting message content must throw an error

Extended length prefix test vectors

The following test vectors exercise the boundary between the 2-byte u16 prefix and the 6-byte extended prefix. Since the payloads are too large to include inline, SHA-256 checksums of the plaintext and base64-encoded payload are provided (following the `encrypt_decrypt_long_msg` pattern).

All vectors use the same `conversation_key` and `nonce` as above. Plaintext is the byte `0x61` (`'a'`) repeated to the specified length.

```
conversation_key: c41c775356fd92eadc63ff5a0dc1da211b268cbea22316767095b2871ea1412d
nonce:            0000000000000000000000000000000000000000000000000000000000000001
```

plaintext_len	prefix	padded_len	plaintext_sha256	payloa
65535	u16 (2 bytes)	65536	`6e1bebca6a8229364a162a72ef064826c4cd7457bf54f190ef782bd9deff3e42`	`6d8c28`
65536	extended (6 bytes)	65536	`bf718b6f653bebc184e1479f1935b8da974d701b893afcf49e701f3e2f9f9c5a`	`b7b4ed`
65537	extended (6 bytes)	81920	`008ffc88d3c96a9f307524eb361e47c5222a887fc45fa0c1fb8d429c5c23b430`	`eeb7c7`

Note that 65535 and 65536 both have a `padded_len` of 65536, but the total padded-with-prefix sizes differ: 65538 (2 + 65536) vs 65542 (6 + 65536). The jump to 65537 triggers the next padding bucket at 81920.

> **Warning** `unrecommended`: prefer a single nsec

NIP-06

Basic key derivation from mnemonic seed phrase

`draft` `unrecommended` `optional`

[BIP39](https://bips.xyz/39) is used to generate mnemonic seed words and derive a binary seed from them.

[BIP32](https://bips.xyz/32) is used to derive the path `m/44'/1237'/<account>'/0/0` (according to the Nostr entry on [SLIP44](https://github.com/satoshilabs/slips/blob/master/slip-0044.md)).

A basic client can simply use an `account` of `0` to derive a single key. For more advanced use-cases you can increment `account`, allowing generation of practically infinite keys from the 5-level path with hardened derivation.

Other types of clients can still get fancy and use other derivation paths for their own other purposes.

Test vectors

mnemonic: leader monkey parrot ring guide accident before fence cannon height naive bean
private key (hex): 7f7ff03d123792d6ac594bfa67bf6d0c0ab55b6b1fdb6249303fe861f1ccba9a
nsec: nsec10allq0gjx7fddtzef0ax00mdps9t2kmtrldkyjfs8l5xruwvh2dq0lhhkp

public key (hex): 17162c921dc4d2518f9a101db33695df1afb56ab82f5ff3e5da6eec3ca5cd917
npub: npub1zutzeysacnf9rru6zqwmxd54mud0k44tst6l70ja5mhv8jjumytsd2x7nu

---

mnemonic: what bleak badge arrange retreat wolf trade produce cricket blur garlic valid proud rude strong choose busy staff weather area salt hollow arm fade
private key (hex): c15d739894c81a2fcfd3a2df85a0d2c0dbc47a280d092799f144d73d7ae78add
nsec: nsec1c9wh8xy5eqdzln7n5t0ctgxjcrdug73gp5yj0x03gntn67h83twssdfhel
public key (hex): d41b22899549e1f3d335a31002cfd382174006e166d3e658e3a5eecdb6463573
npub: npub16sdj9zv4f8sl85e45vgq9n7nsgt5qphpvmf7vk8r5hhvmdjxx4es8rq74h

NIP-49

Private Key Encryption (`ncryptsec`)

`draft` `optional`

This NIP defines a method by which clients can encrypt (and decrypt) a user's private key with a password.

Symmetric Encryption Key derivation

PASSWORD = Read from the user. The password should be unicode normalized to NFKC format to ensure that the password can be entered identically on other computers/clients.

LOG_N = Let the user or implementer choose one byte representing a power of 2 (e.g. 18 represents 262,144) which is used as the number of rounds for scrypt. Larger numbers take more time and more memory, and offer better protection:

```
| LOG_N | MEMORY REQUIRED | APPROX TIME ON FAST COMPUTER |
|-------|-----------------|----------------------------- |
| 16    | 64 MiB          | 100 ms                       |
| 18    | 256 MiB         |                              |
| 20    | 1 GiB           | 2 seconds                    |
| 21    | 2 GiB           |                              |
| 22    | 4 GiB           |                              |
```

SALT = 16 random bytes

SYMMETRIC_KEY = scrypt(password=PASSWORD, salt=SALT, log_n=LOG_N, r=8, p=1)

The symmetric key should be 32 bytes long.

This symmetric encryption key is temporary and should be zeroed and discarded after use and not stored or reused for any other purpose.

Encrypting a private key

The private key encryption process is as follows:

PRIVATE_KEY = User's private (secret) secp256k1 key as 32 raw bytes (not hex or bech32 encoded!)

KEY_SECURITY_BYTE = one of:

- 0x00 - if the key has been known to have been handled insecurely (stored unencrypted, cut and paste unencrypted, etc)
- 0x01 - if the key has NOT been known to have been handled insecurely (stored unencrypted, cut and paste unencrypted, etc)
- 0x02 - if the client does not track this data

ASSOCIATED_DATA = KEY_SECURITY_BYTE

NONCE = 24 byte random nonce

CIPHERTEXT = XChaCha20-Poly1305(plaintext=PRIVATE_KEY, associated_data=ASSOCIATED_DATA, nonce=NONCE, key=SYMMETRIC_KEY)

VERSION_NUMBER = 0x02

CIPHERTEXT_CONCATENATION = concat(VERSION_NUMBER, LOG_N, SALT, NONCE, ASSOCIATED_DATA, CIPHERTEXT)

ENCRYPTED_PRIVATE_KEY = bech32_encode('ncryptsec', CIPHERTEXT_CONCATENATION)

The output prior to bech32 encoding should be 91 bytes long.

The decryption process operates in the reverse.

Test Data

Password Unicode Normalization

The following password input: "ÅΩẛ̣" - Unicode Codepoints: U+212B U+2126 U+1E9B U+0323 - UTF-8 bytes: [0xE2, 0x84, 0xAB, 0xE2, 0x84, 0xA6, 0xE1, 0xBA, 0x9B, 0xCC, 0xA3]

Should be converted into the unicode normalized NFKC format prior to use in scrypt: "ÅΩṩ" - Unicode Codepoints: U+00C5 U+03A9 U+1E69 - UTF-8 bytes: [0xC3, 0x85, 0xCE, 0xA9, 0xE1, 0xB9, 0xA9]

Encryption

The encryption process is non-deterministic due to the random nonce.

Decryption

The following encrypted private key:

```
ncryptsec1qgg9947rlpvqu76pj5ecreduf9jxhselq2nae2kghhvd5g7dgjtcxfqtd67p9m0w57lspw8gsq6yphnm8623nsl8xn9j4jdzz8
```

When decrypted with password='nostr' and log_n=16 yields the following hex-encoded private key:

`3501454135014541350145413501453fefb02227e449e57cf4d3a3ce05378683`

Discussion

On Key Derivation

Passwords make poor cryptographic keys. Prior to use as a cryptographic key, two things need to happen:

1. An encryption key needs to be deterministically created from the password such that is has a uniform functionally random distribution of bits, such that the symmetric encryption algorithm's assumptions are valid, and
2. A slow irreversible algorithm should be injected into the process, so that brute-force attempts to decrypt by trying many passwords are severely hampered.

These are achieved using a password-based key derivation function. We use scrypt, which has been proven to be maximally memory hard and which several cryptographers have indicated to the author is better than argon2 even though argon2 won a competition in 2015.

On the symmetric encryption algorithm

XChaCha20-Poly1305 is typically favored by cryptographers over AES and is less associated with the U.S. government. It (or it's earlier variant without the 'X') is gaining wide usage, is used in TLS and OpenSSH, and is available in most modern crypto libraries.

Recommendations

It is not recommended that users publish these encrypted private keys to nostr, as cracking a key may become easier when an attacker can amass many encrypted private keys.

It is recommended that clients zero out the memory of passwords and private keys before freeing that memory.

NIP-98

HTTP Auth

`draft` `optional`

This NIP defines an ephemeral event used to authorize requests to HTTP servers using nostr events.

This is useful for HTTP services which are built for Nostr and deal with Nostr user accounts.

Nostr event

A `kind 27235` (In reference to [RFC 7235](https://www.rfc-editor.org/rfc/rfc7235)) event is used.

The `content` SHOULD be empty.

The following tags MUST be included.

- `u` - absolute URL
- `method` - HTTP Request Method

Example event:

```
{
  "id": "fe964e758903360f28d8424d092da8494ed207cba823110be3a57dfe4b578734",
  "pubkey": "63fe6318dc58583cfe16810f86dd09e18bfd76aabc24a0081ce2856f330504ed",
  "content": "",
  "kind": 27235,
  "created_at": 1682327852,
  "tags": [
    ["u", "https://api.snort.social/api/v1/n5sp/list"],
    ["method", "GET"]
  ],
  "sig":
      "5ed9d8ec958bc854f997bdc24ac337d005af372324747efe4a00e24f4c30437ff4dd8308684bed467d9d6be3e5a517bb43b

}
```

Servers MUST perform the following checks in order to validate the event: 1. The `kind` MUST be `27235`. 2. The `created_at` timestamp MUST be within a reasonable time window (suggestion 60 seconds). 3. The `u` tag MUST be exactly the same as the absolute request URL (including query parameters). 4. The `method` tag MUST be the same HTTP method used for the requested resource.

When the request contains a body (as in POST/PUT/PATCH methods) clients SHOULD include a SHA256 hash of the request body in a `payload` tag as hex (`["payload", "<sha256-hex>"]`), servers MAY check this to validate that the requested payload is authorized.

If one of the checks was to fail the server SHOULD respond with a 401 Unauthorized response code.

Servers MAY perform additional implementation-specific validation checks.

Request Flow

Using the `Authorization` HTTP header, the `kind 27235` event MUST be `base64` encoded and use the Authorization scheme `Nostr`

Example HTTP Authorization header:

```
Authorization: Nostr
eyJpZCI6ImZlOTY0ZTc1ODkwMzM2MGYyOGQ4NDI0ZDA5MmRhODQ5NGVkMjA3Y2JhODIzMTEwYmUzYTU3ZGZlNGI1Nzg3MzQi
LCJwdWJrZXkiOiI2M2ZlNjMxOGRjNTg1ODNjZmUxNjgxMGY4NmRkMDllMThiZmQ3NmFhYmMyNGEwMDgxY2UyODU2ZjMzMDUw
NGVkIiwiY29udGVudCI6IiIsImtpbmQiOjI3MjM1LCJjcmVhdGVkX2F0IjoxNjgyMzI3ODUyLCJ0YWdzIjpbWyJ1IiwiaHR0
cHM6Ly9hcGkuc25vcnQuc29jaWFsL2FwaS92MS9uNXNwL2xpc3QiXSxbIm1ldGhvZCIsIkdFVCJdXSwic2lnIjoiNWVkOWQ4
ZWM5NThiYzg1NGY5OTdiZGMyNGFjMzM3ZDAwNWFmMzcyMzI0NzQ3ZWZlNGEwMGUyNGY0YzMwNDM3ZmY0ZGQ4MzA4Njg0YmVk
NDY3ZDlkNmJlM2U1YTUxN2JiNDNiMTczMmNjN2QzMzk0OWEzYWFmODY3MDVjMjIxODQifQ
```

Reference Implementations

- C# ASP.NET `AuthenticationHandler` NostrAuth.cs

NIP-62

Request to Vanish

`draft` `optional` `relay`

This NIP offers a Nostr-native way to request a complete reset of a key's fingerprint on the web. This procedure is legally binding in some jurisdictions, and thus, supporters of this NIP should truly delete events from their database.

Request to Vanish from Relay

Kind `62` requests a specific relay to delete everything, including NIP-09 Deletion Events, from the `.pubkey` until its `.created_at`.

```
{
  "kind": 62,
  "pubkey": <32-byte hex-encoded public key of the event creator>,
  "tags": [
    ["relay", "<relay url>"]
  ],
  "content": "<reason or note>",
  //...other fields
}
```

The tag list MUST include at least one `relay` value.

Content MAY include a reason or a legal notice to the relay operator.

Relays MUST fully delete any events from the `.pubkey` if their service URL is tagged in the event.

Relays SHOULD delete all NIP-59 Gift Wraps that p-tagged the `.pubkey` if their service URL is tagged in the event, deleting all DMs to the pubkey.

Relays MUST ensure the deleted events cannot be re-broadcasted into the relay.

Relays MAY store the signed request to vanish for bookkeeping.

Paid relays or relays that restrict who can post MUST also follow the request to vanish regardless of the user's status.

Publishing a deletion request event (Kind `5`) against a request to vanish has no effect. Clients and relays are not obliged to support "unrequest vanish" functionality.

Clients SHOULD send this event to the target relays only.

Global Request to Vanish

To request ALL relays to delete everything, the event MUST include a `relay` tag with the value `ALL_RELAYS` in uppercase.

```
{
  "kind": 62,
  "pubkey": <32-byte hex-encoded public key of the event creator>,
  "tags": [
    ["relay", "ALL_RELAYS"]
  ],
  "content": "<reason>",
  //...other fields
}
```

Clients SHOULD broadcast this event to as many relays as possible.

Developers

The last chapter is for people writing clients. Users should never paste a private key into a website. These NIPs are the seams between an app, a signer, and the rest of the network.

NIP-07 is `window.nostr`, a JavaScript object a browser extension injects so a web app can request a public key, signatures, and encryption without seeing the `nsec`. NIP-55 is the Android equivalent: an intent-based signer application that other apps can call. Together they are how "log in with Nostr" works without turning every frontend into a wallet.

NIP-89 is recommended application handlers. A client publishes the kinds it knows how to open; another client can send a user there instead of rendering a blob it does not understand. That is how a specialized app (a chess board, a wiki, a calendar) gets traffic from a general-purpose feed.

NIP-31 is about dealing with unknown events: a `hint` field so a client can tell a user what an unfamiliar kind is supposed to be. It is unrecommended — unnecessarily bloated — but you will still see the idea discussed, which is why it is here. The better instinct is NIP-89: if you do not know how to render something, point at software that does.

In this chapter

- NIP-07 — `window.nostr` for browsers
- NIP-55 — Android signer application
- NIP-89 — Recommended application handlers
- NIP-31 — Dealing with unknown events (unrecommended; unnecessarily bloated)

NIP-07

`window.nostr` capability for web browsers

`draft` `optional`

The `window.nostr` object may be made available by web browsers or extensions and websites or web-apps may make use of it after checking its availability.

That object must define the following methods:

```
async window.nostr.getPublicKey(): string // returns a public key as hex
async window.nostr.signEvent(event: { created_at: number, kind: number, tags: string[][],
content: string }): Event // takes an event object, adds `id`, `pubkey` and `sig` and returns it
```

Aside from these two basic above, the following functions can also be implemented optionally:

```
async window.nostr.nip04.encrypt(pubkey, plaintext): string // returns ciphertext and iv as
specified in nip-04 (deprecated)
async window.nostr.nip04.decrypt(pubkey, ciphertext): string // takes ciphertext and iv as
specified in nip-04 (deprecated)
async window.nostr.nip44.encrypt(pubkey, plaintext): string // returns ciphertext as specified
in nip-44
async window.nostr.nip44.decrypt(pubkey, ciphertext): string // takes ciphertext as specified in
nip-44
```

Recommendation to Extension Authors

To make sure that the `window.nostr` is available to nostr clients on page load, the authors who create Chromium and Firefox extensions should load their scripts by specifying `"run_at": "document_end"` in the extension's manifest.

Implementation

See https://github.com/aljazceru/awesome-nostr#nip-07-browser-extensions.

NIP-55

Android Signer Application

`draft` `optional`

Rationale

This NIP describes a method for 2-way communication between an Android signer and any Nostr client running on the same device, so that the client never needs to handle the user's private key. The signer is an Android application; the client may be another Android application or a web page.

Terminology

- **user**: A person trying to use Nostr.
- **client**: A user-facing application that sends signing requests.
- **signer**: An Android application that holds the user's private key and answers requests from *client*.
- **user-pubkey**: The public key representing *user*, returned by `get_public_key`.

- **package name**: The Android package name of the *signer* (e.g. `com.example.signer`), used by *client* to address subsequent requests.

All pubkeys in this NIP are in hex format.

Communication methods

A *client* can talk to the *signer* in three ways:

- **Intents** — used by Android clients. The *signer* opens, the *user* approves or rejects the request manually, and the result is returned to *client*.
- **Content Resolver** — used by Android clients. Runs in the background without opening the *signer*, but only works for permissions the *user* has previously chosen to remember.
- **Web** — used by web clients. The result is returned via a `callbackUrl` or copied to the clipboard.

Setup

To detect and call the *signer*, an Android *client* must declare the `nostrsigner` scheme in its `AndroidManifest.xml`:

```xml
<queries>
  <intent>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="nostrsigner" />
  </intent>
</queries>
```

It can then check whether a *signer* is installed:

```kotlin
fun isExternalSignerInstalled(context: Context): Boolean {
  val intent = Intent(Intent.ACTION_VIEW, Uri.parse("nostrsigner:"))
  return context.packageManager.queryIntentActivities(intent, 0).isNotEmpty()
}
```

Initiating a connection

1. *client* sends a `get_public_key` request.
2. *signer* responds with the `user-pubkey` and its `package name`.
3. *client* stores both, and SHOULD NOT call `get_public_key` again while the *user* stays logged in.
4. *client* addresses all further requests to that `package name`.

A *client* MAY pass a list of `permissions` with `get_public_key` so the *user* can pre-authorize methods (used by the Content Resolver to answer in the background). Each permission is an object with a `type` (a method name) and, for `sign_event`, an optional `kind`:

```json
[{ "type": "sign_event", "kind": 22242 }, { "type": "nip44_decrypt" }]
```

Methods

Every request has a `type` (the method name) and a payload. The payload is passed differently per transport (see below) but its meaning is shared:

Method	Payload	Extra params	Result
`get_public_key`	*(empty)*	`permissions`	`user-pubkey`
`sign_event`	event JSON	`current_user`	the signature, and signed event
`nip04_encrypt`	plaintext	`pubkey`, `current_user`	the ciphertext
`nip44_encrypt`	plaintext	`pubkey`, `current_user`	the ciphertext
`nip04_decrypt`	ciphertext	`pubkey`, `current_user`	the plaintext
`nip44_decrypt`	ciphertext	`pubkey`, `current_user`	the plaintext
`decrypt_zap_event`	event JSON	`current_user`	the decrypted event JSON

- `current_user` is the `user-pubkey` currently logged in to *client*, in hex format.
- `pubkey` is the public key of the other party used for encryption/decryption, in hex format.
- `id` is an optional client-chosen string echoed back in the result, used to match responses when several requests are sent without waiting.

Using Intents

The payload is the `nostrsigner:` URI data; all other params are intent extras. The result is returned via `registerForActivityResult` / `rememberLauncherForActivityResult`.

```
val intent = Intent(Intent.ACTION_VIEW, Uri.parse("nostrsigner:$payload")).apply {
  `package` = signerPackageName        // omit only for get_public_key
  putExtra("type", "sign_event")
  putExtra("id", event.id)             // optional, echoed back
  putExtra("current_user", userPubkey)
}
launcher.launch(intent)
```

The result is read from the returned intent's extras:

Extra	Description
`result`	the method result (pubkey, signature, ciphertext, etc.)
`id`	the `id` sent in the request
`event`	the signed event JSON (`sign_event` only)
`package`	the *signer* package name (`get_public_key` only)
`rejected`	`true` when the *user* rejected the request

A `resultCode` other than `Activity.RESULT_OK` means the *signer* failed (e.g. it crashed), not that the *user* rejected the request. When the *user* rejects, the *signer* returns `RESULT_OK` with a `rejected` extra set to `true`.

```
onResult = { result ->
  if (result.resultCode != Activity.RESULT_OK) {
    // signer error (e.g. crash)
  } else if (result.data?.getBooleanExtra("rejected", false) == true) {
    // user rejected the request
  } else {
    val value = result.data?.getStringExtra("result")
  }
}
```

To sign multiple events without reopening the *signer* for each one, *client* adds the flags below and the *signer* returns an array of results. The *signer* must declare `android:launchMode="singleTop"` for this to work.

```
intent.addFlags(Intent.FLAG_ACTIVITY_SINGLE_TOP or Intent.FLAG_ACTIVITY_CLEAR_TOP)

// signer answers with:
intent.putExtra("results", listOf(Result(package = signerPackageName, result = signature, id =
        intentId)).toJson())
```

Using Content Resolver

The *client* queries `content://<package-name>.<TYPE>` (e.g. `SIGN_EVENT`, `NIP44_ENCRYPT`). The query's `selectionArgs` carry the payload and params in the order `[payload, pubkey, current_user]`:

```
val result = context.contentResolver.query(
  Uri.parse("content://com.example.signer.SIGN_EVENT"),
  listOf(eventJson, "", loggedInUserPubkey),
  null, null, null,
)
```

The cursor returns a `result` column (and, for `sign_event`, an `event` column with the signed event JSON). The query returns `null`, or a `rejected` column is present, when:

- the *user* did not enable "remember my choice" for this request,
- the `user-pubkey` is not present in the *signer*,
- the request `type` is not recognized, or
- the *user* chose to always reject this request (a `rejected` column is returned — *client* SHOULD NOT then fall back to an Intent).

```
if (result == null) return
if (result.getColumnIndex("rejected") > -1) return
if (result.moveToFirst()) {
  val value = result.getString(result.getColumnIndex("result"))
  // for sign_event also: result.getColumnIndex("event")
}
```

Clients SHOULD store the `user-pubkey` locally and avoid calling `get_public_key` after the user is logged in.

Using Web Applications

Web clients can't receive a result from an intent, so the *signer* returns the result via a `callbackUrl` (appended to the URL) or, if none is given, by copying it to the clipboard. The request is a `nostrsigner:` URL whose path is the payload and whose query string carries the params:

```
nostrsigner:<payload>?type=<method>&pubkey=<hex_pubkey>&callbackUrl=https://example.com/?result=
```

Query parameters:

- `type` — the method name.
- `pubkey` — the other party's hex pubkey (encryption/decryption methods).
- `callbackUrl` — where the *signer* appends the result. If omitted, the result is copied to the clipboard.
- `returnType` — `signature` or `event`.
- `compressionType` — `none` (default) or `gzip`. With `gzip` the returned event is `"Signer1"` + Base64(gzip(event json)); useful because intents and URLs have length limits.

```
window.href = `nostrsigner:${eventJson}?
        compressionType=none&returnType=signature&type=sign_event&callbackUrl=https://
        example.com/?event=`;
```

> Consider using [NIP-46: Nostr Remote Signing]() for web applications. With the approach here the web client can't call the *signer* in the background, so the *user* sees a popup for every request.

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <h1>Test</h1>

    <script>
        window.onload = function() {
            var params = new URL(window.location.href).searchParams;
            var result = params.get("event");
            if (result) alert(result);

            var json = { kind: 1, content: "test" };
            var encodedJson = encodeURIComponent(JSON.stringify(json));
            var anchor = document.createElement("a");
            anchor.href = `nostrsigner:${encodedJson}?
        compressionType=none&returnType=signature&type=sign_event&callbackUrl=https://
        example.com/?event=`;
            anchor.textContent = "Open External Signer";
            document.body.appendChild(anchor);
        }
    </script>
```

```html
</body>
</html>
```

NIP-89

Recommended Application Handlers

`draft` `optional`

This NIP describes `kind:31989` and `kind:31990`: a way to discover applications that can handle unknown event-kinds.

Rationale

Nostr's discoverability and transparent event interaction is one of its most interesting/novel mechanics. This NIP provides a simple way for clients to discover applications that handle events of a specific kind to ensure smooth cross-client and cross-kind interactions.

Parties involved

There are three actors to this workflow:

- application that handles a specific event kind (note that an application doesn't necessarily need to be a distinct entity and it could just be the same pubkey as user A)
    - Publishes `kind:31990`, detailing how apps should redirect to it
- user A, who recommends an app that handles a specific event kind
    - Publishes `kind:31989`
- user B, who seeks a recommendation for an app that handles a specific event kind
    - Queries for `kind:31989` and, based on results, queries for `kind:31990`

Events

Recommendation event

```json
{
  "kind": 31989,
  "pubkey": <recommender-user-pubkey>,
  "tags": [
    ["d", <supported-event-kind>],
    ["a", "31990:app1-pubkey:<d-identifier>", "wss://relay1", "ios"],
    ["a", "31990:app2-pubkey:<d-identifier>", "wss://relay2", "web"]
  ],
  // other fields...
}
```

The `d` tag in `kind:31989` is the supported event kind this event is recommending.

Multiple `a` tags can appear on the same `kind:31989`.

The second value of the tag SHOULD be a relay hint. The third value of the tag SHOULD be the platform where this recommendation might apply.

Handler information

```json
{
  "kind": 31990,
  "pubkey": "<application-pubkey>",
  "content": "<optional-kind:0-style-metadata>",
  "tags": [
    ["d", <random-id>],
    ["k", <supported-event-kind>],
    ["latest", "35128:<application-pubkey>:<site-d-identifier>", "wss://relay1"],
    ["next", "35128:<application-pubkey>:<next-site-d-identifier>", "wss://relay1"],
    ["web", "https://..../a/<bech32>", "nevent"],
    ["web", "https://..../p/<bech32>", "nprofile"],
    ["web", "https://..../e/<bech32>"],
    ["ios", ".../<bech32>"]
  ],
  // other fields...
}
```

- `content` is an optional `metadata`-like stringified JSON object, as described in NIP-01. This content is useful when the pubkey creating the `kind:31990` is not an application. If `content` is empty, the `kind:0` of the pubkey should be used to display application information (e.g. name, picture, web, LUD16, etc.)
- `k` tags' value is the event kind that is supported by this `kind:31990`. Using a `k` tag(s) (instead of having the kind of the `d` tag) provides:
    - Multiple `k` tags can exist in the same event if the application supports more than one event kind and their handler URLs are the same.
    - The same pubkey can have multiple events with different apps that handle the same event kind.
- App descriptor events SHOULD tag or otherwise reference related site manifest events when applicable, so clients can discover the deployed site manifest for an application directly from the upstream app descriptor.
- For web applications, `kind:31990` events SHOULD include `latest` and `next` tags referencing related nsite manifests when applicable. These tags use the same shape as an `a` tag: the second element is `<kind>:<pubkey>:<d-tag>` and the third element is an optional relay hint. `latest` SHOULD reference the current site manifest for the application, while `next` SHOULD reference the next site manifest intended for rollout or upgrade.
- `bech32` in a URL MUST be replaced by clients with the NIP-19-encoded entity that should be loaded by the application.

Multiple tags might be registered by the app, following NIP-19 nomenclature as the second value of the array.

A tag without a second value in the array SHOULD be considered a generic handler for any NIP-19 entity that is not handled by a different tag.

Client tag

When publishing events, clients MAY include a `client` tag. Identifying the client that published the note. This tag is a tuple of `name`, `address` identifying a handler event and, a relay `hint` for finding the handler event. This has privacy implications for users, so clients SHOULD allow users to opt-out of using this tag.

```
{
  "kind": 1,
  "tags": [
    ["client", "My Client", "31990:app1-pubkey:<d-identifier>", "wss://relay1"]
  ]
  // other fields...
}
```

User flow

A user A who uses a non-`kind:1`-centric nostr app could choose to announce/recommend a certain kind-handler application.

When user B sees an unknown event kind, e.g. in a social-media centric nostr client, the client would allow user B to interact with the unknown-kind event (e.g. tapping on it).

The client MIGHT query for the user's and the user's follows handler.

Example

User A recommends a `kind:31337`-handler

User A might be a user of Zapstr, a `kind:31337`-centric client (tracks). Using Zapstr, user A publishes an event recommending Zapstr as a `kind:31337`-handler.

```
{
  "kind": 31989,
  "tags": [
    ["d", "31337"],
    ["a", "31990:1743058db7078661b94aaf4286429d97ee5257d14a86d6bfa54cb0482b876fb0:abcd", <relay-
        url>, "web"]
  ],
  // other fields...
}
```

User B interacts with a `kind:31337`-handler

User B might see in their timeline an event referring to a `kind:31337` event (e.g. a `kind:1` tagging a `kind:31337`).

User B's client, not knowing how to handle a `kind:31337` might display the event using its `alt` tag (as described in NIP-31). When the user clicks on the event, the application queries for a handler for this `kind`:

```
["REQ", <id>, { "kinds": [31989], "#d": ["31337"], "authors": [<user>, <users-contact-list>] }]
```

User B, who follows User A, sees that `kind:31989` event and fetches the `a`-tagged event for the app and handler information.

User B's client sees the application's `kind:31990` which includes the information to redirect the user to the relevant URL with the desired entity replaced in the URL.

Alternative query bypassing `kind:31989`

Alternatively, users might choose to query directly for `kind:31990` for an event kind. Clients SHOULD be careful doing this and use spam-prevention mechanisms or querying high-quality restricted relays to avoid directing users to malicious handlers.

```
["REQ", <id>, { "kinds": [31990], "#k": [<desired-event-kind>], "authors": [...] }]
```

> **Warning** `unrecommended`: unnecessarily bloated

NIP-31

Dealing with unknown event kinds

`draft` `unrecommended` `optional`

When creating a new custom event kind that is part of a custom protocol and isn't meant to be read as text (like `kind:1`), clients should use an `alt` tag to write a short human-readable plaintext summary of what that event is about.

The intent is that social clients, used to display only `kind:1` notes, can still show something in case a custom event pops up in their timelines. The content of the `alt` tag should provide enough context for a user that doesn't know anything about this event kind to understand what it is.

These clients that only know `kind:1` are not expected to ask relays for events of different kinds, but users could still reference these weird events on their notes, and without proper context these could be nonsensical notes. Having the fallback text makes that situation much better – even if only for making the user aware that they should try to view that custom event elsewhere.

`kind:1`-centric clients can make interacting with these event kinds more functional by supporting NIP-89.

Conclusion

That's the book. The NIPs themselves will keep changing — new kinds, better encryption, ideas that get tried and later marked unrecommended. This compilation is a map of the territory at one point in time, not the territory.

If you want to change a NIP, do not open a pull request here. Go to nostr-protocol/nips and talk to the people who implement this stuff. This repository is only the binding: chapter intros, reading order, and the scripts that turn the specs into an ebook.

If a grouping feels wrong, or a chapter intro undersells a NIP, those words are mine and I am happy to hear about them.

Thanks for reading. Credit for the protocol belongs to the NIP authors. I hope having their work in one place, in an order that tells a story, makes it easier to build the next client, relay, or weird little kind that nobody expected.